Supplement to "A strongly regular graph with parameters $(99, 14, 1, 2)$ has no automorphism of order two"

Texts of the repository as of 7 October 2026

This supplement reproduces, without any rewriting, the texts of the repository `conway99-c2-aws` (GitHub, account `amazing-source`) on which the paper relies for the three lemmas T_{16}, T_{18}, T_{20} (Theorem 7.1 of the paper), for lattice 143 (Proposition 8.3) and for the rules of the search program (Section 8.4). They are working documents: plain text, with the notation of the project (in particular the properties (R-i) to (R-vii) are (R1) to (R7) of the paper, A.0 to A.9 are Sections 3 to 6, and "Theorem R" is Theorem 6.1). The proof of T_{16} is a dossier in French, kept as received; Part 1 begins with its index in English. Long lines of tables are wrapped. The programs, outputs and referee reports named in these texts are in the repository.

Contents

1 The lemma T_{16} **2**
- 1.1 Index, statement and status . . . 2
- 1.2 The equality case 8/7: values, active orbits, the case k = 2 . . . 3
- 1.3 The two cubic graphs; the cube . . . 7
- 1.4 The Wagner graph . . . 11

2 The lemma T_{18} **17**

3 The lemma T_{20} **45**

4 Where the computations of T18 and T20 are **81**

5 Lattice 143 **82**

6 The rules of the search program, derived from the properties of a frame **84**

1 The lemma T_{16}

1.1 Index, statement and status

File: lemma/T16_PROOF.md

```
# T16: no sigma-odd integer (-4)-eigenfunction of squared norm 16 (dual minimum > 8/7)

Index written by the lab session on 2026-10-07 for ONE referee pass (board, hole H5 of ‘lemma/upstream/AUDIT_ENGINE_RULES.md‘).
**The proof text is the dossier of 04/10/2026, in French, kept as received in ‘lemma/t16/dossier_2026-10-04/‘.** It was written by
ChatGPT for the user; it is not rewritten here. This page says what is claimed, where each step is, what is computed, and who
checked what.

## Statement

Gamma = srg(99,14,1,2), sigma an automorphism of order 2 with exactly one fixed vertex p. There is no sigma-odd integer function
phi on the vertices with A phi = -4 phi and sum phi^2 = 16 (i.e. the dual of the frame lattice N has minimum >= 9/7). Consequently, with
Theorem R of ‘lemma/upstream/AUDIT_REDUCTION.md‘: in the chosen envelope U, E(v) = 7 (v,v) - (v,y_1)^2 - (v,y_2)^2 is never 8, so
E(v) >= 9 for v outside Z y_1 + Z y_2. (Only this direction is used: the projection of U may be a proper sublattice of N^*; referee’s
remark, ‘lemma/t20_CODEX/REFEREE_T16_PROOF.md‘.)

## What rests on it

- The engine rule FD_MIN97 of package v10: "norm-3 x with (x,y_1)^2 + (x,y_2)^2 = 13" (E = 21 - 13 = 8), verdict
  "EXCLUDED (dual min 9/7: norm-3 x with Y=13)".
- The quick deferral path uses the engine’s own min97_witness test before deferring.
- T18 and T20 do not use it (they exclude the squared norms 18 and 20 directly); the dual-norm rule of ‘t20_HAND/NOTES.md‘ H6 needs
  only 8/7.

## Structure of the proof, and where each step is

| Step | Content | Where | Kind |
|---|---|---|---|
| 0 | E(v) >= 8 (no function of squared norm <= 14) | ‘lemma/upstream/AUDIT_REDUCTION.md‘ A.5 (redone by the laptop session on 07/10) |
   hand |
| 1 | Equality E = 8: phi has values 0, +1, -1, eight +1 (the set P) and eight -1 (Q = sigma P) | ‘LEMME_EGALITE_8_SUR_7_C2.txt‘ section
    2 | hand |
| 2 | The number k of active inner orbits is 0 or 2; P and Q are independent | same, sections 3, 4 | hand |
| 3 | k = 2 is impossible (the matrix B "with two diagonals" does not exist) | same, section 5 | hand |
| 4 | k = 0: B = A[P, Q] is the complement of a cubic graph; two types remain: the complement of the cube Q_3 and the complement of the
   Wagner graph | ‘C2_PROLONGEMENT_SUPPORT16_PREUVE_CUBE.txt‘ sections 7, 8 | hand |
| 5 | Cube complement: impossible, by a double count (at least 10 incidences, at most 8) | same, sections 2-6 | hand |
| 6 | Wagner complement: impossible. Global constraints and the eight "rectangles"; profile B dies by hand (three demands, two partners)
   ; profile A by an exhaustive exact enumeration with certificates | ‘C2_WAGNER_BORNE_9_SUR_7.txt‘ sections 2-6; programs ‘
   ENUMERER_WAGNER_PROFIL_A.py‘, ‘ENUMERER_WAGNER_PROFIL_B.py‘ (Python standard library), certificates ‘WAGNER_PROFIL_*_CERTIFICATS.json
   ‘, verifier ‘VERIFIER_WAGNER_CERTIFICATS.py‘ | hand + computed |
| 7 | Corollary: the equality case 8/7 is impossible | same, section 8 | - |

## Who checked what (not a referee pass)

- Lab session, 04/10 (‘lemma/t16/lab_check/VERIF_CUBE_OBSTRUCTION_2026-10-04.md‘): steps 4 (the reduction to the two types as used
  there), 5 (every step of the cube double count re-derived by hand), the structural part of step 6, and the derivation of the pair
  criterion Y = 13 with measurements. The active-h lemma (steps 1-3) was re-read the same day.
- An independent agent of the lab session, 04/10 (‘lemma/t16/independent_check/‘): the two Wagner enumerations redone by another
  program (‘crosscheck_stage2.py‘, outputs ‘out_A.txt‘, ‘out_B.txt‘, ‘crosscheck_A.txt‘, ‘crosscheck_B.txt‘).
- The laptop session, 07/10: steps 0-2 redone by hand as far as "e = 0, P independent, every vertex of P has 4 neighbours in Q, the
  outside vertices contribute exactly 8 common-neighbour pairs".

## Referee pass (Codex, 07/10)

Done: ‘lemma/t20_CODEX/REFEREE_T16_PROOF.md‘. No blocking mathematical error. One defect in the dossier’s verifier
(‘VERIFIER_WAGNER_CERTIFICATS.py‘, check_B does not test W <= set(V)); with the missing test added all 32 shipped profile-B
witnesses and all 240 profile-A obstructions pass again (‘referee_t16_checks.py‘). The dossier is left as received.

## For the referee

Errors only, with file and section references. Points that deserve the closest look: (a) step 4, that the cube and the Wagner graph
are the only cubic graphs left, with the exact hypotheses under which the others are discarded; (b) step 3; (c) step 6: that the
two enumerations are exhaustive for what the text says they cover (section 3 of the Wagner text, "couverture de la réduction"), and
that every rejection in the certificates is an exact integer statement.
```

1.2 The equality case 8/7: values, active orbits, the case k = 2

File: lemma/t16/dossier_2026-10-04/LEMME_EGALITE_8_SUR_7_C2.txt

```
Conway-99, C2 : le cas d'égalité 8/7 ne touche aucun h
4 octobre 2026 -- deuxième avis mathématique

RÉSULTAT ET PORTÉE

PROUVÉ ICI, dans le cadre d'un vrai graphe SRG(99,14,1,2) muni de
l'involution à un seul sommet fixe et de son cadre physique 7 h + 42 u :

    v dans le dual du réseau du cadre, v^2 = 8/7
              implique (v,h_c) = 0 pour chacun des sept c.

La preuve ci-dessous est humaine et autonome pour cette implication.
Elle s'appuie sur le raccord externe entre le graphe et son cadre,
notamment le Gram 3I-S et l'identité de cadre serré de constante 7.
Elle ne constitue pas une vérification formelle Lean.

Dans une enveloppe U avec normales y1,y2, on en déduit :

    x dans U, x^2=3, (x,y1)^2+(x,y2)^2=13
              implique (x,h_c)=0 pour tous les h d'une solution.

Il s'agit d'une condition pour EX : elle utilise le vrai graphe, donc
lambda=1 et mu=2. Un cadre relâché ou une matrice signée N seule ne
peuvent pas être exclus par cette preuve sans leur raccord au graphe.
T4 reste le lemme de travail du dossier pour reconstruire ce graphe
depuis une solution (N,D). F0 n'intervient pas dans cette implication.

Aucun des quatorze réseaux ouverts n'est déclaré exclu ici. Cette
preuve ferme une branche du cas d'égalité, pas le cas d'égalité entier.

1. NOTATIONS ET ENTRÉES EXTERNES

Le sommet fixe est p. Les sept orbites intérieures sont les paires
adjacentes dans N(p). Les quarante-deux autres orbites sont extérieures
et leurs deux sommets sont non adjacents.

Les vecteurs du cadre sont f_i : sept h de norme 4 et quarante-deux u
de norme 3. Leur Gram est H=3I-S, où S est la matrice de l'adjacence
sur le secteur impair de l'involution. On a H^2=7H et

    somme_i f_i f_i^t = 7 I

sur l'espace physique de dimension 22. Les h sont orthogonaux entre
eux ; les produits hors diagonale sont dans {0,+1,-1}. Chaque u a
exactement deux produits non nuls avec les h, chacun égal à +1 ou -1.

Sources lues pour ces raccords et conventions :
- notes/56_PROMPT_ASTRA_ROUTE_C_28_09.md, §1.1 ;
- external_inputs/CONWAY_ASTRA_27_09_2026_0836/EXTRAIT/
  attaque_globale_27_0803/AUDIT_NORME1_ET_METRIQUES.md ;
- MODEL.md et HANDOVER_2026-10-04.md.

La borne min N* >= 8/7 est ancienne. Nous travaillons seulement sur
son cas d'égalité. La note 54 du laboratoire traite déjà le sous-cas
sans h actif ; elle le laisse ouvert. Nous ne le réannonçons pas fermé.
La recherche ciblée dans les notes accessibles n'a pas retrouvé la
présente fermeture du sous-cas avec h actifs. Ce constat n'établit
aucune priorité dans la littérature ni dans des notes non accessibles.

2. UN VECTEUR DUAL D'ÉGALITÉ DONNE HUIT COEFFICIENTS UNITAIRES

Posons a_i=(v,f_i). Ces coefficients sont entiers et

    Ha=7a,  Sa=-4a,  somme_i a_i^2=7v^2=8.

Si |a_i|=2, les autres coefficients ont somme de carrés 4 et donc
somme de valeurs absolues au plus 4. La ligne Ha=7a demanderait une
somme hors diagonale de valeur absolue 6 pour un h (diagonale 4),
ou 8 pour un u (diagonale 3). Les coefficients hors diagonale du Gram
ont valeur absolue au plus 1. C'est impossible. Une valeur absolue
au moins 3 est déjà incompatible avec la somme de carrés 8.

Ainsi a appartient à {0,+1,-1}^49 et son support comporte exactement
huit orbites. Son relevé au graphe vaut a_i sur un représentant,
-a_i sur son mate, et 0 au sommet fixe. Il vérifie Af=-4f.

On note P les huit sommets de valeur +1 et Q=sigma(P) ceux de valeur
-1. Choisir ces représentants ne change aucune adjacence du graphe.

3. LE NOMBRE k DE h ACTIFS EST 0 OU 2
```

Écrivons S=[-I,C;C^t,T]. Chaque colonne de C a deux valeurs non nulles,
dans {+1,-1}. La ligne intérieure de Sa=-4a donne

    C a_u = -3 a_h.

La somme d'une colonne de C est paire ; la somme des a_h est donc
paire. Comme les a_h actifs valent +1 ou -1, k est pair.

Chaque h actif demande au moins trois incidences avec les u actifs
dans cette égalité. Un u actif en fournit au plus deux. Il y a 8-k
u actifs, donc

    3k <= 2(8-k), soit 5k <= 16.

Comme k est pair, k=0 ou k=2. Il reste à réfuter k=2.

4. P ET Q SONT INDÉPENDANTS

Soit e le nombre d'arêtes de P, et d_i les degrés internes dans P.
L'involution donne autant d'arêtes dans Q. L'équation Af=-4f impose
deg_Q(i)=d_i+4 pour i dans P.

Comptons les voisins communs des paires non ordonnées de P. Pour
|P|=m, le total est m(m-1)-e, puisque chaque paire adjacente a un
commun et chaque paire non adjacente en a deux. Les sommets de P et Q
contribuent

    somme_i [C(d_i,2)+C(d_i+4,2)]
         = somme_i d_i^2 + 6e + 6m.

La contribution des autres sommets est donc

    D = m(m-7) - 7e - somme_i d_i^2 >= 0.

Ici m=8. Si e>=1, somme d_i^2 >= somme d_i = 2e, donc
D <= 8-9e < 0. Ainsi e=0. P et Q sont indépendants et chaque sommet
de P a exactement quatre voisins dans Q.

5. LA MATRICE B À DEUX DIAGONALES NE PEUT PAS EXISTER

Ordonnons Q par les mates des sommets de P et posons B=A[P,Q].
L'involution rend B symétrique. B est binaire, de somme de ligne 4.
Chaque intersection de deux lignes distinctes a cardinal au plus 2,
car elle compte des voisins communs dans Q de deux sommets de P.

Si k=2, exactement deux diagonales de B valent 1 : ce sont les deux
orbites intérieures actives. Appelons-les 0 et 1. B01=0, car N(p)
induit un couplage parfait et ces deux sommets appartiennent à deux
paires distinctes.

De plus, les sommets 0 et 1 de P ont déjà p pour voisin commun, hors
Q. L'intersection de leurs lignes de B a donc cardinal au plus 1.

Écrivons ces lignes {0} union A et {1} union C, où A et C sont des
triplets parmi les six indices restants. On a |A intersect C|<=1.
La sous-matrice sur ces six indices n'a pas de diagonale.

Cas A intersect C vide.
Chaque a dans A a un voisin 0. L'intersection de sa ligne avec celle
de 0 impose au plus un voisin dans A ; son intersection avec la
ligne de 1 impose au plus deux voisins dans C. Pour atteindre son
degré 4, il doit avoir exactement un voisin dans A et deux dans C.
Le graphe sur le triplet A serait donc 1-régulier, ce qui est
impossible sur un ensemble de cardinal impair.

Cas |A intersect C|=1.
Écrivons A={s,a,b}, C={s,c,d}, et t pour le sixième indice.
Le sommet t n'est relié ni à 0 ni à 1 et a quatre voisins parmi
{s,a,b,c,d}. Les intersections avec les lignes 0 et 1 bornent ses
nombres de voisins dans A et C par 2 chacun. S'il était relié à s,
leur somme vaudrait 5. Donc ts=0 et t est relié à a,b,c,d.

Le sommet s est relié à 0 et 1. Il lui faut deux autres voisins,
au plus un parmi a,b et au plus un parmi c,d. Après permutation,
ce sont a et c. Ainsi sa=sc=1, sb=sd=st=0.

Le sommet a a déjà 0,s,t pour voisins. L'intersection des lignes
a et 0 interdit ab. Symétriquement cd est interdit. Le sommet b,
déjà relié à 0,t, doit donc être relié à c et d ; le sommet d,
déjà relié à 1,t, doit être relié à a et b.

Les lignes de a et b sont alors respectivement

{0,s,t,d} et {0,t,c,d}.

Elles ont trois indices communs : 0,t,d. Ceci contredit mu=2.

Les deux cas sont impossibles. Donc k ne peut pas valoir 2.
On a k=0 : tout vecteur dual de norme 8/7 est orthogonal à tous les h.

6. CONSÉQUENCE DANS CHAQUE ENVELOPPE

Soit x dans U, x^2=3, et a=(x,y1), b=(x,y2), a^2+b^2=13.
Sa projection sur l'espace physique est

v=x-(a y1+b y2)/7,
v^2=3-13/7=8/7.

Les h et u appartiennent à U et sont orthogonaux aux normales.
L'intégralité de U implique (v,h),(v,u) entiers ; v est donc dans
le dual du réseau du cadre. Le résultat prouvé donne

(x,h_c)=(v,h_c)=0 pour chaque c.

Tous les h d'une solution sont donc dans

{y1,y2}^perp intersect span{x dans U3 : Y(x)=13}^perp.

Les projections de ces x sur {y1,y2}^perp doivent avoir rang au plus
15, puisque les sept h indépendants sont orthogonaux à leur span
dans un espace de dimension 22. Un rang certifié au moins 16
exclurait la paire entière. Ce critère n'est pas déclenché par
l'échantillon ci-dessous.

La même preuve montre |(v,u)|<=1 pour chaque u choisi et exactement
huit u choisis avec produit non nul. Ces contraintes supplémentaires
ne doivent pas être assimilées à un témoin explicite du graphe.

7. VÉRIFICATIONS ET MESURE LIMITÉE

CALCULÉ, contrôles arithmétiques indépendants :
- Une génération récursive des matrices symétriques 8x8 à deux
  diagonales donne six matrices lorsque la borne sur les deux lignes
  intérieures est relâchée à 2. Toutes ont intersection 2 sur ces
  lignes : la présence du sommet fixe est une entrée indispensable.
- Une deuxième génération, par les 2^15 graphes sur les six autres
  indices, donne zéro matrice pour intersection 0 et zéro pour
  intersection 1. Elle soutient la preuve humaine, sans la remplacer.
- Le complément du cube fournit un contrôle positif du problème
  abstrait sans diagonale. Ce n'est pas un graphe Conway.

CALCULÉ, première mesure sur exactement les 25 premières paires
stockées dans profiling/011_sample_pairs.json :
- 0 à 3 directions courtes par paire ; rang 0 à 3 ;
- 146 164 occurrences de candidats h après les restrictions usuelles
  de perpendicularité, ombres et frontière caractéristique ;
- 108 023 après la nouvelle perpendicularité, soit 38 141 retirées
  (26,09 % au total ; entre 0 et 43,37 % selon la paire) ;
- aucune des 25 paires n'est rejetée par le rang ou le simple nombre
  de candidats couvrant chaque point ;
- aucun temps de recherche complète ou gain de production mesuré.

Les occurrences ne sont pas toutes des vecteurs distincts, car le
même h peut apparaître pour plusieurs paires. Les 25 paires ne sont
pas un échantillon aléatoire ni une couverture de 011. Aucun nombre
d'heures économisées ne peut être déduit de ces comptes.

Les bases et Gram ont été vérifiés en arithmétique rationnelle et
entière. Les coquilles locales sont énumérées avec bornes flottantes,
leurs normes sont vérifiées exactement et leurs cardinalités sont
comparées aux formules de coque du dossier. Pour un certificat de
couverture de réseau, reprendre l'énumération certifiée du projet.
Pour retirer un h donné, une seule direction x exacte suffit :
sa présence, sa norme 3, Y(x)=13 et (h,x) non nul sont vérifiables
en entiers, sans exiger de compléter la coque.

Toutes les vérifications nouvelles ont duré moins de cinq secondes
par processus. Limites déclarées : un processus local, 240 secondes
maximum par vérification ; budget de 512 Mio pour le test des paires, 64 Mio
pour le contrôle combinatoire. Aucun job cloud ou dépôt n'a été changé.

8. OBSTACLE RESTANT

Le cas k=0 subsiste : une fonction propre de valeur -4 peut encore
avoir son support de huit orbites toutes extérieures. La note 54
signalait déjà deux types possibles de matrice B et de nombreuses
étiquetages locaux compatibles. La présente preuve ne les réfute pas.

Pour fermer un réseau entier sans recherche des h, il manque un
argument couvrant toutes ses paires de normales et montrant que les
sept h ne peuvent vivre dans l'orthogonal imposé, ou que le support
extérieur de taille huit ne peut pas se prolonger au graphe complet.
Le test de rang simple ne fournit pas cet argument sur les 25 paires.

Conclusion de statut : nouvelle restriction mathématique démontrée
dans cette note ; zéro nouvelle exclusion complète de réseau ;
aucune réduction d'heures de calcul annoncée.

1.3 The two cubic graphs; the cube

File: lemma/t16/dossier_2026-10-04/C2_PROLONGEMENT_SUPPORT16_PREUVE_CUBE.txt

```
Conway-99, C2 : obstruction au prolongement du support de taille 16
4 octobre 2026 -- preuve humaine et portée exacte

RÉSULTAT

PROUVÉ ICI : dans un vrai SRG(99,14,1,2), un vecteur propre f pour -4,
à valeurs 0,+1,-1, avec huit +1 et huit -1, impair sous une involution,
ne peut avoir pour matrice B entre ses deux côtés le complément du cube,
lorsque ses huit orbites actives sont extérieures.

La contradiction est un double compte : au moins 10 incidences, au plus 8.
Dans le cadre C2 à un sommet fixe, la borne inférieure se renforce à 12.

Ce résultat ferme entièrement B4 de la note 54, dans un vrai graphe.
Il ne ferme pas B5 (complément du graphe de Wagner). Il ne démontre donc
ni min N* >= 9/7, ni EX faux, ni une nouvelle exclusion complète de réseau.
Une configuration locale qui survit ne constitue pas un prolongement.

1. HYPOTHÈSES ET DICTIONNAIRE

A est l'adjacence d'un SRG(99,14,1,2) :
  A^2+A = 12I+2J.
L'involution σ fixe p et échange les 49 autres paires de sommets.
Elle a sept paires adjacentes, exactement les sept paires de N(p).
Les autres paires, dites extérieures, sont non adjacentes.

On suppose Af=-4f et f∘σ=-f, avec
  P={f=+1}, Q={f=-1}=σP, |P|=|Q|=8.
Les huit orbites actives sont extérieures. P et Q sont indépendants.
Le double compte qui établit cette dernière propriété est rappelé §8.

Numérotons Q par les mates de P. B=A[P,Q] est alors symétrique, binaire,
sans diagonale, de somme de ligne 4. Son carré a ses entrées hors
diagonale au plus 2, puisqu'elles comptent des voisins communs dans Q.

Pour z hors P∪Q, Af(z)=0 donne
  |N(z)∩P|=|N(z)∩Q|=:d_z.
Notons X_d l'ensemble des sommets extérieurs au support avec d_z=d.
Attention : « extérieur au support » inclut p et les sommets de N(p),
et ne signifie pas ici « orbite extérieure à N[p] ».

2. LES TYPES EXTÉRIEURS AU SUPPORT

PROUVÉ AILLEURS, puis redérivé ici : d_z≤2.
Si z voyait trois sommets de P, leurs trois ensembles de quatre voisins
dans Q auraient des intersections deux à deux au plus 1, car z consomme
déjà un des deux voisins communs permis. Leur union aurait cardinal
au moins 12-3=9, dans un ensemble Q de cardinal 8 : impossible.

Les huit sommets de P ont chacun dix voisins hors du support, donc
  Σ_z d_z=80.
Les paires de P ont au total 2·C(8,2)=56 voisins communs. Q en fournit
8·C(4,2)=48, et P n'en fournit aucun. Par conséquent
  Σ_z C(d_z,2)=8.
Il s'ensuit
  |X_0|=11, |X_1|=64, |X_2|=8.

3. LE COMPTE GLOBAL DES ARÊTES X_0--X_2

Pour z∈X_d, notons a_z,b_z,c_z ses nombres de voisins dans X_2,X_1,X_0.
Comptons les chemins de longueur deux entre z et les 16 sommets P∪Q.
Le SRG en demande 2·16-2d=32-2d : les 2d paires adjacentes ont un
commun au lieu de deux.
Les 2d voisins de z situés dans le support contribuent chacun 4.
Les autres contribuent 2 pour un voisin dans X_1, et 4 dans X_2. Ainsi
  8d+2b_z+4a_z=32-2d,
  b_z+2a_z=16-5d.
La somme des degrés hors support vaut 14-2d, donc
  c_z=a_z+3d-2.
En particulier, sur X_0, c_z=a_z-2 et a_z≥2.

Posons t=e(X_2). Sur X_2 on a c_z=a_z+4 et Σ_{X_2}a_z=2t. Donc
  e(X_0,X_2)=Σ_{X_2}c_z=32+2t,
  Σ_{z∈X_0}(a_z-2)=32+2t-22=10+2t≥10.                (1)

Ces identités sont anciennes : elles figurent dans Wave 81, §5.
Elles ne sont pas présentées comme une invention de cette note.

4. L'INVOLUTION FORCE UN CYCLE SUR QUATRE PAIRES DÉFICITAIRES
```

Supposons maintenant B égal au complément du cube Q_3. Numérotons les sommets du cube par les trois bits. Les antipodes i et i⊕111 ont zéro voisin commun dans B ; toutes les autres paires ont deux voisins communs.

Pour une paire {i,j} de P, le nombre de ses voisins communs hors Q est
  $m_{ij}=2-(B^2)_{ij}$.
Puisque $d_z\leq 2$, chaque contribution non nulle vient d'un sommet de X_2 dont les deux voisins dans P sont précisément {i,j}.

Les huit sommets X_2 ont donc pour paires de voisins dans P les quatre paires antipodales T_0,T_1,T_2,T_3, chacune exactement deux fois. La même propriété vaut pour leurs paires de voisins dans Q.

Si z∈X_2 voit T_i dans P et T_j dans Q, σz voit T_j dans P et T_i dans Q. L'orbite {z,σz} donne donc une arête i--j sur les quatre labels T. Les quatre orbites X_2 forment un multigraphe de degré 2 sur ces labels.

Ce multigraphe n'a pas de boucle : i=j donnerait à z et σz quatre voisins communs dans P∪Q. Il n'a pas non plus d'arête répétée : deux orbites couplées aux mêmes T_i,T_j fourniraient deux sommets distincts ayant les mêmes quatre voisins dans le support. Dans les deux cas, on dépasserait $\mu=2$.

Le multigraphe est donc un graphe simple 2-régulier sur quatre sommets : exactement un cycle C_4. Aucun couplage possible n'est omis dans cet argument. Il n'impose pas qu'un rectangle de support soit un couplage PARFAIT : une telle exigence serait injustifiée dans le cas de Wagner.

5. DEUX CYCLES C_4 DE CONCURRENCE SUR X_2

Identifions les huit sommets X_2 aux huit arcs orientés i→j du cycle précédent. Les voisins dans le support de i→j sont T_i dans P et T_j dans Q.

Deux arcs distincts ont deux voisins communs dans le support exactement quand ils ont la même origine ou la même extrémité ; ils en ont zéro sinon. La relation « deux voisins communs dans le support » forme deux cycles C_4 disjoints sur X_2. En effet, si le cycle des labels a les classes bipartites {0,2} et {1,3}, un cycle de concurrence est constitué des quatre arcs de la première classe vers la seconde ; l'autre contient leurs arcs inverses.

Un sommet z∈X_0 n'est voisin d'aucun sommet du support. Il ne peut être voisin simultanément de deux sommets consécutifs d'un de ces cycles : ces deux sommets auraient leurs deux voisins communs du support PLUS z. $N(z)\cap X_2$ est donc indépendant dans chacun des deux C_4.

6. CONTRADICTION : AU MOINS 10, AU PLUS 8

Soient r_z et s_z les nombres de voisins de z∈X_0 dans les deux cycles. On a $r_z,s_z\in\{0,1,2\}$ et $r_z+s_z=a_z$. Deux voisins dans un cycle doivent être opposés. Le nombre de diagonales des cycles vues entièrement par z est
  $q_z=C(r_z,2)+C(s_z,2)\geq r_z+s_z-2=a_z-2$.

En sommant et en appliquant (1),
  $\Sigma_{X_0}q_z\geq 10+2t\geq 10$.
Mais les deux cycles ont ensemble quatre diagonales. Chaque diagonale est une paire de sommets du vrai graphe, qui a au plus deux voisins communs. Elle peut donc être vue entièrement par au plus deux sommets de X_0 :
  $\Sigma_{X_0}q_z\leq 4\cdot 2=8$.

Contradiction. Le support avec B complément du cube ne se prolonge pas. Cette preuve ne dépend ni de l'énumération des 156 réseaux, ni d'un solveur, ni d'une liste d'étiquetages de racines.

7. RÉDUCTION GLOBALE COMPLÉMENTAIRE : t=1 ET DEUX PROFILS

Cette section apporte une restriction valable aussi pour B de Wagner. Elle ne ferme pas ce dernier type.

Chaque sommet du support voit deux X_2 : la somme de ses déficits est $14-4\cdot 3=2$. Le support contribue donc 16 aux communs des paires de X_2. Le total des communs de ces paires est 56-t. Ainsi
  $\Sigma_z C(a_z,2)=40-t$.
De §3 on tire $\Sigma_{X_0}a=32+2t$ et $\Sigma_{X_1}a=48-4t$; avec $\Sigma_{X_2}a=2t$:
  $\Sigma_z a_z=80$, $\Sigma_z a_z^2=160-2t$.
Sur X_0, $a^2\geq 5a-6$ car a est entier et $a\geq 2$.
Sur X_1 et X_2, $a^2\geq a$. Par conséquent
  $160-2t\geq 5(32+2t)-66+(48-4t)+2t=142+8t$,
  $t\leq 1$.
La borne $t\leq 1$ est ancienne (Wave 81 la vérifie par énumération exacte) ; la présente inégalité en donne une preuve humaine courte.

Dans C2, p∈X_0. Comme $c_p=a_p-2\geq 0$, p voit au moins deux X_2.

Ses voisins sont des paires adjacentes de σ. Chaque paire de X_2 voisine
de p est donc une arête interne de X_2. D'où t≥a_p/2≥1.
Ainsi
  t=1, a_p=2, b_p=12, c_p=0.
L'unique arête de X_2 est l'une des sept paires de N(p). Les six autres
paires de N(p) sont dans X_1. Le profil de leurs voisins dans P est
exactement (2,1,1,1,1,1), à permutation près.

Les défauts quadratiques satisfont
  Σ_{X_0}(a-2)(a-3)+Σ_{X_1∪X_2}a(a-1)=18-10t=8.
Le terme X_2 est nul (deux degrés 1, six degrés 0), celui de p aussi.
Après division par les orbites de taille deux, le budget vaut 4.
Les cinq orbites non fixes de X_0 ont Σ a=16. Leur a est 2,3 ou 4,
car a≥5 consommerait à lui seul au moins 6. Si n_j compte ces orbites,
  n_4-n_2=1, n_4≤2.
On obtient exactement deux profils, où les nombres ci-dessous comptent
des SOMMETS, et a signifie toujours « nombre de voisins dans X_2 » :

  Profil A : X_0 : a=2×1, a=3×8, a=4×2 ;
             X_1 : a=0×22, a=1×40, a=2×2 ;
             X_2 : a=0×6, a=1×2.

  Profil B : X_0 : a=2×3, a=3×4, a=4×4 ;
             X_1 : a=0×20, a=1×44 ;
             X_2 : a=0×6, a=1×2.

Le profil A donne X_0 = p isolé + deux P_3 échangés par σ + deux K_2
échangés par σ. Le profil B donne soit deux P_4 échangés plus trois
isolés, soit un C_4 avec σ demi-tour, deux K_2 échangés et trois isolés.
Ces graphes ne peuvent contenir une arête z--σz, car aucun X_0 ne voit p.
Un triangle invariant aurait un sommet fixe, et deux triangles échangés
nécessiteraient six sommets de degré 2, que les profils n'ont pas.

8. RAPPEL : P ET Q INDÉPENDANTS ; LES DEUX TYPES DE B

Pour un support de m sommets positifs, e=e(P) et d_i=deg_P(i), l'équation
propre donne deg_Q(i)=d_i+4. Le double compte des communs des paires de P
donne pour les sommets hors P∪Q
  D=m(m-7)-7e-Σ_i d_i^2≥0.
Avec m=8 et Σ d_i^2≥2e, un e≥1 donnerait D≤8-9e<0. Donc e=0.

Le complément C de B est un graphe cubique sur huit sommets.
Pour i≠j, (B^2)_ij=2C_ij+(C^2)_ij. La borne (B^2)_ij≤2 interdit tout
triangle dans C. Un graphe cubique sans triangle sur huit sommets est
le cube ou le graphe de Wagner ; voici une petite classification humaine.

Fixons un sommet v avec trois voisins indépendants a,b,c, et les quatre
sommets restants R. Chacun de a,b,c a deux voisins dans R, donc R
induit trois arêtes. Sans triangle, ces trois arêtes forment une étoile
K_{1,3} ou un chemin P_4.
Dans le premier cas, le centre n'a pas de voisin parmi a,b,c et chaque
feuille en a deux. Les six incidences a,b,c--feuilles forment un cycle
de longueur 6 : on retrouve le cube.
Dans le cas R=r_0--r_1--r_2--r_3, les degrés vers a,b,c sont 2,1,1,2.
Les seules paires non adjacentes utilisables pour un de a,b,c sont
{r_0,r_2}, {r_0,r_3}, {r_1,r_3}. Les degrés imposent de les utiliser
chacune une fois : on retrouve Wagner, c'est-à-dire le cycle de huit
sommets avec les quatre arêtes antipodales.

9. VÉRIFICATIONS, SOURCES ET LIMITE

CALCULÉ en entiers, contrôle indépendant de §4-6 :
- les 105 appariements parfaits de huit occurrences déficitaires ont
  été parcourus ; après exclusion des boucles et arêtes répétées,
  ils donnent les trois cycles C_4 possibles sur les quatre labels ;
- pour chacun, les huit rectangles ont bien la concurrence 2C_4 ;
- les 256 sous-ensembles de X_2 ont été vérifiés pour l'inégalité q≥a-2 ;
- contrôle positif local : les rectangles respectent les capacités
  2J-B du support et laissent bien 64 incidences de type X_1.
Ce contrôle positif ne construit pas le graphe complet.
Résultats : C2_PREUVE_CUBE_CONTROLE.json, dans le même dossier outputs.
Un processus, moins d'une seconde, budget prévu inférieur à 32 Mio.

Les diagnostics sur Wagner laissent des configurations partielles.
Les tests de Gram ont aussi laissé des candidats et ne donnent pas
de contradiction supplémentaire générale. Aucune approximation
spectrale n'est utilisée pour prouver le théorème sur le cube.

Sources lues pour éviter de redonner les anciennes pistes :
- laboratoire c2_experimental, notes/54_SOLO_MODELE_JOINT_TEMOIN_28_09.md
  et scripts/k0_n8_config.py, k0_n8_fast.py, k0_n8_numba.py ;
- archive conway99-lab-archive/temp_session_eeaf2d22/scratchpad/
  conway-99-research, attempts/wave78-short-vector-packing/derivation.md,
  attempts/wave81-norm16-labeled-design/derivation.md et exact_check.py,

  verification/wave81-norm16-graphical-refinement/derivation.md ;
- dossier de recherche : MODEL.md, TASK.md, AUDIT_STATUS.md,
  DUPLICATION_NOTES.md, HANDOVER_2026-10-04.md, les deux CASCADE_STUDY ;
- laboratoire : notes/PROOF_SKELETON_C2.md et dossier Astra,
  attaque_globale_27_0803/AUDIT_NORME1_ET_METRIQUES.md.

La recherche dans ces sources n'a pas retrouvé l'obstruction §4-6.
Cela n'établit aucune priorité dans la littérature ni dans des sources
non accessibles. Le comptage global §2-3 et $t \leq 1$ étaient déjà connus.

La preuve porte sur un VRAI graphe : on ne l'applique pas à une matrice
signée relâchée N qui n'a pas de prolongement établi. Le raccord d'une
solution du modèle à un graphe utilise T4 avec son statut dans MODEL.md.
F0, entrée distincte, n'est pas nécessaire pour le théorème conditionnel
ci-dessus ; il intervient seulement pour passer des involutions à un
sommet fixe à toutes les involutions.

Le cas encore à fermer est B égal au complément de Wagner, avec les
contraintes globales de §7. La borne min N* >= 9/7 reste non prouvée.
Aucune modification de moteur, de dépôt ou de job AWS n'a été faite.

1.4 The Wagner graph

File: lemma/t16/dossier_2026-10-04/C2_WAGNER_BORNE_9_SUR_7.txt

```
Conway-99, C2 : obstruction de Wagner et nouvelle borne duale 9/7
4 octobre 2026 -- dossier mathématique et certificats reproductibles

RÉSULTAT ET STATUT

Dans un vrai SRG(99,14,1,2) avec une involution sigma fixant seulement p,
il n'existe pas de fonction propre f pour -4, impaire sous sigma, à
valeurs 0,+1,-1, avec huit sommets +1, huit sommets -1, toutes les
orbites actives extérieures, dont B=A[P,sigma(P)] est le complément
du graphe de Wagner.

STATUT : preuve avec classification finie calculée exactement. La partie
structurelle est écrite ci-dessous ; les deux assertions finies restantes
sont accompagnées de leur énumération exhaustive, des tableaux retenus
et d'un vérificateur indépendant. Ce n'est pas une preuve intégralement
à la main, ni une formalisation Lean.

Avec l'obstruction du cube et le lemme séparé sur les h actifs, cela donne

             min L* >= 9/7,

où L est le réseau engendré par le cadre physique de dimension 22,
sept h de norme 4 et quarante-deux u de norme 3. Cette assertion porte
sur le cadre d'un VRAI graphe, et non sur chaque candidat signé N isolé.

Conséquence pour une enveloppe U et une paire normale y1,y2 :

  x dans U, x^2=3, (x,y1)^2+(x,y2)^2=13  =>  paire impossible pour EX.

Un seul x suffit. Des témoins rationnels/entiers exacts sont joints pour
363 des 400 paires historiques du réseau 011. Il n'y a dans ce fichier
aucune paire K=0. Ce résultat ne ferme aucun réseau entier et ne mesure
aucun gain d'heures AWS. Les jobs et le moteur n'ont pas été modifiés.

1. ENTRÉES ET QUANTIFICATEURS

A est l'adjacence d'un graphe régulier de degré 14 sur 99 sommets, avec
un voisin commun par paire adjacente et deux par paire non adjacente :

             A^2+A=12I+2J.

N(p) est constitué des sept paires adjacentes de sigma. Les 42 autres
paires sont non adjacentes. Le raccord de ce graphe au modèle EX est
celui de MODEL.md, utilisant T4 avec son statut relu à la main, non
formalisé. F0 n'est pas utilisé pour cette obstruction conditionnelle.

On pose P={f=+1}, Q={f=-1}=sigma(P). Ici |P|=|Q|=8. P et Q sont
indépendants et B est symétrique, sans diagonale, de degré 4. Pour
chaque paire distincte i,j, ses lignes ont au plus deux indices communs.
Ces réductions ont été établies dans la note sur le cube, §8, et dans
la note 54 du laboratoire. Elles ne sont pas annoncées nouvelles ici.

IMPORTANT : le k de la coupe dans la note 54, encore ouvert pour 1,2,3,
est distinct du nombre de h actifs dans le lemme d'égalité 8/7. Nous
fermons ici le cas de coupe k=0,n=8. Nous ne fermons pas les autres coupes.

2. CONTRAINTES GLOBALES UTILISÉES

Pour z hors du support P union Q, les nombres de voisins dans P et Q
sont égaux ; on les note d_z. Ils valent 0,1 ou 2. Les ensembles X_d
ont les tailles suivantes :

             |X0|=11, |X1|=64, |X2|=8.

Pour z dans X_d, notons a_z,b_z,c_z les nombres de ses voisins dans
X2,X1,X0. Le comptage des chemins vers le support donne

             b_z+2a_z=16-5d,
             c_z=a_z+3d-2.

Notons t le nombre d'arêtes dans X2. Les comptes globaux donnent

  e(X0,X2)=32+2t ; e(X1,X2)=48-4t ;
  somme_{X0 union X1 union X2} a_z=80 ; somme a_z^2=160-2t.

Sur X0, a>=2 et a^2>=5a-6. Sur X1 et X2, a^2>=a. Ainsi

             160-2t >= 142+8t, donc t<=1.

p appartient à X0. Ses voisins sont des paires adjacentes de sigma,
```

et a_p>=2 : on a donc t>=a_p/2>=1. Par conséquent

t=1, a_p=2, b_p=12, c_p=0.

Appelons x,y=sigma(x) l'unique arête de X2. Ils sont voisins de p.
Les six autres sommets de X2 appartiennent à des orbites extérieures.
Les six autres paires de N(p) appartiennent à X1.

Le budget de défaut est

somme_{X0}(a-2)(a-3)+somme_{X1 union X2}a(a-1)=8.

Il impose exactement deux profils, les multiplicateurs comptant ici
des SOMMETS, pas des orbites :

Profil A : X0 : a=2 ×1 (p), a=3 ×8, a=4 ×2 ;
X1 : a=0 ×22, a=1 ×40, a=2 ×2 ;
X2 : a=0 ×6, a=1 ×2.

Profil B : X0 : a=2 ×3, a=3 ×4, a=4 ×4 ;
X1 : a=0 ×20, a=1 ×44 ;
X2 : a=0 ×6, a=1 ×2.

Ces identités, t=1 et les profils sont la section 7 de la note sur le
cube, que Claude a revérifiée à la main dans sa note du 04/10. Le fait
ancien t<=1 avait aussi été calculé dans Wave 81. Les obstructions des
sections 5 et 6 ci-dessous sont la nouvelle étape.

3. LES HUIT RECTANGLES DE WAGNER : COUVERTURE DE LA RÉDUCTION

Numérotons les sommets de Wagner par Z/8Z, avec les arêtes i~i+1 et
i~i+4. Pour B complémentaire, sa ligne i est

{i+2,i+3,i+5,i+6} modulo 8.

Les huit paires déficitaires sont {i,i+2}, i dans Z/8Z : elles ont
exactement un voisin commun dans B, contre deux pour les autres paires.
Chacune a donc exactement un voisin commun hors du support, dans X2.

Un sommet z de X2 voit une paire U de P et une paire V de Q. Les paires
U et V appartiennent aux huit paires déficitaires. Son mate voit V dans
P et U dans Q. Comme chaque paire déficitaire apparaît exactement une
fois du côté P, les quatre orbites de X2 appairent les huit paires
déficitaires distinctes. Il y a exactement 7×5×3×1=105 appariements.

Pour chacun, les huit rectangles (U,V),(V,U) doivent respecter :

(i) chaque ligne ET chaque colonne de B[U,V] a au plus une entrée 1 ;
(ii) deux rectangles ont au plus deux voisins communs du support ;
(iii) leur nombre total d'occurrences sur chaque cellule (i,j) est
au plus 2-B_ij, le budget des voisins communs de P_i,Q_j.

(i) vient de lambda=1 pour un sommet de X2 adjacent à un sommet du
support. Il s'agit d'un COUPLAGE PARTIEL, qui peut être vide. Exiger un
couplage parfait serait une hypothèse supplémentaire injustifiée.

Ces tests exacts retiennent 31 appariements. Il faut ensuite choisir
quelle orbite est x,y. Comme p est déjà leur unique voisin commun,
leurs rectangles doivent avoir U intersection V vide. Marquer toutes
les orbites satisfaisant cette condition donne 76 cas. Aucun choix
d'orbite n'est imposé arbitrairement.

4. LE PETIT GRAPHE INDUIT ET SES MATRICES D'INCIDENCE

On retient les 35 sommets

Y=P union Q union X0 union X2.

p est dans X0. Numérotons les autres sommets de X0 par cinq paires
(0,1),(2,3),(4,5),(6,7),(8,9), et les sommets de X2 par quatre paires
(0,1),(2,3),(4,5),(6,7), la première étant x,y.

C est l'incidence binaire 10×8 de X0 privé de p vers X2. Chaque colonne
de C a somme 4 : x,y ont cinq voisins dans X0, dont p ; les six autres
sommets en ont quatre et ne voient pas p. Les lignes vont par paires
échangées par sigma. Leurs sommes sont :

A : (4,4,3,3,3,3,3,3,3,3),
B : (4,4,4,4,3,3,3,3,2,2).

Pour deux sommets distincts s,t de X2, soient R_st le nombre de leurs
voisins communs dans le support, et E_st le nombre dans X1. Alors

(C^t C)_st = 2 - [s~t] - R_st - [p voit s et t] - E_st.

Pour x,y, cela vaut 0, car ils sont adjacents, R_xy=0 et p les voit.
Pour toute autre paire, cela vaut 2-R_st-E_st. La diagonale vaut 4.

Dans le profil B, E_st=0, tous les sommets de X1 ayant a<=1.
Dans A, E est la contribution d'une seule paire de sommets X1 de a=2 :
une paire {s,t} et son image par sigma. Cette contribution concerne
seulement les entrées hors diagonale ; elle ne réduit pas la diagonale
4 de C^t C. La paire {s,t} peut être invariante sous sigma, auquel cas
sa contribution hors diagonale est 2. Ce cas est inclus.

Chaque ligne est énumérée parmi les 2^8 masques. Dans chaque orbite,
on prend le masque <= son image ; cela ne fait que nommer les deux
sommets de X0. Les orbites de même poids sont ensuite ordonnées. Deux
orbites identiques de poids 3 ou 4 ne sont pas possibles : elles
donneraient trois ou quatre voisins communs entre deux sommets de X0.
Les combinaisons de masques distincts couvrent donc toutes les matrices.

Soit G=A[X0 privé de p]. Ses degrés sont les sommes de lignes de C
moins 2. Il a six arêtes, toutes dans des orbites de taille 2, car les
sommets non fixes de X0 ne peuvent pas être adjacents à leur mate.
Il suffit donc d'énumérer trois orbites d'arêtes parmi les vingt
possibles entre les cinq paires, puis de retenir les degrés requis.
Cela couvre tout G, sans imposer une forme particulière ou une
symétrie supplémentaire. p est isolé dans A[X0].

Dans Y, toutes les adjacences sont maintenant connues. On teste chaque
paire i,j avec la condition nécessaire

$$|N_Y(i) \text{ intersection } N_Y(j)| <= 2-A_{ij}.$$

Les résultats des deux énumérations sont :

	Profil A	Profil B
Appariements initiaux	105	105
Appariements admissibles	31	31
Choix marqués de x,y	76	76
Matrices C	48	16
Graphes G de degrés requis	48	18
Couples (C,G) valides dans Y	640	32

Les valeurs 48/16 et 640/32 sont CALCULÉES. Les arguments précédents
justifient la couverture des boucles finies des deux scripts joints.

5. PROFIL B : TROIS DEMANDES, SEULEMENT DEUX PARTENAIRES

x voit quatre sommets de X1. Chacun a a=1 et donc c=2 : il voit deux
sommets de X0. Pour z dans X0, le nombre de voisins communs x,z qui
restent à fournir hors Y est exactement

$$F_z=2-[x\sim z]-|N_Y(x) \text{ intersection } N_Y(z)|.$$

Dans chacun des 32 tableaux, F est constitué de huit 1 et de trois 0
(dont celui de p). Les quatre voisins X1 de x doivent donc apparier
ces huit demandes, chaque sommet demandé étant utilisé une fois.

Pour tester si {z,z'} peut être une paire de partenaires, ajoutons à Y
un seul sommet provisoire adjacent à x,z,z'. On ne spécifie aucun de
ses autres voisins. Si ce graphe partiel dépasse un budget de voisins
communs, cette paire est impossible. S'il reste valide, on l'autorise.
Ce test donne un SUR-ENSEMBLE des paires utilisables dans le vrai graphe.

ASSERTION FINIE, CALCULÉE : dans chacun des 32 graphes de partenaires
autorisés, il existe trois sommets demandés W, sans arête entre eux,
dont l'ensemble des partenaires possibles N(W) a seulement deux
sommets. Les trois sommets de W doivent être appariés à trois sommets
distincts hors W ; ils n'en ont que deux. Contradiction.

Chaque certificat fournit W, N(W), le graphe autorisé et les données
pour reconstruire Y. Le vérificateur recalcule toutes les paires
autorisées, puis contrôle l'indépendance de W et |N(W)|<|W|.

Exemple, premier tableau, labels de X0 privé de p :

  Demandes : {1,2,3,4,6,7,8,9}.
  Arêtes autorisées : (1,9),(2,8),(2,9),(3,8),(3,9),(4,6),
                      (4,9),(6,7),(6,8),(7,9),(8,9).
  W={1,2,3}, N(W)={8,9}.

Cette obstruction ferme les 32 tableaux du profil B.

6. PROFIL A : LE VOISINAGE DE x NE PEUT PAS ÊTRE ASSEMBLÉ

Notons T=N(x) intersection (X0 privé de p), de taille 4, et
Z=N(x) intersection X1, également de taille 4. Soit q le nombre des
deux sommets exceptionnels de X1, de a=2, qui appartiennent à Z.

Appliquons A^2=12I+2J-A à l'indicatrice de X2, au sommet x. La somme des a_z sur les voisins de x vaut 12+2×8-a_x=27. Ses quatre voisins du support contribuent 8, p contribue 2, y contribue 1, et Z contribue 4+q. Par conséquent

somme_{z dans T} a_z=12-q,
somme_{z dans T} (a_z-2)=4-q.

Dans A, chaque sommet non fixe de X0 a a>=3. Les quatre termes de la dernière somme valent donc au moins 1. Il s'ensuit q=0 et a_z=3 pour chaque z de T. Autrement dit, T est composé de feuilles de G, les deux centres de G ne voient ni x ni y, et les sommets exceptionnels de X1 ne voient ni x ni y. Chaque sommet de Z a donc a=1,c=2.

Le voisinage N(x) doit induire un couplage parfait : pour chaque voisin w de x, lambda=1 impose exactement un voisin de w dans N(x). L'arête p-y en constitue déjà une paire. Les autres sommets sont les quatre sommets O du support voisins de x, les quatre feuilles T et les quatre sommets Z. Il n'y a aucune arête O-T.

Notons e le nombre d'arêtes sur O et f le nombre d'arêtes sur T. Il reste 4-2e sommets de O et 4-2f sommets de T à apparier avec Z. Il faut donc

8-2e-2f <= 4, soit e+f>=2.

ASSERTION FINIE, CALCULÉE : sur les 640 tableaux de A, e=1 toujours. Dans 400 tableaux, f=0 : la dernière inégalité les exclut. Dans les 240 autres, e=f=1. Tous les sommets de Z sont alors utilisés par le couplage vers O ou T, et il n'y a aucune arête entre deux sommets de Z.

Un sommet z de Z est extérieur : s'il voyait p, il serait dans N(p), où x ne peut être adjacent qu'à y. Son mate n'est donc pas adjacent à z. Ses voisins dans Y sont exactement : x, un sommet P_i, un sommet Q_j, et deux sommets de X0 privé de p. Il n'a pas d'autre voisin dans X2 (a=1), ni dans X0 (c=2), ni dans le support (d=1), ni p. Les voisins de son mate dans Y sont les images de ces cinq sommets.

Pour chaque tableau, on énumère toutes ces possibilités. On ajoute la paire z,sigma(z), sans arête entre eux, et on retient :

- les budgets de voisins communs de toutes les paires ;
- exactement un voisin commun pour x,z ;
- une demande encore positive pour chacune des deux positions X0 et des positions P_i,Q_j.

Cela donne entre 17 et 36 orbites candidates par tableau, 6 544 en tout. Ce sont des candidats locaux, pas des solutions du graphe.

On relie deux orbites candidates si leurs quatre sommets, ajoutés simultanément à Y SANS arête entre ces deux orbites, respectent encore tous les budgets de voisins communs. Omettre d'éventuelles arêtes entre les nouveaux sommets est une relaxation sûre : supprimer des arêtes réduit les nombres de voisins communs et augmente, ou conserve, le plafond 2-A_ij. Un ensemble réalisable dans le vrai graphe doit donc être une clique dans ce graphe de compatibilité.

ASSERTION FINIE, CALCULÉE : chacun des 240 graphes de compatibilité est sans clique de taille quatre. Or x a besoin de quatre sommets de Z, appartenant à quatre orbites distinctes, puisque leurs mates voient y et ne peuvent voir x (a=1). Contradiction.

Cette obstruction ferme les 240 tableaux restants de A, et donc A tout entier. Avec la section 5, elle exclut Wagner dans le sous-cas déclaré.

7. COMMENT VÉRIFIER LA PARTIE CALCULÉE

Deux scripts autonomes, utilisant uniquement la bibliothèque standard Python, réénumèrent les tableaux depuis les huit lignes de B :

ENUMERER_WAGNER_PROFIL_A.py
ENUMERER_WAGNER_PROFIL_B.py

Ils fournissent à la fois la couverture finie et les obstructions, sans charger de catalogue de solutions, de sortie de solveur ou de résultats de la première recherche. Ils écrivent les deux fichiers WAGNER_PROFIL_*_CERTIFICATS.json dans leur propre dossier.

VERIFIER_WAGNER_CERTIFICATS.py utilise NumPy en entiers pour recompter les voisins communs par multiplication de matrices. Il recalcule les candidats et toutes les compatibilités des 240 tableaux A, puis vérifie l'absence de clique de taille quatre. Pour les 32 tableaux B, il reconstruit Y, recalcule le graphe des partenaires et contrôle chaque obstruction trois contre deux. Il vérifie les obstructions, tandis que

la couverture vient des deux énumérateurs précédents.

Contrôles effectivement terminés :

- énumération A par combinaisons directes : environ 153 secondes ;
- énumération B : les 32 tableaux ont leur certificat trois contre deux ;
- autre assemblage récursif du voisinage A : 26 608 nœuds, zéro assemblage des quatre voisins, environ 8 secondes ;
- vérification indépendante par matrices entières : les 240 et 32 certificats passent, 6 544 candidats recomptés, environ 6 secondes.

Les graphes de compatibilité et les budgets sont calculés en entiers. Les produits matriciels du vérificateur restent inférieurs à 100 par entrée : son type int16 ne déborde pas. Aucune approximation spectrale ou flottante n'entre dans l'obstruction de Wagner. Limites : un seul processus, plafond 180 secondes par vérification, mémoire prévue inférieure à 128 Mio pour cette classification.

8. COROLLAIRE : LE CAS D'ÉGALITÉ 8/7 EST IMPOSSIBLE

Soient f_i les 49 vecteurs du cadre physique, H leur Gram, et L leur réseau. H^2=7H et somme f_i f_i^t=7I. Pour v dans L*, les coefficients a_i=(v,f_i) sont entiers et

$$\text{somme } a_i^2 = 7v^2.$$

ENTRÉE EXTERNE : la borne min L*>=8/7 est celle du dossier, avec son statut et ses sources. La présente note améliore son cas d'égalité.

Si v^2=8/7, les équations du Gram excluent |a_i|=2 et donnent huit coefficients ± 1. Le lemme séparé LEMME_EGALITE_8_SUR_7_C2.txt démontre que tous les coefficients des sept h sont nuls. Son relevé est donc une fonction f comme en section 1, sur huit orbites extérieures.

Le complément de B est un graphe cubique sans triangle sur huit sommets, donc le cube ou Wagner : classification humaine de la note cube, §8. Le cube est exclu par cette note ; Wagner est exclu ici. Ainsi aucun v dans L* n'a norme 8/7. Comme $7v^2$ est entier et la norme est au moins 8/7, elle est au moins 9/7 pour tout v non nul.

Cette conclusion ne nécessite PAS la borne calculée n<=8 utilisée pour obtenir le témoin spécial de la route de la note 54 : elle est une assertion sur n'importe quel vecteur dual de norme 8/7 d'un vrai cadre. Pour conclure EX faux par la route témoin entière, en revanche, les coupes 1,2,3 et les entrées externes de cette route restent à traiter.

9. CONSÉQUENCE IMMÉDIATE POUR LES PAIRES NORMALES

U est une enveloppe intégrale de dimension 24, y1,y2 sont orthogonaux de norme 7. Les vecteurs du cadre d'une solution appartiendraient à U et à l'orthogonal des deux normales.

Pour x dans U de norme 3, posons a=(x,y1), b=(x,y2). Si a^2+b^2=13, sa projection v sur l'espace physique aurait

$$v = x - (a\,y1 + b\,y2)/7,$$
$$v^2 = 3 - 13/7 = 8/7.$$

Pour chaque vecteur f_i d'un tel cadre, (v,f_i)=(x,f_i) est entier, par intégralité de U. Ainsi v serait dans L*, contrairement à la borne 9/7. La paire normale est donc exclue AVANT tout choix des h.

Ce critère ne demande ni de générer tous les vecteurs de norme 3 ni de prouver la complétude d'une coque : un seul témoin exact suffit. L'absence de témoin trouvé ne prouve pas la survie de la paire.

CALCULÉ sur le fichier historique profiling/011_sample_pairs.json :

400 paires examinées ; 363 avec un témoin exact Y=13 ;
125 504 055 nœuds historiques sur ces 363 paires,
contre 159 598 993 sur l'ensemble du fichier.

Cela représente 90,75 % des paires et environ 78,64 % de ces nœuds historiques. Ce fichier est un échantillon ordonné déjà calculé ; ses K vont de 10 à 28 et il ne contient aucun K=0. Ces fractions ne sont ni une couverture de 011, ni un taux sur les jobs ouverts, ni une mesure de vitesse ou d'heures économisées.

PAIRES_011_TEMOINS_Y13.json donne la base ambiante rationnelle, son Gram, les coordonnées de x,y1,y2 et les deux produits pour chaque témoin. Deux copies des entrées d'origine et leurs empreintes sont fournies.

VERIFIER_PAIRES_011_Y13.py, bibliothèque standard Python seulement, contrôle exactement : le même réseau que les générateurs du catalogue

(changements de base entiers dans les deux sens), det Gram=1,
les deux normales originales, leurs normes et orthogonalité, x^2=3,
a^2+b^2=13 et $(7v)^2$=56 avec 7v orthogonal aux deux normales.
Les 363 témoins passent, en moins d'une seconde dans cette exécution.
Cette vérification ne dépend d'aucune énumération complète de coque.

10. PORTÉE, ANTÉCÉDENTS ET TRAVAIL RESTANT

La note 54 du labo avait laissé B4 et B5 ouverts, avec des millions
d'étiquetages localement compatibles. Notre obstruction du cube ferme
B4. La présente classification et ses contraintes globales ferment B5.
Ensemble, elles ferment le cas de coupe k=0,n=8 de cette route, et donc
ses cas k=0,n=5..8 avec l'ancienne borne 8/7.

Le mécanisme supplémentaire ici utilise la configuration globale des
11 sommets X0, le sommet fixe, l'unique arête X2, et le couplage dans
N(x). Il n'est pas une réénumération des étiquettes de racines de la
note 54. Les sources consultées n'avaient pas ces obstructions ; ce
constat ne prétend pas établir une priorité dans toute la littérature.

Sources et dépendances conservées :
  - MODEL.md, TASK.md, AUDIT_STATUS.md, DUPLICATION_NOTES.md ;
  - HANDOVER_2026-10-04.md, les deux CASCADE_STUDY et PROOF_SKELETON_C2.md ;
  - note 54 et ses scripts k0_n8 du laboratoire ;
  - Wave 78/81 et la vérification graphique de Wave 81 dans l'archive ;
  - la note de vérification de Claude VERIF_CUBE_OBSTRUCTION_2026-10-04.md ;
  - les deux preuves antérieures jointes : cube et h actifs.

RESTE OUVERT : les coupes k=1,2,3 de la route témoin, EX lui-même,
l'exclusion de chaque réseau encore ouvert, et le gain effectif du
nouveau critère Y=13 sur les paires des jobs actuels. Le total d'heures
AWS ne change pas sur la seule base de cet échantillon. Aucun nouveau
lancement, arrêt de job, changement de dépôt ou de moteur n'a été fait.

Pour revue, commencer par les sections 3-6 et les énumérateurs : ce sont
les nouvelles obligations finies. Les anciens documents joints gardent
leur état historique (ils disent Wagner ouvert) ; la présente note
fournit précisément la fermeture qui leur manquait.

2 The lemma T_{18}

File: lemma/T18_PROOF.md

```
# T18: no sigma-odd integer (-4)-eigenfunction of squared norm 18 on a srg(99,14,1,2)

Self-contained proof text, written on 2026-10-07 from the working reports of the repository ‘conway99-c2-aws‘
(folders ‘lemma/t18‘, ‘lemma/t18_STRUCT‘, ‘lemma/t18_VERIF331‘, ‘lemma/t18_B2‘, ‘lemma/t18_B0‘, ‘lemma/t18_B0_VERIFC‘,
‘lemma/t18_REFEREE‘, and ‘LEMMA_TARGET_2026-10-06.md‘). The working reports are sources, not part of the proof: every
hand statement used below is proved below, from the definitions. The finite computations are described precisely
(what is enumerated, by which tests, with which result), and for each of them the programs and their output files
are listed in the appendix.

Revised on 2026-10-07 after the report of a second referee (‘lemma/t18_REFEREE2/REPORT.md‘), who read this text only
and re-did every Computation with programs of his own, written from it. He found no error and listed eleven gaps or
problems of presentation. The corrections are listed in section 9.6; apart from them no count, statement or proof
has been changed. In this text "the referee", without qualification, always means the first referee (report
‘lemma/t18_REFEREE/REPORT.md‘, written on the working reports before this text existed); the author of the second
report is always called "the second referee".

## 0. Front matter

### 0.1 GAP boxes

**GAP boxes in this document: none.** Every hand statement was re-derived while it was written and no step was found
that could not be justified. What is *not* a hand proof is listed in section 9.2 (the finite computations) and what
has not been independently verified is listed in section 9.3. Three points of section 9.3 deserve to be known before
reading:

- the list of 16 isomorphism classes of 4-regular graphs on 9 vertices now comes from two programs written
  separately (in the first version of this text it came from one program only; second referee’s remark 2); the
  selection of the three classes that matter is done by four programs, two of which share an isomorphism routine
  (section 4.10);
- the programs of the first implementation read a file ‘types.json‘ that was produced with a filter depending on
  another lemma (called H16 in the working reports); the saved output ‘rows.out‘ shows that this filter removed
  nothing on the entries that are read, the second referee has recomputed these entries from the common-neighbour
  budgets alone and found the same lists, and the second implementations do not read the file (section 9.3, item 6;
  this point is not mentioned in the working reports nor in the referee’s report);
- a survivor of a computation is defined by properties of the graph (budgets satisfied, both matrices PSD on the
  set), but a "kept" decision of a positive-semidefiniteness test is a floating-point decision in most programs. It
  is never used as a proof: only rejections are used, and they are exact. Section 6.3 says which lists have been
  verified to be exactly PSD, and by whom (see also section 9.3, item 1).

### 0.2 Notation and conventions

- ‘u‘ between two sets is the union, ‘n‘ the intersection, ‘\‘ the difference; ‘|T|‘ is the number of elements.
- ‘x ~ y‘ means "x and y are adjacent"; ‘x not~ y‘ means "x and y are not adjacent".
- ‘N(x)‘ is the set of neighbours of x, ‘N[x] = N(x) u {x}‘.
- ‘C(a,2) = a(a-1)/2‘. ‘[P]‘ is 1 if the statement P is true and 0 otherwise.
- ‘I‘ is the identity matrix, ‘J‘ the all-ones matrix, ‘1_T‘ the indicator vector of a vertex set T, ‘e_x = 1_{x}‘.
- For a matrix M and a vertex set Y, ‘M[Y]‘ is the principal submatrix on Y; for two lists R, R’ of vertices,
  ‘A[R,R’]‘ is the submatrix with rows R and columns R’.
- "PSD" means positive semidefinite.
- Labels in square brackets such as [A3], [N3], [P2], [C-b] are the names of the same statements in the working
  reports. They are given only so that a reader of those reports can find the correspondence (appendix C); no
  proof below refers to them.
- A statement marked **Computation** is the result of a finite enumeration by programs. A statement marked
  **Lemma**, **Proposition**, **Theorem** or **Corollary** without that word is proved by hand in the text, except
  that the three final statements (Propositions 7.7 and 8.9, Theorem 9.1) combine hand statements with
  Computations and say so.
- Letters with several uses (second referee’s remark 11). In the first version of this text the letters B, C, E, V,
  R and H each named a mathematical object and also a program or a stage. Now none of them names a program (next
  item), and their meanings are these. B is the orbit graph of section 4.9 (B0, B1, B2 are its three possible
  values). C(a,2) is a binomial coefficient, and C is a set in the proof of Lemma 2.1. E_0, E_3, E_{-4} are the
  projectors and V_3, V_{-4} the eigenspaces of section 1.1 (V is also a deficient pair in section 7.1). R is a set
  of orbit representatives (section 6.4), and R_q is a row (section 7.3). H is a set of edges inside X_2 (sections
  7.3 and 8.2); H’ occurs only in Lemma 6.2; H16 is the name given in the working reports to an earlier lemma (no
  configuration of squared norm <= 16), which is not used here. The stages of the enumeration of section 8 keep
  their names "stage B", "stage C", "stage D", because the programs and their output files carry them (‘B250‘,
  ‘C250‘, ‘stageC‘, ‘stageD‘); they are always written with the word "stage" (as in "stage-C survivor"). "Case A",
  "Case B", "Case C" are the three cases of the proof of Lemma 4.7. A label in round brackets with a digit, such as
  (I1), (R2), (F7), (C4), (D3), is the name of a numbered identity or condition.
- Programs and families of programs are never called by a single letter. They are called by their file names or by
  the tags I1, I2, I3 (section 7.5), ENUM, VERIF, CMINE (sections 8.3 and 8.4; they were E, V, C in the first
  version of this text), REF1 (the programs of the referee; R in the first version) and REF2 (the programs of the
  second referee, section 9.6).
- Three symbols of the first version have been renamed, so that they no longer share a letter with another object:
  the inner indicator of section 4.7 is eps_i (it was h_i, while h_0, h_1, h_2 are the profile counts of section
  5.5); the numbers attached to the orbits in section 3 are kappa_0 (they were c_0); and the number of neighbours of
  v in X_0 is written a_0(v) everywhere (it was also written c(v), while c is the index of the orbits of N(p): a_c,
  b_c, w_c, K_c).


## 1. Setting and statement
```

1.1 The graph

Throughout, G is a strongly regular graph with parameters (99, 14, 1, 2): it has 99 vertices, every vertex has exactly 14 neighbours, two adjacent vertices have exactly 1 common neighbour (lambda = 1), and two distinct non-adjacent vertices have exactly 2 common neighbours (mu = 2). Whether such a graph exists is Conway's 99-graph problem; everything below is about a hypothetical G. A is its adjacency matrix.

The (x,y) entry of A^2 is the number of common neighbours of x and y, so

    A^2 = 14 I + 1 * A + 2 (J - I - A) = 12 I - A + 2 J.   (1.1)

A 1 = 14 * 1 (1 = all-ones vector). On the orthogonal complement of 1, (1.1) reads A^2 + A - 12 I = 0, i.e. (A - 3I)(A + 4I) = 0, so the other eigenvalues are 3 and -4. If f and g are their multiplicities, f + g = 98 and trace A = 0 = 14 + 3f - 4g, hence f = 54 and g = 44:

    eigenvalue 14 (multiplicity 1), 3 (multiplicity 54), -4 (multiplicity 44).

V_3 and V_{-4} are the eigenspaces, and E_0 = J/99, E_3, E_{-4} the orthogonal projectors on the three eigenspaces: E_0 + E_3 + E_{-4} = I and A = 14 E_0 + 3 E_3 - 4 E_{-4}.

1.2 The involution

sigma is an automorphism of G such that sigma^2 is the identity and sigma has **exactly one fixed vertex**, called p.

This is all that is assumed about sigma. Nothing is assumed about other automorphisms of G, and nothing from the lattice or "frame" picture that motivated the lemma is used.

For a vertex x != p the vertex sigma x is called the **mate** of x. The sets {x, sigma x} with x != p are the 49 **orbits**; {p} is the fixed orbit. An edge {x, sigma x} is called a **mate edge**. A vertex set Y is **sigma-invariant** if sigma Y = Y. A function phi on the vertices is **sigma-odd** if phi(sigma x) = -phi(x) for every vertex x; then phi(p) = 0.

1.3 Statement

> **Theorem T18.** Let G be a strongly regular graph with parameters (99,14,1,2) and sigma an automorphism of G of
> order 2 with exactly one fixed vertex. There is no function phi from the vertex set of G to the integers such that
> (i) phi(sigma x) = -phi(x) for every vertex x, (ii) A phi = -4 phi, (iii) sum over all vertices of phi(x)^2 = 18.

The proof is by contradiction: a triple (G, sigma, phi) satisfying the hypotheses is called a **configuration**, and the document shows that no configuration exists. Its logical chain is:

    phi integer-valued
      -> section 3 (hand): phi takes the values 0, +1, -1 only; S+ = {phi = +1} has 9 vertices
      -> section 4 (hand + Computation 4.10): S+ is independent, no vertex of the support is in N[p], and the
                          "orbit graph" B of the support is one of three explicit graphs B0, B1, B2
      -> section 5 (hand): no outside vertex has 3 neighbours in S+; the neighbourhood of p has one of three
                          "profiles" (2,5,0), (3,3,1), (4,1,2); profile (4,1,2) is impossible
      -> section 7 (hand model + Computations 7.3, 7.6): profile (3,3,1) is impossible for B0, B1, B2
      -> section 8 (hand model + Computations 8.4, 8.5, 8.6, 8.8): profile (2,5,0) is impossible for B0, B1, B2
      -> contradiction.

Section 6 contains the three tests used by the computations (common-neighbour budgets, two PSD matrices, a rank bound), with their proofs.

1.4 Context (not part of the lemma)

In the project, T18 is the graph form of a statement about a lattice: ‘LEMMA_TARGET_2026-10-06.md‘, section 2, gives a dictionary under which "every non-zero sigma-odd integer function phi with A phi = -4 phi has squared norm >= 2m" is equivalent to a lower bound m/7 on the minimum of a dual lattice; the case of squared norm <= 16 is an earlier lemma, T18 is the case 18. That dictionary is presented in its own source as reconstructed during a session, with the warning "check it against the project's own notes before building on it", and it was not verified by the referee. **It is outside this document**: what is proved here is the graph statement of section 1.3 and nothing else. The campaign only needs phi with values 0, +1, -1; section 3 shows that this is not a restriction.

2. Basic facts about sigma [A0]

Lemma 2.1. Let x be a vertex, x != p.

(a) x ~ sigma x if and only if x ~ p.

(b) N(p) consists of 7 orbits {a_c, b_c}, c = 1..7, with b_c = sigma a_c; the 7 mate edges {a_c, b_c} are the only edges with both ends in N(p). In other words N(p) induces a perfect matching, made of mate edges.

(c) For c = 1..7 the only common neighbour of a_c and b_c is p. Hence no vertex other than p is adjacent to both a_c and b_c.

(d) If x is outside N[p], then x not~ sigma x, and N(x) n N(sigma x) = {z, sigma z} for a vertex z which is outside N[p] (and z != sigma z).

Proof. Let C = N(x) n N(sigma x). Since sigma is an automorphism and an involution, sigma C = N(sigma x) n N(x) = C: the set C is sigma-invariant. Also x != sigma x, because p is the only fixed vertex.

(a) If x ~ sigma x, then |C| = 1 (lambda = 1); a sigma-invariant set with one element consists of a fixed vertex, so C = {p} and x ~ p. Conversely let x ~ p. Then sigma x ~ sigma p = p, so p is in C. If x were not adjacent to sigma x we would have |C| = 2 (mu = 2), C = {p, c} with c != p, and sigma c would be an element of C other than sigma p = p, i.e. sigma c = c: a second fixed vertex, which does not exist. So x ~ sigma x.

(b) If a is in N(p) then sigma a is in N(p) (sigma p = p), a != sigma a, and a ~ sigma a by (a). So N(p) is a union of orbits, 14/2 = 7 of them, each carrying a mate edge. The pair a, p is adjacent, so a and p have exactly one common neighbour; a common neighbour of a and p is exactly a neighbour of a inside N(p). Hence a has exactly one neighbour in N(p), and it is sigma a.

(c) In the proof of (a) we saw that C = {p} when x ~ sigma x; apply it to x = a_c.

(d) x is not adjacent to p, so x not~ sigma x by (a) and |C| = 2. p is not in C (x not~ p). A sigma-invariant 2-set that does not contain the fixed vertex is an orbit: C = {z, sigma z}, z != sigma z. If z were in N(p), then by (c) the only common neighbour of z and sigma z would be p; but x is a common neighbour of z and sigma z (z and sigma z are both in C, hence both adjacent to x), so x = p, a contradiction. And z != p. So z is outside N[p]. QED

Corollary 2.2. Every vertex x outside N[p] has exactly 2 neighbours in N(p).

Proof. x and p are distinct and non-adjacent, so they have exactly 2 common neighbours, and the common neighbours of x and p are the neighbours of x in N(p). QED

Corollary 2.2 is used many times below in the form "the two common neighbours of x and p".

Remark (not used below). By (d), sending the orbit of x to the orbit {z, sigma z} defines a map iota on the (99 - 1 - 14)/2 = 42 orbits outside N[p]; it is a fixed-point-free involution, because x is a common neighbour of z and sigma z.

3. From integer values to the values 0, +1, -1

This step is missing in the working reports for squared norm 18 (referee's remark 1); the proof below is the referee's, checked.

Proposition 3.1. Let phi be an integer-valued sigma-odd function with A phi = -4 phi and sum phi(x)^2 = 18. Then phi takes only the values 0, +1, -1, and the set S+ = {x : phi(x) = +1} has exactly 9 vertices. Moreover S- = {x : phi(x) = -1} = sigma S+, and p is in neither.

Proof. phi(p) = 0. For an orbit O = {x, sigma x} put kappa_O = |phi(x)| = |phi(sigma x)|, an integer >= 0. Then sum over the 49 orbits of 2 kappa_O^2 = 18, i.e.

$$\sum_O \kappa_O^2 = 9. \tag{3.1}$$

Claim: if kappa_O = kappa > 0, then the sum of kappa_O' over the orbits O' != O is at least 3 kappa. Choose x in O with phi(x) = kappa > 0. The eigen-equation at x is

$$-4\kappa = \sum_{y \sim x} \varphi(y).$$

Sort the neighbours of x by orbit. The neighbour p (if x ~ p) contributes 0. The mate sigma x, if it is a neighbour of x, contributes phi(sigma x) = -kappa. An orbit O' = {y, sigma y} different from O contributes 0 if x is adjacent to none of its two vertices, +-phi(y) if x is adjacent to exactly one of them, and phi(y) + phi(sigma y) = 0 if x is adjacent to both; in all cases at least -kappa_O'. Hence -4 kappa >= -kappa - sum_{O' != O} kappa_O', which is the claim.

By (3.1) the multiset of the non-zero values kappa_O is one of {3}, {2,2,1}, {2,1,1,1,1,1}, {1,1,1,1,1,1,1,1,1} (the ways of writing 9 as a sum of squares 9, 4, 1). The claim excludes the first three:

- {3}: kappa = 3, the others sum to 0 < 9;
- {2,2,1}: kappa = 2, the others sum to 3 < 6;
- {2,1,1,1,1,1}: kappa = 2, the others sum to 5 < 6.

So exactly 9 orbits have kappa_O = 1 and the other 40 have kappa_O = 0. In each of the 9 orbits one vertex has phi = +1 and its mate has phi = -1. QED

From now on a configuration is a triple (G, sigma, phi) with phi as in Proposition 3.1, and

$$S^+ = \{\varphi = +1\}\ (9 \text{ vertices}), \quad S^- = \sigma S^+ = \{\varphi = -1\}, \quad S = S^+ \cup S^- \ \text{(the support, 18 vertices)}.$$

4. Reductions on the support

4.1 Notation and the eigen-equation

For a vertex v let d+(v) = |N(v) n S+| and d-(v) = |N(v) n S-|. Since sigma exchanges S+ and S-,

$$d^+(\sigma v) = d^-(v) \quad \text{and} \quad d^-(\sigma v) = d^+(v) \quad \text{for every vertex } v. \tag{4.1}$$

(A phi)(v) = d+(v) - d-(v), so A phi = -4 phi says exactly:

$$d^-(x) = d^+(x) + 4 \quad \text{for } x \text{ in } S^+; \tag{4.2}$$
$$d^+(v) = d^-(v) \quad \text{for } v \text{ outside } S; \text{ this common value is written } d(v). \tag{4.3}$$

By (4.1), d(sigma v) = d(v) for v outside S. Three numbers are attached to the configuration:

$$k = d(p) = \text{the number of vertices of } S^+ \text{ in } N(p);$$

```
    e = the number of edges with both ends in S+;
    Q = sum over x in S+ of d+(x)^2.

An orbit {a_c, b_c} of N(p) is **active** if phi(a_c) != 0. Then exactly one of a_c, b_c is in S+ and the other is
in S-, so k is the number of active orbits of N(p), S n N(p) has 2k vertices and S \ N(p) has 18 - 2k vertices.
p is not in S.

### 4.2 k is 0 or 2

**Lemma 4.2.** k is in {0, 2}.

*Proof.* Fix c. By Lemma 2.1 (b) the neighbours of a_c are p, b_c and 12 vertices outside N[p]; call the latter the
outer neighbours of a_c. The eigen-equation at a_c is

    -4 phi(a_c) = phi(p) + phi(b_c) + (sum of phi over the 12 outer neighbours) = 0 - phi(a_c) + (that sum),

so the sum of phi over the outer neighbours of a_c is -3 phi(a_c). Let n_c be the number of outer neighbours of
a_c that lie in S. As phi is +-1 on S and 0 elsewhere,

    n_c = phi(a_c) (mod 2)   and   n_c >= 3 |phi(a_c)|.

So n_c is odd and >= 3 if the orbit c is active, and n_c is even otherwise. By sigma, b_c has the same number n_c of
outer neighbours in S.

Now count the pairs (s, v) with s in S \ N(p), v in N(p), s ~ v. By Corollary 2.2 each s contributes 2, so the
number is 2 (18 - 2k). Counting by v it is sum over c of 2 n_c. Hence sum_c n_c = 18 - 2k.

Parity: sum_c n_c = k (mod 2) and 18 - 2k is even, so k is even. Size: sum_c n_c >= 3k, so 18 - 2k >= 3k and
k <= 3. Hence k is 0 or 2. QED

### 4.3 e is at most 2

**Lemma 4.3.** e <= 2.

*Proof.* Let e(S) be the number of edges with both ends in S. By sigma there are e edges inside S-, and the number
of edges between S+ and S- is sum_{x in S+} d-(x) = sum_{x in S+} (d+(x) + 4) = 2e + 36 by (4.2). So e(S) = 4e + 36
and 1_S^T A 1_S = 2 e(S) = 8e + 72. On the other hand A = 14 E_0 + 3 E_3 - 4 E_{-4} <= 14 E_0 + 3 (I - E_0) as
quadratic forms, and 1_S^T E_0 1_S = 18^2/99 = 36/11, so

    1_S^T A 1_S <= 14 * 36/11 + 3 (18 - 36/11) = 54 + 11 * 36/11 = 90.

Hence 8e + 72 <= 90 and e <= 2. QED

### 4.4 Two counting identities

**Lemma 4.4.** With the sums taken over all vertices z outside S (p included, with d(p) = k):

    (I1)   sum_z C(d(z), 2) = 18 - 7e - Q;
    (I2)   sum_z d(z)       = 90 - 4e.

*Proof.* (I2). A vertex x of S+ has d+(x) neighbours in S+, d+(x) + 4 in S-, hence 10 - 2 d+(x) outside S. Summing
over S+, the number of edges between S+ and the outside of S is 90 - 2 * 2e = 90 - 4e, and it is also
sum_z d+(z) = sum_z d(z).

(I1). Count the triples ({x, x'}, v) where {x, x'} is a pair of distinct vertices of S+ and v is a common neighbour
of x and x'. There are C(9,2) = 36 pairs; the e adjacent ones have 1 common neighbour and the others 2, so the
number of triples is 72 - e. Counting by v: a vertex v contributes C(d+(v), 2). For v = x in S+ this is
C(d+(x), 2). For v = sigma x in S- it is C(d-(x), 2) = C(d+(x) + 4, 2) by (4.1) and (4.2). For v = z outside S it is
C(d(z), 2). Since C(d,2) + C(d+4,2) = d^2 + 3d + 6,

    72 - e = sum_{x in S+} (d+(x)^2 + 3 d+(x) + 6) + sum_z C(d(z),2) = Q + 6e + 54 + sum_z C(d(z),2).   QED

### 4.5 The parity law  [A1]

**Lemma 4.5.** e = C(k,2) (mod 2). So k = 0 forces e even and k = 2 forces e odd.

*Proof.* The complement of S is sigma-invariant and p is its only fixed vertex, so the 80 vertices z != p outside S
are paired by sigma, with d(sigma z) = d(z). Hence sum_{z outside S, z != p} C(d(z),2) is even, and by (I1)
18 - 7e - Q = C(k,2) (mod 2). Finally Q = sum d+(x)^2 = sum d+(x) = 2e = 0 (mod 2). So 7e + C(k,2) is even. QED

By Lemmas 4.2, 4.3 and 4.5 the pair (k, e) is one of (0,0), (0,2), (2,1).

### 4.6 e = 2 is impossible  [A2]

**Lemma 4.6.** e != 2.

*Proof.* Let e = 2. Either the two edges inside S+ are disjoint, and then four vertices have d+ = 1 and Q = 4; or
they share a vertex, and then Q = 4 + 1 + 1 = 6. By (I1), sum_z C(d(z),2) = 4 - Q. If Q = 6 this is negative,
impossible. If Q = 4 it is 0, so d(z) <= 1 for every z outside S; but by (I2) sum_z d(z) = 82, and there are only
81 vertices outside S. QED

### 4.7 e = 1 is impossible  [A3]

Label S+ = {x_0, ..., x_8} and put y_i = sigma x_i, so S- = {y_0, ..., y_8}. Call the index i **inner** if x_i is in
N(p) and **outer** otherwise, and put eps_i = 1 if i is inner, 0 if i is outer. By Lemma 2.1 (a),
```

    x_i ~ y_i if and only if i is inner.   (4.4)

For an index j let M_j = {i != j : x_j ~ y_i}. Applying sigma, x_j ~ y_i if and only if x_i ~ y_j, so

    i is in M_j if and only if j is in M_i.   (4.5)

d-(x_j) = |M_j| + eps_j by (4.4), so by (4.2)

    |M_j| = d+(x_j) + 4 - eps_j.   (4.6)

Lemma 4.7. e != 1.

Proof. Let e = 1, and let the unique edge inside S+ be {x_a, x_b}. Then d+(x_a) = d+(x_b) = 1 and d+ = 0 on the other seven vertices of S+. There are three cases.

Case C: a and b are both inner. Then x_a and x_b are adjacent vertices of N(p). By Lemma 2.1 (b) the only edges inside N(p) are mate edges, so x_b would be the mate of x_a; but the mate of x_a is y_a, which is in S-, not in S+. Impossible. (This case is not mentioned in the working report.)

Case A: a and b are both outer. Renumber so that a = 0, b = 1. By (4.6), |M_0| = |M_1| = 5 and |M_j| = 4 - eps_j for j >= 2.

The pair x_0, x_1 is adjacent, so it has exactly one common neighbour. For i in M_0 n M_1 the vertex y_i is adjacent to x_0 and to x_1. So

    |M_0 n M_1| <= 1.   (4.7)

Suppose first that 1 is not in M_0. By (4.5), 0 is not in M_1. Then M_0 and M_1 are 5-subsets of the 7-set {2,...,8}, so |M_0 n M_1| >= 3, contradicting (4.7).

So 1 is in M_0 and 0 is in M_1. Put P = M_0 \ {1} and P' = M_1 \ {0}: two 4-subsets of {2,...,8} with P n P' = M_0 n M_1. By (4.7), |P n P'| <= 1; and |P u P'| <= 7 gives |P n P'| >= 1. Hence

    P n P' = {r} for one index r,   and   P u P' = {2,...,8}.

Let j be in P \ {r}. Then j is in M_0 and not in M_1; by (4.5), 0 is in M_j and 1 is not in M_j.

- The vertices x_j and x_0 are not adjacent (the only edge inside S+ is {x_0, x_1} and j >= 2), so they have at most 2 common neighbours. For i in M_j n M_0 the vertex y_i is a common neighbour; and y_j is a common neighbour when eps_j = 1 (x_j ~ y_j by (4.4), and x_0 ~ y_j because j is in M_0). These vertices are distinct (i != j for i in M_j). As 1 is not in M_j, M_j n M_0 = M_j n P. Hence |M_j n P| + eps_j <= 2.
- The vertices x_j and x_1 are not adjacent. y_0 is a common neighbour (0 is in M_j and 0 is in M_1), and so is y_i for every i in M_j n P' (P' is contained in M_1). Hence 1 + |M_j n P'| <= 2, i.e. |M_j n P'| <= 1.
- M_j contains 0, does not contain 1, and has 4 - eps_j elements, so |M_j n {2,...,8}| = 3 - eps_j. Since {2,...,8} = P u P' and P n P' = {r},

      3 - eps_j = |M_j n P| + |M_j n P'| - [r in M_j] <= (2 - eps_j) + 1 - [r in M_j].

  Hence r is not in M_j.

The same argument with the roles of 0 and 1 (and of P and P') exchanged shows that r is not in M_j for j in P' \ {r}. So r is not in M_j for every j in {2,...,8} \ {r}; by (4.5), M_r contains no such j, i.e. M_r is a subset of {0, 1} and |M_r| <= 2. But |M_r| = 4 - eps_r >= 3 by (4.6). Contradiction.

Case B: a is outer and b is inner. By Lemma 4.2 the number of inner indices is k = 2 (it is not 0 since b is inner). Renumber so that a = 0, b = 7, and the other inner index is 8. (The working report uses the label 8 without saying that this relies on k = 2.) By (4.6), |M_0| = 5 and |M_7| = 1 + 4 - 1 = 4.

- 7 is not in M_0: otherwise x_0 would be adjacent to x_7 and to its mate y_7, which is excluded by Lemma 2.1 (c) because x_0 != p. By (4.5), 0 is not in M_7.
- 8 is not in M_7: x_7 and y_8 are two vertices of N(p) that are not mates of each other, so they are not adjacent (Lemma 2.1 (b)). (This justification is missing in the working report.)

So M_0 is a 5-subset of {1,...,6,8} and M_7 is a 4-subset of {1,...,6}; both lie in the 7-set {1,...,6,8}, so |M_0 n M_7| >= 2. For i in M_0 n M_7 the vertex y_i is a common neighbour of the adjacent vertices x_0 and x_7, which have only one. Contradiction. QED

4.8 The support is independent and avoids N[p] [A4, A5]

Corollary 4.8. k = 0 and e = 0. Consequently:

(a) S+ and S- are independent sets;
(b) no vertex of S is in N[p]; in particular d(p) = 0;
(c) x_i not~ y_i for every i;
(d) every vertex of S has exactly 4 neighbours in S, and they are on the other side: d-(x_i) = 4, d+(x_i) = 0.

Proof. (k, e) is one of (0,0), (0,2), (2,1) (section 4.5), and e is neither 2 (Lemma 4.6) nor 1 (Lemma 4.7). So (k, e) = (0, 0). (a) e = 0, and S- = sigma S+. (b) k = 0 says that no vertex of S+ is in N(p); apply sigma for S-; p is not in S. (c) is (4.4) with all indices outer. (d) is (4.2) with d+ = 0. QED

Remark. The case k = 2, called "pattern (ii)" in the working reports, is thus excluded by the parity law together with Lemmas 4.6 and 4.7. Said directly for e = 0: (I1) gives sum_z C(d(z),2) = 18, p contributes C(2,2) = 1 and the 80 other outside vertices contribute an even number, but 17 is odd.

4.9 The orbit graph B

Keep the labelling S+ = {x_0,...,x_8}, y_i = sigma x_i. Define a graph B on the index set {0,...,8} by

    i ~_B j   if and only if   x_i ~ y_j.

Lemma 4.9. B is a well-defined simple graph, it is 4-regular, and any two distinct indices i, j have at most 2 common neighbours in B. Moreover, for i != j the number of common neighbours of x_i and x_j that lie in S is exactly the codegree codeg_B(i,j) = |B(i) n B(j)|, where B(i) is the set of B-neighbours of i.

Proof. The relation is symmetric by (4.5), and there is no loop by Corollary 4.8 (c). The B-neighbours of i are the indices j with y_j in N(x_i), and d-(x_i) = 4, so B is 4-regular. Let i != j. The vertices x_i and x_j are distinct and non-adjacent (Corollary 4.8 (a)), so they have exactly 2 common neighbours in G. Their common neighbours inside S are in S- (S+ is independent), and y_l is a common neighbour exactly when l is in B(i) n B(j). So codeg_B(i,j) <= 2. QED

A relabelling of the nine orbits (a permutation g of {0,...,8}, x_i becoming x_{g i} and y_i becoming y_{g i}) replaces B by an isomorphic graph. So B matters only up to isomorphism, and once B is fixed, the labellings that give exactly this B differ by the elements of Aut(B).

4.10 The three possible graphs B

> **Computation 4.10.** Up to isomorphism there are exactly three 4-regular graphs on 9 vertices in which any two
> distinct vertices have at most 2 common neighbours. In each of them every pair of distinct vertices has exactly 1
> or exactly 2 common neighbours (never 0). Representatives, with the vertices 0..8 and the edges written as pairs
> of digits:
>
>   B0: 01 02 03 04 14 17 18 23 26 28 35 37 45 46 56 57 68 78
>   B1: 01 02 03 04 14 17 18 23 26 28 35 36 45 47 56 57 68 78
>   B2: 01 02 03 04 14 17 18 23 25 26 34 36 48 56 57 58 67 78

Further data on these graphs (identification only; not used in the proof except for the codegrees):

	automorphisms	triangles	codegrees of adjacent pairs	codegrees of non-adjacent pairs	spectrum	name
B0	72	6	18 pairs with 1	18 pairs with 2	4, 1 (x4), -2 (x4)	K3 x K3 (Paley graph on 9 vertices)
B1	12	8	12 with 1, 6 with 2	6 with 1, 12 with 2	4, 2, 1 (x2), -1 (x2), -2 (x3)	-
B2	18	9	9 with 1, 9 with 2	9 with 1, 9 with 2	4, -2 (x2), the roots of t^3 - 3t - 1 twice	circulant C9(1,2)

What exactly was computed, and by whom:

- ‘lemma/t18_REFEREE/ref1_quartic9.py‘ enumerates all 1,024,380 labelled 4-regular graphs on 9 vertices and sorts them into isomorphism classes: 16 classes, all connected. Exactly 3 classes have all codegrees <= 2; they contain 5040 + 30240 + 20160 = 55440 = 9!/72 + 9!/12 + 9!/18 labelled graphs, they have minimum codegree 1, and they are the classes of B0, B1, B2. Each of the other 13 classes has a pair of vertices with 3 or 4 common neighbours. In the first version of this text the list of 16 classes came from this program only.
- ‘lemma/t18_REFEREE2/q9.py‘ (the second referee’s program: pure Python, no code of the repository) does the same enumeration again and gives the same result: 1,024,380 labelled graphs; 16 isomorphism classes, all connected; exactly 3 classes with all codegrees <= 2, containing 5040 + 30240 + 20160 = 55440 labelled graphs, with minimum codegree 1, isomorphic to B0, B1, B2; maximum codegree 3 or 4 in each of the other 13 classes. **So the list of 16 classes now comes from two programs written separately.**
- ‘lemma/t18/enum_S.py‘ enumerates directly the labelled 4-regular graphs with all codegrees <= 2 in which the neighbours of vertex 0 are 1, 2, 3, 4: 792 graphs (= 55440 / C(8,4)), in 3 isomorphism classes.
- ‘lemma/t18/verify_S.py‘ enumerates them without that normalisation: 55440 labelled graphs, 3 classes.

So the selection "4-regular, all codegrees <= 2 => B0, B1 or B2" is a finite computation done by four programs. Two caveats: ‘verify_S.py‘ imports the isomorphism-class routine of ‘enum_S.py‘ (its enumeration is independent, its class count is not), and ‘enum_S.py‘ / ‘verify_S.py‘ impose the condition in the form "common-neighbour budgets on the 18 vertices of S", which for a bipartite graph between S+ and S- given by B is exactly "all codegrees of B are <= 2" (two vertices on different sides have no common neighbour in S).

Hand remark. A 4-regular graph on 9 vertices is connected (a component has at least 5 vertices), in agreement with the output of ‘ref1_quartic9.py‘.

Corollary 4.11. In a configuration, after a suitable labelling of the nine orbits of the support, B is equal to one of B0, B1, B2. For each of the three graphs, 18 of the 36 pairs {i, j} have codegree 1 and 18 have codegree 2.

Proof. Lemma 4.9 and Computation 4.10. If a pairs have codegree 1 and b pairs codegree 2, then a + b = 36 and a + 2b = sum_l C(4,2) = 54 (count the pairs inside the neighbourhoods B(l)), so a = b = 18. QED

A pair {i, j} with codeg_B(i,j) = 1 is called a **deficient pair**. In the order used by all the programs (the pairs {i,j}, i < j, in lexicographic order; the position in this list, from 0 to 17, is the *index* of the pair):

    B0: 01 02 03 04 14 17 18 23 26 28 35 37 45 46 56 57 68 78   (for B0 the deficient pairs are the edges)
    B1: 01 02 03 04 13 16 18 24 27 28 35 37 45 46 56 57 68 78
    B2: 01 02 05 07 12 16 17 25 28 34 36 37 38 45 46 48 58 67

5. The vertices outside the support

From here on: k = 0, e = 0, S+ = {x_i}, S- = {y_i} are independent, S is disjoint from N[p], x_i ~ y_j if and only if i ~_B j, and B is one of B0, B1, B2 (Corollaries 4.8 and 4.11). There are 99 - 18 = 81 vertices outside S, and p is one of them.

5.1 The classes X_d and the path-count identity

For d >= 0 let X_d be the set of vertices z outside S with d(z) = d, i.e. with exactly d neighbours in S+ and d in S- (see (4.3)), and n_d = |X_d|. Each X_d is sigma-invariant, because d(sigma z) = d(z). p is in X_0 (Corollary 4.8). For a vertex v and an integer j let

    a_j(v) = |N(v) n X_j|.

Lemma 5.1 (path count). Let z be outside S with d(z) = d. Then

    sum over the neighbours w of z outside S of d(w) = 18 - 5d,   i.e.   a_1(z) + 2 a_2(z) + 3 a_3(z) + ... = 18 - 5d.

In particular d <= 3: X_d is empty for d >= 4. Moreover

    a_0(z) = a_2(z) + 2 a_3(z) + 3d - 4.   (5.1)

Proof. Count the pairs (w, s) with s in S+, w ~ z and w ~ s. For a fixed s (s != z since z is outside S) the number of w is the number of common neighbours of z and s: 1 if s ~ z, 2 otherwise. So the count is d + 2 (9 - d) = 18 - d. For a fixed neighbour w of z the number of s is d+(w): this is 0 if w is in S+ (S+ is independent), 4 if w is in S- (Corollary 4.8 (d) and sigma), and d(w) if w is outside S. z has d neighbours in S-. So 18 - d = 4d + sum_{w ~ z, w outside S} d(w).

The sum is >= 0, so d <= 3; then all outside vertices have d in {0,1,2,3} and the sum is a_1 + 2 a_2 + 3 a_3. The degree of z is 14 = 2d + a_0 + a_1 + a_2 + a_3. Subtracting the two relations gives (5.1). QED

5.2 The sizes n_d

Lemma 5.2. n_0 = 9 - n_3, n_1 = 54 + 3 n_3, n_2 = 18 - 3 n_3.

Proof. n_0 + n_1 + n_2 + n_3 = 81. By (I2) with e = 0: n_1 + 2 n_2 + 3 n_3 = 90. By (I1) with e = 0 and Q = 0: n_2 + 3 n_3 = 18. Solve. QED

5.3 No outside vertex has d = 3 [N3]

This is placed here, before anything that uses it (referee's remark 2: the working report used it in the proof of the bound of section 5.6 through a computed fact; the hand proof below removes that dependence).

Theorem 5.3. X_3 is empty. Hence (n_0, n_1, n_2) = (9, 54, 18) and every outside vertex has d in {0, 1, 2}.

Proof. Suppose z is in X_3. Its 14 - 6 = 8 neighbours outside S have d-values summing to 18 - 15 = 3 (Lemma 5.1), so at least 5 of them have d = 0: T = N(z) n X_0 has at least 5 elements. The vertex z' = sigma z is in X_3 too, and z' != z because the only fixed vertex is p and d(p) = 0. So n_3 >= 2 and |X_0| = 9 - n_3 <= 7 (Lemma 5.2). T' = N(z') n X_0 also has at least 5 elements. T and T' are subsets of X_0, so |T n T'| >= 5 + 5 - 7 = 3: the distinct vertices z and z' have at least 3 common neighbours, but two distinct vertices have at most 2. QED

The argument uses neither the graph B, nor the neighbourhood of p, nor any computation.

5.4 The identities when X_3 is empty [A8]

Since a_3 = 0 everywhere, write for a vertex v outside S:

    a(v) = a_2(v) (neighbours in X_2),   b(v) = a_1(v) (in X_1);   a_0(v) (in X_0) keeps its name.

Lemma 5.4.

(a) For z in X_0: a_0 = a - 4, b = 18 - 2a; in particular a >= 4.
    For z in X_1: a_0 = a - 1, b = 13 - 2a; in particular a >= 1.
    For z in X_2: a_0 = a + 2, b = 8 - 2a;   in particular a <= 4.

(b) Every vertex of S has exactly 4 neighbours in X_2 and 6 in X_1 (and 4 in S, none in X_0).

(c) Let t be the number of edges with both ends in X_2. Then the number of edges inside X_0 is also t, and

        sum_{X_2} a = 2t,   sum_{X_0} a = 36 + 2t,   sum_{X_1} a = 144 - 4t.

(d) sum over all 81 outside vertices of a^2 = 576 - 2t, and

    (*)   sum_{X_0} (a - 4)^2 + sum_{X_1} (a - 2)(a - 3) + sum_{X_2} a (a - 1) = 36.

    Each of the three sums is >= 0, and t <= 9.

Proof. (a) Lemma 5.1 with a_3 = 0 gives b + 2a = 18 - 5d and a_0 = a + 3d - 4. The inequalities follow from a_0 >= 0 and b >= 0.

(b) Let x be in S+. It has 4 neighbours in S- and none in S+, hence 10 neighbours outside S, each with d >= 1 (it is adjacent to x), so none in X_0. Count the pairs (w, s) with s in S+ \ {x}, w ~ x and w ~ s: each of the 8 vertices s is non-adjacent to x, so the count is 16. A neighbour w of x in S- has 3 neighbours in S+ other than x; an outside neighbour w has d(w) - 1. So 16 = 12 + sum_{w ~ x, w outside} (d(w) - 1), i.e. the ten values d(w), each 1 or 2, sum to 14: four are 2 and six are 1. Apply sigma for S-.

(c) sum_{X_2} a counts each edge inside X_2 twice. The number of edges between X_0 and X_2 is sum_{s in X_2} a_0(s) = sum_{X_2} (a + 2) = 2t + 36, and it is also sum_{z in X_0} a(z). The number of edges inside X_0 is half of sum_{X_0} a_0 = sum_{X_0} (a - 4) = 36 + 2t - 36 = 2t. The number of edges between X_1 and X_2 is sum_{X_2} b = sum_{X_2} (8 - 2a) = 144 - 4t = sum_{X_1} a.

(d) sum over all 99 vertices v of |N(v) n X_2|^2 is the number of triples (v, s, s') with s, s' in X_2 and v adjacent

to both, i.e. sum over the ordered pairs (s, s') of X_2 of |N(s) n N(s')|: 14 for each of the 18 pairs with s = s', and 2 - [s ~ s'] for each of the 18 * 17 ordered pairs with s != s'. Total 252 + 612 - 2t = 864 - 2t. The 18 vertices of S contribute 16 each by (b). So the outside vertices contribute 576 - 2t. Then, using (c) and n_0 = 9, n_1 = 54,

    sum_{X_0}(a-4)^2 + sum_{X_1}(a-2)(a-3) + sum_{X_2} a(a-1)
       = (576 - 2t) - 8 (36 + 2t) + 16 * 9 - 5 (144 - 4t) + 6 * 54 - 2t = 36.

For an integer a, (a-2)(a-3) >= 0 and a(a-1) >= 0. So sum_{X_0} (a-4)^2 <= 36; by the Cauchy-Schwarz inequality on the 9 terms, (2t)^2 = (sum_{X_0} (a - 4))^2 <= 9 * 36, so t <= 9. QED

Identity (*) is used below only as a filter inside some programs ((F7) of Lemma 7.5 and (C4) of Lemma 8.3), where it turns out to remove nothing. It is not needed for the hand part.

5.5 The profile of the neighbourhood of p

N(p) is disjoint from S and consists of the 7 orbits {a_c, b_c} (Lemma 2.1). Put

    delta_c = d(a_c) = d(b_c)   (in {0,1,2} by Theorem 5.3),   h_j = the number of c with delta_c = j   (j = 0,1,2).

So N(p) n X_j consists of h_j orbits (2 h_j vertices), and a_2(p) = 2 h_2.

Lemma 5.5. sum_c delta_c = 9, hence h_2 = 2 + h_0 and h_1 = 5 - 2 h_0, and (h_2, h_1, h_0) is one of

    (2,5,0),   (3,3,1),   (4,1,2).

This triple is called the **profile** of the configuration. The list is exhaustive.

Proof. Each of the 9 vertices of S+ is outside N[p], so it has exactly 2 neighbours in N(p) (Corollary 2.2). Hence sum over v in N(p) of d+(v) = 18, i.e. 2 sum_c delta_c = 18. So h_1 + 2 h_2 = 9 and h_0 + h_1 + h_2 = 7, which gives h_2 = 2 + h_0, h_1 = 5 - 2 h_0, and h_1 >= 0 forces h_0 in {0, 1, 2}. The list is exhaustive because delta_c <= 2, which is Theorem 5.3. QED

5.6 The bound h_0 <= 1; profile (4,1,2) is impossible [A9]

Lemma 5.6. sum over the 14 neighbours w of p of a_2(w) = 36 - 2 h_2. Moreover, for a neighbour w of p:

- if w is in X_2 then a_2(w) >= 1;
- if w is in X_1 then a_2(w) >= 2;
- if w is in X_0 then a_2(w) >= 6.

Proof. sum_{w ~ p} a_2(w) is the number of pairs (w, s) with s in X_2, w ~ p, w ~ s, i.e. the sum over s in X_2 of the number of common neighbours of p and s. p is not in X_2, so this number is 2 - [s ~ p], and the sum is 2 * 18 - a_2(p) = 36 - 2 h_2.

If w is in N(p) n X_2, its mate sigma w is adjacent to w (Lemma 2.1 (a)) and lies in X_2: a_2(w) >= 1.
If w is in N(p) n X_1, then a_0(w) = a_2(w) - 1 by Lemma 5.4 (a), and p is a neighbour of w in X_0: a_0(w) >= 1.
If w is in N(p) n X_0, then a_0(w) = a_2(w) - 4, and p and sigma w are two neighbours of w in X_0: a_0(w) >= 2. QED

Proposition 5.7. h_0 <= 1. So profile (4,1,2) is impossible, for B0, B1 and B2. In profile (3,3,1) all the inequalities of Lemma 5.6 are equalities: a_2 = 1 on N(p) n X_2, a_2 = 2 on N(p) n X_1, a_2 = 6 on N(p) n X_0.

Proof. By Lemma 5.6, 2 h_2 * 1 + 2 h_1 * 2 + 2 h_0 * 6 <= 36 - 2 h_2, i.e. 4 h_2 + 4 h_1 + 12 h_0 <= 36. With h_2 = 2 + h_0 and h_1 = 5 - 2 h_0 this reads 28 + 8 h_0 <= 36, so h_0 <= 1. For h_0 = 1, i.e. profile (3,3,1), the sum of the lower bounds is 6 * 1 + 6 * 2 + 2 * 6 = 30 and the exact value is 36 - 2 * 3 = 30: the two are equal, so every lower bound is attained. For h_0 = 2, i.e. profile (4,1,2), the lower bounds sum to 8 + 4 + 24 = 36 while the exact value is 36 - 8 = 28: impossible. QED

This is a pure hand proof: it uses Theorem 5.3 (through Lemma 5.4 (a) and delta_c <= 2) and nothing computed. What remains is profile (3,3,1) (section 7) and profile (2,5,0) (section 8).

6. The tests used by the computations

A computation of sections 7 and 8 enumerates the possible induced subgraphs of G on certain vertex sets and discards those that violate a necessary condition. Three kinds of necessary conditions are used. **Both PSD matrices of 6.1 are used** (referee's remark 3: the working report says in one place that the violated matrix is always the first one; that sentence is wrong, see section 7.6).

6.1 Two positive-semidefinite matrices

    M1 = 9 (3I - A) + J = 27 I - 9 A + J,   M2 = 11 (4I + A) - 2J = 44 I + 11 A - 2 J.

Lemma 6.1. M1 = 63 E_{-4} and M2 = 77 E_3. Both are PSD, M1 has rank 44 and M2 has rank 54. For every vertex set Y the principal submatrices M1[Y] and M2[Y] are PSD.

Proof. M1 and M2 are polynomials in A and J, so they act as scalars on the three eigenspaces. On the all-ones vector (A = 14, J = 99): M1 = 27 - 126 + 99 = 0 and M2 = 44 + 154 - 198 = 0. On V_3 (A = 3, J = 0): M1 = 27 - 27 = 0 and M2 = 44 + 33 = 77. On V_{-4} (A = -4, J = 0): M1 = 27 + 36 = 63 and M2 = 44 - 44 = 0. A principal submatrix of a PSD matrix is PSD. QED

M1[Y] and M2[Y] depend only on the induced subgraph of G on Y: with A_Y its adjacency matrix, M1[Y] = 27 I - 9 A_Y + J and M2[Y] = 44 I + 11 A_Y - 2J (I and J of size |Y|).

A **PSD violation** on Y is an integer vector x indexed by Y with x^T M1[Y] x < 0 or x^T M2[Y] x < 0. Checking a

given violation is one integer computation. Since M1[Y] and M2[Y] have integer entries, a PSD violation on Y exists if and only if M1[Y] or M2[Y] is not PSD (a real vector with negative quadratic form can be replaced by a rational vector close to it, and then multiplied by a common denominator). So "there is no PSD violation on Y" and "M1[Y] and M2[Y] are both PSD" are the same statement about the graph on Y, and it does not depend on any program.

6.2 The common-neighbour budget

Lemma 6.2. Let Y be a vertex set and let H' be a graph on Y all of whose edges are edges of G (a spanning subgraph of the induced subgraph G[Y]). Then for any two distinct vertices u, v of Y,

    |N_H'(u) n N_H'(v)| <= 2 - [u ~ v in H'].

Proof. N_H'(u) n N_H'(v) is contained in N(u) n N(v), which has 1 element if u ~ v in G and 2 otherwise. If u ~ v in H' then u ~ v in G and the bound is 1. If u not~ v in H' the bound 2 holds in both cases. QED

A pair u, v violating the inequality is a **budget violation**. The point of allowing a spanning subgraph is that a partially specified graph (some adjacencies not yet decided and provisionally absent) may be rejected by a budget violation: adding edges can only increase the numbers of common neighbours and decrease the bound. A PSD violation, on the contrary, is meaningful only on a vertex set all of whose adjacencies are decided.

6.3 Determined sets, and what a computation proves

Let D be some data extracted from a configuration (for instance the matching tau of section 7.1). A vertex set Y (described in terms of the configuration, e.g. "S u {p} u X_2") is **determined by D** if the induced subgraph G[Y], with the names of its vertices, can be written down from D alone. For every determined set used below, the text gives a table that says, for every pair of classes of vertices of Y, whether the adjacency is a parameter in D or is forced, and by which lemma.

The computations have the following form. A finite list L of candidate data D is defined: all values of the parameters that satisfy the forced numbers named in the statement of the Computation. For D in L, write G_D for the graph on Y built from D.

Definition (survivor). A candidate D in L is a **survivor** if G_D has no budget violation and the matrices M1[Y] and M2[Y] of G_D are both PSD. This is a property of D alone: it does not refer to a program, to the order of the tests or to a numerical threshold. (In the first version of this text a survivor was a candidate for which "no violation is found", which depended on the program; second referee's remark 1.) The data of a configuration satisfy the forced numbers, which are proved in the text, and pass both tests by Lemmas 6.1 and 6.2. Hence:

    the data D of any configuration belong to the list of survivors. (6.1)

So an empty list of survivors is a contradiction, and a non-empty list reduces the problem to its members.

What a program does. Apart from the referee's programs REF1, which are used only as count checks (see "Exactness of the rejections" below), a program **rejects** a candidate D only when it has an exact witness that D cannot be the data of a configuration. There are three kinds of witnesses:

- a budget violation on G_D, or on a partial graph, i.e. a graph on a subset of Y all of whose edges are edges of G_D (this is how partial data are tested, see section 6.2);
- a PSD violation on Y, or on a subset of Y all of whose adjacencies are decided;
- a forced number proved in the text that fails. Besides the forced numbers named in the Computation, some programs of the repository use other ones, also proved in the text. Those mentioned in this text are (F2) and (F7) (section 7.3), and (C3), (C4) and the existence of five types satisfying (C5) (section 8.3); the text reports each time that they remove nothing.

The candidates that a program does not reject form its **kept list**. The data of a configuration admit no witness of any of the three kinds: a principal submatrix of a PSD matrix is PSD (Lemma 6.1), a graph all of whose edges are edges of G has no budget violation (Lemma 6.2), and the forced numbers are proved. Hence, for every program whose rejections are exact:

    the data D of any configuration belong to the kept list. (6.2)

(6.2) rests on exact rejections only, and it is all that the exclusions of this text use. A computation excludes something in one of two ways:

- its kept list is empty: then no configuration has its data in L;
- its kept list is not empty, and the next step is run over the kept list (not over the list of survivors): by (6.2) the data of a configuration are still among the candidates.

A witness of the first or of the second kind also shows that D is not a survivor, for the same two reasons. So a program with exact rejections that uses no forced number other than those named in the Computation never rejects a survivor, and its kept list contains the list of survivors; according to their headers this is the case of the programs of the second referee (`lemma/t18_REFEREE2`). In the other direction a kept list may be larger than the list of survivors, because of the way "kept" is decided (see below). This never matters for an exclusion. It matters only for the question whether a number printed in this text is the number of survivors or only the size of a kept list.

Exactness of the rejections. According to their sources (for the programs read for this text) or their headers (for the others; see section 9.3, item 3), in all the programs of the repository a rejection is exact: a budget violation is an integer count, and a PSD violation is an integer vector x whose quadratic form is evaluated in integer arithmetic (or, in some programs, an exact elimination or an exact characteristic polynomial). The same holds for the programs of the second referee (integer vector evaluated in Python integers, or exact characteristic polynomial). The referee's own programs (REF1) are different: they reject on the floating-point eigenvalue alone, without an exact certificate; they are used here only as count checks (section 9.3).

"Kept" decisions. A candidate for which no violation is found is kept; for most programs this "kept" rests on a

floating-point eigenvalue (smallest eigenvalue above a threshold of about -10^-7; the threshold is -10^-6 in ‘r3_verify.py‘) and **is not a proof of positive semidefiniteness and is never used as one**: keeping too much can only make the kept list larger, which is harmless by (6.2).

For the following lists, which are the lists with a PSD condition whose sizes are printed in the Computations, it has been verified exactly that every member is a survivor, i.e. that both matrices are PSD on the whole set (the budgets are integer counts in every program):

list	vertex set	every member exactly PSD: verified by
39-vertex survivors of profile (3,3,1) (383 / 481 / 7)	Y1	I2, which decides its acceptances at 39, 45 and 51 vertices by the exact characteristic polynomial (section 7.5). I1 (and I3 for B2) keep on a floating-point eigenvalue and obtain the same sets. Not re-checked by the second referee
tables of line 3b of section 7.5 (23 / 164 / 0)	Y (45 vertices)	I2 (as above). I1 keeps on a floating-point eigenvalue and obtains the same sets. Not re-checked by the second referee
pairs (tau, F) of the second row of the table of Computation 8.4 (4904 / 11509 / 3863)	S u {p} u A_p (23 vertices)	the second referee (‘recount.py‘, exact characteristic polynomial). ENUM and VERIF keep on a floating-point eigenvalue
stage-B triples (5595 / 4977 / 196)	Y37	the second referee (‘exactB.py‘: exact characteristic polynomials of M1[Y37] and M2[Y37] for every triple, and all budgets), on his own list, which is the list of VERIF up to the action of Aut(B) (‘orbitsB_<s>.out‘). ENUM and VERIF keep on a floating-point eigenvalue
the four stage-C survivors of B1	Y47	‘r3_survivors.py‘ and ‘ref8_survivors.py‘ (both matrices), ‘r4_verifyD.py‘ (M1), and the second referee (‘survivors.py‘, both matrices); see section 8.5

All the other "kept" decisions are floating-point and remain so: the intermediate decisions taken during a search on a subset of Y or on partial data (they only prune); the counters specific to one program that are quoted in the notes (for instance the numbers of sets K "without PSD violation" on 39 vertices in section 8.4); the kept lists of REF1; and the kept lists of the second referee’s programs other than those of the table. The lists of the remarks of section 7.6 (enumeration with M1 alone) are not used in the proof and are not covered by the table. None of this enters an exclusion: the exclusions rest on empty kept lists (line 4b of the table of section 7.5; stage C for B0 and B2; stage D), which involve no acceptance at all, and on Computation 8.6, which is an exact integer computation on four explicit graphs.

6.4 The rank bound [P2]

Let P be the permutation matrix of sigma (P e_x = e_{sigma x}). P is symmetric, P^2 = I, and P commutes with A (sigma is an automorphism) and with J.

Lemma 6.3. trace(P E_{-4}) = 0 and trace(P E_3) = 0. Hence sigma acts on V_{-4} with the eigenvalues +1 and -1, each with multiplicity 22, and on V_3 with the eigenvalues +1 and -1, each with multiplicity 27.

Proof. trace P is the number of fixed vertices, 1. trace(PA) = sum_x A_{x, sigma x} is the number of vertices x with x ~ sigma x, which is |N(p)| = 14 by Lemma 2.1 (a). trace(PJ) = 99. By Lemma 6.1,

$$\mathrm{trace}(P E_{-4}) = (27 * 1 - 9 * 14 + 99) / 63 = 0, \qquad \mathrm{trace}(P E_3) = (44 * 1 + 11 * 14 - 2 * 99) / 77 = 0.$$

P preserves V_{-4} and V_3 and has eigenvalues +-1 there; the trace is the difference of the two multiplicities, and their sums are 44 and 54. QED

Proposition 6.4 (rank bound). Let Y be a sigma-invariant vertex set, and let R be a set containing exactly one vertex of each orbit {x, sigma x} contained in Y (x != p).

(a) (odd part) The |R| x |R| integer matrix

$$G_{odd} = 3\,I - A[R,R] + A[R, \sigma R] \qquad (\text{entry } (x,y)\text{: } 3\,[x = y] - [x \sim y] + [x \sim \sigma y])$$

has rank at most 22. In particular, if |R| >= 23 then every 23 x 23 principal minor of G_odd is 0; if |R| = 23, det G_odd = 0.

(b) (even part) Let the vectors g_x = e_x + e_{sigma x} (x in R), together with g_p = e_p if p is in Y, be the "even generators". The matrix G_even with entries g^T M1 g’ has rank at most 22. Explicitly, for x != y in R:
G_even(x,y) = 4 - 18 ([x ~ y] + [x ~ sigma y]), G_even(x,x) = 58 - 18 [x ~ sigma x],
G_even(p,x) = 2 - 18 [p ~ x], G_even(p,p) = 28.

Proof. (a) For x in R put u_x = e_x - e_{sigma x}. Since M1 commutes with P, M1(sigma x, sigma y) = M1(x, y) and M1(sigma x, y) = M1(x, sigma y), so

$$u_x^T M1\, u_y = 2\,(M1(x,y) - M1(x, \sigma y)) = 2\,(27\,[x = y] - 9\,[x \sim y] + 9\,[x \sim \sigma y]) = 18\,G_{odd}(x,y)$$

(the terms +1 coming from J cancel, and x = sigma y does not happen for x, y in R). On the other hand M1 = 63 E_{-4} and E_{-4} is a symmetric idempotent, so u_x^T M1 u_y = 63 (E_{-4} u_x) . (E_{-4} u_y): the matrix 18 G_odd is 63 times the Gram matrix of the vectors E_{-4} u_x. These vectors lie in V_{-4} and satisfy P E_{-4} u_x = E_{-4} P u_x = - E_{-4} u_x, so they lie in the (-1)-eigenspace of sigma on V_{-4}, which has dimension 22 by Lemma 6.3. A Gram matrix of vectors in a space of dimension 22 has rank at most 22.

(b) The same computation with g_x: g^T M1 g’ = 63 (E_{-4} g) . (E_{-4} g’), and the vectors E_{-4} g lie in the (+1)-eigenspace of sigma on V_{-4}, of dimension 22. The explicit entries are 2 (M1(x,y) + M1(x, sigma y)), M1(p,x) + M1(p, sigma x) and M1(p,p). QED

G_odd and G_even depend only on the induced subgraph on Y and on sigma. So for a sigma-invariant *determined* set Y with at least 23 orbits, "some 23 x 23 principal minor of G_odd is non-zero" is a contradiction, checked by one exact integer determinant. Positive semidefiniteness does not see this condition: the matrix can be positive definite (see section 8.6). The analogous bounds for M2 (odd and even parts of rank at most 27) are true by the same proof and are not used.

7. Profile (3,3,1)

Section 7.1 is common to sections 7 and 8: it describes X_2 and the first stage of both enumerations.

7.1 X_2, the matching tau of the deficient pairs, and stage 1

Notation: for a set U of indices, x_U = {x_i : i in U} and y_U = {y_i : i in U}.

Lemma 7.1. (a) For every deficient pair U = {i, j} there is exactly one vertex outside S adjacent to both x_i and x_j. Call it s_U. It lies in X_2 and N(s_U) n S+ = x_U. The map U -> s_U is a bijection from the 18 deficient pairs onto X_2. A pair {i, j} that is not deficient has no common neighbour outside S.

(b) There is a permutation tau of the set of deficient pairs such that, for every deficient pair U,

    N(s_U) n S- = y_{tau U}   and   sigma s_U = s_{tau U}.

tau is an involution without fixed point.

Proof. (a) Let i != j. The vertices x_i and x_j have exactly 2 common neighbours, of which codeg_B(i,j) lie in S (Lemma 4.9); codeg_B(i,j) is 1 or 2 (Corollary 4.11). So they have exactly one common neighbour outside S if {i,j} is deficient and none otherwise. A vertex outside S adjacent to x_i and x_j has d >= 2, hence d = 2 (Theorem 5.3): it is in X_2 and its neighbours in S+ are exactly x_i and x_j. Conversely a vertex z of X_2 has exactly two neighbours x_i, x_j in S+; it is an outside common neighbour of them, so {i,j} is deficient and z = s_{i,j}. The map is injective because U is read off from N(s_U) n S+.

(b) sigma s_U is in X_2 and N(sigma s_U) n S+ = sigma (N(s_U) n S-). So if V denotes the set of the two indices l with y_l ~ s_U, then sigma s_U is a vertex of X_2 whose neighbours in S+ are x_V; by (a), V is a deficient pair and sigma s_U = s_V. Put tau U = V. Since sigma^2 = 1, tau^2 = 1. If tau U = U, then sigma s_U = s_U, so s_U = p; but p is in X_0. QED

So the 9 orbits of X_2 are the 9 pairs {U, tau U}; such a pair of deficient pairs is called a **rectangle**, and the vertex s_U is said to be of type (U ; tau U): it is adjacent to x_U and y_{tau U}; its mate is of type (tau U ; U). tau is a perfect matching of the 18 deficient pairs into 9 rectangles. Once B is fixed, the adjacency between S and X_2 is determined by tau.

Lemma 7.2 (necessary conditions on tau; these are the stage-1 tests).

(R1) For every deficient pair U, with V = tau U: every index i of U has at most one B-neighbour in V, and every index l of V has at most one B-neighbour in U.

(R2) For all indices i, l in {0,...,8}, the case i = l included: the number of deficient pairs U with i in U and l in tau U is at most 2 - [i ~_B l].

(R3) The orbit {s_U, s_V} (V = tau U) is contained in N(p) if and only if s_U ~ s_V; and in that case U n V is empty.

Proof. (R1) Let s = s_U and i in U, so s ~ x_i. Every y_l with l in V n B(i) is adjacent to s (l in V) and to x_i (l in B(i)): it is a common neighbour of the adjacent pair s, x_i, which has only one. The second half is the same with the adjacent pair s, y_l and the vertices x_i, i in U n B(l).

(R2) Every such s_U is adjacent to x_i and to y_l. These two vertices are distinct, and they are adjacent if and only if i ~_B l (for i = l they are not adjacent, by Corollary 4.8 (c), and [i ~_B i] = 0). So they have exactly 2 - [i ~_B l] common neighbours, and the vertices s_U are distinct for distinct U.

(R3) The first statement is Lemma 2.1 (a). If s_U ~ s_V, their only common neighbour is p (Lemma 2.1 (c)); for i in U n V the vertex x_i would be adjacent to s_U and to s_V. QED

Remarks on the tests. The header of the first stage-1 program lists four tests, by reference to a note that is not in the repository; here is what they are. Its test (i) is (R1). Its test (iii), "cell budgets", is (R2): the "cell" (i,l) is the pair of vertices x_i, y_l. Its test (i'), |U n tau U| <= 1, is automatic, since U and tau U are two distinct 2-sets. Its test (P) requires in addition that the type (U ; tau U) be in the list ‘types.json‘; that list was produced by a test on the 21 vertices S u {p, s_U, s_{tau U}}, in at least one of the two statuses of the orbit (inside or outside N(p)); see section 9.3, item 6, for what that test contains. Computation 7.3 below shows that (P) removes nothing.

Call an unordered pair {U, V} of distinct deficient pairs an **admissible rectangle** if (R1) holds for it (the condition is symmetric in U and V), and **usable in N(p)** if moreover U n V is empty. Call a fixed-point-free involution tau of the 18 deficient pairs an **admissible matching** if its 9 rectangles are admissible and (R2) holds. By Lemma 7.2 the tau of a configuration is an admissible matching, and the orbits of X_2 contained in N(p) correspond to rectangles of tau that are usable in N(p).

Aut(B) acts on the deficient pairs and hence on the admissible matchings. By the remark after Lemma 4.9, relabelling the nine orbits of the support by an element g of Aut(B) keeps B and replaces tau by its image under g. Hence:

    in a configuration with orbit graph B one may assume, after relabelling, that tau is any chosen
    representative of its orbit under Aut(B).   (7.1)

> **Computation 7.3 (stage 1).** For each of B0, B1, B2 there are 99 admissible rectangles, 63 of which are usable
> in N(p). The numbers of admissible matchings and of their orbits under Aut(B) are:
>
> | | B0 | B1 | B2 |
> |---|---|---|---|
> | admissible matchings tau | 25514 | 13799 | 9229 |
> | order of Aut(B) | 72 | 12 | 18 |
> | orbits under Aut(B) | 425 | 1259 | 528 |

>
> One representative of each orbit is stored in ‘lemma/t18/r2_tau_<s>.json‘ (s = 0, 1, 2 for B0, B1, B2). "Matching
> #n" below means the n-th entry (from 0) of that file.

Programs (four, in agreement on every number of the table):

- ‘lemma/t18/r2_stage1.py‘: tests (R1), (i’), (R2) and (P); it writes ‘r2_tau_<s>.json‘.
- ‘lemma/t18/r3_verify.py stage1‘: another algorithm, with its own 21-vertex test; it also checks that the stored representatives are admissible matchings, pairwise inequivalent, one in each orbit.
- ‘lemma/t18_VERIF331/v1_stage1.py‘: runs twice, with (R1) and (R2) only, and with an exact 21-vertex PSD test in addition; the two runs give the same 99 rectangles and the same numbers of matchings and orbits. It builds its own list of representatives.
- ‘lemma/t18_REFEREE/ref2_stage1.py‘: (R1) and (R2) only; ‘ref10_tau_files.py‘ checks that the stored representatives are the referee’s own, in the same order.

Test (P) removes nothing: the first program, which applies it, finds 99 admissible rectangles and the same numbers of matchings as the programs that apply (R1) and (R2) only.

All the later programs except those of ‘lemma/t18_VERIF331‘ and ‘lemma/t18_REFEREE‘ read the representatives from ‘r2_tau_<s>.json‘.

7.2 The forced structure of profile (3,3,1) [A10]

In this section the profile is (h_2, h_1, h_0) = (3, 3, 1). Notation:

```
    A_p  = N(p) n X_2      (6 vertices, 3 orbits);
    W    = N(p) n X_1      (6 vertices, 3 orbits);
    {z_0, sigma z_0} = N(p) n X_0;
    X_2' = X_2 \ A_p       (12 vertices, 6 orbits);
    X_0' = X_0 \ {p, z_0, sigma z_0}   (6 vertices, 3 orbits; X_0 has 9 vertices by Theorem 5.3);
    A_{z_0} = N(z_0) n X_2;
    f_i  = the number of vertices of A_p adjacent to x_i.
```

Lemma 7.4.

(a) A vertex of A_p has no neighbour in X_2 except its mate. It has exactly 3 neighbours in X_0: p and two vertices of X_0’.

(b) A vertex w of W has exactly 2 neighbours in X_2; they lie in X_2’ and they are not mates of each other. Its only neighbour in X_0 is p. It has exactly one neighbour x_i in S+ and one neighbour y_j in S-, and i != j; w is said to be of type (i ; j), and sigma w is of type (j ; i). The three orbits of W have three different unordered types {i, j}.

(c) z_0 has no neighbour in S, in A_p or in W; its neighbours in X_0 are exactly p and sigma z_0; A_{z_0} has 6 elements, is contained in X_2’ and contains exactly one vertex of each of the 6 orbits of X_2’; N(sigma z_0) n X_2 = sigma A_{z_0} is the complement of A_{z_0} in X_2’.

(d) A vertex s of X_2’ is adjacent to exactly one of z_0, sigma z_0, to exactly one vertex of W, to no vertex of A_p, and to exactly a_2(s) + 1 vertices of X_0’.

(e) A vertex q of X_0’ has no neighbour in S and is not adjacent to p, to sigma q, to z_0, to sigma z_0, or to any vertex of W. It has exactly 2 neighbours in A_p, and they lie in two different orbits of A_p. a_2(q) >= 4, and q has exactly a_2(q) - 4 neighbours in X_0’.

(f) For every i, x_i has exactly f_i neighbours in A_p and 2 - f_i in W. In particular f_i <= 2.

Proof. By Proposition 5.7, a_2 = 1 on A_p, a_2 = 2 on W and a_2 = 6 on z_0 and sigma z_0. Recall that inside N(p) a vertex is adjacent only to its mate (Lemma 2.1 (b)), and that mates lie in the same class X_d.

(a) The mate is a neighbour in X_2 and a_2 = 1. By Lemma 5.4 (a), a_0 = a + 2 = 3; p is one of the three neighbours in X_0; z_0 and sigma z_0 are vertices of N(p) other than the mate, so they are not neighbours; the two remaining ones are in X_0’.

(b) a_2(w) = 2 and a_0(w) = a_2(w) - 1 = 1, and p is a neighbour of w in X_0. The two neighbours in X_2 are not in A_p, which is inside N(p). If they were s and sigma s, then w would be a common neighbour of s and sigma s, where s is outside N[p]; by Lemma 2.1 (d) the common neighbours of s and sigma s are outside N[p], but w is in N(p). The same argument shows i != j: if i = j, then w would be a common neighbour of x_i and its mate y_i, and x_i is outside N[p]. sigma w is adjacent to sigma x_i = y_i and sigma y_j = x_j. Finally let w and w’ be in two different orbits of W and suppose they have the same unordered type; replacing w’ by its mate we may assume that both are of type (i ; j). Then w and w’ are distinct, not adjacent (they are in N(p) and not mates), and have the three common neighbours p, x_i, y_j: impossible.

(c) z_0 is in X_0, so it has no neighbour in S. Inside N(p) it is adjacent only to sigma z_0. a_0(z_0) = 6 - 4 = 2 and p, sigma z_0 are two neighbours in X_0. So A_{z_0} is a 6-subset of X_2’. Applying sigma, the neighbours of sigma z_0 in X_2 form sigma A_{z_0}. A vertex in both A_{z_0} and sigma A_{z_0} would be a common neighbour of z_0 and sigma z_0 other than p, which Lemma 2.1 (c) forbids. So A_{z_0} and sigma A_{z_0} are disjoint 6-subsets of the 12-set X_2’: they are complementary, and A_{z_0} contains exactly one vertex of each orbit.

(d) The first statement is (c). s is outside N[p], so it has exactly 2 neighbours in N(p) (Corollary 2.2): none in A_p by (a), exactly one in {z_0, sigma z_0}, hence exactly one in W. By Lemma 5.4 (a) s has a_2(s) + 2 neighbours in X_0; p is not one of them, exactly one of z_0, sigma z_0 is; the other a_2(s) + 1 are in X_0’.

(e) q is in X_0: no neighbour in S. q is not in N(p), so q not~ p and q not~ sigma q (Lemma 2.1 (a)). By (a) there are exactly 6 * 2 = 12 edges between A_p and X_0’. By Corollary 2.2 each of the 6 vertices of X_0’ has exactly 2 neighbours in N(p), hence at most 2 in A_p; as the total is 12, each has exactly 2 neighbours in A_p, and

therefore **none in the rest of N(p), i.e. none in W u {z_0, sigma z_0}**. (This consequence is used for the 45- and 51-vertex sets and is not stated in the working report: referee's remark 7.) The two neighbours in A_p are not mates of each other, by Lemma 2.1 (c). By Lemma 5.4 (a), a_2(q) >= 4 and q has a_2(q) - 4 neighbours in X_0; they are none of p, z_0, sigma z_0, so they are in X_0'.

(f) x_i is outside N[p]: it has exactly 2 neighbours in N(p) = A_p u W u {z_0, sigma z_0}, and none of them is z_0 or sigma z_0. QED

7.3 The determined sets of 39, 45 and 51 vertices

```
    Y1    = S u {p} u X_2 u {z_0, sigma z_0}        (18 + 1 + 18 + 2 = 39 vertices);
    Y     = Y1 u X_0' = S u X_0 u X_2               (45 vertices);
    Y u W                                            (51 vertices).
```

All three are sigma-invariant. The data:

- **tau**, the matching of section 7.1 (a representative, by (7.1));
- **F**, the set of the three rectangles of tau whose orbits form A_p; each must be usable in N(p) (Lemma 7.2 (R3));
- **H**, the set of edges inside X_2'. It is invariant under sigma (which acts on X_2 by s_U -> s_{tau U}) and contains no mate edge (Lemma 2.1 (a));
- **A_{z_0}**, a choice of one vertex in each of the 6 orbits of X_2'. Exchanging the names z_0 and sigma z_0 replaces A_{z_0} by its complement; the programs fix the names by requiring that A_{z_0} contain a prescribed vertex of the first orbit, so there are 2^5 choices;
- **the rows**: for each q in X_0' the set R_q = N(q) n X_2, with R_{sigma q} = sigma R_q; and **G'**, the set of edges inside X_0', invariant under sigma. (rows, G') is called a **table**. As an object to be enumerated and counted (second referee's remark 5), a table is defined as follows. The six vertices of X_0' have no names given by the earlier data, so a table is an unordered set of three row-orbits {R, sigma R}, R being a subset of X_2, together with a sigma-invariant set G' of edges on the six vertices that carry these rows. A row may contain both vertices of an orbit of X_2'. Two tables that differ only by the order of the three orbits of X_0', or by the exchange of q and sigma q inside an orbit, are the same table. (As soon as there is no budget violation on Y the six rows are pairwise distinct, because a row has at least 4 elements by (F4) below and two distinct vertices have at most 2 common neighbours; so the rows identify the six vertices.) This is the object counted in lines 3a and 3b of section 7.5;
- **for W**: the three types (i ; j), and for each w in W the 2-set K_w = N(w) n X_2', with K_{sigma w} = sigma K_w.

Lemma 7.5. Y1 is determined by (tau, F, H, A_{z_0}); Y is determined by these data and the table; Y u W is determined by all the data. More precisely every adjacency between two vertices of Y u W is given by the following list, in which "parameter" means "read from the data" and every other entry is forced by the lemma quoted.

pair of classes	adjacency	reason
S+ with S+, S- with S-	none	Corollary 4.8 (a)
S+ with S-	x_i ~ y_j iff i ~_B j	definition of B
S with p	none	Corollary 4.8 (b)
S with X_2	s_U ~ x_i iff i in U; s_U ~ y_l iff l in tau U	parameter tau (Lemma 7.1)
p with X_2	exactly the 6 vertices of A_p	parameter F
A_p with A_p	the 3 mate edges only	Lemma 2.1 (b)
A_p with X_2'	none	Lemma 7.4 (a)
X_2' with X_2'	parameter H; no mate edge	Lemma 2.1 (a)
z_0, sigma z_0 with p; z_0 with sigma z_0	adjacent	definition; Lemma 2.1 (a)
z_0, sigma z_0 with S, with A_p	none	Lemma 7.4 (c)
z_0 with X_2'	exactly A_{z_0}; sigma z_0: the complement	parameter A_{z_0} (Lemma 7.4 (c))
X_0' with S, p, z_0, sigma z_0	none	Lemma 7.4 (e)
q with sigma q (q in X_0')	not adjacent	Lemma 7.4 (e)
X_0' with X_2	parameter rows	-
X_0' with X_0'	parameter G'	-
W with p	adjacent	definition
W with W	the 3 mate edges only	Lemma 2.1 (b)
W with A_p, with z_0, sigma z_0	none	Lemma 2.1 (b)
W with X_0'	none	Lemma 7.4 (e) (or (b))
W with S	w ~ x_i, y_j only; sigma w ~ x_j, y_i only	parameter types (Lemma 7.4 (b))
W with X_2'	parameter K_w	-

Moreover the data satisfy the following **forced numbers**:

- (F1) each of the three rectangles of F is usable in N(p);
- (F2) every vertex of X_2 has at most 4 neighbours in X_2 (Lemma 5.4 (a));
- (F3) every s in A_p has exactly 2 neighbours in X_0'; every s in X_2' has exactly (number of its H-neighbours) + 1;
- (F4) every q in X_0' has exactly 2 neighbours in A_p, |R_q| >= 4, and exactly |R_q| - 4 neighbours in G';
- (F5) the three unordered types are distinct, i != j in each, and every index i occurs in exactly 2 - f_i of the three unordered types (x_i has exactly one neighbour in each orbit of W whose unordered type contains i);
- (F6) each K_w has 2 elements and contains no mate pair; every vertex of X_2' lies in K_w for exactly one w in W;
- (F7) sum over X_2 of a_2 (a_2 - 1) <= 24.

Proof. The table is a restatement of the lemmas quoted; note that on X_2 one has a_2(s) = 1 for s in A_p and a_2(s) = the number of H-neighbours of s for s in X_2'. (F1) is Lemma 7.2 (R3). (F3) is Lemma 7.4 (a), (d). (F4) is Lemma 7.4 (e). (F5) is Lemma 7.4 (b), (f): a vertex of W of type (i ; j) contributes one neighbour in W to x_i and its mate, of type (j ; i), one to x_j. (F6) is Lemma 7.4 (b), (d). (F7): in identity (*) of Lemma 5.4 the three vertices p, z_0, sigma z_0 of X_0 have a_2 = 6 (for p: a_2(p) = 2 h_2 = 6) and contribute 3 * (6 - 4)^2 = 12 to the first sum; all other terms of the first two sums are >= 0; so the third sum is at most 36 - 12 = 24. QED

(F2) and (F7) are the only filters, in the programs read for this text (`r2_hfirst.py`, `v2_stage2.py`, `v3_stage34.py`), that are not budgets, PSD violations or the structure of Lemma 7.4: `r2_hfirst.py` uses (F2) and (F7) in step 2; `v2_stage2.py` uses neither (its 39-vertex survivors are defined by budgets and PSD on Y1 alone); `v3_stage34.py` uses (F2) in step 3 and only records (*). Since the two programs produce the same 39-vertex

survivors (section 7.5), (F2) and (F7) remove nothing there.

7.4 What is enumerated, and the result

> **Computation 7.6.** For each B in {B0, B1, B2} and each representative tau of Computation 7.3:
>
> - step 1: all sets F of three rectangles of tau that are usable in N(p), such that the graph on the 25 vertices
>   S u {p} u A_p has no budget violation;
> - step 2: for each such (tau, F), all (H, A_{z_0}) such that the graph on Y1 (39 vertices) has no budget violation
>   and M1[Y1] and M2[Y1] are both PSD; these (tau, F, H, A_{z_0}) are the **39-vertex survivors**;
> - step 3: for each 39-vertex survivor, all tables (in the sense of section 7.3) satisfying (F3), (F4); among them
>   those for which the graph on Y (45 vertices) has no budget violation; among those, the ones for which M1[Y]
>   and M2[Y] are both PSD;
> - step 4: for each table of the last kind, all (types, K) satisfying (F5), (F6); among them those for which the
>   graph on Y u W (51 vertices) has no budget violation; among those, the ones for which M1 and M2 are both PSD
>   on Y u W.
>
> Result: the last list is empty for B0, B1 and B2. The counts are in the table of section 7.5.

All these lists are defined by properties of the graphs: they are lists of survivors in the sense of section 6.3 and do not depend on a program. What the exclusion uses is only (6.2): in every implementation each step is run over the kept list of the previous step, every rejection is exact, and the kept list of step 4 is empty; by (6.2) no configuration has its data there, whatever the floating-point "kept" decisions were. (For a program that uses only the forced numbers named above, such as the second referee's, the empty kept list also gives the Result as it is stated, because such a program never rejects a survivor.) That the kept lists of steps 2 and 3 are exactly the lists of survivors defined here, so that 383 / 481 / 7 and 23 / 164 / 0 in section 7.5 are numbers of survivors, rests on the exact acceptances of I2 (section 6.3).

Proposition 7.7. Profile (3,3,1) is impossible for B0, B1 and B2.

Proof. Suppose a configuration has profile (3,3,1) and orbit graph B. By (7.1) its matching tau may be taken to be one of the representatives. By Lemma 7.5 its sets Y1, Y, Y u W are determined by its data, which satisfy (F1)-(F7), and by Lemmas 6.1 and 6.2 the corresponding graphs have no budget violation and no PSD violation. So its data survive all four steps (this is (6.1)), contradicting Computation 7.6. QED

Status: hand part (Lemmas 7.1, 7.2, 7.4, 7.5) plus Computations 7.3 and 7.6.

7.5 Counts, and the number of implementations of each line

Implementations:

- **I1** = `lemma/t18/r2_hfirst.py` (steps 1-2: H is completed first, then A_{z_0}), `lemma/t18/r2_stage2.py` run with the switch R2_SURV (step 3, restricted to the 39-vertex survivors), `lemma/t18/r2_ext2.py` (step 4, by a pruned search). These programs read `structures.json`, `types.json` and `r2_tau_<s>.json`.
- **I2** = `lemma/t18_VERIF331/`: `v1_stage1.py` (own stage 1 and own representatives), `v2_stage2.py` (steps 1-2: the orbits of X_2' are added one at a time, each with its side relative to z_0, so A_{z_0} is chosen before H is complete), `v3_stage34.py` (step 3; it also adds W directly to the 39 vertices and combines), `v7_step4_direct.py` (step 4: every (types, K), budgets as the only pre-filter, then an exact PSD decision). Written from the mathematical description; no program of `lemma/t18` was opened; the output files of I1 were read only for the comparison of the sets (`v4_compare39.py`, `v4_compare45.py`). In I2 a rejection is an integer vector re-evaluated in Python integers or an exact characteristic polynomial, and the acceptances at 39, 45 and 51 vertices are decided by the exact characteristic polynomial.
- **I3** = `lemma/t18_B2/e331.py` (B2 only, steps 1-3); shares no code with I1 and does not read `types.json`.
- **REF1** = the referee's programs `lemma/t18_REFEREE/ref2_stage1.py`, `ref6_p331.py`, `ref7_p331_ext.py`, `ref7b_p331_ext.py` (own model, no repository code; floating-point PSD rejections: a count check only).

line	B0	B1	B2	programs
0. admissible matchings / orbits (Computation 7.3)	25514 / 425	13799 / 1259	9229 / 528	4 programs (section 7.1)
1. pairs (tau, F) with no budget violation on the 25 vertices S u {p} u A_p	3510	7224	2406	I1, I2, REF1
2. 39-vertex survivors (tau, F, H, A_{z_0})	383	481	7	I1, I2 (identical sets, see below), REF1 (same counts); I3 for B2 (identical graphs)
2'. matchings / pairs (tau, F) carrying them	50 / 95	55 / 76	1 / 1	I1, I2, REF1
3a. tables with no budget violation on Y (45 vertices)	8639	27203	0	I1, I2, REF1; I3 for B2
3b. of which no PSD violation on Y	23	164	0	I1, I2 (identical sets), REF1
4a. graphs on the 51 vertices with no budget violation, over the tables of 3b	52	1839	-	I2, REF1
4b. of which no PSD violation	**0**	**0**	-	I1, I2 (twice: `v3_stage34.py` and `v7_step4_direct.py`), REF1

So every line except 4a has two exact implementations (I1 and I2) that share no code, plus the referee's count check; for B2, where the enumeration stops at line 3a, there are three exact implementations. Line 4a is an intermediate count, not needed for the result (line 4b); it has one exact implementation (I2) and the referee's.

Notes on the table.

- Lines 2 and 3b, "identical sets": I1 and I2 do not use the same representative in each orbit of matchings, so every survivor is brought to a canonical form under Aut(B) and the multisets of canonical forms are compared (`lemma/t18_VERIF331/v4_compare.out`): identical for the 39-vertex survivors of the three graphs (383 configurations in 232 canonical forms for B0, 481 in 383 for B1, 7 in 7 for B2) and for the tables of line 3b (23 in 15 forms for B0, 164 in 116 for B1). For B2, I3 and I1 give the same 7 labelled graphs up to the exchange of z_0 and sigma z_0, all in matching #385 (`lemma/t18_B2/e331_compare_2.out`).
- Line 3b: the 23 tables of B0 and the 164 tables of B1 lie in 9 matchings each.
- Line 4a is not produced by I1, whose step 4 prunes during the search; its own counters are: 242 type solutions and 14520 search nodes for B0 (14165 rejections by a budget violation, 355 by a PSD violation, 0 by exact elimination alone), 2060 and 124992 for B1 (119016, 5918, 0), and no graph left in both cases. A separate brute-force check of I1 (`r2_ext2_bruteforce.py`, 9 tables, 86400 complete assignments per table) finds 12, 12, 2

  (B0) and 39, 39, 39, 0, 0, 0 (B1) graphs without budget violation, none without PSD violation.
- For B1, step 4 is confirmed by a different test: ‘lemma/t18/r4_rank331.py‘ applies the rank bound (Proposition 6.4) to the 47 vertices Y u {w, sigma w}, a determined sigma-invariant set with 23 orbits, for one orbit of W at a time. Over the 164 tables there are 295200 candidates (type, K_w), 16334 without budget violation; G_odd is singular for 2 of them only, and none satisfies both rank conditions. So every one of the 164 tables dies as soon as one orbit of W is added, by the rank bound alone.

7.6 Remarks and corrections to the working reports

Both matrices are needed. The working report states that "the violated matrix is 9(3I - A) + J". This is false in general. I2 and REF1 ran the whole enumeration with M1 alone:

with M1 alone	B0	B1	B2
39-vertex survivors	887	672	7
tables with no violation on 45 vertices	102	190	0
graphs with no violation on 51 vertices	0	10	0

(39 and 51 vertices: I2 and REF1; 45 vertices: I2.) So with M1 alone ten configurations of B1 would survive on 51 vertices. They all lie in one orbit of matchings, have t = 3 and no edge of H, satisfy every budget, and M1 is exactly PSD on them; each is rejected by an explicit integer vector x with x^T M2 x = -88, -594 or -726 (‘lemma/t18_VERIF331/v6_m1_final.out‘). In the direct step 4 of I2, 1829 of the 1839 graphs of line 4a are rejected by M1 and exactly these 10 by M2 only. At 39 vertices, 504 configurations of B0 and 191 of B1 are rejected by M2 only. I1 tests both matrices (this is visible in the source of ‘r2_hfirst.py‘, ‘r2_stage2.py‘, ‘r2_ext2.py‘), which is why its counts agree with the two-matrix runs.

Line 1 (referee’s remark 6; corrected after the second referee’s remark 7). The working report describes the numbers 3510, 7224, 2406 as the pairs (tau, F) "passing the N(p) tests ... [such that] the 3 pairs of N(p) n X_1 exist". They are in fact the numbers of pairs with no budget violation on S u {p} u A_p, as defined in line 1: the existence test coded in ‘r2_hfirst.py‘ only asks for three types satisfying (F5), and it removes nothing (every one of the 3510, 7224, 2406 pairs admits such types). A complete existence test asks in addition, for each type (i ; j) of an orbit {w, sigma w} of W, the two following conditions:

- (cell room) the number of deficient pairs U with i in U and j in tau U is at most 1 - [i ~_B j], and the same with i and j exchanged. This is condition (C6) of section 8.2: w and these vertices s_U are distinct common neighbours of x_i and y_j;
- (no vertex of A_p in the cell) no vertex s_U of A_p has i in U and j in tau U, or j in U and i in tau U. Such a vertex and w (respectively sigma w) would be two non-adjacent vertices of N(p) with the three common neighbours p, x_i, y_j (respectively p, x_j, y_i).

The numbers of pairs (tau, F) that admit three types are then:

existence test	B0	B1	B2
(F5) only (the test of ‘r2_hfirst.py‘)	3510	7224	2406
(F5) and cell room	1958	4332	1309
(F5), cell room, and no vertex of A_p in the cell	1834	4096	1265

(Second referee: ‘lemma/t18_REFEREE2/recount.py‘, ‘recount2.py‘.) The last row is what the referee counted (‘ref2_stage1_B*.out‘). The first version of this text said that the cell-room condition alone gives 1834, 4096, 1265; this was imprecise: cell room alone gives 1958, 4332, 1309, and the second condition is needed to reach the referee’s numbers. This changes nothing downstream: the two omitted conditions are valid necessary conditions and the extra pairs die later.

Counters that are not comparable between programs. The working report gives "completions C": 23155 (B0), 93684 (B1), 44 (B2). I2 gives 11797, 45362, 88, and so does REF1 ("covers"). These are two different objects: I1 counts the choices of rows before H is fixed, I2 and REF1 count, for each 39-vertex survivor, the triples of rows with the right numbers (F3). Neither is used in the proof; lines 3a and 3b, which are well defined, agree. The same holds for the counters "after the pair tests" (624, 750, 114) and "(H, A_{z_0}) tested" (3810, 4416, 121) of the working report, which depend on the order of the tests in ‘r2_hfirst.py‘; the number of complete H on the 37 vertices S u {p} u X_2 (1618, 927, 16) is found by both I1 and REF1.

Profile (3,3,1) does not need X_0’ for B0 and B2: in I2, adding W directly to the 39 vertices leaves 0, 2, 0 configurations for B0, B1, B2 (‘v5_summary.out‘, "dual order"). This is an observation, not used.

8. Profile (2,5,0)

In this section the profile is (h_2, h_1, h_0) = (2, 5, 0). Notation:

```
    A_p  = N(p) n X_2      (4 vertices, 2 orbits);
    W    = N(p) n X_1      (10 vertices, 5 orbits);          N(p) n X_0 is empty, so N(p) = A_p u W;
    X_2' = X_2 \ A_p       (14 vertices, 7 orbits);
    X_0 \ {p}              (8 vertices, 4 orbits);
    f_i  = the number of vertices of A_p adjacent to x_i;
    t    = the number of edges inside X_2 (as in Lemma 5.4).
```

tau, the vertices s_U, the rectangles and the representatives are those of section 7.1.

8.1 The forced structure [A14]

Lemma 8.1.

(a) Inside N(p) = A_p u W the only edges are the 7 mate edges: there is no edge between the two orbits of A_p, none between A_p and W, and none between two different orbits of W.

(b) A vertex w of W has exactly one neighbour x_i in S+ and one neighbour y_j in S-, and i != j; w is of type
    (i ; j) and sigma w of type (j ; i). The five orbits of W have five different unordered types {i, j}.
    The set K_w = N(w) n X_2 is contained in X_2', has a_2(w) >= 2 elements, contains no mate pair, and
    K_{sigma w} = sigma K_w. w has exactly a_2(w) - 1 neighbours in X_0, one of which is p.

(c) sum over A_p of (a_2 - 1) + sum over W of (a_2 - 2) = 8, and all the terms are >= 0.

(d) Every s in X_2' satisfies |N(s) n A_p| + |N(s) n W| = 2.

(e) For every i, x_i has exactly f_i neighbours in A_p and 2 - f_i in W; in particular f_i <= 2.

(f) p has no neighbour in X_0. A vertex z of X_0 \ {p} has no neighbour in S, is not adjacent to p nor to sigma z,
    and satisfies |N(z) n A_p| + |N(z) n W| = 2.

Proof. (a) Lemma 2.1 (b). (b) d(w) = 1 gives x_i and y_j. If i = j, w would be a common neighbour of x_i and its
mate, with x_i outside N[p], whereas by Lemma 2.1 (d) such common neighbours are outside N[p]. If w and w' lie in
different orbits of W and have the same unordered type, replace w' by its mate so that both are of type (i ; j):
then w and w' are distinct, non-adjacent by (a), with the three common neighbours p, x_i, y_j. K_w avoids A_p by (a).
a_2(w) >= 2 is Lemma 5.6. If K_w contained s and sigma s, then w would be a common neighbour of s and sigma s with s
outside N[p], contradicting Lemma 2.1 (d) again. K_{sigma w} = sigma K_w because sigma is an automorphism. The last
statement is Lemma 5.4 (a), a_0 = a_2 - 1.
(c) By Lemma 5.6 the sum of a_2 over N(p) is 36 - 2 * 2 = 32, and 32 - 4 * 1 - 10 * 2 = 8; the lower bounds of
Lemma 5.6 give the signs. (d) and (e): s and x_i are outside N[p]; Corollary 2.2. (f) N(p) contains no vertex of
X_0. z is in X_0: no neighbour in S. z is not in N(p), so z not~ sigma z by Lemma 2.1 (a); Corollary 2.2. QED

8.2 The determined sets of 37 and 47 vertices [C-a to C-d]

    Y37 = S u {p} u X_2   (18 + 1 + 18 = 37 vertices; 18 orbits and p);
    Y47 = Y37 u W   (47 vertices; 9 + 9 + 5 = 23 orbits and p).

Both are sigma-invariant. The data:

- **tau** (a representative, by (7.1));
- **F**, the set of the two rectangles of tau whose orbits form A_p; each must be usable in N(p);
- **H**, the set of all edges inside X_2 (here H includes the two mate edges of A_p); it is invariant under sigma.
  The four pairs between the two orbits of A_p are not parameters: they are non-adjacent by Lemma 8.1 (a) (an edge
  {a, a'} between the two orbits would give the adjacent vertices p and a the two common neighbours sigma a and a').
  So the free pairs of H are those with at least one vertex in X_2';
- **the types**: five unordered pairs {i_c, j_c}, c = 1..5; w_c denotes the vertex of type (i_c ; j_c) and sigma w_c
  the vertex of type (j_c ; i_c);
- **K_c** = N(w_c) n X_2, a subset of X_2', for c = 1..5.

Lemma 8.2. Y37 is determined by (tau, F, H) and Y47 by (tau, F, H, types, K_1, ..., K_5). More precisely:

pair of classes	adjacency	reason
S+ with S+, S- with S-; S+ with S-; S with p; S with X_2	as in Lemma 7.5	Corollary 4.8, definition of B, Lemma 7.1
p with X_2	exactly the 4 vertices of A_p	parameter F
A_p with A_p	the 2 mate edges only: no edge between the two orbits of A_p	Lemma 8.1 (a)
A_p with X_2', X_2' with X_2'	parameter H; no mate edge inside X_2'	Lemma 2.1 (a)
W with p	adjacent	definition
W with W	the 5 mate edges only	Lemma 8.1 (a)
W with A_p	none	Lemma 8.1 (a)
W with S	w_c ~ x_{i_c}, y_{j_c} only; sigma w_c ~ x_{j_c}, y_{i_c} only	parameter types (Lemma 8.1 (b))
W with X_2'	w_c ~ s iff s in K_c; sigma w_c ~ s iff s in sigma K_c	parameter K_c (Lemma 8.1 (b))

The same list shows that the following subsets are determined by the corresponding part of the data; they are the
sets on which the programs test partial data:

- S u {p} u A_p (23 vertices), by (tau, F);
- S u N[p] = S u {p} u A_p u W (33 vertices), by (tau, F, types);
- S u {p} u X_2 u (some orbits of W) (39, 41, ... vertices), by (tau, F, H) and the types and sets K_c of these orbits;
- S u {p} u A_p u (some orbits of X_2') u (one orbit {w, sigma w} of W) (27 or 29 vertices for one or two orbits of
  X_2'), by (tau, F), the edges of H inside the set, the type of w and the intersection of K_w with the set. In
  particular the hypothesis "K_w contains neither s nor sigma s" determines the 27-vertex set of the orbit of s
  with no edge between {w, sigma w} and {s, sigma s}; if that graph has a violation, then K_w must contain s or
  sigma s (this is the "forced orbit" test of the first program).

Lemma 8.3 (forced numbers). The data of a configuration satisfy:

- (C1) both rectangles of F are usable in N(p);
- (C2) H contains the mate edge {s, sigma s} if and only if s is in A_p, H contains no pair between the two orbits
  of A_p, and every vertex of X_2 has at most 4 neighbours in X_2;
- (C3) FL := sum over A_p of (a_2 - 1) is the number of edges between A_p and X_2'; it is even and FL <= 8;
- (C4) t <= 9 and sum over X_2 of a_2 (a_2 - 1) + m(t) <= 36, where m(t) is twice the minimum of
  y_1^2 + y_2^2 + y_3^2 + y_4^2 over the non-negative integers with y_1 + y_2 + y_3 + y_4 = t
  (m = 0, 2, 4, 6, 8, 14, 20, 26, 32, 42 for t = 0, ..., 9);
- (C5) the five unordered types are distinct, i != j in each, and every index i occurs in exactly 2 - f_i of them;
- (C6) ("cell room") for every c, the number of deficient pairs U with i_c in U and j_c in tau U is at most
  1 - [i_c ~_B j_c], and the same with i_c and j_c exchanged;
- (C7) K_c is a subset of X_2' without mate pair, 2 <= |K_c| <= 2 + slack with slack = 4 - FL/2, and
  sum over c of (|K_c| - 2) = slack;
- (C8) every s in X_2' belongs to exactly req(s) = 2 - |N(s) n A_p| of the ten sets K_1, sigma K_1, ..., K_5,
  sigma K_5. In particular req(s) >= 0, and req(sigma s) = req(s).

Proof. (C1) Lemma 7.2 (R3). (C2) Lemma 2.1 (a), Lemma 8.1 (a) and Lemma 5.4 (a). (C3) A vertex of A_p has a_2 - 1 neighbours in X_2 other than its mate, and they are in X_2' by Lemma 8.1 (a). a_2 takes the same value on mates, so FL is even.
FL <= 8 by Lemma 8.1 (c). (C4) t <= 9 is Lemma 5.4 (d). In identity (*), the vertex p has a_2(p) = 4 and contributes 0 to the first sum; the other 8 vertices of X_0 form 4 orbits, a_2 - 4 is a non-negative integer y_q constant on each orbit, and 2 (y_1 + y_2 + y_3 + y_4) = sum_{X_0} (a_2 - 4) = 2t by Lemma 5.4 (c); so the first sum is 2 (y_1^2 + ... + y_4^2) >= m(t); the second sum is >= 0. (C5) Lemma 8.1 (b), (e): x_i has exactly one neighbour in each orbit of W whose unordered type contains i. (C6) w_c and the vertices s_U with i_c in U and j_c in tau U are distinct common neighbours of x_{i_c} and y_{j_c}, which have 2 - [i_c ~_B j_c] common neighbours; the same for sigma w_c with x_{j_c} and y_{i_c}. (C7) Lemma 8.1 (b); by Lemma 8.1 (c), sum over W of (a_2 - 2) = 8 - FL, and a_2(w_c) = a_2(sigma w_c) = |K_c|, so 2 sum_c (|K_c| - 2) = 8 - FL. (C8) Lemma 8.1 (d): s is adjacent to w_c iff s is in K_c, and to sigma w_c iff s is in sigma K_c. QED

8.3 Stage B: the 37 vertices

> **Computation 8.4 (stage B).** For each B in {B0, B1, B2} and each representative tau, list all (F, H) where F is
> a set of two rectangles of tau usable in N(p) and H is a sigma-invariant set of edges on X_2 satisfying (C2) (so
> H does not range over the four pairs between the two orbits of A_p: they are absent), such that the graph on Y37
> has no budget violation and M1[Y37] and M2[Y37] are both PSD. The triples (tau, F, H) obtained are the
> **stage-B triples**:
>
> | | B0 | B1 | B2 |
> |---|---|---|---|
> | pairs (tau, F) with no budget violation on the 23 vertices S u {p} u A_p | 5545 | 12366 | 4347 |
> | of which M1 and M2 are both PSD on these 23 vertices | 4904 | 11509 | 3863 |
> | **stage-B triples (tau, F, H)** | **5595** | **4977** | **196** |
> | matchings / pairs (tau, F) carrying them | 133 / 901 | 275 / 1015 | 53 / 74 |
> | triples with t = 2 / 4 / 6 | 459 / 2584 / 2552 | 386 / 2351 / 2240 | 6 / 80 / 110 |
> | triples with largest a_2 on X_2 equal to 1 / 2 | 5232 / 363 | 3791 / 1186 | 141 / 55 |

Programs:

- **ENUM** ("the enumerator") = `lemma/t18/r3_p250.py`, stage B: the orbits of X_2' are added one at a time, with the states of their edges to the earlier orbits; budgets on partial graphs, PSD when an orbit is complete; it uses (C2), (C3) during the search and applies (C4) to the complete H. It reads `types.json`.
- **VERIF** ("the verifier") = `lemma/t18/r3_verify.py B250`: no shared code, graphs re-entered as edge lists, orbits added in the reverse order, budgets by integer matrix products; it uses (C2), (C3) and not (C4).
- **REF1** = `lemma/t18_REFEREE/ref4_stageB.py` (own model; floating-point PSD).

ENUM and VERIF produce **identical sets of triples** for the three graphs (`lemma/t18/r3_finalB.out` for B1, `lemma/t18_B0/b0_compareB.out` for B0, `lemma/t18_B2/b2_compareB.out` for B2). REF1 finds the same numbers of triples, of pairs (tau, F), of matchings, and the same distributions by t and by largest a_2. So the list of stage-B triples has two exact implementations and one count check.

Notes. The first row is given by ENUM and REF1 for B0 and B2, and by REF1 alone for B1 (for B1 the runs of ENUM overlap and its counter cannot be summed). The second row is given by VERIF and REF1 for the three graphs and by ENUM for B0 and B2. ENUM and VERIF also discard a pair (tau, F) when some f_i exceeds 2 or when no set of five types satisfies (C5) (Lemma 8.3; ENUM takes the types from `types.json`, VERIF from its own 21-vertex test); REF1 does not. This removes nothing: in every output of ENUM and of VERIF the counter is the same before and after this test (for instance 5545 = 5545 for B0 and 4347 = 4347 for B2 in ENUM), and REF1 obtains the same triples counts without it. Condition (C4) removes nothing: in every run the number of complete H is the same before and after it (ENUM: 5595 = 5595 and 196 = 196; comparison files: "4977 after the counting conditions"; REF1: (C4) true on all triples). The last two rows are data, not proved facts: on all stage-B triples t is 2, 4 or 6 and a_2 <= 2 on X_2. (Hand proofs that t is even and different from 8 exist in the working report `lemma/t18_STRUCT`; they are not needed.)

For B1 the stage-B list of ENUM was produced in five launches, some of them cut by a time limit; the bookkeeping script `lemma/t18_B0_VERIFC/b1_coverage.py` checks that each of the 1259 matchings is completed by exactly one of the 14 files used and that the 4977 triples so obtained are the set of VERIF (`04_b1_coverage.out`).

(Other outputs quote other numbers of files read, 17 and 15; see the notes at the end of appendix A.5.)

That the kept lists of ENUM and VERIF are the list of stage-B triples of the definition, i.e. that every one of the 5595 + 4977 + 196 triples is exactly PSD for both matrices on Y37, was verified by the second referee (`lemma/t18_REFEREE2/exactB.py`; section 6.3).

The pairs between the two orbits of A_p (second referee's remark 4). In the first version of this text the row "X_2 with X_2" of Lemma 8.2 made every pair of X_2 a parameter of H, and (C2) did not record that the four pairs between the two orbits of A_p are non-adjacent (Lemma 8.1 (a)). The list of stage-B triples does not depend on this: a set H containing such a pair {a, a'} has a budget violation on Y37, because the adjacent vertices p and a then have the two common neighbours sigma a and a'. In the programs these pairs are never chosen. This is so in `stageB.py` of the second referee, whose list of triples is the list of VERIF (identical multisets of canonical forms under Aut(B) for the three graphs: `lemma/t18_REFEREE2/orbitsB_<s>.out`); and, as read in their sources for this revision, in ENUM and VERIF, which only choose the edges between an orbit of X_2' and the other orbits.

8.4 Stage C: the 47 vertices

> **Computation 8.5 (stage C).** For every stage-B triple (tau, F, H), list all (types, K_1, ..., K_5) satisfying
> (C5), (C7), (C8) such that the graph on Y47 has no budget violation and M1[Y47] and M2[Y47] are both PSD.
> The (tau, F, H, types, K) obtained are the **stage-C survivors**.
>
> - For **B0**: there is no stage-C survivor.
> - For **B2**: there is no stage-C survivor.
> - For **B1**: there are exactly 4 stage-C survivors: matching #813 with F = [2, 9], matching #813 with
>   F = [3, 6], matching #975 with F = [1, 2], matching #975 with F = [3, 6]. (F is written with the indices, in the

> list of deficient pairs of B1 given after Corollary 4.11, of one deficient pair of each of its two rectangles.
> The four graphs are given explicitly in appendix B.)

(C6) is not listed among the conditions because it is implied by the absence of budget violation on Y47; the programs use it, or an equivalent budget test on a partial graph, as a filter.

As in section 6.3, what the programs establish by exact rejections is that their kept lists of stage C are empty for B0 and B2 and consist of the same four graphs for B1. That these four graphs are survivors in the sense of the definition, i.e. exactly PSD on Y47, is verified separately (section 8.5); and the stage-B triples over which stage C is run have been verified to be exactly PSD on Y37 (section 6.3).

Programs:

- **ENUM** = ‘lemma/t18/r3_p250.py‘, stage C: the orbits of W are added one at a time, each with its set K_c; tests on the 27-, 29- and 33-vertex subsets of section 8.2 and on S u {p} u X_2 u (the orbits of W already placed).
- **CMINE** = ‘c_mine.py‘ (written in ‘lemma/t18_B2‘; the copy used for B0 and B1 is in ‘lemma/t18_B0_VERIFC‘): for every type, all sets K with no violation on the 39 vertices S u {p} u X_2 u {w, sigma w}; then an exact cover of the numbers req(s) by one K per orbit of W, with a test on 41 vertices for every two orbits; then the test on 47 vertices. It shares no code with ENUM or VERIF, does not read ‘types.json‘, and takes the stage-B triples as input. A PSD rejection is an integer vector evaluated in Python integers, or else an exact fraction-free elimination.
- **VERIF** = ‘lemma/t18/r3_verify.py C250‘: "dual" order (one orbit of X_2’ at a time: which vertices of W it sees). Complete for B2, partial for B0 and B1. Two further partial programs for B1: ‘lemma/t18/r4_verifyC.py‘ and ‘lemma/t18/r4_verifyC2.py‘.
- **REF1** = ‘lemma/t18_REFEREE/ref5_stageC.py‘, on the referee’s own stage-B triples (floating-point PSD).

graph	program	scope	stage-C survivors	where
B0	ENUM (as ‘b0_stageC.py‘, ‘b0_stageC2.py‘)	2667 representatives of the 5595 triples modulo the stabilisers of the matchings	0	‘lemma/t18_B0/b0_stageC_done_*.jsonl‘ and ‘b0_stageC_survivors.json‘ (checked by the second referee: 2667 lines, every index 0..2666 exactly once, no survivor); ‘lemma/t18_B0/REPORT.md‘, section "FINAL STATE", is only a summary generated from these logs
B0	CMINE	all 5595 triples, each exactly once (24 parts)	0	‘lemma/t18_B0_VERIFC/09_B0_check_complete.out‘
B0	REF1	all 5595 triples	0	‘lemma/t18_REFEREE/ref9_totals.out‘
B0	VERIF	29 triples: the run of part 0 of 150, cut by its time limit after 29 triples (not a designed sample)	0	‘lemma/t18_B0/b0_verC_sample.out‘
B1	ENUM	all 1259 matchings (4977 triples)	4	‘lemma/t18/r3_finalB.out‘, ‘r3_round5.out‘
B1	CMINE	all 4977 triples, each exactly once (24 parts)	4, the same 47-vertex graphs as ENUM	‘lemma/t18_B0_VERIFC/11_B1_check_complete.out‘, ‘13_B1_survivors_compare.out‘
B1	REF1	all 4977 triples	4, same matchings and F	‘lemma/t18_REFEREE/ref9_totals.out‘
B1	VERIF, ‘r4_verifyC.py‘, ‘r4_verifyC2.py‘	2086 of the 4977 triples, including the 170 triples of matchings #813 and #975	the same 4, no other	‘lemma/t18/r4_compareC_final.out‘
B2	ENUM	all 528 matchings (196 triples)	0	‘lemma/t18_B2/b2_BC_0of2.out‘, ‘b2_BC_1of2.out‘
B2	VERIF	all 196 triples, in four runs: the first was cut by its time limit after 86 of the 98 triples of its part, and the four runs together treat 86 + 43 + 43 + 24 = 196 triples	0	‘lemma/t18_B2/b2v_C250_*.out‘
B2	CMINE	all 196 triples	0	‘lemma/t18_B2/c_mine_2_all.out‘
B2	REF1	all 196 triples	0	‘lemma/t18_REFEREE/ref5_stageC_B2.out‘

So the stage-C result has two complete exact implementations sharing no code for B1 and for B2 (ENUM and CMINE; for B2 also VERIF), plus the referee’s complete count check (REF1). For B0 the first version of this text also claimed "two complete exact implementations"; this was too strong (second referee’s remark 6). CMINE treats all 5595 triples of B0, but ENUM treats 2667 representatives: it is complete only modulo the reduction to orbits under the stabilisers, which exists in one program of the repository (see the note below). This is harmless: besides CMINE and REF1, the stage C of the second referee, which rejects exactly, treats all 5595 triples of B0 and finds no survivor (section 9.6).

Notes.

- *Status in the working reports.* ‘LEMMA_TARGET_2026-10-06.md‘ (section 8) and ‘lemma/t18/T18_REPORT.md‘ say that B0 is open and that stage C of B1 has one complete implementation, the second covering 2086 triples; ‘lemma/t18_B0/REPORT.md‘ says that stage C of B0 has one implementation. These statements are superseded by the later complete runs of CMINE stored in ‘lemma/t18_B0_VERIFC‘ (run on 2026-10-06/07, after those reports) and by the referee’s runs.
- *The reduction modulo stabilisers (B0, program ENUM only).* An automorphism g of B fixing tau maps a stage-B triple (tau, F, H) to (tau, gF, gH) and maps the completions of the first to the completions of the second (it is a relabelling of the orbits of the support, as in (7.1)). So it is enough to complete one triple in each orbit. In the repository this reduction exists in one implementation (‘b0_orbitsB.py‘, which asserts that the set of triples is closed). The second referee has recounted it on his own list of triples (‘lemma/t18_REFEREE2/orbitsB0.py‘, ‘orbitsB_0.out‘): the 5595 triples are closed under the stabilisers and form 2667 orbits, and the stabiliser orders of the 133 matchings carrying triples are those of ‘b0_orbitsB.out‘ (order 1 for 85 matchings, 2 for 36, 4 for 9, and 6, 8, 72 for one matching each). He did not compare the 2667 stored representatives (‘b0_stageB_reps.json‘) themselves. The reduction is not needed for the result, since CMINE, REF1 and the stage C of the second referee treat all 5595 triples.
- *Inputs of CMINE.* For B0, CMINE reads the stage-B file of ENUM, which is the set of VERIF (‘08_b0_input_crosscheck.out‘). For B1 it reads the list built by ‘b1_coverage.py‘, equal to the set of VERIF; ‘12_B1_check_complete_verifier_list.out‘ checks the completeness of the run against the list of VERIF directly. A control run of CMINE on the 170 triples of matchings #813 and #975, read from the files of VERIF, gives the same four graphs (‘05_control_B1_check.out‘, ‘06_control_B1_compare.out‘).
- *Work done* (program-specific counters, not comparable between programs). ENUM on the 2667 representatives of B0: 262850 (triple, type solution) pairs examined, 170382 of them dead by the counting and forced-orbit tests, 701270 search nodes, 340605 rejections by a budget violation and 704789 by a PSD violation. CMINE on B0: 132132 types tested, 603053 sets K without budget violation on 39 vertices, 52580 without PSD violation, 3165 cover nodes, no cover reaching the 47-vertex test, 611642 PSD certificates and no use of the exact fallback. CMINE on B1: 115473 types, 905578 and 75295 sets K, 9248 cover nodes, 43 tests on 47 vertices, 4 survivors, 959041 certificates, no fallback. CMINE on B2: 4222 types, 39107 and 1535 sets K, 173 cover nodes, no 47-vertex test, 40845 certificates.
- *"Type solutions" (referee’s remark 6).* The B2 report gives "11235 type solutions over the 74 pairs (tau, F), of which 4227 pass on the 33 vertices S u N[p]", found by ENUM and by CMINE. These numbers count the sets of five types

satisfying (C5) only, i.e. **without the cell-room condition (C6)**; with (C6) the referee counts 770 type solutions, 514 of which pass on the 33 vertices (‘ref5b_types_variants_B2.out‘). Both programs apply (C6) later, as a budget test, so nothing changes downstream.

8.5 The four survivors of B1

For the four stage-C survivors of B1 (appendix B), the following is established exactly by programs that share no code with the enumerations (‘lemma/t18/r3_survivors.py‘, by rational elimination, for both matrices; ‘lemma/t18/r4_verifyD.py‘, for M1; ‘lemma/t18_REFEREE/ref8_survivors.py‘, by exact characteristic polynomials, for both matrices; and, since the second referee’s report, ‘lemma/t18_REFEREE2/survivors.py‘, which rebuilds the four graphs from appendix B alone, by exact characteristic polynomials, for both matrices):

survivor	t	a_2 on A_p	orbits of W with a_2 = 3 / with a_2 = 2	M1 on the 47 vertices	M2 on the 47 vertices
#813, F = [2, 9]	4	1, 2, 2, 1	3 / 2	PSD, rank 46	PSD, rank 45
#813, F = [3, 6]	4	1, 2, 2, 1	3 / 2	PSD, rank 46	PSD, rank 45
#975, F = [1, 2]	2	1, 1, 1, 1	4 / 1	PSD, rank 46	PSD, rank 45
#975, F = [3, 6]	2	1, 1, 1, 1	4 / 1	PSD, rank 46	PSD, rank 45

They satisfy every budget, they are sigma-invariant, and both matrices are exactly PSD. So budgets and semidefiniteness cannot exclude them on 47 vertices. They are excluded in two independent ways (sections 8.6 and 8.7); each way alone suffices.

The working report of round 3 presents these four configurations as "an open case" (PSD, rank 46 of 47). That wording is superseded: rank 46 is itself impossible, by the rank bound.

8.6 First exclusion: the rank bound

Y47 is a sigma-invariant determined set (Lemma 8.2) with 23 orbits and the fixed vertex p. Take for R the 9 vertices x_i, one vertex of each of the 9 orbits of X_2 and the 5 vertices w_c. By Proposition 6.4 (a), in a configuration the 23 x 23 integer matrix G_odd = 3I - A[R,R] + A[R, sigma R] is singular, and by Proposition 6.4 (b) the 24 x 24 matrix G_even has rank at most 22.

> **Computation 8.6.** For the four stage-C survivors of B1:
>
> | survivor | det G_odd (23 x 23) | rank G_odd | rank G_even (24 x 24) |
> |---|---|---|---|
> | #813, F = [2, 9] | 47684 | 23 | 23 |
> | #813, F = [3, 6] | 47684 | 23 | 23 |
> | #975, F = [1, 2] | 874944 | 23 | 23 |
> | #975, F = [3, 6] | 874944 | 23 | 23 |

det G_odd != 0 contradicts Proposition 6.4 (a); rank G_even = 23 > 22 contradicts Proposition 6.4 (b) independently. So none of the four graphs is the induced subgraph on Y47 of a configuration.

Programs (five codes written separately, all exact): ‘lemma/t18/r4_rank.py‘ (fraction-free Bareiss determinant, ranks by rational elimination), ‘lemma/t18_STRUCT/rank_survivors.py‘ (ranks) and ‘rank_survivors_check2.py‘ (Bareiss determinant; rank modulo two primes), ‘lemma/t18_B2/rank_B1_survivors.py‘ with ‘rankbound.py‘ (determinant and ranks, on the survivors as found by CMINE), ‘lemma/t18_B0/b0_rank47.py‘ (ranks), ‘lemma/t18_REFEREE/ref8_survivors.py‘ (determinant and ranks, on the survivors as found by REF1). All give the values of the table. The smallest eigenvalue of G_odd is about 0.011 (#813) and 0.012 (#975) in floating point: the matrix is positive definite, which is why a PSD test cannot see the obstruction.

8.7 Second exclusion: the vertices of X_0 other than p (stage D) [A18, A19]

X_0 has 9 vertices (Theorem 5.3), so X_0 \ {p} has 8 vertices: at least one vertex z of X_0 \ {p} exists.

Lemma 8.7. Let z be a vertex of X_0 \ {p} and Z = N(z) n (X_2 u W). Then:

- (D1) z has no neighbour in S u {p} and is not adjacent to sigma z; N(sigma z) n (X_2 u W) = sigma Z. Hence Y47 u {z} (48 vertices) and Y47 u {z, sigma z} (49 vertices) are determined by the data of Y47 together with Z;
- (D2) |Z n (A_p u W)| = 2;
- (D3) 4 <= |Z n X_2| <= 4 + t;
- (D4) every s in X_2 has exactly need(s) = a_2(s) + 2 - [s in A_p] neighbours in X_0 \ {p}, and every w in W has exactly need(w) = a_2(w) - 2. Hence, for every vertex v of X_2 u W, [v in Z] + [v in sigma Z] <= need(v); in particular Z contains no vertex with need = 0.

Proof. (D1) and (D2) are Lemma 8.1 (f). (D3): |Z n X_2| = a_2(z) >= 4 by Lemma 5.4 (a), and z has a_2(z) - 4 neighbours in X_0; so has sigma z. By Lemma 5.4 (c) the sum over X_0 of the numbers of neighbours in X_0 is 2t; z and sigma z are two distinct vertices of X_0, so 2 (a_2(z) - 4) <= 2t. (D4): by Lemma 5.4 (a), s in X_2 has a_2(s) + 2 neighbours in X_0, and p is one of them exactly when s is in A_p; w in W has a_2(w) - 1 neighbours in X_0, one of which is p (Lemma 8.1 (b)). v is adjacent to z iff v is in Z and to sigma z iff v is in sigma Z, and these are neighbours of v in X_0 \ {p}. QED

The upper bound in (D3) is not in the list of forced numbers of the working report, but the three programs of the repository apply it (referee’s remark 6); this is why they report 846 below where the referee, without it, counts 849.

> **Computation 8.8 (stage D).** For each of the four stage-C survivors of B1, list all subsets Z of X_2 u W
> satisfying (D2), (D3), (D4) such that the graph on the 49 vertices Y47 u {z, sigma z} has no budget violation.
> The numbers of unordered pairs {Z, sigma Z} are
>
> #813, F = [2, 9]: 678; #813, F = [3, 6]: 678; #975, F = [1, 2]: 846; #975, F = [3, 6]: 846.
>
> For every one of them a PSD violation is found on the 49 vertices (an integer vector x with x^T M x < 0;

> 3048 = 2 (678 + 846) certificates), and according to the second program already on the 48 vertices Y47 u {z}.

By (6.1) and Lemma 8.7, the set Z of the vertex z of a configuration would be one of the listed sets and would have no PSD violation. So none of the four graphs is the induced subgraph on Y47 of a configuration.

Programs (three exact, plus the referee):

- ‘lemma/t18/r4_stageD.py‘: 678, 678, 846, 846 candidates, 0 without PSD violation; its 3048 certificates (‘r4cert_stageD_B1.jsonl‘) were re-checked by ‘r3_verify.py certs‘: 3048 checked, 0 invalid, all on 49 vertices;
- ‘lemma/t18/r4_verifyD.py‘ (no shared code, own generator): 1356, 1356, 1692, 1692 oriented sets Z with an admissible mate (twice the numbers above), a PSD violation found for each on the 48 vertices, budgets satisfied on all the 49-vertex graphs, no ambiguous floating-point case;
- ‘lemma/t18_B2/d_stage.py‘ (no shared code): 678, 678, 846, 846 candidates, 0 without PSD violation, 3048 certificates, re-checked (‘lemma/t18_B2/b2_certs_check.out‘);
- ‘lemma/t18_REFEREE/ref8_survivors.py‘: 678 and 678; 849 and 849 without the upper bound of (D3), of which 846 satisfy it; none passes the (floating-point) PSD test.

On "55 vertices" (second referee’s remark 9). The outputs of ‘r4_stageD.py‘ and of ‘d_stage.py‘ contain the words "0 configurations on 55 vertices left". No set of 55 vertices is used, or shown to be determined, in this text: (D1) covers only the 48 vertices Y47 u {z} and the 49 vertices Y47 u {z, sigma z}, and Computation 8.8 stops there. For the 55 vertices Y47 u (X_0 \ {p}), the adjacencies between vertices of two different orbits of X_0 \ {p} are neither forced by a lemma of this text nor listed as a parameter. The printed words only mean that, no set Z being left for one orbit {z, sigma z}, nothing is left that could be extended to the four orbits of X_0 \ {p}.

Remark (not used). On the four survivors the kernel of M1[Y47] is spanned by the vector 4 e_p + 1_{N(p)}, which is in the kernel of M1 for the whole graph; so the condition "the row of a new vertex is orthogonal to the kernel" mentioned in the working report reduces to Corollary 2.2 and gives nothing.

8.8 Conclusion for profile (2,5,0)

Proposition 8.9. Profile (2,5,0) is impossible for B0, B1 and B2.

Proof. Suppose a configuration has profile (2,5,0) and orbit graph B. By (7.1) take tau among the representatives. By Lemmas 8.2 and 8.3 its (tau, F, H) is a stage-B triple (Computation 8.4) and its (tau, F, H, types, K) is a stage-C survivor (Computation 8.5): this is (6.1). For B0 and B2 there is no stage-C survivor: contradiction. For B1 the graph induced on Y47 is one of four graphs, each of which contradicts Proposition 6.4 by Computation 8.6 (and, independently, contradicts Lemma 8.7 by Computation 8.8). QED

Status: hand part (Lemmas 7.1, 7.2, 8.1, 8.2, 8.3, Proposition 6.4, Lemma 8.7) plus Computations 7.3, 8.4, 8.5 and 8.6 (or 8.8).

Summary of section 8:

	B0	B1	B2
matchings / orbits	25514 / 425	13799 / 1259	9229 / 528
stage-B triples on 37 vertices	5595	4977	196
stage-C survivors on 47 vertices	0	4	0
excluded by the rank bound (det G_odd != 0)	-	4 of 4	-
excluded by stage D (no vertex of X_0 \ {p} can be added)	-	4 of 4	-

9. Conclusion and status

9.1 Conclusion

Theorem 9.1 (= T18). There is no configuration: for G and sigma as in section 1, no integer-valued sigma-odd function phi satisfies A phi = -4 phi and sum phi^2 = 18.

Proof, given the Computations. Suppose (G, sigma, phi) is a configuration. By Proposition 3.1, phi takes the values 0, +1, -1 and its support is S = S+ u S- with |S+| = 9. By Corollary 4.8, S+ is independent and S avoids N[p]; by Lemma 4.9 and Computation 4.10, the orbit graph B is one of B0, B1, B2 after a suitable labelling. By Theorem 5.3 no outside vertex has three neighbours in S+, and by Lemma 5.5 the profile is (2,5,0), (3,3,1) or (4,1,2). Profile (4,1,2) is impossible by Proposition 5.7, profile (3,3,1) by Proposition 7.7, profile (2,5,0) by Proposition 8.9. Contradiction. QED

The theorem therefore rests on the hand statements of sections 2-8 and on six finite computations: 4.10 (the three graphs), 7.3 (stage 1), 7.6 (profile (3,3,1)), 8.4 and 8.5 (stages B and C of profile (2,5,0)), and 8.6 (the rank of the four survivors of B1; Computation 8.8 is an independent alternative to 8.6).

9.2 What is proved by hand and what is computed

statement	content	nature	implementations
Lemma 2.1, Corollary 2.2	sigma and N(p)	hand	-
Proposition 3.1	integer values => 0, +1, -1, nine at +1	hand	-
Lemmas 4.2-4.7, Corollary 4.8, Lemma 4.9	k = 0, e = 0, B is 4-regular with codegrees <= 2	hand	-
Computation 4.10	B is B0, B1 or B2; their codegrees are 1 or 2	computed	4 programs (‘enum_S.py‘, ‘verify_S.py‘, ‘ref1_quartic9.py‘, and REF2: ‘q9.py‘), see 9.3 item 5
Lemmas 5.1, 5.2, Theorem 5.3	path count; n_3 = 0	hand	-
Lemmas 5.4, 5.5, 5.6, Proposition 5.7	identities; the three profiles; (4,1,2) impossible	hand	-
Lemmas 6.1, 6.2, 6.3, Proposition 6.4	the two PSD matrices, the budget, the rank bound	hand	-
Lemmas 7.1, 7.2	X_2 and the matching tau; stage-1 tests	hand	-
Computation 7.3	the admissible matchings and their orbits	computed	4 programs, same numbers; representatives checked by 2 of them; REF2: same numbers, stored representatives checked

Lemmas 7.4, 7.5	structure and determined sets of profile (3,3,1)	hand	-
Computation 7.6	profile (3,3,1): nothing survives on 51 vertices (B0, B1), on 45 (B2)	computed	2 exact implementations sharing no code (I1, I2), identical intermediate sets; a third for B2 (I3); referee's count check (REF1); REF2: same counts on every line of section 7.5, exact rejections
Lemmas 8.1, 8.2, 8.3	structure and determined sets of profile (2,5,0)	hand	-
Computation 8.4	stage B: 5595 / 4977 / 196 triples	computed	2 exact implementations (ENUM, VERIF), identical sets; REF1: same counts; REF2: same list as VERIF up to Aut(B), every triple verified exactly PSD on Y37
Computation 8.5	stage C: 0 / 4 / 0 survivors	computed	B1 and B2: 2 complete exact implementations sharing no code (ENUM, CMINE), and a third for B2 (VERIF); B0: CMINE complete, ENUM complete modulo the stabiliser reduction (section 8.4); partial runs of VERIF for B0, B1; REF1: same result; REF2: same result on all triples, exact rejections
Computation 8.6	the four survivors violate the rank bound	computed	5 exact codes written separately; REF2: same values, from appendix B alone
Lemma 8.7	forced numbers of stage D	hand	-
Computation 8.8	no vertex of X_0 \ {p} can be added to the four survivors	computed	3 exact programs sharing no code; REF1; REF2: same numbers, every Z rejected exactly

The hand part was audited by a referee who re-derived every statement from the definitions before reading the given proof (`lemma/t18_REFEREE/REPORT.md`): every load-bearing hand statement was found correct, and the referee's list of defects of the written text is addressed in section 9.5. The referee also tested the general facts (Lemma 2.1, Lemma 6.3, the counting identities, the parity law, the path count) on two real graphs with lambda = 1, mu = 2 and an involution with one fixed point, the Paley graph on 9 vertices and the srg(243,22,1,2), with x -> -x (`ref3_bvls.out`).

A second referee then read this text itself and re-did every Computation with his own programs (REF2 in the table above); what he covered and what he did not check is in section 9.6.

9.3 What has not been independently verified, and other caveats

1. **Floating-point "kept" decisions.** In ENUM (`r3_p250.py`), VERIF (`r3_verify.py`), CMINE (`c_mine.py`), I1, `e331.py`, `d_stage.py`, `r4_stageD.py`, a candidate is kept when the smallest floating-point eigenvalue is above about -10^-7 (-10^-6 in `r3_verify.py`); this is never used as a proof of semidefiniteness. A rejection requires an exact witness: an integer vector with negative quadratic form (64-bit integers in `r3_p250.py` and `r4_stageD.py`, with the bound |x^T M x| < 2 * 10^15 stated in the source of `r3_p250.py`; Python integers elsewhere), or an exact elimination (fallback of `c_mine.py`, `r2_hfirst.py`, `r2_ext2.py`; it produced no rejection in the runs reported), or an exact characteristic polynomial (I2). When the eigenvalue is negative and no witness is found, `r3_p250.py` keeps the candidate and counts it ("psd_float_neg_no_cert"): 0 cases in every run reported; `r3_verify.py` and `r4_verifyD.py` report 0 "ambiguous" cases. The places where a positive statement of semidefiniteness is made are exact; they are listed in the table of section 6.3: the four survivors of B1 (section 8.5), the acceptances of I2 at 39, 45, 51 vertices, and, since the second referee's report, the stage-B triples on Y37 and the pairs (tau, F) on 23 vertices (`exactB.py`, `recount.py`). None of these positive statements is needed for Theorem 9.1: by (6.2) the exclusions use exact rejections only. They are needed only to know that the numbers printed for these lists are numbers of survivors in the sense of the definition of section 6.3. Every other "kept" decision is floating-point.
2. **The referee's runs use floating-point PSD rejections** (threshold -10^-7) with no exact certificate. They are checks of the model and of the counts, not further exact proofs. Exception: the ranks and determinants of the four survivors in `ref8_survivors.py` are exact. (This item is about the first referee; the programs of the second referee reject exactly, see section 9.6.)
3. **The programs were not audited line by line.** The referee states that it did not verify "the repo's programs themselves and their exact certificates (only that they agree with the referee's model at every checkpoint)". For this text the following were read in full: `r2_stage1.py`, `r2_hfirst.py`, `r3_p250.py`, `c_mine.py`, `v1_stage1.py`, `v3_stage34.py`, `vcommon.py`, `rows.py`; in part: `r3_verify.py`, `v2_stage2.py`, `r4_verifyD.py`, `ref4_stageB.py`; for the others only the header. That each program enumerates exactly the list described in the text is supported by the agreement of implementations that share no code, not by a proof. The second referee did not audit the programs of the repository either (section 9.6).
4. **Shared inputs.** Every program except those of I2, REF1 and REF2 reads the matching representatives from `lemma/t18/r2_tau_<s>.json` (file checked by `r3_verify.py stage1`, by `ref10_tau_files.py` and by the second referee's `stage1.py`). CMINE takes the stage-B triples as input (lists shown identical to the set of VERIF). I1 and ENUM read `structures.json` (the three graphs; the other programs re-enter them as edge lists).
5. **Computation 4.10.** In the first version of this text the list of the 16 classes of 4-regular graphs on 9 vertices came from one program (`ref1_quartic9.py`). It is now also the output of the second referee's enumeration (`lemma/t18_REFEREE2/q9.py`, no shared code), with the same result. The three classes with codegrees <= 2 are found by four programs, but `verify_S.py` imports the isomorphism routine of `enum_S.py`. The printed output of `enum_S.py` is quoted in the working report and not saved in a file; its result file `structures.json` is in the repository.
6. **`types.json` and the lemma H16.** I1 and ENUM (`r2_stage1.py`, `r2_hfirst.py`, `r2_stage2.py`, `r2_ext2.py`, `r3_p250.py` and its copies `b0_stageC.py`, `b0_stageC2.py`) read lists of admissible "types" from `lemma/t18/types.json`, produced by `lemma/t18/rows.py`. So does the check program `r2_ext2_bruteforce.py`, and so do four programs that are not used in this text (`level1.py`, `level1_count.py`, `r2_gram.py`, `star.py`); the first version of this text omitted these five from the list of readers (second referee's remark 3). The last filter of `rows.py` (called L in its header) is conditional on H16, an earlier lemma (no configuration of squared norm <= 16) that is **not proved in this document**. The saved output `lemma/t18/rows.out` shows that, for the entries that are read (types with one or two neighbours in S+), the three counters "graph filters", "graph filters + PSD", "graph filters + PSD + L" are equal for B0, B1 and B2 (81 = 81 = 81 and 72 = 72 = 72 for d = 1; 198 = 198 = 198 and 126 = 126 = 126 for d = 2). So neither the PSD filter nor L removed anything there, and the lists that are read are those of the graph filters alone, which are instances of Lemma 6.2 and Lemma 2.1. This no longer rests on `rows.out` alone: the second referee recomputed the lists directly (`lemma/t18_REFEREE2/recount.py`, `recount.out`) and found that, for each of B0, B1, B2, the entries of `types.json` with d = 0, 1, 2 are exactly the types with no budget violation on the 21 vertices S u {p, w, sigma w}: 1, 81 and 198 types (for d = 0, 1, 2) when the orbit {w, sigma w} is outside N(p), and 1, 72 and 126 types when it is inside N(p). Neither the PSD filter nor the H16 filter plays any role there. (For d = 3, which is not among the entries that are read, his output shows a difference for B1: 12 types by budgets alone, none in `types.json`.) In addition, every computed line has an implementation that does not read `types.json` (VERIF, CMINE, I2, I3, `d_stage.py`, REF1, REF2); the second referee confirmed this by searching the sources (`r3_verify.py`, `c_mine.py`, everything in `lemma/t18_VERIF331`, `e331.py`, `d_stage.py`, everything in `lemma/t18_REFEREE`, `r4_stageD.py`, `r4_verifyD.py`, `r4_verifyC.py`, `r4_verifyC2.py`: the file name occurs

there in comments only). H16 is therefore not used; this was not pointed out in the working reports nor by the referee.
7. **Saved certificates.** Only samples of the PSD witnesses were saved and re-checked: 50000 for stages B and C of B1 (out of roughly 1.6 million rejections), 12000 for B2, all valid; the 3048 witnesses of stage D are complete and were re-checked in both implementations; the witnesses saved for B0 (‘r3cert_0_*.jsonl‘) were not re-checked. Files larger than 1 MB (among them ‘r4cert_stageD_B1.jsonl‘ and ‘t18_B0/r3_p250_0_0of1_B.json‘) are not in git; ‘lemma/LARGE_FILES_NOT_IN_GIT.txt‘ lists them, and they are regenerated by the scripts.
8. **Runs cut by time limits.** The B1 runs of ENUM were made in five launches; the stage-B run of VERIF for B0 in two (matchings 0..417, then 418..424); the stage-C run of VERIF for B2 in four. Completeness is checked by bookkeeping scripts (‘b1_coverage.py‘, ‘b0_input_crosscheck.py‘, ‘check_complete.py‘), not by a rerun in one piece. The runs of CMINE are complete, every triple being processed exactly once.
9. **Patched scripts.** Several scripts were edited during the sessions (‘_patch_*.py‘). The working reports state that the scripts in place are the final ones; that the stored outputs were produced by exactly these versions was not verified.
10. **Single implementations** (none is load-bearing): the reduction of the B0 stage-B triples modulo stabilisers (section 8.4), which exists in one program of the repository (its closure, its number of orbits and its stabiliser orders have been recounted by the second referee; the stored representatives were not compared); the rank-bound confirmation of step 4 of profile (3,3,1) for B1 (‘r4_rank331.py‘). Line 4a of the table of section 7.5 has one exact implementation (I2) and the referee’s; the second referee’s program finds the same numbers.
11. **Not verified at all, and not used:** the dictionary between T18 and the lattice statement (section 1.4), the lemma H16, everything about the analogous statement for squared norm 20.
12. **Parts of this text that only its author had checked** when the first version was written: Proposition 3.1 as written here (it is the referee’s proof); the completed case analysis of Lemma 4.7; the proofs of (F7) in Lemma 7.5 and of the upper bound of (D3) in Lemma 8.7; item 6 above. All of these have since been checked by the second referee (section 3 of his report for the hand proofs; his remark 3 and ‘recount.py‘ for item 6). Two small scripts written for the first version (‘_check_text_defs.py‘, ‘_check_survivors.py‘, in ‘lemma/t18_PROOF/‘) confirm that the definitions (R1), (R2) of section 7.1 reproduce the numbers 99, 63, 25514, 13799, 9229, and that the data printed in appendix B reproduce the determinants 47684 and 874944; they are transcription checks, not part of the proof. What only its author has checked now is the text written for the present revision (the passages listed in section 9.6).

9.4 Statements of the working reports that are not used and are left out

- **A7** (consequences of H16 for independent triples of B): conditional on H16, not verified by the referee, and no enumeration uses it.
- **P6, P7, P8** of ‘lemma/t18_STRUCT‘ (a_2 <= 2 on X_2; a_2 in {2,3,4} on W; a multigraph reformulation at p): no written proof, not verified by the referee, no enumeration prunes with them.
- P1, P3, P4, P5 of ‘lemma/t18_STRUCT‘ (three integer eigenvectors; a rank-nullity count; t is even; t != 8): found correct by the referee, not needed. Identity (*) is proved directly in Lemma 5.4.
- The by-product of the n_3 = 0 report (a_0 <= 5 outside S, a_2 <= 3 on X_2, a_2 <= 6 on X_1, a_2 <= 8 on X_0 \ {p}): found correct by the referee; used only in a partial side route for B0 (‘b0_x0.py‘), not here.
- The computed bullets of A6 (‘rows.py‘: d(a_c) <= 2; no vertex with d = 3 for B1; "6 classes of d = 3 types remain for B0, 9 for B2"): superseded by Theorem 5.3.
- The enumerations of the cases e = 1, e = 2 and k = 2 by ‘enum_S.py‘, ‘verify_S.py‘, ‘level1.py‘ (all empty or dead): machine confirmations of Lemmas 4.5-4.7, not needed.
- The "X_0-first" route for B0 (‘b0_x0.py‘, 81 of 2667 representatives), the lattice tests, the "level-1" and "star" experiments, the kernel condition of stage D: partial or inconclusive, not used.

9.5 Corrections made with respect to the working reports (the referee’s nine remarks)

1. First step missing for squared norm 18: Proposition 3.1.
2. The bound h_0 <= 1 depended on a computed fact: Theorem 5.3 is now placed before Lemma 5.4-Proposition 5.7, which are pure hand proofs.
3. Wrong sentence on the violated matrix: both matrices are defined in section 6.1 and section 7.6 gives the counts with M1 alone.
4. Lemma 4.7: case C added; the two missing justifications of case B added; the use of k = 2 made explicit.
5. Stage-1 tests: defined and proved in Lemma 7.2, with the meaning of the four tests of the program header.
6. Counters: line 1 of section 7.5 (budget-only counts), the "type solutions" of section 8.4 (counted without (C6)), and the 846 of section 8.7 (with the upper bound of (D3)) are described as what they count.
7. Lemma 7.4 (e) states that the six other vertices of X_0 have no neighbour in W u {z_0, sigma z_0}.
8. Superseded statements removed: the "open case" of the four survivors (section 8.5), "B0 is open" and "one implementation for stage C of B1" (section 8.4), the d = 3 classes of A6 (section 9.4).
9. Unverified and unused statements left out: section 9.4.

9.6 The second referee: what his reproduction covered, what he did not check, the eleven corrections

A second referee read the first version of this text (‘lemma/t18_REFEREE2/REPORT.md‘). He read this text only (not the working reports, not the first referee’s report). He checked every hand statement line by line (section 3 of his report: every statement marked "hand" in section 9.2, and the remark on the kernel in section 8.7, is found correct, and nothing is used before it is proved). He traced every pair of classes of every determined set used by the proof to a lemma or to a listed parameter (section 4 of his report: the sets of 37, 39, 45, 47, 48, 49 and 51 vertices, and the sets of 39 and 41 vertices with one or two orbits of W). And he re-did every Computation with programs written from this text alone. His verdict: no error that breaks the proof, and every Computation (4.10, 7.3, 7.6, 8.4, 8.5, 8.6, 8.8) reproduced with every count equal.

His programs, called REF2 in this text, are in ‘lemma/t18_REFEREE2‘ (appendix A.7). They form one more implementation of every computed line of section 9.2, written from this text alone, with no code of the repository: Python with numpy and python-flint, no solver. In REF2 a rejection is exact (an integer count; an integer vector with negative quadratic form, evaluated in Python integers; or an exact characteristic polynomial). A "kept" decision of REF2 is floating-point (threshold -10^-7), except in the exact checks named below. Four of these programs open data files of the repository, for comparison only: ‘r2_tau_<s>.json‘ (‘stage1.py‘, ‘survivors.py‘, ‘orbitsB0.py‘), ‘types.json‘ (‘recount.py‘) and the stage-B files of VERIF (‘orbitsB0.py‘).

What REF2 covered (section 5 of the report; the numbers below were compared with his output files for this revision).

Computation 4.10 (‘q9.py‘): 1,024,380 labelled 4-regular graphs on 9 vertices; 16 isomorphism classes, all connected; exactly 3 classes with all codegrees <= 2 (5040 + 30240 + 20160 = 55440 labelled graphs, minimum codegree 1, isomorphic to B0, B1, B2); maximum codegree 3 or 4 in each of the other 13 classes.

Data on B0, B1, B2 and Computation 7.3 (‘stage1.py‘): 72 / 12 / 18 automorphisms and 6 / 8 / 9 triangles; 99 admissible rectangles, 63 of them usable in N(p); 25514 / 13799 / 9229 admissible matchings; 425 / 1259 / 528 orbits under Aut(B), also by Burnside’s count. His output also shows that the lists of deficient pairs printed after Corollary 4.11 are the right ones, and that the representatives stored in ‘r2_tau_<s>.json‘ are admissible matchings, one in each orbit.

Computation 7.6 (‘p331.py‘; it uses (F1), (F3), (F4), (F5), (F6), and neither (F2) nor (F7)):

line of section 7.5	B0	B1	B2
1. pairs (tau, F), budgets on 25 vertices	3510	7224	2406
2. kept on 39 vertices	383	481	7
2’. matchings / pairs (tau, F) carrying them	50 / 95	55 / 76	1 / 1
3a. tables, budgets on 45 vertices	8639	27203	0
3b. of which kept after the PSD test on 45 vertices	23	164	0
4a. graphs on 51 vertices, budgets	52	1839	-
4b. of which kept after the PSD test on 51 vertices	0	0	-

With M1 alone (‘p331_m1.py‘): 887 / 672 / 7 kept on 39 vertices, then 102 / 190 / 0 tables, then 0 / 10 / 0 graphs on 51 vertices, as in the table of section 7.6.

Computation 8.4 (‘stageB.py‘; the pairs between the two orbits of A_p are absent; the degree cap of (C2), (C3) and (C4) are not used, only recorded): 5545 / 12366 / 4347 pairs (tau, F) with no budget violation on the 23 vertices, of which 4904 / 11509 / 3863 are exactly PSD (‘recount.py‘); 5595 / 4977 / 196 stage-B triples, with the same numbers of matchings and of pairs (tau, F) carrying them, and the same distributions by t and by largest a_2, as in the table of Computation 8.4. This list is the list of VERIF up to the action of Aut(B) (‘orbitsB0.py‘). Every triple is exactly PSD for both matrices and has no budget violation on Y37 (‘exactB.py‘), and every triple satisfies the degree cap of (C2), (C3) and (C4).

Computation 8.5 (‘stageC.py‘, run on all the 5595 + 4977 + 196 triples of his stage-B list; it uses (C5), (C7), (C8)): no survivor for B0, none for B2, exactly 4 for B1, which are the four graphs of appendix B (‘compareC.py‘).

Computations 8.6 and 8.8 (‘survivors.py‘, which rebuilds the four graphs from appendix B alone): on each of them (C1)-(C8) hold and there is no budget violation; M1[Y47] is PSD of rank 46 and M2[Y47] is PSD of rank 45, exactly; det G_odd = 47684, 47684, 874944, 874944, rank G_odd = 23 and rank G_even = 23; in stage D there are 678, 678, 846, 846 pairs {Z, sigma Z} (849 and 849 for the two survivors of matching #975 without the upper bound of (D3)); none of the oriented sets Z is exactly PSD on the 49 vertices, nor already on the 48 vertices Y47 u {z}, and on the 49 vertices the violated matrix is always M1.

Checks made for his remarks 3, 6 and 7: ‘recount.py‘ (the entries of ‘types.json‘; section 9.3, item 6), ‘orbitsB0.py‘ (the reduction modulo stabilisers for B0; section 8.4) and ‘recount2.py‘ (the remark on line 1 in section 7.6).

Comparison with output files of the repository: his report says that 18 groups of counts quoted in this text were compared with the output files and that all agree, among them ‘ref1_quartic9.out‘, ‘verify_S.out‘, ‘r2_stage1_*.out‘, ‘v4_compare.out‘, ‘r2_summary_*.out‘, ‘v7_step4_*.out‘, ‘r4_rank331.out‘, ‘09_B0_check_complete.out‘, ‘11_B1_check_complete.out‘, ‘13_B1_survivors_compare.out‘, ‘rows.out‘. The complete list of the 18 groups is not in his folder (the report refers to a file ‘SPOTCHECK.txt‘, which is not there).

What the second referee did not check (section 6 of his report):

- the programs of the repository line by line; that the stored outputs were produced by the scripts now in place; the PSD certificates saved in the repository (he produced rejections of his own instead);
- the exact positive semidefiniteness of the 39- and 45-vertex survivors of profile (3,3,1) (only rejections matter for the proof; section 6.3 says who verified it);
- section 1.4 of this text (the lattice dictionary), the lemma H16, and the case of squared norm 20.

So items 3, 7, 9 and 11 of section 9.3 stand as they were. The passages written for the present revision (listed below) have not been read by any referee.

The eleven corrections made after his report (numbered as in section 2 of the report; apart from them no count, statement or proof has been changed).

1. *Definition of a survivor.* A survivor is now defined by properties of the graph (no budget violation, M1 and M2 PSD on the set) and no longer by "no violation is found": sections 6.1 and 6.3, Computations 7.6, 8.4, 8.5. Section 6.3 states (6.2), on which every exclusion rests and which uses exact rejections only, and gives the table of the lists verified to be exactly PSD, with the name of who verified them; every other "kept" decision is floating-point (also section 9.3, item 1).
2. *The 16 classes of Computation 4.10* now come from two programs written separately (section 4.10; section 9.3, item 5).
3. *‘types.json‘ and H16.* The entries that are read have been recomputed from the budgets alone; the list of the programs that read the file is completed (section 9.3, item 6).
4. *Pairs between the two orbits of A_p.* They are recorded as non-adjacent in the data H, in Lemma 8.2 and in (C2); Computation 8.4 says that H does not range over them; note in section 8.3.
5. *Tables* are defined as counted objects (section 7.3; step 3 of Computation 7.6).
6. *Stage C of B0.* The statement on the number of complete implementations is corrected (section 8.4; section 9.2; section 9.3, item 10); the reduction modulo stabilisers has been recounted.
7. *Remark on line 1 of section 7.5*: the two conditions and the three sets of numbers (section 7.6).
8. *Location of the two check scripts*: ‘lemma/t18_PROOF/‘ (section 9.3, item 12; appendix B).
9. *"55 vertices"*: note in section 8.7.

10. *Appendix.* Location of the B0 result of ENUM; the run of 29 triples of VERIF for B0; the four B2 runs of VERIF;
    the numbers of files 14, 15 and 17 (table of section 8.4; appendix A.5 and its notes).
11. *Notation.* eps_i for the inner indicator of section 4.7; kappa_0 in section 3; a_0 instead of c(v); the tags
    ENUM, VERIF, CMINE, REF1 instead of the letters E, V, C, R for programs; conventions of section 0.2; threshold
    of `r3_verify.py` (-10^-6, not -10^-7).

9.7 Possible simplifications (not used; unrefereed)

The second referee added four remarks that would simplify the text. They are his own and have not been refereed, and
nothing in this document uses them: (a) rank M1[Y47] = 46 > 44 = rank M1 would already exclude the four
survivors by Lemma 6.1 alone, without Lemma 6.3, Proposition 6.4 or stage D; (b) the four survivors are two graphs
up to the stabiliser of the matching; (c) (D3) can be sharpened to 3 (a_2(z) - 4) <= 2t; (d) (F2), (F7), the degree
cap of (C2), (C3), (C4) and the upper bound of (D3) are never needed: runs without them give the same lists.

Appendix A. Programs, outputs and commands

Paths are relative to the root of the repository `conway99-c2-aws`. "python" is Python 3 with numpy; the programs
of `lemma/t18_VERIF331` and `ref8_survivors.py` also need python-flint, and `show_structs.py` needs sympy. Each
command is run in the folder of the script (the scripts read their inputs next to themselves). The command lines
are those recorded in the working reports, in the driver logs, or in the headers of the scripts; where the exact
arguments of a run were not recorded, the usage line is given and the names of the output files show how the work
was split. s (or g) is 0, 1, 2 for B0, B1, B2.

The programs of the second referee (`lemma/t18_REFEREE2`, tag REF2) run with Python 3, numpy and python-flint, and
sympy for `stage1.py`. Their command lines were not recorded: the usage line of the script is given, and each `.out`
file is the saved standard output of the run whose arguments appear in its name. `stage1.py` writes
`my_tau_<s>.json` (his own representatives), which his later enumeration programs read. All his files are listed in
A.7.

A.1 Computation 4.10 (the three graphs)

program	command	output
`lemma/t18/enum_S.py`	`python enum_S.py`	`structures.json` (printed counts quoted in `T18_REPORT.md`, section D: "(i) e=0: labelled 792, iso classes 3")
`lemma/t18/verify_S.py`	`python verify_S.py`	`verify_S.out` ("(i) e=0: labelled 55440, iso classes 3")
`lemma/t18/show_structs.py`, `check_iso.py`	`python show_structs.py`, `python check_iso.py`	`show_structs.out` (edges, automorphisms, codegrees), `check_iso.out`
`lemma/t18_REFEREE/ref1_quartic9.py`	`python ref1_quartic9.py`	`ref1_quartic9.out` (1024380 labelled graphs, 16 classes, 3 with codegrees <= 2)
`lemma/t18_REFEREE2/q9.py` (REF2)	`python q9.py`	`q9.out` (1024380 labelled graphs, 16 classes, 3 with codegrees <= 2, isomorphic to B0, B1, B2)

A.2 Computation 7.3 (stage 1)

program	command	output
`lemma/t18/rows.py` (prerequisite of the next line)	`python rows.py`	`rows.out`, `types.json`
`lemma/t18/r2_stage1.py`	`python r2_stage1.py <s>`	`r2_stage1_<s>.out`, `r2_tau_<s>.json`
`lemma/t18/r3_verify.py`	`python r3_verify.py stage1 <s>`	`r3v_stage1_<s>.out` (for B2 also `lemma/t18_B2/b2v_stage1.out`)
`lemma/t18_VERIF331/v1_stage1.py`	`python v1_stage1.py <g>`	`v1_stage1_<g>.out`, `v1_tau_<g>.json`, `v1_tau_<g>_nopsd.json`
`lemma/t18_REFEREE/ref2_stage1.py`	`python ref2_stage1.py B0 B1 B2`	`ref2_stage1_B<g>.out`
`lemma/t18_REFEREE/ref10_tau_files.py`	`python ref10_tau_files.py` (after `ref4_stageB.py`)	`ref10_tau_files.out`
`lemma/t18_REFEREE2/stage1.py` (REF2)	`python stage1.py`	`stage1.out`, `my_tau_<s>.json`

A.3 Computation 7.6 (profile (3,3,1))

line of section 7.5	program	command	output
1, 2	I1: `lemma/t18/r2_hfirst.py`	`python r2_hfirst.py <s> <part> <nparts> <time limit>` (B0, B2: one part; B1: three parts; R2_CAP large enough to save all survivors)	`r2hf_0_all.out`, `r2hf_1_0.out`, `r2hf_1_1.out`, `r2hf_1_2.out`, `r2hf_2_all.out`; `r2_hfirst_0_0of1.json`, `r2_hfirst_1_{0,1,2}of3.json`, `r2_hfirst_2_0of1.json`
3a, 3b	I1: `lemma/t18/r2_stage2.py`	`R2_SURV=<files of r2_hfirst.py> [R2_IDX=<matchings>] python r2_stage2.py <s> <part> <nparts> <time limit>`	B0: `r2ext_0_g0.out`, `r2ext_0_g1.out`, `r2ext_0_g2.out`, `r2_stage2_0_0of1_surv_idx{0,1,2}.json`, summary `r2_summary_0.out`; B1: `r2ext_1_0.out`, `r2ext2_1_g0.out`, `r2ext2_1_g1.out`, `r2ext2_1_g2.out`, `r2_stage2_1_0of3_surv.json`, `r2_stage2_1_0of1_surv_idx{1,2,4}.json`, summary `r2_summary_1.out` (`python r2_summary.py`); B2: `r2ext_2.out`, `r2_stage2_2_0of1_surv.json`
4b	I1: `lemma/t18/r2_ext2.py`	`python r2_ext2.py <s> <files of r2_stage2.py>`	B0: `r2_ext2_B0.out`; B1: `r2_ext2_part0.out`, `r2_ext2_g0.out`, `r2_ext2_g12.out`
(check of 4)	I1: `lemma/t18/r2_ext2_bruteforce.py`	`python r2_ext2_bruteforce.py <s> <file> <number of tables>`	`r2_bf_B0.out`, `r2_bf_B1.out`, `r2_bf_B1b.out`
(check of 4, B1)	`lemma/t18/r4_rank331.py`	`python r4_rank331.py <max tables> <time limit> <files of r2_stage2.py>`	`r4_rank331.out`
1, 2	I2: `lemma/t18_VERIF331/v2_stage2.py`	`python v2_stage2.py <g> <part> <nparts>` (default T18_MATS=2; T18_MATS=1 for M1 alone)	`v2_stage2_<g>_m2.out`, `v2_surv_<g>_m2_0of1.json` (and `_m1` for M1 alone)
3a, 3b, 4b	I2: `lemma/t18_VERIF331/v3_stage34.py`	`python v3_stage34.py <g> <part> <nparts>` (T18_ROWPSD=0 for the raw counts)	`v3_stage34_<g>_m2*.out`, `v3_out_<g>_m2_*.json`
4a, 4b	I2: `lemma/t18_VERIF331/v7_step4_direct.py`	`python v7_step4_direct.py <g>` (g = 0, 1)	`v7_step4_0.out`, `v7_step4_1.out`
comparison of the sets	I2: `v4_compare39.py`, `v4_compare45.py`	`python v4_compare39.py <g>`, `python v4_compare45.py <g>`	`v4_compare.out`
summary; M1 alone	I2: `v5_summary.py`, `v6_m1_final.py`	`python v5_summary.py`, `python v6_m1_final.py 1`	`v5_summary.out`, `v6_m1_final.out`

1-3 (B2)	I3: ‘lemma/t18_B2/e331.py‘	‘python e331.py 2 0 1 280‘ ; ‘python e331_compare.py 2 r2_hfirst_2_0of1.json‘	‘e331_2.out‘, ‘e331_compare_2.out‘
1, 2	REF1: ‘lemma/t18_REFEREE/ref6_p331.py‘	‘python ref6_p331.py <B0, B1 or B2>‘ (REF_M1ONLY=1 for M1 alone)	‘ref6_p331_B<g>.out‘, ‘ref6_p331_B<g>_0of1.json‘ (and ‘_M1only‘)
3a-4b	REF1: ‘ref7_p331_ext.py‘ (B2), ‘ref7b_p331_ext.py‘ (B0, B1)	‘python ref7b_p331_ext.py <B0 or B1> ref6_p331_B<g>_0of1.json <part> 3‘	‘ref7_p331_ext_B2.out‘, ‘ref7b_p331_ext_B<g>_<part>of3.out‘; totals in ‘ref9_totals.out‘ (‘python ref9_totals.py‘)
1, 2, 2’, 3a, 3b, 4a, 4b	REF2: ‘lemma/t18_REFEREE2/p331.py‘	‘python p331.py <s> <part> <nparts>‘ (B0: 3 parts; B1: 4 parts; B2: 1 part)	‘p331_0_{0,1,2}.out‘, ‘p331_1_{0,1,2,3}.out‘ (no log was saved for B2); ‘my331_0_{0,1,2}of3.json‘, ‘my331_1_{0,1,2,3}of4.json‘, ‘my331_2_0of1.json‘
M1 alone (section 7.6)	REF2: ‘lemma/t18_REFEREE2/p331_m1.py‘	‘python p331_m1.py <s> <part> <nparts>‘ with the environment variable M1ONLY set	‘p331m1_0_{0,1,2}.out‘, ‘p331m1_1_{0,1,2,3}.out‘, ‘p331m1_2_0.out‘; ‘my331_M1_0_{0,1,2}of3.json‘, ‘my331_M1_1_{0,1,2,3}of4.json‘, ‘my331_M1_2_0of1.json‘
1; the remark on line 1 (section 7.6)	REF2: ‘lemma/t18_REFEREE2/recount.py‘, ‘recount2.py‘	‘python recount.py‘ ; ‘python recount2.py‘	‘recount.out‘, ‘recount2.out‘

A.4 Computation 8.4 (stage B of profile (2,5,0))

graph	program	command	output
B1	ENUM: ‘lemma/t18/r3_p250.py‘	‘python r3_p250.py 1 <part> <nparts> <time limit> BC‘, with R3_IDX=<matchings> for the later launches	14 files ‘r3_p250_1_*_BC.json‘ (listed, with the launch of each, in ‘lemma/t18_B0_VERIFC/04_b1_coverage.out‘); logs ‘r3_1_*.out‘
B1	VERIF: ‘lemma/t18/r3_verify.py‘	‘python r3_verify.py B250 1 <part> 3 <time limit>‘	‘r3v_B250_1_{0,1,2}.out‘, ‘r3v_B250_1_{0,1,2}of3.json‘
B1	comparison	‘python r3_finalB.py‘	‘lemma/t18/r3_finalB.out‘
B0	ENUM: ‘lemma/t18_B0/r3_p250.py‘ (unchanged copy)	‘python r3_p250.py 0 0 1 540 B‘	‘b0_enumB.out‘, ‘r3_p250_0_0of1_B.json‘ (not in git)
B0	VERIF: ‘lemma/t18_B0/r3_verify.py‘, ‘b0_verify.py‘	‘python r3_verify.py B250 0 0 1 540‘, then ‘b0_verify.py‘ (the same program with the list R3V_TAUS of matchings 418..424)	‘b0_verB.out‘, ‘b0_verB_t418.out‘, ‘r3v_B250_0_0of1.json‘, ‘r3v_B250_0_0of1_t418.json‘
B0	comparison	‘python b0_compareB.py‘	‘lemma/t18_B0/b0_compareB.out‘
B2	ENUM: ‘lemma/t18_B2/r3_p250.py‘ (unchanged copy)	‘python r3_p250.py 2 0 2 560 BC‘ ; ‘python r3_p250.py 2 1 2 560 BC‘	‘b2_BC_0of2.out‘, ‘b2_BC_1of2.out‘, ‘r3_p250_2_{0,1}of2_BC.json‘, statistics ‘b2_stats.out‘
B2	VERIF: ‘lemma/t18_B2/r3_verify.py‘	‘python r3_verify.py B250 2 0 1 540‘	‘b2v_B250.out‘, ‘r3v_B250_2_0of1.json‘
B2	comparison	‘python r3_verify.py compareB 2 r3_p250_2_0of2_BC.json r3_p250_2_1of2_BC.json‘	‘lemma/t18_B2/b2_compareB.out‘
all	REF1: ‘lemma/t18_REFEREE/ref4_stageB.py‘	‘python ref4_stageB.py <B0, B1 or B2>‘	‘ref4_stageB_B<g>.out‘, ‘ref4_stageB_B<g>_0of1.json‘
all	observed distributions	‘python check_lemmas_stageB.py‘ (‘lemma/t18_STRUCT‘)	‘check_lemmas_stageB.out‘
all	REF2: ‘lemma/t18_REFEREE2/stageB.py‘	‘python stageB.py <s> [first] [last]‘ (B0 in four runs, B1 in three, B2 in one)	‘stageB_0_{a,b,c,d}.out‘, ‘stageB_1_{a,b,c}.out‘, ‘stageB_2.out‘; ‘myB_0_0_300.json‘, ‘myB_0_300_380.json‘, ‘myB_0_380_410.json‘, ‘myB_0_410_425.json‘, ‘myB_1_0_420.json‘, ‘myB_1_420_840.json‘, ‘myB_1_840_1259.json‘, ‘myB_2_0_528.json‘
all	REF2: every stage-B triple exactly PSD on Y37 (‘exactB.py‘)	‘python exactB.py‘	‘exactB.out‘
all	REF2: orbits under the stabilisers; comparison with the files of VERIF (‘orbitsB0.py‘)	‘python orbitsB0.py <s>‘	‘orbitsB_<s>.out‘
all	REF2: first two rows of the table of Computation 8.4 (‘recount.py‘)	‘python recount.py‘	‘recount.out‘

A.5 Computation 8.5 (stage C of profile (2,5,0))

graph	program	command	output
B1	ENUM	the runs of A.4 (stages "BC")	‘r3_finalB.out‘, ‘r3_round5.out‘; survivors in ‘r3_p250_1_0of3_idx752_BC.json‘ (#813) and ‘r3_p250_1_2of3_idx758_BC.json‘ (#975)
B1	CMINE: ‘lemma/t18_B0_VERIFC/c_mine.py‘	‘python b1_coverage.py in_B1 in_B1v B1/b1_stageB_all.json‘ ; ‘python run_parts.py B1 6 1 24 3000 all B1/b1_stageB_all.json‘ (which runs ‘python c_mine.py 1 <part> 24 3000 b1_stageB_all.json‘ for the 24 parts) ; ‘python check_complete.py B1 1 4977 275 - B1/b1_stageB_all.json‘ ; ‘python b1_survivors_compare.py B1 in_B1 control_B1/c_mine_1_0of1_t813.json‘	‘04_b1_coverage.out‘; ‘B1/c_mine_1_<part>of24.json‘, ‘.out‘; ‘11_B1_check_complete.out‘, ‘12_B1_check_complete_verifier_list.out‘, ‘13_B1_survivors_compare.out‘
B1	CMINE, control on #813 and #975	‘C_TAUS=813,975 python run_parts.py control_B1 1 1 1 3000 all in_B1v/r3v_B250_1_{0,1,2}of3.json‘ ; ‘python check_complete.py control_B1 1 170 2 813,975 <the same three files>‘	‘control_B1/c_mine_1_0of1_t813.json‘, ‘05_control_B1_check.out‘, ‘06_control_B1_compare.out‘, ‘07_control_B1_vs_B2session.out‘
B1	partial: VERIF, ‘r4_verifyC.py‘, ‘r4_verifyC2.py‘	‘python r3_verify.py C250 1 ...‘ ; ‘R4V_FULL=1 python r4_verifyC2.py 1 <part> 2 470‘ ; ‘python r4_compareC_final.py‘	‘lemma/t18/r4_compareC_final.out‘
B0	ENUM: ‘lemma/t18_B0/b0_stageC.py‘, ‘b0_stageC2.py‘	‘python b0_orbitsB.py‘ ; ‘bash b0_loopC.sh‘ (slices of at most 560 s) ; ‘python b0_summaryC.py‘	‘b0_orbitsB.out‘, ‘b0_stageB_reps.json‘; the result is in ‘b0_stageC_done_*.jsonl‘ (one line per representative) and ‘b0_stageC_survivors.json‘; logs ‘b0_stageC_r*_*.out‘; ‘lemma/t18_B0/REPORT.md‘ ("FINAL STATE") is only a summary generated from them
B0	CMINE: ‘lemma/t18_B0_VERIFC/c_mine.py‘	‘python run_parts.py B0 6 0 24 3000 all in_B0/r3_p250_0_0of1_B.json‘ ; ‘python check_complete.py B0 0 5595 133 - in_B0/r3_p250_0_0of1_B.json‘ ; ‘python b0_input_crosscheck.py <enumerator file> <the two verifier files> r2_tau_0.json‘	‘B0/c_mine_0_<part>of24.json‘, ‘.out‘; ‘09_B0_check_complete.out‘; ‘08_b0_input_crosscheck.out‘
B0	VERIF, part 0 of 150, cut by its time limit after 29 triples (not a designed sample)	‘python b0_verify.py C250 0 0 150 270‘	‘lemma/t18_B0/b0_verC_sample.out‘
B2	ENUM	the runs of A.4	‘b2_BC_0of2.out‘, ‘b2_BC_1of2.out‘
B2	VERIF	‘python r3_verify.py C250 2 0 2 550‘ (cut by its time limit after 86 of the 98 triples of its part), then with R3V_SKIP / R3V_TAKE (0/86 and 86/86 with part 1 of 2, 172/24 with part 0 of 1); the four runs together treat 86 + 43 + 43 + 24 = 196 triples	‘b2v_C250_0of2.out‘, ‘b2v_C250_odd_skip0.out‘, ‘b2v_C250_odd_skip86.out‘, ‘b2v_C250_skip172.out‘
B2	CMINE: ‘lemma/t18_B2/c_mine.py‘	‘python c_mine.py 2 0 1 520 r3_p250_2_0of2_BC.json r3_p250_2_1of2_BC.json‘	‘c_mine_2_all.out‘, ‘c_mine_2_0of1.json‘
all	REF1: ‘lemma/t18_REFEREE/ref5_stageC.py‘	‘python ref5_stageC.py <B0, B1 or B2> ref4_stageB_B<g>_0of1.json full ...‘ ; ‘python ref9_totals.py‘	‘ref5_stageC_B0_all_*of3.out‘, ‘ref5_stageC_B1_all_*of3.out‘, ‘ref5_stageC_B2.out‘, ‘ref9_totals.out‘; type-solution variants in ‘ref5b_types_variants_B2.out‘
all	REF2: ‘lemma/t18_REFEREE2/stageC.py‘	‘python stageC.py <s> <part> <nparts> <files myB_<s>_*.json>‘ (B0 and B1: 8 parts; B2: 6 parts)	‘stageC_0_<part>.out‘ and ‘stageC_1_<part>.out‘ (part 0..7), ‘stageC_2_<part>.out‘ (part 0..5); ‘myC_0_<part>of8.json‘, ‘myC_1_<part>of8.json‘, ‘myC_2_<part>of6.json‘
B1	REF2: his four survivors compared with appendix B (‘compareC.py‘)	‘python compareC.py‘	‘compareC.out‘

The input file ‘in_B0/r3_p250_0_0of1_B.json‘ of the B0 run of CMINE is the stage-B file of ENUM (1.2 MB, not in git); the log ‘00_setup_hashes.log‘ records its hash and that of the copies of ‘c_mine.py‘ and ‘r2_tau_<s>.json‘.

Notes on A.4 and A.5 (second referee’s remark 10).

- *Numbers of files of ENUM for B1.* Three outputs quote three numbers: 14 files in ‘04_b1_coverage.out‘, 17 in ‘lemma/t18/r3_finalB.out‘, 15 in ‘13_B1_survivors_compare.out‘. This text relies on the 14 files of ‘04_b1_coverage.out‘, in which every matching is taken from the one file that completed it. The scripts of the two other outputs take every file ‘r3_p250_1_*.json‘ whose name contains ‘_BC‘ (respectively ends in ‘_BC.json‘), so they also read files that ‘04_b1_coverage.out‘ lists as not used: byte-identical copies (‘_round1‘) of three of the 14 files, and the early trial ‘r3_p250_1_0of40_BC.json‘. Today the folder ‘in_B1‘ contains 15 files ending in ‘_BC.json‘ (the 14 and that trial), and ‘lemma/t18‘ contains 18 files whose name contains ‘_BC‘ (the 14, the three copies and the trial); ‘r3_finalB.out‘ does not list the 17 files it read. The extra files add nothing: ‘r3_finalB.out‘ reports the same 4977 triples and the same four survivors, and ‘13_B1_survivors_compare.out‘ the same four survivors.
- *B0, ENUM.* The result is in the files ‘b0_stageC_done_*.jsonl‘ and ‘b0_stageC_survivors.json‘ of ‘lemma/t18_B0‘, not in ‘REPORT.md‘, whose section "FINAL STATE" is generated from them. Checked by the second referee: 2667 lines, every index from 0 to 2666 exactly once, no survivor, and the counters 262850 / 170382 / 701270 / 340605 / 704789 quoted in the notes of section 8.4.
- *B0, VERIF.* ‘b0_verC_sample.out‘ is not a designed sample: it is the run of part 0 of 150, which was cut by its time limit after 29 triples.
- *B2, VERIF.* The first of the four runs was cut by its time limit after 86 of the 98 triples of its part; the three other runs take the rest, and the four runs together treat 86 + 43 + 43 + 24 = 196 triples.

A.6 Computations 8.6 and 8.8 (the four survivors of B1)

what	program	command	output
exact re-examination	‘lemma/t18/r3_survivors.py‘	‘python r3_survivors.py 1 <files r3_p250_1_*_BC.json>‘	‘r3_survivors.out‘
rank bound	‘lemma/t18/r4_rank.py‘	‘python r4_rank.py 1 <files r3_p250_1_*_BC.json>‘	‘r4_rank.out‘
rank bound	‘lemma/t18_STRUCT/rank_survivors.py‘, ‘rank_survivors_check2.py‘	‘python rank_survivors.py‘ ; ‘python rank_survivors_check2.py‘	‘rank_survivors.out‘, ‘rank_survivors_check2.out‘
rank bound	‘lemma/t18_B2/rank_B1_survivors.py‘ (uses ‘rankbound.py‘)	‘python rank_B1_survivors.py‘	‘rank_B1_survivors.out‘
rank bound	‘lemma/t18_B0/b0_rank47.py‘	‘python b0_rank47.py 1 ../t18/r3_p250_1_*_BC.json‘	‘b0_rank47_B1control.out‘
rank bound, stage D	‘lemma/t18_REFEREE/ref8_survivors.py‘	‘python ref8_survivors.py‘	‘ref8_survivors.out‘
stage D	‘lemma/t18/r4_stageD.py‘	‘python r4_stageD.py 1 <files r3_p250_1_*_BC.json>‘ ; ‘python r3_verify.py certs r4cert_stageD_B1.jsonl‘	‘r4_stageD_B1.out‘, ‘r4_stageD_B1.json‘, ‘r4cert_stageD_B1.jsonl‘ (not in git)
stage D	‘lemma/t18/r4_verifyD.py‘	‘python r4_verifyD.py 1 <files r3_p250_1_*_BC.json>‘	‘r4_verifyD.out‘
stage D	‘lemma/t18_B2/d_stage.py‘	‘python d_stage.py 1 B1check <file with the survivors>‘ ; ‘python r3_verify.py certs ...‘	‘d_stage_B1check.out‘, ‘d_stage_B1check.json‘, ‘b2_certs_check.out‘
exact re-examination, rank bound, stage D	REF2: ‘lemma/t18_REFEREE2/survivors.py‘ (the four graphs rebuilt from appendix B)	‘python survivors.py‘	‘survivors.out‘

A.7 The files of the second referee (‘lemma/t18_REFEREE2‘)

The folder contains 118 files: the report ‘REPORT.md‘ and the 117 files of the following table.

files	what they are	computed line reproduced
‘common.py‘	helpers: the graphs B0, B1, B2 and the deficient pairs as printed in this text, automorphisms, budget test	-
‘q9.py‘, ‘q9.out‘	enumeration of the 4-regular graphs on 9 vertices	Computation 4.10 (A.1)
‘stage1.py‘, ‘stage1.out‘; ‘my_tau_0.json‘, ‘my_tau_1.json‘, ‘my_tau_2.json‘	data on B0, B1, B2; rectangles, matchings, orbits; his representatives	Computation 7.3 (A.2)
‘p331.py‘; ‘p331_0_0.out‘, ‘p331_0_1.out‘, ‘p331_0_2.out‘; ‘p331_1_0.out‘, ‘p331_1_1.out‘, ‘p331_1_2.out‘, ‘p331_1_3.out‘; ‘my331_0_0of3.json‘, ‘my331_0_1of3.json‘, ‘my331_0_2of3.json‘; ‘my331_1_0of4.json‘, ‘my331_1_1of4.json‘, ‘my331_1_2of4.json‘, ‘my331_1_3of4.json‘; ‘my331_2_0of1.json‘	profile (3,3,1), steps 1-4, both matrices. No ‘.out‘ log was saved for B2: its counters are in ‘my331_2_0of1.json‘	Computation 7.6, lines 1 to 4b of section 7.5 (A.3)
‘p331_m1.py‘; ‘p331m1_0_0.out‘, ‘p331m1_0_1.out‘, ‘p331m1_0_2.out‘; ‘p331m1_1_0.out‘, ‘p331m1_1_1.out‘, ‘p331m1_1_2.out‘, ‘p331m1_1_3.out‘; ‘p331m1_2_0.out‘; ‘my331_M1_0_0of3.json‘, ‘my331_M1_0_1of3.json‘, ‘my331_M1_0_2of3.json‘; ‘my331_M1_1_0of4.json‘, ‘my331_M1_1_1of4.json‘, ‘my331_M1_1_2of4.json‘, ‘my331_M1_1_3of4.json‘; ‘my331_M1_2_0of1.json‘	the same with M1 alone	the table of section 7.6 (A.3)
‘recount.py‘, ‘recount.out‘	‘types.json‘ against budgets alone; recount of line 1 of section 7.5 and of the first two rows of the table of Computation 8.4	section 9.3, item 6; A.3, A.4
‘recount2.py‘, ‘recount2.out‘	pairs (tau, F) admitting types, under three existence tests	the remark on line 1 in section 7.6 (A.3)
‘stageB.py‘; ‘stageB_0_a.out‘, ‘stageB_0_b.out‘, ‘stageB_0_c.out‘, ‘stageB_0_d.out‘; ‘stageB_1_a.out‘, ‘stageB_1_b.out‘, ‘stageB_1_c.out‘; ‘stageB_2.out‘; ‘myB_0_0_300.json‘, ‘myB_0_300_380.json‘, ‘myB_0_380_410.json‘, ‘myB_0_410_425.json‘; ‘myB_1_0_420.json‘, ‘myB_1_420_840.json‘, ‘myB_1_840_1259.json‘; ‘myB_2_0_528.json‘	stage B	Computation 8.4 (A.4)
‘exactB.py‘, ‘exactB.out‘	exact PSD and budgets of every stage-B triple on Y37; ranks	section 6.3 (A.4)
‘orbitsB0.py‘; ‘orbitsB_0.out‘, ‘orbitsB_1.out‘, ‘orbitsB_2.out‘	orbits of the stage-B triples under the stabilisers; comparison with the files of VERIF	sections 8.3 and 8.4 (A.4)
‘stageC.py‘; ‘stageC_0_0.out‘ to ‘stageC_0_7.out‘; ‘stageC_1_0.out‘ to ‘stageC_1_7.out‘; ‘stageC_2_0.out‘ to ‘stageC_2_5.out‘; ‘myC_0_0of8.json‘ to ‘myC_0_7of8.json‘; ‘myC_1_0of8.json‘ to ‘myC_1_7of8.json‘; ‘myC_2_0of6.json‘ to ‘myC_2_5of6.json‘	stage C	Computation 8.5 (A.5)
‘compareC.py‘, ‘compareC.out‘	his four stage-C survivors of B1 against appendix B	Computation 8.5 (A.5)
‘survivors.py‘, ‘survivors.out‘	the four graphs of appendix B: forced numbers, budgets, exact PSD and ranks, G_odd, G_even, stage D	section 8.5; Computations 8.6 and 8.8 (A.6)

Appendix B. The four stage-C survivors of B1, explicitly

The data below are copied from the output files of CMINE (‘lemma/t18_B0_VERIFC/B1/c_mine_1_{3,4,5}of24.json‘), which contain the same four 47-vertex graphs as the output files of ENUM. They allow Computation 8.6 to be redone independently of all the files; the second referee did so (‘lemma/t18_REFEREE2/survivors.py‘, section 9.6).

Graph B1: edges 01 02 03 04 14 17 18 23 26 28 35 36 45 47 56 57 68 78. Its deficient pairs, with their indices k:

    0:01 1:02 2:03 3:04 4:13 5:16 6:18 7:24 8:27 9:28 10:35 11:37 12:45 13:46 14:56 15:57 16:68 17:78

Vertices of Y47: x_0..x_8, y_0..y_8, p, s_0..s_17 (s_k is the vertex of X_2 attached to the deficient pair of index k), w_0..w_4 and their mates. Adjacencies:

- x_i ~ y_j iff ij is an edge of B1; no other edge inside S; p has no neighbour in S;
- s_k ~ x_i for the two indices i of the deficient pair k, and s_k ~ y_l for the two indices l of the deficient pair tau k; sigma s_k = s_{tau k};
- p ~ s_k for k in A_p; p ~ w_c and p ~ sigma w_c for all c; w_c ~ sigma w_c;
- s_k ~ s_l for the pairs k-l listed under "H";
- w_c of type (i ; j): w_c ~ x_i, w_c ~ y_j, sigma w_c ~ x_j, sigma w_c ~ y_i; w_c ~ s_k for k in K_c and sigma w_c ~ s_k for k in tau K_c;
- no other edges.

sigma exchanges x_i and y_i, s_k and s_{tau k}, w_c and sigma w_c, and fixes p. For the rank bound take R = {x_0, ..., x_8} u {s_k : k < tau k} u {w_0, ..., w_4} (23 vertices).

Matching #813: tau = 0-5, 1-8, 2-15, 3-14, 4-7, 6-10, 9-12, 11-17, 13-16 (pairs of indices), i.e. the rectangles 01|16, 02|27, 03|57, 04|56, 13|24, 18|35, 28|45, 37|78, 46|68.

Survivor #813, F = [2, 9]. A_p = {s_2, s_9, s_12, s_15}. H: 0-12, 2-15, 5-9, 9-12 (t = 4).

    w_0: type (0;4), K = {6, 11, 13}   (tau K = {10, 16, 17})
    w_1: type (1;3), K = {3, 8, 16}   (tau K = {1, 13, 14})
    w_2: type (1;6), K = {6, 7}   (tau K = {4, 10})
    w_3: type (2;8), K = {3, 4, 17}   (tau K = {7, 11, 14})
    w_4: type (6;7), K = {0, 8}   (tau K = {1, 5})

Survivor #813, F = [3, 6]. A_p = {s_3, s_6, s_10, s_14}. H: 1-10, 3-14, 6-8, 6-10 (t = 4).

    w_0: type (0;3), K = {9, 11, 13}   (tau K = {12, 16, 17})
    w_1: type (1;8), K = {2, 7, 16}   (tau K = {4, 13, 15})
    w_2: type (2;4), K = {2, 5, 17}   (tau K = {0, 11, 15})
    w_3: type (2;7), K = {4, 9}   (tau K = {7, 12})
    w_4: type (6;7), K = {0, 8}   (tau K = {1, 5})

Matching #975: tau = 0-5, 1-12, 2-17, 3-14, 4-11, 6-10, 7-9, 8-15, 13-16, i.e. the rectangles 01|16, 02|45, 03|78, 04|56, 13|37, 18|35, 24|28, 27|57, 46|68.

Survivor #975, F = [1, 2]. A_p = {s_1, s_2, s_12, s_17}. H: 1-12, 2-17 (t = 2).

    w_0: type (1;2), K = {6, 11, 13}   (tau K = {4, 10, 16})
    w_1: type (1;8), K = {3, 9, 15}   (tau K = {7, 8, 14})
    w_2: type (3;4), K = {5, 8, 13}   (tau K = {0, 15, 16})
    w_3: type (5;6), K = {5, 7, 11}   (tau K = {0, 4, 9})
    w_4: type (6;7), K = {3, 10}   (tau K = {6, 14})

Survivor #975, F = [3, 6]. A_p = {s_3, s_6, s_10, s_14}. H: 3-14, 6-10 (t = 2).

    w_0: type (0;2), K = {4, 8, 13}   (tau K = {11, 15, 16})
    w_1: type (1;2), K = {1, 17}   (tau K = {2, 12})
    w_2: type (3;4), K = {5, 7, 8}   (tau K = {0, 9, 15})
    w_3: type (6;7), K = {2, 9, 11}   (tau K = {4, 7, 17})
    w_4: type (7;8), K = {0, 12, 16}   (tau K = {1, 5, 13})

Transcription check made for this text (‘_check_survivors.py‘, in ‘lemma/t18_PROOF/‘; not part of the proof): the four graphs built from these lists are sigma-invariant, satisfy every budget, coincide with the graphs stored by ENUM, and give det G_odd = 47684, 47684, 874944, 874944 (fraction-free Bareiss in Python integers) and rank G_even = 23.

Appendix C. Correspondence with the labels of the working reports

label in the working reports	here
A0 (‘T18_REPORT.md‘)	Lemma 2.1, Corollary 2.2
first step, integer => 0/+-1 (‘t18_REFEREE/REPORT.md‘, section 3)	Proposition 3.1
items 1-5 of ‘LEMMA_TARGET‘, section 3	sections 4.1-4.4 and 4.9
A1 (parity law)	Lemma 4.5
A2, A3, A4	Lemmas 4.6, 4.7; Corollary 4.8 and the remark after it
A5	Corollary 4.8, Lemma 4.9, Computation 4.10, Corollary 4.11
A6, hand bullets	Lemmas 5.1, 5.2
N3: Lemma 1, Lemma 2, Theorem (‘N3_REPORT.md‘)	Lemma 5.1, Lemma 5.2, Theorem 5.3
A8	Lemma 5.4
A9; the profile split	Lemma 5.6, Proposition 5.7; Lemma 5.5
P2 (‘t18_STRUCT/REPORT.md‘)	Lemma 6.3, Proposition 6.4
stage-1 tests (i), (i’), (iii), (P) (header of ‘r2_stage1.py‘)	Lemma 7.2 and the remarks after it; Computation 7.3
A10	Lemmas 7.4, 7.5
A11, A12, A13	Computation 7.6, Proposition 7.7, sections 7.5 and 7.6
A14 a-h; C-a to C-d (‘t18_B2/REPORT.md‘)	Lemmas 8.1, 8.2, 8.3
A15, A16, A17, A20	Computations 8.4, 8.5
A18	Lemma 8.7
A19; A19b	Computation 8.8; Computation 8.6

| A21 | note in section 7.5 (‘r4_rank331.py‘) |
| A7; P1, P3-P8 | not used (section 9.4) |

Appendix D. Numbering of the vertices in the files

- Programs of ‘lemma/t18‘, ‘lemma/t18_B0‘, ‘lemma/t18_B2‘, ‘lemma/t18_B0_VERIFC‘ and of the referee: x_i = i and y_i = 9 + i (i = 0..8); p = 18; the vertex s_k of X_2 attached to the deficient pair of index k is 19 + k, its mate is 19 + tau k; in profile (2,5,0), w_c = 37 + 2c and sigma w_c = 38 + 2c (c = 0..4), and in stage D the vertices of X_0 \ {p} are 47..54; in profile (3,3,1) (‘r2_hfirst.py‘, ‘e331.py‘, ‘ref6_p331.py‘), z_0 = 37 and sigma z_0 = 38. ‘r2_ext2.py‘ and ‘r4_rank331.py‘ renumber: support 0..17, X_0 = 18..26 (p = 18), X_2 = 27..44, W from 45.
- Programs of ‘lemma/t18_VERIF331‘: x_i = i, y_i = 9 + i, p = 18, z_0 = 19, sigma z_0 = 20, A_p = 21..26 (three mate pairs), X_2’ = 27..38 (orbit number k is 27 + 2k, 28 + 2k), then the added vertices.
- "F = [a, b]" (or [a, b, c]) lists, for each rectangle of F, the smaller of the two indices of its deficient pairs.
- "Matching #n" is the entry n (from 0) of the list "reps" of ‘lemma/t18/r2_tau_<s>.json‘; an entry is the list (tau 0, tau 1, ..., tau 17) of the images of the 18 indices.

3 The lemma T_{20}

File: lemma/T20_PROOF.md

```
# T20: no sigma-odd integer (-4)-eigenfunction of squared norm 20 on a srg(99,14,1,2)

> **State of this text (main session, 07/10, 14:10 UTC).**
> 1. Every case of the proof is closed: the last one, case I with n_3 = 2, is section 9.B.6 (Computation 9.29, two
>    programs, written on 07/10 at 14:00 UTC). Section 9.B.6 and `SPEC_ROUTE_B_N3.txt` had their referee pass on 07/10
>    (addendum to `lemma/t20_CODEX/REFEREE_T20_PROOF.md`): no blocking error; its two presentation points are merged.
>    The referee did not audit the search code. Its reserve G1 (provenance) is answered by `lemma/ARTIFACTS_T18_T20.md`
>    (run branches, program versions, archive branch of the raw outputs); G2 (wording of the verifier contract) is
>    merged since 07/10 evening: the contract, where it is in the four verifiers, and three negative controls are at
>    the end of section 6.5; item 3 of section 10.3 no longer says "node by node". Not yet re-read by the referee.
> 2. III.1 has a second verification: a verifier re-traversed the whole tree (1,628,864 nodes, same counts per level, same
>    kept count at every node, 0 complete), and the single node the first program rejected by the rank bound is also dead
>    without it (`lemma/t20_III1/VERIF.md`). Scope of its independence, as that report says: its budget test, its PSD test and
>    its bookkeeping are its own; it shares the model file `cases.py` and parts of the enumeration skeleton and of the
>    candidate-generation loops with the first program. Sections 0.2, 7.3, 7.4, 10.2 and 10.3 still say "one program".
> 3. Case I, n_3 = 0: the second program (`in_verify.py`, no code shared with the first) is complete on all twelve cases
>    (07/10, 14:33 UTC): stage 1 identical, stage 2 agreeing at every node, 0 complete configurations; where the first
>    program used the rank bound, the second closes the node by its own search. I.1 (3,4,0): 7,279,961 nodes, branch
>    `lemma-run/t20_I1_340-i-019e86d399c17954f`; I.1 (4,2,1): 1,406,917 nodes, branch
>    `lemma-run/t20_Iv_G-i-05703a74fbb027423`; the ten others on the branches `lemma-run/t20_Iv_{B,C,D,E,H,I,J}-*`.
>    Sections 0.2, 9.A and 10.2 still give the earlier counts (two of twelve).
> 4. Corrections of the referee report `lemma/t20_CODEX/REFEREE_T20_PROOF.md` merged on 07/10: E1 (sign in the proof of
>    Corollary 5.2), E2 (split parameter 3 means tasks of level 4), C2 (paths), G3 (item 2 above), G2 (item 1). G1 is
>    answered by `lemma/ARTIFACTS_T18_T20.md`. Still to merge: C3.

## Proof text, state of 2026-10-07. Every case is closed and refereed; every computed step has a second program.

Self-contained proof text, written on 2026-10-07 from the working reports of the repository `conway99-c2-aws`
(`lemma/T20_WORK.md`, `lemma/T20_BOARD.md`, `lemma/t20/T20_REPORT.md`, `lemma/t20_R2/REPORT.md`, `lemma/t20_III1/`,
`lemma/t20_R3_I/REPORT.md`, `lemma/t20_HAND/NOTES.md`, `lemma/t20_HAND/SPEC_N3.md`, `lemma/t20_CODEX/*.md`), in the
same form as `lemma/T18_PROOF.md`. The working reports are sources, not part of the proof: every hand statement used
below is proved below, from the definitions. The finite computations are described precisely (what is enumerated, by
which tests, with which result, by how many programs), and for each of them the programs and their output files are
listed in appendix A.

**This text does not prove T20.** It proves, by hand statements and finite computations, Theorem 10.1: a
configuration, if there is one, has no active inner orbit (k = 0), has exactly two outside vertices with three
neighbours in S+ (n_3 = 2), and has profile (3,4,0) or (4,2,1). The exclusion of that last sub-case is Computation 9.29
(section 9.B.6, two programs).

(This file is `lemma/T20_PROOF.md`; its working files are in `lemma/t20_PROOF/`.)

## 0. Front matter

### 0.1 GAP boxes

**GAP boxes in this document: none.** Every hand statement was re-derived while it was written, and no step was
found that could not be justified. This sentence is about the hand proofs only. It says nothing about the
finite computations (section 10.2) or about what has not been independently
verified (section 10.4). Four points of section 10.4 deserve to be known before reading:

- several proofs of this text are not in the working reports and were written for it (list in section 10.4,
  item 12). Nobody but their author has read them;
- a "kept" decision of a positive-semidefiniteness test is a floating-point decision in every enumeration program. It
  is never used as a proof: only rejections are used, and they are exact (section 6.3). Every number of "nodes",
  "bases" or "kept" candidates printed below is therefore the size of a kept list of the named program, not a number
  of survivors in the mathematical sense, unless the text says otherwise;
- in every search (Computations 7.4, 8.9, 9.20) that has a second program, the second program re-traverses the
  search tree of the first one:
  it reads, at every node, which unit the first program chose, regenerates and tests all candidates of that unit
  with its own code, and stops if its number of kept candidates differs. It does not choose the units by itself
  (sections 7.4, 8.4, 9.A.5);
- the rank bound (Proposition 6.4) is used by the computations: at 1 node of configuration III.1 and at 15 nodes of
  the graph I.1 (section 10.3).

### 0.2 What is NOT covered

1. **Case k = 0, n_3 = 2** (section 9.B): closed on 07/10 by Computation 9.29 (two programs). The section has not
   yet been refereed.
2. **Computed lines that have one program only at the time of writing** (state of the sources: `T20_BOARD.md` and
   `lemma/t20_R3_I/REPORT.md` of 07/10; runs in progress are not anticipated):
   - configuration III.1 (Computation 7.5): one program; the independent verifier is running;
   - case k = 0, n_3 = 0 (Computations 9.19 and 9.20): two programs, both complete, for the two cases I.0 (4,2,1)
     and I.5 (4,2,1) only. For I.0 (3,4,0) the second program has done stage 1 (identical lists) and stage 2 on 372 of
     the 11,150 bases. For the nine other cases there is the first program only; the second program is running.
3. The lattice context (section 1.4), the three earlier small-norm statements (squared norms <= 14, 16 and 18) and
   the "dual-norm rule" built on the first of them: not used, not proved here (section 10.3).

### 0.3 Notation and conventions
```

- ‘u‘ between two sets is the union, ‘n‘ the intersection, ‘\‘ the difference; ‘|T|‘ is the number of elements.
- ‘x ~ y‘ means "x and y are adjacent"; ‘x not~ y‘ means "x and y are not adjacent".
- ‘N(x)‘ is the set of neighbours of x, ‘N[x] = N(x) u {x}‘.
- ‘C(a,2) = a(a-1)/2‘. ‘[P]‘ is 1 if the statement P is true and 0 otherwise.
- ‘I‘ is the identity matrix, ‘J‘ the all-ones matrix, ‘1_T‘ the indicator vector of a vertex set T, ‘e_x = 1_{x}‘.
- For a matrix M and a vertex set Y, ‘M[Y]‘ is the principal submatrix on Y; for two lists R, R’ of vertices, ‘A[R,R’]‘ is the submatrix with rows R and columns R’.
- "PSD" means positive semidefinite.
- For two vertex sets X, Y: ‘e(X)‘ is the number of edges with both ends in X, ‘e(X,Y)‘ the number of edges with one end in X and the other in Y (X, Y disjoint).
- A vertex of N(p) is called **inner**; a vertex outside N[p] is called a **word** (the name used by all the working reports). An orbit {x, sigma x} is inner or a word orbit accordingly.
- Labels in square brackets such as [A7], [G0], [II.4], [C6], [H1] are the names of the same statements in the working reports. They are given only so that a reader of those reports can find the correspondence (appendix B); no proof below refers to them.
- A statement marked **Computation** is the result of a finite enumeration by programs. A statement marked **Lemma**, **Proposition**, **Theorem** or **Corollary** without that word is proved by hand in the text, except that the final statements of the cases (Corollary 4.13, Propositions 7.6, 8.10, 9.21, Theorem 10.1) combine hand statements with Computations and say so.
- Letters with several uses. The working reports use the letters W, R, T, U, t, s, c, Q, H for several objects each. In this text:
  - ‘W‘ is the set N(p) n X_1 (section 8: ten vertices; section 9: 2 h_1 vertices). The set called W in the report on profile (4,2,1) is called ‘L‘ here (Theorem 9.14).
  - ‘R‘ is a set of orbit representatives (section 6.4 only), and in section 9.B.4 the set N(z) n X_1. The set X_0 \ N[p], called Q or R in the reports, is called ‘R_0‘ here.
  - ‘T‘ is always N(z) n X_0 for the vertex z under discussion. The triangle called T in one report is ‘Delta‘ here.
  - ‘U(s)‘, ‘V(s)‘ are the two index sets of the type of a vertex (section 5.5); ‘U‘, ‘V‘ without argument are those of the vertex z of X_3 in section 9.B.
  - ‘t‘ is the level of a node in a search (number of orbits placed). The number of edges inside X_2, called t in the reports, is written e(X_2).
  - ‘s‘, ‘s’‘ are vertices of X_2. The numbers called s(w) in one report are ‘rho(w)‘ here.
  - ‘c‘, ‘c’‘ index the seven orbits {a_c, b_c} of N(p). ‘kappa_0‘ are the absolute values of section 3.
  - ‘Q‘ is the number sum d+(x)^2 of section 4. The identities called (Q0), (Q2) in the reports are (9.6) and (9.11).
  - ‘K‘ is the set of inner indices of the support (section 4.1); the reports call it H.
  - ‘E_0, E_3, E_{-4}‘ are the projectors of section 1.1; ‘e‘ is the number of edges inside S+.
- The nine support configurations keep the names of the working reports: I.0, ..., I.5, IIa.0, III.0, III.1. "Case I", "case II", "case III" are the three cases of Corollary 4.5 (k = 0, 2, 4).
- Programs are called by their file names.

1. Setting and statement

1.1 The graph

Throughout, G is a strongly regular graph with parameters (99, 14, 1, 2): it has 99 vertices, every vertex has exactly 14 neighbours, two adjacent vertices have exactly 1 common neighbour (lambda = 1), and two distinct non-adjacent vertices have exactly 2 common neighbours (mu = 2). Whether such a graph exists is Conway’s 99-graph problem; everything below is about a hypothetical G. A is its adjacency matrix.

The (x,y) entry of A^2 is the number of common neighbours of x and y, so

    A^2 = 14 I + 1 * A + 2 (J - I - A) = 12 I - A + 2 J.   (1.1)

A 1 = 14 * 1 (1 = all-ones vector). On the orthogonal complement of 1, (1.1) reads A^2 + A - 12 I = 0, i.e. (A - 3I)(A + 4I) = 0, so the other eigenvalues are 3 and -4. If f and g are their multiplicities, f + g = 98 and trace A = 0 = 14 + 3f - 4g, hence f = 54 and g = 44:

    eigenvalue 14 (multiplicity 1), 3 (multiplicity 54), -4 (multiplicity 44).

V_3 and V_{-4} are the eigenspaces, and E_0 = J/99, E_3, E_{-4} the orthogonal projectors on the three eigenspaces: E_0 + E_3 + E_{-4} = I and A = 14 E_0 + 3 E_3 - 4 E_{-4}.

1.2 The involution

sigma is an automorphism of G such that sigma^2 is the identity and sigma has **exactly one fixed vertex**, called p.

This is all that is assumed about sigma. Nothing is assumed about other automorphisms of G, and nothing from the lattice or "frame" picture that motivated the lemma is used.

For a vertex x != p the vertex sigma x is called the **mate** of x. The sets {x, sigma x} with x != p are the 49 **orbits**; {p} is the fixed orbit. An edge {x, sigma x} is called a **mate edge**. A vertex set Y is **sigma-invariant** if sigma Y = Y. A function phi on the vertices is **sigma-odd** if phi(sigma x) = -phi(x) for every vertex x; then phi(p) = 0.

1.3 Statement

> **T20.** Let G be a strongly regular graph with parameters (99,14,1,2) and sigma an automorphism of G of order 2
> with exactly one fixed vertex. There is no function phi from the vertex set of G to the integers such that
> (i) phi(sigma x) = -phi(x) for every vertex x, (ii) A phi = -4 phi, (iii) sum over all 99 vertices of
> phi(x)^2 = 20.

A triple (G, sigma, phi) satisfying the hypotheses is called a **configuration**. T20 says that no configuration exists. **It is not proved here.** What is proved is the following chain; the last line is where it stops.

    phi integer-valued

-> section 3 (hand): phi takes the values 0, +1, -1 only; S+ = {phi = +1} has 10 vertices
-> section 4 (hand + Computation 4.12): the number k of vertices of S+ in N(p) is 0, 2 or 4; the support, with the vertex p, is one of nine explicit graphs: I.0 ... I.5 (k = 0), IIa.0 (k = 2), III.0, III.1 (k = 4)
-> section 7 (hand model + Computations 7.4, 7.5): III.0 and III.1 are impossible (III.1: one program at the time of writing)
-> section 8 (hand lemmas + Computations 8.8, 8.9): IIa.0 is impossible
-> section 9 (hand): for I.0 ... I.5 the number n_3 of outside vertices with three neighbours in S+ is 0 or 2, and the neighbourhood of p has profile (3,4,0) or (4,2,1)
-> section 9.A (hand model + Computations 9.19, 9.20): n_3 = 0 is impossible for the six graphs and the two profiles (two programs for two of the twelve cases, one for the others)
-> section 9.B (hand model + Computation 9.29): n_3 = 2 is impossible for the six graphs and the two profiles (two programs)

Section 5 contains the general facts on the vertices outside the support, and section 6 the tests used by the computations (common-neighbour budgets, two PSD matrices, a rank bound) and the form of the searches, with their proofs.

1.4 Context (not part of the lemma)

In the project, T20 is the graph form of a statement about a lattice (‘LEMMA_TARGET_2026-10-06.md‘, section 2, and ‘lemma/BRIDGE.md‘): T18 is the same statement with squared norm 18 (‘lemma/T18_PROOF.md‘), and according to ‘lemma/T20_WORK.md‘ T20 is what remains to complete the exclusion of the 156 envelopes of the search. That dictionary and that bridge are **outside this document**: what is treated here is the graph statement of section 1.3 and nothing else.

One remark on the statement itself (round 1, section 0): the task text given to the first working session said "sum of phi(x)^2 = 40 (20 per side)". Everything else in the project says squared norm 20 on the 99 vertices, i.e. 10 on each of S+ and S-, and this is the statement treated in all the working reports and here. Nothing below applies to squared norm 40.

The theorem T18 is not used anywhere in this text (section 10.3). Some general facts are common to the two texts (Lemma 2.1, section 6.1, 6.2, 6.4); they are proved again here, so that this text stands alone.

2. Basic facts about sigma [round 1, A0; (M) of round 2]

Lemma 2.1. Let x be a vertex, x != p.

(a) x ~ sigma x if and only if x ~ p.

(b) N(p) consists of 7 orbits {a_c, b_c}, c = 1..7, with b_c = sigma a_c; the 7 mate edges {a_c, b_c} are the only edges with both ends in N(p). In other words N(p) induces a perfect matching, made of mate edges.

(c) For c = 1..7 the only common neighbour of a_c and b_c is p. Hence no vertex other than p is adjacent to both a_c and b_c.

(d) If x is outside N[p], then x not~ sigma x, and N(x) n N(sigma x) = {z, sigma z} for a vertex z which is outside N[p] (and z != sigma z).

Proof. (Identical to T18_PROOF, Lemma 2.1.) Let C = N(x) n N(sigma x). Since sigma is an automorphism and an involution, sigma C = N(sigma x) n N(x) = C: the set C is sigma-invariant. Also x != sigma x, because p is the only fixed vertex.

(a) If x ~ sigma x, then |C| = 1 (lambda = 1); a sigma-invariant set with one element consists of a fixed vertex, so C = {p} and x ~ p. Conversely let x ~ p. Then sigma x ~ sigma p = p, so p is in C. If x were not adjacent to sigma x we would have |C| = 2 (mu = 2), C = {p, c} with c != p, and sigma c would be an element of C other than sigma p = p, i.e. sigma c = c: a second fixed vertex, which does not exist. So x ~ sigma x.

(b) If a is in N(p) then sigma a is in N(p) (sigma p = p), a != sigma a, and a ~ sigma a by (a). So N(p) is a union of orbits, 14/2 = 7 of them, each carrying a mate edge. The pair a, p is adjacent, so a and p have exactly one common neighbour; a common neighbour of a and p is exactly a neighbour of a inside N(p). Hence a has exactly one neighbour in N(p), and it is sigma a.

(c) In the proof of (a) we saw that C = {p} when x ~ sigma x; apply it to x = a_c.

(d) x is not adjacent to p, so x not~ sigma x by (a) and |C| = 2. p is not in C (x not~ p). A sigma-invariant 2-set that does not contain the fixed vertex is an orbit: C = {z, sigma z}, z != sigma z. If z were in N(p), then by (c) the only common neighbour of z and sigma z would be p; but x is a common neighbour of z and sigma z (z and sigma z are both in C, hence both adjacent to x), so x = p, a contradiction. And z != p. So z is outside N[p]. QED

Corollary 2.2. Every word x (vertex outside N[p]) has exactly 2 neighbours in N(p).

Proof. x and p are distinct and non-adjacent, so they have exactly 2 common neighbours, and the common neighbours of x and p are the neighbours of x in N(p). QED

Corollary 2.3.

(a) The two neighbours in N(p) of a word lie in two different orbits of N(p).

(b) Let w be in N(p) and let {q, sigma q} be an orbit other than {w, sigma w}. If the orbit is inner, w is adjacent to neither q nor sigma q. If it is a word orbit, w is adjacent to at most one of q, sigma q.

(c) Let x be a word and {q, sigma q} an orbit with q != x, sigma x. If x ~ q and x ~ sigma q, then {q, sigma q} is a word orbit and N(q) n N(sigma q) = {x, sigma x}.

Proof. (a) By Lemma 2.1 (c) no vertex other than p is adjacent to both a_c and b_c. (b) The first statement is Lemma 2.1 (b). For the second, suppose w ~ q and w ~ sigma q with q a word. Applying sigma to w ~ q gives sigma w ~ sigma q. So sigma q is adjacent to both w and sigma w, the two ends of a mate edge of N(p), and sigma q != p: this contradicts Lemma 2.1 (c). (c) x is adjacent to both q and sigma q and x != p, so by Lemma 2.1 (c) the orbit is not inner; it is a word orbit, q not~ sigma q, and q, sigma q have exactly two common neighbours. x is one of them, and so is sigma x (apply sigma to x ~ q and to x ~ sigma q). QED

Corollaries 2.2 and 2.3 are used many times below, often without mention, in the form "a word has exactly two neighbours in N(p), in two different pairs" and "an inner vertex is adjacent to at most one end of each word orbit".

3. From integer values to the values 0, +1, -1 [round 1, A1, A2]

For an orbit O = {x, sigma x} put kappa_O = |phi(x)| = |phi(sigma x)|, an integer >= 0. As phi(p) = 0, hypothesis (iii) reads: sum over the 49 orbits of 2 kappa_O^2 = 20, i.e.

    sum_O kappa_O^2 = 10. (3.1)

Lemma 3.1. Let O be an orbit with kappa_O = kappa > 0. Then

    sum over the orbits O' != O of kappa_O' >= 3 kappa if O is inner, >= 4 kappa if O is a word orbit.

If O is inner and equality holds, then for x in O with phi(x) = kappa and for every orbit O' != O with kappa_O' > 0, x is adjacent to exactly one vertex y of O', and phi(y) = -kappa_O'.

Proof. Choose x in O with phi(x) = kappa > 0. The eigen-equation at x is

    -4 kappa = sum over the neighbours y of x of phi(y).

Sort the neighbours of x by orbit. The neighbour p (if x ~ p) contributes 0. An orbit O' = {y, sigma y} different from O contributes 0 if x is adjacent to none of its two vertices, +-kappa_O' if x is adjacent to exactly one of them, and phi(y) + phi(sigma y) = 0 if x is adjacent to both; in all cases at least -kappa_O', with equality (when kappa_O' > 0) only if x is adjacent to exactly one vertex of O' and that vertex has the value -kappa_O'. If O is inner, sigma x is a neighbour of x (Lemma 2.1 (a)) and contributes -kappa; hence
-4 kappa >= -kappa - sum_{O' != O} kappa_O'. If O is a word orbit, sigma x is not a neighbour of x, and
-4 kappa >= - sum_{O' != O} kappa_O'. In the inner case, equality in the sum forces equality in every term. QED

Lemma 3.2 [A2]. There is no configuration in which one orbit has kappa = 2, six orbits have kappa = 1 and the others have kappa = 0.

Proof. The other orbits sum to 6 < 8, so by Lemma 3.1 the orbit O with kappa = 2 is inner, and equality holds in Lemma 3.1 (6 = 3 * 2). Let x be in O with phi(x) = 2, so x is in N(p), sigma x ~ x and phi(sigma x) = -2. Name the vertices of the six other orbits of the support x_1, ..., x_6 (phi = +1) and y_i = sigma x_i (phi = -1). By the equality case of Lemma 3.1, x is adjacent to y_1, ..., y_6 and to none of x_1, ..., x_6. Applying sigma, sigma x is adjacent to x_1, ..., x_6.

Consider x_1. It is adjacent to sigma x, and it is not p, so by Lemma 2.1 (c) it is not adjacent to x. Hence x_1 and x are distinct and non-adjacent: they have exactly two common neighbours. One of them is sigma x (sigma x ~ x_1 and sigma x ~ x). All of y_1, ..., y_6 are neighbours of x, so at most one of them is a neighbour of x_1. The eigen-equation at x_1 reads -4 = sum over the neighbours w of x_1 of phi(w). The neighbours of x_1 with a non-zero value are among sigma x (value -2), x_2, ..., x_6 (value +1 each) and y_1, ..., y_6 (value -1 each, at most one of them is a neighbour); x is not a neighbour. So the sum is at least -2 - 1 = -3 > -4. Contradiction. QED

The proof uses only lambda = 1, mu = 2 and the eigen-equation.

Proposition 3.3. Let phi be an integer-valued sigma-odd function with A phi = -4 phi and sum phi(x)^2 = 20. Then phi takes only the values 0, +1, -1, and the set S+ = {x : phi(x) = +1} has exactly 10 vertices. Moreover S- = {x : phi(x) = -1} = sigma S+, and p is in neither.

Proof. By (3.1) the multiset of the non-zero values kappa_O is one of

    {3,1}, {2,2,1,1}, {2,1,1,1,1,1,1}, {1,1,1,1,1,1,1,1,1,1}

(the ways of writing 10 as a sum of squares 9, 4, 1). Lemma 3.1 excludes the first two:

    {3,1}: kappa = 3, the others sum to 1 < 9;
    {2,2,1,1}: kappa = 2, the others sum to 4 < 6.

The third is excluded by Lemma 3.2. So exactly 10 orbits have kappa_O = 1 and the other 39 have kappa_O = 0. In each of the 10 orbits one vertex has phi = +1 and its mate has phi = -1. QED

So the general integer case adds nothing to the case of values 0, +1, -1. From now on a configuration is a triple (G, sigma, phi) with phi as in Proposition 3.3, and

    S+ = {phi = +1} (10 vertices), S- = sigma S+ = {phi = -1}, S = S+ u S- (the support, 20 vertices).

There are 99 - 20 = 79 vertices outside S, and p is one of them.

4. The support

4.1 Notation and the eigen-equation

For a vertex v let d+(v) = |N(v) n S+| and d-(v) = |N(v) n S-|. Since sigma exchanges S+ and S-,

    d+(sigma v) = d-(v)   and   d-(sigma v) = d+(v)   for every vertex v.   (4.1)

(A phi)(v) = d+(v) - d-(v), so A phi = -4 phi says exactly:

    d-(x) = d+(x) + 4   for x in S+;   (4.2)
    d+(v) = d-(v)   for v outside S; this common value is written d(v).   (4.3)

By (4.1), d(sigma v) = d(v) for v outside S. Three numbers are attached to the configuration:

    k = d(p) = the number of vertices of S+ in N(p);
    e = the number of edges with both ends in S+;
    Q = sum over x in S+ of d+(x)^2.

An orbit {a_c, b_c} of N(p) is **active** if phi(a_c) != 0. Then exactly one of a_c, b_c is in S+ and the other is in S-, so k is the number of active orbits of N(p), S n N(p) has 2k vertices and S \ N(p) has 20 - 2k vertices. p is not in S.

Label S+ = {x_0, ..., x_9} and put y_i = sigma x_i, so S- = {y_0, ..., y_9}. Call the index i **inner** if x_i is in N(p) and **outer** otherwise; K is the set of inner indices (|K| = k), and eps_i = 1 if i is inner, 0 if i is outer. By Lemma 2.1 (a),

    x_i ~ y_i if and only if i is inner.   (4.4)

For an index j let M_j = {i != j : x_j ~ y_i}. Applying sigma, x_j ~ y_i if and only if x_i ~ y_j, so

    i is in M_j if and only if j is in M_i.   (4.5)

d-(x_j) = |M_j| + eps_j by (4.4), so by (4.2)

    |M_j| = d+(x_j) + 4 - eps_j.   (4.6)

A pair {i, j} with x_i ~ y_j (i in M_j) is called a **minus edge**, and a pair {i, j} with x_i ~ x_j (equivalently y_i ~ y_j) a **plus edge**; e is the number of plus edges.

4.2 k is 0, 2 or 4 [A3]

Lemma 4.1. k is in {0, 2, 4}. If k = 4, then every active inner vertex has exactly 3 neighbours in S \ N(p), and no inactive inner vertex has a neighbour in S.

Proof. Fix c. By Lemma 2.1 (b) the neighbours of a_c are p, b_c and 12 words; call the latter the outer neighbours of a_c. The eigen-equation at a_c is

    -4 phi(a_c) = phi(p) + phi(b_c) + (sum of phi over the 12 outer neighbours) = 0 - phi(a_c) + (that sum),

so the sum of phi over the outer neighbours of a_c is -3 phi(a_c). Let n_c be the number of outer neighbours of a_c that lie in S. As phi is +-1 on S and 0 elsewhere,

    n_c = phi(a_c) (mod 2)   and   n_c >= 3 |phi(a_c)|.

So n_c is odd and >= 3 if the orbit c is active, and n_c is even otherwise. By sigma, b_c has the same number n_c of outer neighbours in S.

Now count the pairs (s, v) with s in S \ N(p), v in N(p), s ~ v. By Corollary 2.2 each s contributes 2, so the number is 2 (20 - 2k). Counting by v it is sum over c of 2 n_c. Hence sum_c n_c = 20 - 2k.

Parity: sum_c n_c = k (mod 2) and 20 - 2k is even, so k is even. Size: sum_c n_c >= 3k, so 20 - 2k >= 3k and k <= 4. Hence k is 0, 2 or 4. If k = 4, then sum_c n_c = 12 = 3k, so n_c = 3 for the four active orbits and n_c = 0 for the three inactive ones; an inactive inner vertex has the neighbours p, its mate (not in S) and 12 words, none of them in S. QED

4.3 Three counting identities [A4]

Lemma 4.2. With the sums taken over all vertices z outside S (p included, with d(p) = k), and with n_d the number of z outside S with d(z) = d:

    (I1)   sum_z C(d(z), 2) = 30 - 7e - Q;
    (I2)   sum_z d(z)   = 100 - 4e;
    (I3)   sum_z C(d(z) - 1, 2) = n_0 + n_3 + 3 n_4 + 6 n_5 + ... = 9 - 3e - Q.

Proof. (I2). A vertex x of S+ has d+(x) neighbours in S+, d+(x) + 4 in S-, hence 10 - 2 d+(x) outside S. Summing over S+, the number of edges between S+ and the outside of S is 100 - 2 * 2e = 100 - 4e, and it is also sum_z d+(z) = sum_z d(z).

(I1). Count the triples ({x, x'}, v) where {x, x'} is a pair of distinct vertices of S+ and v is a common neighbour of x and x'. There are C(10,2) = 45 pairs; the e adjacent ones have 1 common neighbour and the others 2, so the number of triples is 90 - e. Counting by v: a vertex v contributes C(d+(v), 2). For v = x in S+ this is C(d+(x), 2). For v = sigma x in S- it is C(d-(x), 2) = C(d+(x) + 4, 2) by (4.1) and (4.2). For v = z outside S it is C(d(z), 2). Since C(d,2) + C(d+4,2) = d^2 + 3d + 6,

    90 - e = sum_{x in S+} (d+(x)^2 + 3 d+(x) + 6) + sum_z C(d(z),2) = Q + 6e + 60 + sum_z C(d(z),2).

(I3). C(d,2) - (d - 1) = (d-1)(d-2)/2 = C(d-1, 2), which is 1, 0, 0, 1, 3, 6 for d = 0, 1, 2, 3, 4, 5. There are 79 vertices outside S, so (I1) - (I2) + 79 gives sum_z C(d(z)-1, 2) = (30 - 7e - Q) - (100 - 4e) + 79 = 9 - 3e - Q. QED

4.4 e is at most 1

Lemma 4.3. e <= 1. If e = 1, then Q = 2.

Proof. Q = sum d+(x)^2 >= sum d+(x) = 2e. The left side of (I3) is >= 0, so 0 <= 9 - 3e - Q <= 9 - 5e and e <= 1. If e = 1, the two ends of the unique edge inside S+ have d+ = 1 and the other eight vertices have d+ = 0. QED

Remark. The bound obtained from A <= 14 E_0 + 3 (I - E_0), which gave e <= 2 for squared norm 18, gives only e <= 3 here; (I3) is what is used.

4.5 The parity law, and the three cases [A5]

Lemma 4.4. e = C(k,2) (mod 2).

Proof. The complement of S is sigma-invariant and p is its only fixed vertex, so the 78 vertices z != p outside S are paired by sigma, with d(sigma z) = d(z). Hence sum_{z outside S, z != p} C(d(z),2) is even, and by (I1) 30 - 7e - Q = C(k,2) (mod 2). Finally Q = sum d+(x)^2 = sum d+(x) = 2e = 0 (mod 2). So 7e + C(k,2) is even. QED

Corollary 4.5. The pair (k, e) is one of

    (0, 0) (case I),   (2, 1) (case II),   (4, 0) (case III).

Proof. Lemmas 4.1 and 4.3 leave k in {0, 2, 4} and e in {0, 1}; C(0,2) = 0, C(2,2) = 1 and C(4,2) = 6, so by Lemma 4.4 e is even for k = 0 and k = 4, and odd for k = 2. QED

4.6 Case II: where the plus edge is [A7, A8]

In case II there are two inner indices and eight outer ones, and exactly one plus edge {a, b}: d+(x_a) = d+(x_b) = 1 and d+ = 0 on the other eight vertices of S+.

Lemma 4.6. In case II the two ends of the plus edge are not both inner.

Proof. If a and b were both inner, x_a and x_b would be adjacent vertices of N(p). By Lemma 2.1 (b) the only edges inside N(p) are mate edges, so x_b would be the mate of x_a; but the mate of x_a is y_a, which is in S-. QED

Lemma 4.7 [A7]. In case II the plus edge does not join an outer index and an inner index.

Proof. Suppose it does. Name the indices so that the plus edge is {0, c} with 0 outer and c inner; let c' be the other inner index and W' the set of the seven other outer indices. By (4.6): |M_0| = 5, |M_c| = 1 + 4 - 1 = 4, and |M_j| = 4 for j in W'.

- c is not in M_0: otherwise x_0 would be adjacent to x_c and to its mate y_c, which Lemma 2.1 (c) forbids because x_0 != p. By (4.5), 0 is not in M_c.
- c' is not in M_c: x_c and y_c' are two vertices of N(p) that are not mates, so they are not adjacent (Lemma 2.1 (b)).

So M_c is a 4-subset of W' and M_0 is a 5-subset of W' u {c'}, a set of 8 elements. For i in M_0 n M_c the vertex y_i is a common neighbour of the adjacent vertices x_0 and x_c, which have only one. So |M_0 n M_c| <= 1, hence |M_0 u M_c| >= 8, and therefore

    M_0 u M_c = W' u {c'},   M_0 n M_c = {u} for one index u of W',   c' is in M_0.

Put U_0 = M_c \ {u}, a set of three outer indices. Let j be in U_0. Then j is in M_c and not in M_0; by (4.5), c is in M_j and 0 is not in M_j. M_j has 4 elements, none of which is j or 0; so M_j \ {c} is a 3-subset of W' u {c'} = M_0 u M_c.

- x_j and x_c are not adjacent (the only plus edge is {0, c}). y_c is a common neighbour of them (x_c ~ y_c by (4.4), and x_j ~ y_c because c is in M_j), and so is y_i for every i in M_j n M_c; these vertices are distinct (c is not in M_c). Hence 1 + |M_j n M_c| <= 2, i.e. |M_j n M_c| <= 1.
- x_j and x_0 are not adjacent, and y_i is a common neighbour for every i in M_j n M_0. Hence |M_j n M_0| <= 2.
- c is in neither M_0 nor M_c, and M_0 n M_c = {u}, so

      3 = |M_j \ {c}| = |M_j n M_0| + |M_j n M_c| - [u in M_j] <= 2 + 1 - [u in M_j].

  Hence u is not in M_j and |M_j n M_c| = 1. As u is not in M_j and j is not in M_j, the set M_j n M_c is contained in U_0 \ {j}: the index j has exactly one minus-neighbour in U_0 \ {j}.

By (4.5) the relation "i is in M_j" is symmetric. So it defines a graph on the 3-set U_0 in which every vertex has degree exactly 1. A graph has an even number of vertices of odd degree. Contradiction. QED

Lemma 4.8 [A8]. In case II let {a, b} be the plus edge (a, b outer, by Lemmas 4.6 and 4.7). Then x_a ~ y_b and x_b ~ y_a: the four vertices x_a, x_b, y_a, y_b induce the 4-cycle x_a - x_b - y_a - y_b - x_a.

Proof. |M_a| = |M_b| = 5 by (4.6). Suppose b is not in M_a. By (4.5), a is not in M_b. Then M_a and M_b are 5-subsets of the 8 indices other than a and b, so |M_a n M_b| >= 2: two vertices y_i are common neighbours of the adjacent vertices x_a and x_b, which have only one. So b is in M_a and a is in M_b. The edges are x_a x_b, x_b y_a, y_a y_b (the image of x_a x_b) and y_b x_a; x_a not~ y_a and x_b not~ y_b by (4.4). QED

Lemma 4.8 is not needed for what follows (the enumeration of section 4.9 does not assume it and finds it); it is stated because the working reports describe case II by it.

4.7 Case III: the words of the support are the edges of K4 [A9, first part]

Lemma 4.9. In case III let K be the set of the four inner indices.

(a) For c in K, the neighbours of x_c in S are y_c and exactly three vertices y_j with j outer.

(b) For j outer, the two neighbours of x_j in N(p) are y_c and y_c' for two distinct indices c, c' of K. Put kappa(j) = {c, c'}. For c in K and j outer: x_c ~ y_j if and only if c is in kappa(j).

(c) kappa is a bijection from the six outer indices onto the six 2-subsets of K.

(d) The six inactive inner vertices have no neighbour in S.

Proof. (d) is the last statement of Lemma 4.1. (a) By Lemma 4.1, x_c has exactly 3 neighbours in S \ N(p). By the proof of Lemma 4.1 the sum of phi over them is -3 phi(x_c) = -3, so all three are in S-. Inside N(p) the only neighbour of x_c is its mate y_c. (b) x_j is a word: it has exactly two neighbours in N(p), in two different orbits (Corollaries 2.2, 2.3). By (d) they are active inner vertices, i.e. vertices x_c or y_c with c in K. As e = 0, x_j has no neighbour in S+. So they are y_c and y_c' with c != c'. Applying sigma, y_j ~ x_c if and only if x_j ~ y_c.
(c) By (a) every c of K lies in kappa(j) for exactly three outer j. Let c != c' be in K. The vertices x_c and x_c' are in N(p), they are not mates, so they are non-adjacent and have exactly two common neighbours, one of which is p. For every j with kappa(j) = {c, c'} the vertex y_j is a common neighbour. So at most one j has kappa(j) = {c, c'}. There are six outer indices and six 2-subsets of K. QED

4.8 Case I: the orbit graph B [A10, first part]

Lemma 4.10. In case I (k = 0, e = 0):

(a) S+ and S- are independent sets;
(b) no vertex of S is in N[p]; in particular d(p) = 0;
(c) x_i not~ y_i for every i;
(d) every vertex of S has exactly 4 neighbours in S, and they are on the other side: d-(x_i) = 4, d+(x_i) = 0.

Define a graph B on the index set {0, ..., 9} by: i ~_B j if and only if x_i ~ y_j. Then B is a well-defined simple graph, it is 4-regular, and any two distinct indices i, j have at most 2 common neighbours in B. For i != j the number of common neighbours of x_i and x_j that lie in S is exactly the codegree codeg_B(i,j) = |B(i) n B(j)|, where B(i) is the set of B-neighbours of i. For all i, l (i = l allowed), x_i and y_l have no common neighbour in S.

Proof. (a) e = 0, and S- = sigma S+. (b) k = 0 says that no vertex of S+ is in N(p); apply sigma for S-; p is not in S. (c) is (4.4) with all indices outer. (d) is (4.2) with d+ = 0. The relation ~_B is symmetric by (4.5), and there is no loop by (c). The B-neighbours of i are the indices j with y_j in N(x_i), and d-(x_i) = 4, so B is 4-regular. Let i != j. The vertices x_i and x_j are distinct and non-adjacent, so they have exactly 2 common neighbours in G. Their common neighbours inside S are in S- (S+ is independent), and y_l is a common neighbour exactly when l is in B(i) n B(j). So codeg_B(i,j) <= 2. Finally a common neighbour of x_i and y_l inside S would be in S+ and adjacent to x_i, or in S- and adjacent to y_l; both are excluded by (a). QED

4.9 Support structures and their enumeration [round 1, B1, B2]

Definition. Let K be a subset of {0, ..., 9} and P a set of 2-subsets of {0, ..., 9}. A **support structure of type (K, P)** is a graph on the 21 vertices x_0, ..., x_9, y_0, ..., y_9, p such that:

- (S1) it is invariant under the involution that exchanges x_i and y_i for every i and fixes p;
- (S2) p ~ x_i and p ~ y_i and x_i ~ y_i if i is in K; none of the three if i is not in K;
- (S3) for i != j: x_i ~ x_j if and only if {i, j} is in P;
- (S4) (the eigen-equation) for every i: (number of j != i with x_i ~ y_j) + [i in K] = (number of j with {i, j} in P) + 4;
- (S5) (the budgets) two adjacent vertices have at most 1 common neighbour among the 21 vertices, and two distinct non-adjacent vertices at most 2.

By (S1) the adjacencies between two orbits i != j are given by two bits: "minus" (x_i ~ y_j and x_j ~ y_i) and "plus" (x_i ~ x_j and y_i ~ y_j). So a support structure of type (K, P) is the same thing as a set of minus edges. Two support structures are **isomorphic** if one is obtained from the other by a permutation of the ten indices (applied to the x_i and to the y_i together); such a permutation carries the type (K, P) to its image.

Lemma 4.11. In a configuration, with any labelling S+ = {x_0, ..., x_9}, y_i = sigma x_i, the induced subgraph of G on S u {p} is a support structure of type (K, P), where K is the set of inner indices and P the set of plus edges. Moreover, after a suitable relabelling of the ten orbits of the support, the type is one of

```
case I:    K empty,          P empty;
case IIa:  K = {0, 1},       P = {{2, 3}};
case III:  K = {0, 1, 2, 3}, P empty.
```

Proof. (S1): sigma is an automorphism exchanging x_i and y_i and fixing p. (S2): p ~ x_i iff x_i is in N(p) iff i is in K; p ~ y_i likewise by sigma; x_i ~ y_i iff i is in K by (4.4). (S3) is the definition of P. (S4) is (4.6). (S5): a common neighbour inside S u {p} is a common neighbour in G, and G has lambda = 1 and mu = 2. For the types: by Corollary 4.5 the pair (k, e) is (0,0), (2,1) or (4,0). In case II the plus edge joins two outer indices (Lemmas 4.6, 4.7). Relabel so that the inner indices come first and, in case II, the plus edge is {2, 3}. QED

The type K = {0, 1}, P = {{0, 2}} (plus edge between an inner and an outer index) is called IIb in the working reports; by Lemma 4.7 it does not occur, and the programs confirm that it has no support structure.

> **Computation 4.12.** The numbers of labelled support structures of the following types are:
>
> | type | labelled support structures |
> |---|---|
> | I, with the additional normalisation "the minus-neighbours of the index 0 are 1, 2, 3, 4" | 27720 |
> | IIa: K = {0,1}, P = {{2,3}} | 2880 |
> | IIb: K = {0,1}, P = {{0,2}} | 0 |
> | III: K = {0,1,2,3}, P empty | 10080 |
>

> Up to isomorphism there are exactly six support structures of type I, one of type IIa and two of type III. They
> are listed in section 4.10 under the names I.0, ..., I.5, IIa.0, III.0, III.1.

Why the list is complete, given the counts. Let G_0 be the group of the permutations of the ten indices that preserve the type: all 10! permutations for type I, 2 * 2 * 6! = 2880 for IIa (exchange of the two inner indices, exchange of 2 and 3, permutations of 4..9) and 4! * 6! = 17280 for III. The labelled structures of a type that are isomorphic to a given one, with automorphism group of order a inside G_0, are |G_0| / a in number; for type I with the normalisation of the table they are 10! / (126 a), because the C(9,4) = 126 possible sets of minus-neighbours of the index 0 are permuted transitively by the permutations fixing 0 (each class is 4-regular at 0). With the orders a of section 4.10:

    type I: 10!/126 * (1/16 + 1/4 + 1/10 + 1/4 + 1/4 + 1/20) = 1800 + 7200 + 2880 + 7200 + 7200 + 1440 = 27720;
    type IIa: 2880 / 1 = 2880;
    type III: 17280 * (1/3 + 1/4) = 5760 + 4320 = 10080.

The nine graphs of section 4.10 are support structures of their types, they are pairwise non-isomorphic (the six of type I have six different tables of codegrees; III.0 and III.1 differ by the graph on the outer indices), and their classes already account for the totals of the table. So there is no other class. This argument uses, as computed data, the four totals and the nine orders a.

Programs (two, written separately):

- ‘lemma/t20/t20_enum.py‘ with ‘t20_kernel.py‘ and ‘t20_common.py‘ (enumerator 1): backtracking over the 45 pairs of orbits with an incremental test of (S5) and degree pruning for (S4); every structure found is re-checked; the structures are sorted into isomorphism classes and the automorphism orders computed. Results: 27720 (1,841,249 search nodes), 2880 (6,414,667), 0 (1,463,867), 10080 (469,755) labelled structures; 6, 1, 0, 2 classes, with the orders 16, 4, 10, 4, 4, 20; 1; 3, 4; the count |G_0| / a is checked for every class (‘enum_I.out‘, ‘enum_III_IIb_IIa.out‘).
- ‘lemma/t20/t20_verify.py‘ (enumerator 2; no shared code: pure Python, the pairs in the reverse order, the inner orbits on the highest labels, for type I the minus-neighbours of the index 9 fixed instead of those of 0; every structure found is re-tested from scratch with A @ A and A phi = -4 phi). Results: 27720, 2880, 0, 10080 labelled structures (‘verify_III_I.out‘, ‘verify_IIa.out‘, ‘verify_II.out‘, ‘verify_IIb.out‘). It has **no isomorphism test**: it sorts the labelled structures by an invariant (traces of powers of a weighted orbit matrix) and finds 6, 1, 2 distinct invariants with the multiplicities 1800 / 2880 / 7200 / 7200 / 7200 / 1440, 2880, and 5760 / 4320, equal to the numbers |G_0| / a predicted from enumerator 1.

So the four totals come from two programs; the nine automorphism orders come from enumerator 1 only. (For this text they were recomputed, together with the conditions (S1)-(S5), the codegrees and the units of section 4.10, by the transcription script ‘lemma/t20_PROOF/_check_text_data.py‘; it is a check of the printed data, not a third enumeration.) Two details of the runs: for type IIa the verifier was run with the plus edge on the labels 6, 7 (3,208,774 nodes); its run with the plus edge on the labels 0, 1 was cut by its time limit and gave nothing. For type IIb it was run twice (plus edge on the labels 0, 9: 10,596,116 nodes; on 7, 9: 731,934 nodes), with 0 structures both times. The number of labelled structures of a type does not depend on which labels carry the plus edge.

Corollary 4.13 (hand statements + Computation 4.12). In a configuration, after a suitable labelling of the ten orbits of the support, the induced subgraph of G on S u {p} is one of the nine graphs I.0, ..., I.5, IIa.0, III.0, III.1 of section 4.10. Once this is arranged, the labellings that give exactly the same graph differ by the automorphisms of the structure.

Proof. Lemma 4.11 and Computation 4.12. QED

4.10 The nine configurations [round 1, section C]

The vertices are x_0..x_9, y_0..y_9, p. A minus edge "ij" means x_i ~ y_j and x_j ~ y_i. An inner index c means x_c ~ y_c, p ~ x_c, p ~ y_c. Nothing else is adjacent in S u {p}, except the plus edge of IIa.0.

Case I: no inner index; the minus edges are the edges of B (Lemma 4.10).

	edges of B	automorphisms	triangles	codegrees of adjacent pairs	codegrees of non-adjacent pairs
I.0	01 02 03 04 17 18 19 25 26 29 34 36 38 45 47 58 59 67 68 79	16	4	8 pairs with 0, 12 with 1	1 with 0, 24 with 2
I.1	01 02 03 04 17 18 19 25 26 29 34 36 38 45 47 57 59 68 69 78	4	6	3 with 0, 16 with 1, 1 with 2	8 with 1, 17 with 2
I.2	01 02 03 04 17 18 19 25 26 29 34 36 38 45 47 57 59 67 68 89	10	5	5 with 0, 15 with 1	5 with 1, 20 with 2
I.3	01 02 03 04 17 18 19 25 26 29 34 36 38 45 47 56 59 68 78 79	4	6	4 with 0, 14 with 1, 2 with 2	1 with 0, 6 with 1, 18 with 2
I.4	01 02 03 04 17 18 19 24 26 29 34 35 38 46 56 57 58 69 78 79	4	8	2 with 0, 12 with 1, 6 with 2	3 with 0, 8 with 1, 14 with 2
I.5	01 02 03 04 14 18 19 23 26 27 34 37 49 56 57 58 59 67 68 89	20	10	10 with 1, 10 with 2	5 with 0, 10 with 1, 10 with 2

Characteristic polynomials of B (identification only; not used):

    I.0: (t-4)(t-1)^2(t+1)(t^2+t-4)(t^2+2t-1)^2
    I.1: (t-4)(t+2)(t^2-2)(t^3-4t+2)(t^3+2t^2-2t-2)
    I.2: t(t-4)(t^4+2t^3-4t^2-5t+5)^2
    I.3: (t-4)(t-1)(t+1)^2(t^2-3)(t^2+t-4)(t^2+2t-1)
    I.4: (t-4)(t-1)(t+1)^4(t^2-5)(t^2+t-4)
    I.5: t(t-4)(t+1)^4(t^2-5)^2

According to ‘names.out‘, I.5 is isomorphic to the circulant graph C10(1,2) and the five others are none of the 4-valent circulants on 10 vertices (not used).

Case II: inner indices 0, 1; plus edge {2, 3}.

| | minus edges | plus edge | automorphisms |

|---|---|---|---|
| IIa.0 | 07 08 09 13 15 16 23 24 26 28 29 35 37 39 45 46 48 57 68 79 | 23 | 1 |

In IIa.0 the minus edge 23 is present, as Lemma 4.8 says; the common neighbour of x_2 and x_3 is y_9.

Case III: inner indices 0, 1, 2, 3; the outer indices 4..9 are the 2-subsets of {0,1,2,3} (Lemma 4.9): kappa(4) = {2,3}, kappa(5) = {1,3}, kappa(6) = {1,2}, kappa(7) = {0,3}, kappa(8) = {0,2}, kappa(9) = {0,1}.

	minus edges	minus edges between outer indices	automorphisms
III.0	07 08 09 15 16 19 24 26 28 34 35 37 48 49 57 58 67 69	the hexagon 4-8-5-7-6-9	3
III.1	07 08 09 15 16 19 24 26 28 34 35 37 48 49 56 57 67 89	the two triangles 4-8-9 and 5-6-7	4

Units. For i < j let

    need(i, j) = 2 - [x_i ~ x_j] - (the number of common neighbours of x_i and x_j in S u {p}). (4.7)

It is the number of common neighbours of x_i and x_j outside S u {p}; a pair with need > 0 is called a **deficient pair** or a **unit**. For case I, need(i,j) = 2 - codeg_B(i,j) (Lemma 4.10). The units of the nine structures (‘deficiency.out‘; recomputed by ‘_check_text_data.py‘):

	total	pairs with need = 2	pairs with need = 1
I.0	30	01 02 18 26 39 45 47 58 67	03 04 17 19 25 29 34 36 38 59 68 79
I.1	30	01 02 19	03 04 17 18 25 26 27 28 34 35 36 38 39 45 46 47 49 57 58 59 67 68 69 78
I.2	30	01 02 17 26 67	03 04 18 19 25 29 34 35 36 38 39 45 47 48 49 57 58 59 68 89
I.3	30	01 02 39 45 47	03 04 15 16 18 19 26 27 28 29 34 36 38 56 58 59 67 68 78 79
I.4	30	01 28 39 47 56	02 03 05 07 14 16 18 19 25 27 29 34 35 38 46 48 57 68 69 79
I.5	30	05 17 29 38 46	01 02 06 08 12 16 18 26 28 34 35 37 39 45 47 49 57 59 68 79
IIa.0	20	45 67	01 04 05 06 08 16 18 19 29 38 47 49 57 58 69 78
III.0	24	49 58 67	07 08 09 14 15 18 25 26 27 34 36 39 47 48 57 59 68 69
III.1	24	58	05 06 07 14 18 19 25 26 27 34 38 39 45 46 47 49 59 67 68 69 78 79

(The totals are forced: for case I, the sum over the 45 pairs of 2 - codeg_B is 90 - 10 * C(4,2) = 30.)

Remark (not used). On S u {p}, for each of the nine structures, the matrix M1 of section 6.1 is positive definite and M2 is positive semidefinite of rank 20, exactly (‘enum_I.out‘, ‘enum_III_IIb_IIa.out‘). So the two PSD tests exclude nothing at the level of the support.

5. The vertices outside the support

From here on a configuration is in one of the three cases of Corollary 4.5, and e <= 1.

5.1 The classes X_d

For d >= 0 let X_d be the set of vertices z outside S with d(z) = d, i.e. with exactly d neighbours in S+ and d in S- (see (4.3)), and n_d = |X_d|. Each X_d is sigma-invariant, because d(sigma z) = d(z). p is in X_k. For a vertex v and an integer j let

    a_j(v) = |N(v) n X_j|.

A vertex of S is called a **plus end** if it is an end of an edge inside S+ or of an edge inside S-. There is no plus end in cases I and III, and there are four in case II (x_a, x_b, y_a, y_b for the plus edge {a, b}). For a vertex z let q(z) be the number of neighbours of z that are plus ends. A vertex w of S has exactly

    4 + 2 [w is a plus end] (5.1)

neighbours in S: for w = x in S+ this is d+(x) + d-(x) = 2 d+(x) + 4 by (4.2), and d+(x) is 1 for a plus end and 0 otherwise (e <= 1); for w in S- apply sigma.

5.2 The path-count identity [G0; round 1, A6]

Lemma 5.1. Let z be outside S with d(z) = d. Then

    sum over the neighbours w of z outside S of d(w) = 20 - 5d - q(z), (5.2)

i.e. a_1(z) + 2 a_2(z) + 3 a_3(z) + 4 a_4(z) + ... = 20 - 5d - q(z), and

    a_0(z) - a_2(z) - 2 a_3(z) - 3 a_4(z) - ... = 3d + q(z) - 6. (5.3)

Proof. Count the pairs (w, s) with s in S, w ~ z and w ~ s. For a fixed s (s != z since z is outside S) the number of w is the number of common neighbours of z and s: 1 if s ~ z, 2 otherwise. z has 2d neighbours in S, so the count is 2d + 2 (20 - 2d) = 40 - 2d. For a fixed neighbour w of z the number of s is the number of neighbours of w in S: 4 + 2 [w is a plus end] if w is in S, by (5.1), and d+(w) + d-(w) = 2 d(w) if w is outside S. So

    40 - 2d = 4 * 2d + 2 q(z) + 2 sum_{w ~ z, w outside S} d(w),

which is (5.2). The degree of z is 14 = 2d + a_0 + a_1 + a_2 + ...; subtracting this from (5.2) gives sum_j (j - 1) a_j = 6 - 3d - q(z), which is (5.3). QED

Corollary 5.2. Every vertex outside S has d <= 4. A vertex z outside S with d(z) = 4 has q(z) = 0 and its six neighbours outside S are in X_0.

Proof. The left side of (5.2) is >= 0, so 5d <= 20. If d = 4 the equation reads

0 = q(z) + (the sum of d(w) over the neighbours w of z outside S), a sum of two non-negative terms: so q(z) = 0 and every neighbour of z outside S has d = 0; there are 14 - 8 = 6 of them. QED

5.3 The sizes of the classes

Lemma 5.3.

(I) In case I: X_4 is empty, p is in X_0, n_3 is even, and

$$n_0 = 9 - n_3, \quad n_1 = 40 + 3\,n_3, \quad n_2 = 30 - 3\,n_3.$$

(II) In case II: X_4 is empty, p is in X_2, and

$$n_0 = 4 - n_3, \quad n_1 = 54 + 3\,n_3, \quad n_2 = 21 - 3\,n_3.$$

(III) In case III: X_4 = {p}, X_3 is empty, X_0 is the set of the six inactive inner vertices, n_2 = 24 and n_1 = 48. In particular every vertex of X_1 u X_2 is a word.

Proof. By Corollary 5.2, n_d = 0 for d >= 5, so (I3) reads n_0 + n_3 + 3 n_4 = 9 - 3e - Q. A class X_d that does not contain p has an even number of elements (it is sigma-invariant and p is the only fixed vertex).

(I) e = 0, Q = 0: n_0 + n_3 + 3 n_4 = 9. p is in X_0 (k = 0), so n_3 and n_4 are even. If n_4 >= 2, then n_0 <= 3; but a vertex of X_4 has six neighbours in X_0 (Corollary 5.2). So n_4 = 0 and n_0 = 9 - n_3. Then n_0 + n_1 + n_2 + n_3 = 79 and, by (I2), n_1 + 2 n_2 + 3 n_3 = 100; solving, n_2 = 30 - 3 n_3 and n_1 = 40 + 3 n_3.

(II) e = 1, Q = 2 (Lemma 4.3): n_0 + n_3 + 3 n_4 = 4. p is in X_2 (k = 2), so n_4 is even, hence n_4 = 0 and n_0 = 4 - n_3. By (I1), n_2 + 3 n_3 = 30 - 7 - 2 = 21. Then n_1 = 79 - n_0 - n_2 - n_3 = 54 + 3 n_3. (Check with (I2): n_1 + 2 n_2 + 3 n_3 = 96 = 100 - 4.)

(III) e = 0, Q = 0: n_0 + n_3 + 3 n_4 = 9. p is in X_4 (k = 4), so n_4 >= 1. The six inactive inner vertices are outside S and have no neighbour in S (Lemma 4.1), so they are in X_0 and n_0 >= 6. Hence n_3 + 3 n_4 <= 3: n_4 = 1, n_3 = 0, n_0 = 6. So X_4 = {p} and X_0 is the set of the six inactive inner vertices. Then n_1 + n_2 = 79 - 6 - 1 = 72 and, by (I2), n_1 + 2 n_2 + 4 = 100; so n_2 = 24 and n_1 = 48. Finally N(p) consists of the eight active inner vertices, which are in S, and of X_0; so no vertex of X_1 u X_2 is in N(p), and none is p. QED

5.4 A bound on the neighbours in X_0 [G1]

Lemma 5.4. Let z be a vertex outside S, z != p, and T = N(z) n X_0.

(a) If z is a word, then p is not in T, T u sigma T is contained in X_0 \ {p}, and

$$2\,|T| - 2 \le |X_0 \setminus \{p\}|.$$

(b) If z is in N(p), then 2 |T| - 1 <= |X_0|.

Proof. sigma T = N(sigma z) n X_0, because X_0 is sigma-invariant. T n sigma T is contained in N(z) n N(sigma z). (a) z not~ p, so p is in neither T nor sigma T. z and sigma z are distinct and non-adjacent (Lemma 2.1 (a)), so |N(z) n N(sigma z)| = 2 and |T u sigma T| >= 2 |T| - 2. (b) N(z) n N(sigma z) = {p} by Lemma 2.1 (c), so |T u sigma T| >= 2 |T| - 1, and T u sigma T is contained in X_0. QED

5.5 Types, and the units [round 1, section C; C2]

For a vertex s outside S, the **type** of s is the pair (U(s) ; V(s)), where

$$U(s) = \{i : s \sim x_i\}, \qquad V(s) = \{l : s \sim y_l\}; \qquad |U(s)| = |V(s)| = d(s).$$

Applying sigma: U(sigma s) = V(s) and V(sigma s) = U(s).

Lemma 5.5. Let s be a vertex outside S, s != p.

(a) U(s) n V(s) contains no inner index. If s is in N(p), then U(s) n V(s) is empty. If s is a word, then |U(s) n V(s)| <= 1. In particular U(s) != V(s) when d(s) >= 2.

(b) Suppose that every vertex outside S u {p} has d <= 2. Then for every pair {i, j} of distinct indices, the number of vertices s of X_2 \ {p} with U(s) = {i, j} is exactly need(i, j) (see (4.7)); every vertex of X_2 \ {p} is counted for exactly one pair; and the sum of need over all pairs is |X_2 \ {p}|.

Proof. (a) Let i be in U(s) n V(s): s ~ x_i and s ~ y_i. If i is inner, s is adjacent to both ends of a mate edge of N(p) and s != p, against Lemma 2.1 (c). So i is outer. Applying sigma, sigma s ~ y_i and sigma s ~ x_i: x_i and y_i are common neighbours of s and sigma s. If s is in N(p), the only common neighbour of s and sigma s is p (Lemma 2.1 (c)): impossible, so U(s) n V(s) is empty. If s is a word, N(s) n N(sigma s) has exactly two elements, so at most one index i has {x_i, y_i} contained in it. If d(s) >= 2 and U(s) = V(s), then |U(s) n V(s)| >= 2.

(b) The vertices x_i and x_j have exactly 2 - [x_i ~ x_j] common neighbours, so exactly need(i, j) of them are outside S u {p}. A vertex outside S u {p} adjacent to x_i and x_j has d >= 2, hence d = 2 by the hypothesis: it is in X_2 \ {p} and its neighbours in S+ are exactly x_i and x_j. Conversely a vertex s of X_2 \ {p} has exactly two neighbours in S+ and is a common neighbour of them outside S u {p}. QED

The hypothesis of (b) holds in case III (Lemma 5.3), in case II (Lemma 8.3 below) and in case I when n_3 = 0.

6. The tests used by the computations

A computation of sections 7, 8 and 9 enumerates the possible induced subgraphs of G on certain vertex sets and

discards those that violate a necessary condition. Three kinds of necessary conditions are used: budgets, the two PSD matrices, and (at a few nodes, section 10.3) the rank bound. **Both PSD matrices of 6.1 are used.**

6.1 Two positive-semidefinite matrices

$$M1 = 9\,(3I - A) + J = 27\,I - 9\,A + J, \qquad M2 = 11\,(4I + A) - 2J = 44\,I + 11\,A - 2\,J.$$

Lemma 6.1. M1 = 63 E_{-4} and M2 = 77 E_3. Both are PSD, M1 has rank 44 and M2 has rank 54. For every vertex set Y the principal submatrices M1[Y] and M2[Y] are PSD.

Proof. (Identical to T18_PROOF, Lemma 6.1.) M1 and M2 are polynomials in A and J, so they act as scalars on the three eigenspaces. On the all-ones vector (A = 14, J = 99): M1 = 27 - 126 + 99 = 0 and M2 = 44 + 154 - 198 = 0. On V_3 (A = 3, J = 0): M1 = 27 - 27 = 0 and M2 = 44 + 33 = 77. On V_{-4} (A = -4, J = 0): M1 = 27 + 36 = 63 and M2 = 44 - 44 = 0. A principal submatrix of a PSD matrix is PSD. QED

M1[Y] and M2[Y] depend only on the induced subgraph of G on Y: with A_Y its adjacency matrix, M1[Y] = 27 I - 9 A_Y + J and M2[Y] = 44 I + 11 A_Y - 2J (I and J of size |Y|).

A **PSD violation** on Y is an integer vector x indexed by Y with x^T M1[Y] x < 0 or x^T M2[Y] x < 0. Checking a given violation is one integer computation. Since M1[Y] and M2[Y] have integer entries, a PSD violation on Y exists if and only if M1[Y] or M2[Y] is not PSD (a real vector with negative quadratic form can be replaced by a rational vector close to it, and then multiplied by a common denominator). So "there is no PSD violation on Y" and "M1[Y] and M2[Y] are both PSD" are the same statement about the graph on Y, and it does not depend on any program.

6.2 The common-neighbour budget

Lemma 6.2. Let Y be a vertex set and let H' be a graph on Y all of whose edges are edges of G (a spanning subgraph of the induced subgraph G[Y]). Then for any two distinct vertices u, v of Y,

$$|N_{H'}(u) \cap N_{H'}(v)| \le 2 - [u \sim v \text{ in } H'].$$

Proof. (Identical to T18_PROOF, Lemma 6.2.) N_H'(u) n N_H'(v) is contained in N(u) n N(v), which has 1 element if u ~ v in G and 2 otherwise. If u ~ v in H' then u ~ v in G and the bound is 1. If u not~ v in H' the bound 2 holds in both cases. QED

A pair u, v violating the inequality is a **budget violation**. The point of allowing a spanning subgraph is that a partially specified graph (some adjacencies not yet decided and provisionally absent) may be rejected by a budget violation: adding edges can only increase the numbers of common neighbours and decrease the bound. A PSD violation, on the contrary, is meaningful only on a vertex set all of whose adjacencies are decided.

6.3 Determined sets, and what a computation proves

Let D be some data extracted from a configuration (for instance the types of the vertices of N(p)). A vertex set Y (described in terms of the configuration, e.g. "S u N[p]") is **determined by D** if the induced subgraph G[Y], with the names of its vertices, can be written down from D alone. For every determined set used below, the text gives a table that says, for every pair of classes of vertices of Y, whether the adjacency is a parameter in D or is forced, and by which lemma.

The computations have the following form. A finite list of candidate data D is defined: all values of the parameters that satisfy the forced numbers named in the text. For a candidate D, write G_D for the graph on Y built from D.

Definition (survivor). A candidate D is a **survivor** if G_D has no budget violation and the matrices M1[Y] and M2[Y] of G_D are both PSD. This is a property of D alone: it does not refer to a program, to the order of the tests or to a numerical threshold. The data of a configuration satisfy the forced numbers, which are proved in the text, and pass both tests by Lemmas 6.1 and 6.2. Hence:

$$\text{the data D of any configuration are a survivor.} \qquad (6.1)$$

What a program does. A program **rejects** a candidate D only when it has an exact witness that D cannot be the data of a configuration. There are four kinds of witnesses:

- a budget violation on G_D, or on a partial graph, i.e. a graph on a subset of Y all of whose edges are edges of G_D (section 6.2);
- a PSD violation on Y, or on a subset of Y all of whose adjacencies are decided;
- a forced number proved in the text that fails (in the searches below these are upper bounds, "caps", on numbers of neighbours, and the multiplicities of the units);
- a violation of the rank bound of section 6.4 on a sigma-invariant subset of Y all of whose adjacencies are decided.

The candidates that a program does not reject form its **kept list**. The data of a configuration admit no witness of any of the four kinds: a principal submatrix of a PSD matrix is PSD (Lemma 6.1), a graph all of whose edges are edges of G has no budget violation (Lemma 6.2), the forced numbers are proved, and so is the rank bound (Proposition 6.4). Hence, for every program whose rejections are exact:

$$\text{the data D of any configuration belong to the kept list.} \qquad (6.2)$$

(6.2) rests on exact rejections only, and it is all that the exclusions of this text use: every exclusion below has the form "the kept list of the last step is empty".

Exactness of the rejections. According to their sources (for the parts read for this text; see section 10.4, item 2) and their headers, in all the enumeration programs a rejection is exact: a budget violation is an integer count (the fast engines select with floating-point products of 0/1 matrices and recount every budget rejection in 64-bit integers); a PSD violation is an integer vector whose quadratic form is evaluated in Python integers, or, when no such vector is found, an exact rational elimination; a rank is an exact integer rank (python-flint). The

quadratic form is often evaluated on a "sigma-block": for x = sum c_v (e_v - e_{sigma v}) one has
x^T M1 x = 18 c^T G_odd c with the matrix G_odd of Proposition 6.4, and x^T M2 x = 22 c^T (7I - G_odd) c (because
M1 + (9/11) M2 = 63 I - (7/11) J and J x = 0); for x = sum c_v (e_v + e_{sigma v}) + c_p e_p the form is that of
the "even block". A block vector with negative form is a PSD violation on the vertices.

"Kept" decisions. A candidate for which no violation is found is kept; this "kept" rests on a floating-point
eigenvalue or Schur complement (tolerances of about 10^-7 to 10^-9) and **is not a proof of positive semidefiniteness
and is never used as one**: keeping too much can only make the kept list larger, which is harmless by (6.2).

Consequently, in this text:

- a number of survivors would be a property of the mathematics; **no list of this text has been verified to consist
  of survivors exactly**, except the nine support structures on S u {p} (remark at the end of section 4.10) and,
  according to round 1, the two base graphs of case III on S u N[p] (both matrices exactly PSD, of rank 26 and 25);
- every count of "bases", "nodes", "kept" or "candidates" printed in sections 7, 8, 9 is a count of one named
  program. Counts of nodes depend, in addition, on the rule by which the program chooses the next unit (section
  6.5). Two programs can be compared on such counts only if they treat the same units in the same order; this is
  how the second programs are built (section 0.1).

6.4 The rank bound [C9; T18_PROOF 6.4]

Let P be the permutation matrix of sigma (P e_x = e_{sigma x}). P is symmetric, P^2 = I, and P commutes with A
(sigma is an automorphism) and with J.

Lemma 6.3. trace(P E_{-4}) = 0 and trace(P E_3) = 0. Hence sigma acts on V_{-4} with the eigenvalues +1 and -1,
each with multiplicity 22, and on V_3 with the eigenvalues +1 and -1, each with multiplicity 27.

Proof. (Identical to T18_PROOF, Lemma 6.3.) trace P is the number of fixed vertices, 1. trace(PA) =
sum_x A_{x, sigma x} is the number of vertices x with x ~ sigma x, which is |N(p)| = 14 by Lemma 2.1 (a).
trace(PJ) = 99. By Lemma 6.1,

    trace(P E_{-4}) = (27 * 1 - 9 * 14 + 99) / 63 = 0,   trace(P E_3) = (44 * 1 + 11 * 14 - 2 * 99) / 77 = 0.

P preserves V_{-4} and V_3 and has eigenvalues +-1 there; the trace is the difference of the two multiplicities,
and their sums are 44 and 54. QED

Proposition 6.4 (rank bound). Let Y be a sigma-invariant vertex set, and let R be a set containing exactly one
vertex of each orbit {x, sigma x} contained in Y (x != p).

(a) (odd part) The |R| x |R| integer matrix

        G_odd = 3 I - A[R,R] + A[R, sigma R]   (entry (x,y): 3 [x = y] - [x ~ y] + [x ~ sigma y])

    has rank at most 22. In particular, if |R| >= 23 then every 23 x 23 principal minor of G_odd is 0.

(b) (even part) Let the vectors g_x = e_x + e_{sigma x} (x in R), together with g_p = e_p if p is in Y, be the
    "even generators". The matrix G_even with entries g^T M1 g' has rank at most 22. Explicitly, for x != y in R:
    G_even(x,y) = 4 - 18 ([x ~ y] + [x ~ sigma y]), G_even(x,x) = 58 - 18 [x ~ sigma x],
    G_even(p,x) = 2 - 18 [p ~ x], G_even(p,p) = 28.

Proof. (Identical to T18_PROOF, Proposition 6.4.) (a) For x in R put u_x = e_x - e_{sigma x}. Since M1 commutes
with P, M1(sigma x, sigma y) = M1(x, y) and M1(sigma x, y) = M1(x, sigma y), so

    u_x^T M1 u_y = 2 (M1(x,y) - M1(x, sigma y)) = 2 (27 [x = y] - 9 [x ~ y] + 9 [x ~ sigma y]) = 18 G_odd(x,y)

(the terms +1 coming from J cancel, and x = sigma y does not happen for x, y in R). On the other hand
M1 = 63 E_{-4} and E_{-4} is a symmetric idempotent, so u_x^T M1 u_y = 63 (E_{-4} u_x) . (E_{-4} u_y): the matrix
18 G_odd is 63 times the Gram matrix of the vectors E_{-4} u_x. These vectors lie in V_{-4} and satisfy
P E_{-4} u_x = E_{-4} P u_x = - E_{-4} u_x, so they lie in the (-1)-eigenspace of sigma on V_{-4}, which has
dimension 22 by Lemma 6.3. A Gram matrix of vectors in a space of dimension 22 has rank at most 22.

(b) The same computation with g_x: g^T M1 g' = 63 (E_{-4} g) . (E_{-4} g'), and the vectors E_{-4} g lie in the
(+1)-eigenspace of sigma on V_{-4}, of dimension 22. The explicit entries are 2 (M1(x,y) + M1(x, sigma y)),
M1(p,x) + M1(p, sigma x) and M1(p,p). QED

G_odd and G_even depend only on the induced subgraph on Y and on sigma. So for a sigma-invariant determined set Y,
"rank G_odd > 22" or "rank G_even > 22" is a contradiction, checked by one exact integer rank. Positive
semidefiniteness does not see this condition: G_odd can be positive definite. The proposition is a general fact
about G and sigma. It is stated and proved in T18_PROOF as well, but it does not depend on the theorem T18.

6.5 The search over units [round 2, section B; SPEC_N3, section 4]

All the enumerations of sections 7, 8 and 9 have the same form. It is described here once, with the proof that it
is exhaustive.

Setting. In a configuration let

- Y_0 be a sigma-invariant vertex set containing S u {p} (the **base**);
- F be a sigma-invariant set of vertices outside Y_0, each of which has exactly two neighbours in S+ (the **free
  vertices**); F is a union of n orbits, since p is in Y_0;
- for every pair {i, j} of indices, m(i, j) be the number of vertices f of F with U(f) = {i, j}; these numbers are
  assumed to be known from the graph on Y_0 (they are the multiplicities of the units, minus the units realised
  inside Y_0).

A **state of level t** is a labelled graph with an involution, on |Y_0| + 2t vertices: the graph on Y_0, and t

further orbits {f, sigma f}, each with the adjacency of f to all the earlier vertices (that of sigma f follows by the involution). The adjacency between two different added orbits {f, sigma f} and {f', sigma f'} is invariant under the involution, so it is one of four **patterns**: the two bits [f ~ f'] (then sigma f ~ sigma f') and [f ~ sigma f'] (then sigma f ~ f'). A state is **true** if it is the induced subgraph of G on Y_0 and t orbits of F, with the involution sigma.

A **model** is a finite list of conditions on a state that are proved to hold for every true state (for instance: "f is adjacent to exactly two vertices of N(p), in two different pairs", "V(f) != U(f)", "f has at most 5 neighbours among the other added orbits").

A **search** is a tree of states. Its root is the graph on Y_0. Let N be a node of level t < n. A unit {i, j} is chosen, by any rule, among the pairs for which fewer than m(i, j) of the 2t added vertices have U = {i, j}. The **candidates** of N are all the states of level t + 1 obtained from N by adding one orbit {f, sigma f} with U(f) = {i, j} and satisfying the model. The children of N are the candidates that are not rejected, a rejection being an exact witness in the sense of section 6.3.

Lemma 6.5. Suppose that the graph on Y_0 of a configuration is the root of a search. Then, whatever the rule that chooses the units, the search has a true node at every level t <= n. In particular it has a node of level n.

Proof. By induction on t. The root is true. Let N be a true node of level t < n. Such a pair {i, j} exists: the 2t added vertices are fewer than the 2n = sum of m(i, j) vertices of F. Let {i, j} be the unit chosen at N. Exactly m(i, j) vertices of F have U = {i, j}, and fewer than m(i, j) of them are among the added vertices of N; so some vertex f of F with U(f) = {i, j} is not in N, and neither is sigma f. The induced subgraph of G on the vertices of N together with f and sigma f is a state of level t + 1; it is true, so it satisfies the model, and it is a candidate of N. It admits no exact witness (section 6.3). So it is a child of N. QED

Three remarks.

- *The rule is free.* The first programs choose the unit by a floating-point look-ahead (the unit with the fewest candidates that seem to survive). Nothing is concluded from this look-ahead: at the chosen unit all candidates are generated, and each is kept or rejected exactly.
- *Tests on parts of a candidate.* A candidate may be rejected by a witness on a subset of its vertices on which all adjacencies are decided, for instance the base and the new orbit alone, or the base and two orbits with their pattern (the "single" and "pair" tests of the engines). These are witnesses of the kinds of section 6.3.
- *Unit of the mate.* For the new orbit {f, sigma f}, U(sigma f) = V(f) (section 5.5). So the orbit uses one realisation of the unit U(f) and one of the unit V(f), and a true candidate has V(f) != U(f) (Lemma 5.5 (a)) and V(f) among the units that still have a realisation outside N and different from f. The engines enumerate V(f) among these units only.

By Lemma 6.5 and (6.2): **if a search whose rejections are exact has no node of level n, then no configuration has the root as its graph on Y_0.**

What a second program that follows a plan must do (the verifier contract; referee's reserve G2). In Computations 7.4, 8.9, 9.20 and 9.29, and in the second verification of III.1, the second program does not choose the units: at every node it reads, from a plan written by the first program, which unit the first program chose. By Lemma 6.5 any choice of an unfilled unit is allowed, so this is sound provided that the second program, in its own current state,

1. checks that the unit read from the plan is unfilled (it still has a positive remaining multiplicity);
2. generates all the candidates of that unit with its own model, and tests them with its own tests;
3. descends into every candidate that its own tests keep;
4. ends with an error on a plan that is out of step, too short or too long, an error never being counted as a completed traversal.

Under 1 to 4 the second program performs a complete search of its own in the sense of Lemma 6.5, whatever the first program did, and its conclusion "no node of level n" does not depend on the two programs keeping the same candidates. The equality of the numbers of kept candidates at every node, which the second programs also check, is a cross-check between the two programs: equal numbers do not show that the two kept lists are equal, and the exclusion does not need it.

Where 1 to 4 are in the code (read for this text on 07/10; line numbers of the files of that day):

program (computation)	1. unit unfilled	2. own candidates, own tests	3. descent on its own kept list	4. plan errors
`t20_R2/engv.py` (7.4: III.0)	`assert U0 in remaining`, l. 389	`gen`, l. 294 and 390; its own `reject` and budget recount	`for ... in good`, l. 428	`next(planit)` l. 386; depth l. 387; kept count l. 427; lines left, printed, l. 474-476
`t20_R2/engv2.py` (III.1)	l. 426	l. 427, same structure	l. 465	l. 423, 424, 464; lines left, printed, l. 513
`t20_R2/iian_verify.py` (8.9: IIa.0)	`assert U0 in remaining`, l. 240	l. 241-294	recursion l. 307	`next(plan)` l. 237; depth l. 238; kept count l. 295; lines left l. 310, printed
`t20_R3_I/in_verify.py` (9.20 and 9.29: case I)	`assert U0 in remaining`, l. 971	`gen_cands` l. 858, `test_cands` l. 901	`for ... in good`, l. 982-1008	`next(plan)` l. 968; depth l. 969; kept count l. 976; `assert left == 0`, l. 1044

The remaining multiplicities are kept in a dictionary from which a unit is deleted when its multiplicity reaches 0, so "in remaining" means "positive remaining multiplicity". In `engv.py`, `engv2.py` and `iian_verify.py` the number of plan lines left is printed, not asserted: the outputs cited in sections 7 and 8 all show 0. A plan that is too short makes `next` fail, and the program ends without a result line. `engv.py` and `engv2.py` also have a time limit: a run that reaches it stops descending, leaves plan lines unread, and is not a completed traversal.

Negative controls of the contract (07/10; `iian_verify.py follow` on the type set 39, whose plan has 7 lines; with the true plan: "nodes 7 ... kept 6 complete 0 | plan lines left 0"). (a) The unit of the second line replaced by the unit of the first line, which is filled at that node: `AssertionError` at l. 240. (b) A unit that does not exist, (0, 0), at a line of depth 3: `AssertionError` at l. 240. (c) The last line removed: `StopIteration` at l. 237. In the three cases the program ends with an error and prints no result line. The controls of the III.1 verifier (a plan with one kept count changed, and one with a line deleted) are in `lemma/t20_III1/VERIF.md`. Conditions 2 and 3 were read in the code, not tested by a control.

7. Case III (k = 4): the configurations III.0 and III.1

In this section the configuration is in case III. By Corollary 4.13 the ten orbits of the support are labelled so that the graph on S u {p} is III.0 or III.1: the inner indices are K = {0, 1, 2, 3}, and the outer indices 4..9 are the 2-subsets kappa(j) of K (section 4.10).

7.1 The model [A9; round 2, case III]

Lemma 7.1.

(a) X_4 = {p}; X_3 is empty; X_0 is the set of the six inactive inner vertices (three mate edges); |X_2| = 24, |X_1| = 48; every vertex of X_1 u X_2 is a word.

(b) Let s be in X_2 and r(s) = |U(s) n K| + |V(s) n K|, the number of neighbours of s among the eight active inner vertices. Then s not~ p, s not~ sigma s, r(s) <= 2, and s has exactly 2 - r(s) neighbours in X_0; if there are two, they are not mates. Moreover s has exactly 2 - r(s) neighbours in X_2.

(c) Every vertex of X_0 has exactly 4 neighbours in X_2. Its other neighbours are p, its mate and 8 vertices of X_1.

(d) For every pair {i, j} of indices, exactly need(i, j) vertices s of X_2 have U(s) = {i, j} (the units of III.0, resp. III.1, in section 4.10; 24 in all). For s in X_2: V(s) = U(sigma s), V(s) != U(s), |U(s) n V(s)| <= 1 and U(s) n V(s) contains no inner index.

Proof. (a) is Lemma 5.3 (III). (b) s is a word by (a), so s not~ p and s not~ sigma s (Lemma 2.1 (a)), and s has exactly two neighbours in N(p), in two different orbits (Corollaries 2.2, 2.3). N(p) consists of the eight active inner vertices x_c, y_c (c in K) and of X_0. The neighbours of s among the former are the x_c with c in U(s) and the y_c with c in V(s): r(s) of them. So the other 2 - r(s) are in X_0, in different orbits. By (5.3) with d = 2 and q = 0: a_0(s) - a_2(s) - 2 a_3(s) - 3 a_4(s) = 0; a_3(s) = 0 because X_3 is empty and a_4(s) = [s ~ p] = 0. So a_2(s) = a_0(s) = 2 - r(s).

(c) Let u be in X_0. By Lemma 2.1 (b) its neighbours in N[p] are p and its mate. So a_0(u) = 1 (the mate; the other vertices of X_0 are inner) and a_4(u) = 1 (p); a_3(u) = 0. By (5.3) with d = 0, q = 0:
1 - a_2(u) - 3 = -6, so a_2(u) = 4. By (5.2), a_1(u) + 2 * 4 + 4 * 1 = 20, so a_1(u) = 8.

(d) By (a) every vertex outside S u {p} has d <= 2, so Lemma 5.5 (b) applies; X_2 does not contain p. The rest is section 5.5 and Lemma 5.5 (a). QED

7.2 The determined sets and the search

    Y_0 = S u N[p] = S u {p} u X_0   (20 + 1 + 6 = 27 vertices).

The free vertices are F = X_2 (24 vertices, n = 12 orbits), with the multiplicities m(i, j) = need(i, j).

Lemma 7.2. Y_0 is determined by the support structure (III.0 or III.1). A state of level t (section 6.5) is determined by the support structure and, for each added orbit {s, sigma s}: the sets U(s), V(s), the set N0(s) = N(s) n X_0, and one pattern to each earlier orbit. Every adjacency is given by the following list.

pair of classes	adjacency	reason
S u {p} with S u {p}	the support structure	Corollary 4.13
X_0 with p	all adjacent	X_0 is contained in N(p)
X_0 with X_0	the three mate edges only	Lemma 2.1 (b)
X_0 with S	none	Lemma 5.3 (III) (d = 0)
s with S+, with S-	x_i for i in U(s), y_l for l in V(s)	parameters U(s), V(s)
s with p	none	Lemma 7.1 (b)
s with sigma s	not adjacent	Lemma 7.1 (b)
s with X_0	exactly N0(s)	parameter N0(s)
sigma s with anything	the images under sigma: U and V exchanged, N0 replaced by sigma N0	sigma is an automorphism
two different added orbits	one of the four patterns	parameter

The **model** (conditions satisfied by every true state):

- (T1) U(s) is the unit chosen at the node; V(s) is a unit different from U(s) (Lemma 7.1 (d));
- (T2) r(s) = |U(s) n K| + |V(s) n K| <= 2; N0(s) has exactly 2 - r(s) elements, not two mates (Lemma 7.1 (b));
- (T3) s has at most 2 - r(s) neighbours among the other added orbits (Lemma 7.1 (b));
- (T4) every vertex of X_0 has at most 4 neighbours among the added vertices (Lemma 7.1 (c));
- (T5) at level 12 the numbers of (T3) and (T4) are exactly 2 - r(s) and 4.

Proof. The table is a restatement of the lemmas quoted. In a true state the added vertices are vertices of X_2, so the numbers of Lemma 7.1 (b), (c), which count neighbours in the whole of X_2, are upper bounds for the neighbours among the added vertices, and are attained when all 12 orbits are added. QED

Symmetry. Let H_0 be the group of the 3! * 2^3 = 48 permutations of the six names of the vertices of X_0 that permute the three mate pairs and commute with the exchange of mates. By the table, the vertices of X_0 have the same neighbours in Y_0 up to these permutations (p and the mate), so every element of H_0, extended by the identity on S u {p}, is an automorphism of the graph on Y_0 commuting with the involution. The programs use it as follows: at a node N let H_N be the subgroup of the elements of H_0 that fix the set N0(s') of every added orbit of N (as a set in the runs of III.1; pointwise in the runs of III.0, which gives a smaller subgroup). A candidate is omitted if the sorted list of its N0(s) is not the lexicographically smallest among its images under H_N.

Lemma 7.3. With this rule, Lemma 6.5 holds in the following form: if a configuration has the support structure III.0 (resp. III.1), then the search with root the graph on Y_0 has a node of every level t <= 12.

Proof. The names of the six vertices of X_0 are not fixed by the configuration: any naming compatible with the

mate pairs makes the root true. We show by induction on t that there are a node N of level t and a naming of X_0 for which N is true. Let N and the naming be given, t < 12, and let f be as in the proof of Lemma 6.5. Choose g in H_N such that g(N0(f)) is lexicographically smallest, and rename the vertices of X_0 by g. An element of H_N fixes the set N0(s') of every added vertex s' of N, and, as it commutes with the exchange of mates, also the set sigma N0(s') of its mate; it fixes S u {p}. So it is an automorphism of the labelled graph N that moves only vertices of X_0, and N is still true for the new naming. For the new naming the candidate defined by f has the smallest N0, so it is not omitted; as in Lemma 6.5 it is a child of N, and it is true. QED

7.3 The two computations

> **Computation 7.4 (III.0).** For the support structure III.0, the search of section 7.2 (root: the 27 vertices of
> Y_0; model (T1)-(T5); symmetry rule with the pointwise stabilisers; rejections by budget violations and PSD
> violations) has no node of level 10. With the rule of the first program for the choice of the units, the numbers
> of nodes are:
>
> | level | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
> |---|---|---|---|---|---|---|---|---|---|---|---|
> | nodes | 1 | 13 | 116 | 881 | 7,109 | 30,102 | 54,327 | 31,126 | 6,086 | 243 | 0 |
>
> Total: 130,004 nodes and 4,094,709 candidates, of which 671,315 are rejected by a budget violation, 3,293,391 by a
> PSD violation, and 130,003 are kept. No rejection by the rank bound. No time-out.

> **Computation 7.5 (III.1).** For the support structure III.1, the same search (symmetry rule with the setwise
> stabilisers; rejections by budget violations, PSD violations and the rank bound) has no node of level 10. With
> the rule of the program for the choice of the units, the numbers of nodes are:
>
> | level | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
> |---|---|---|---|---|---|---|---|---|---|---|---|
> | nodes | 1 | 13 | 208 | 2,156 | 21,165 | 132,172 | 492,124 | 764,789 | 211,921 | 4,315 | 0 |
>
> Total: 1,628,864 nodes and 75,949,406 candidates, of which 17,487,147 are rejected by a budget violation and
> 56,833,395 by a PSD violation. Exactly one candidate of a node of level 9 passes the budget and PSD tests; it is
> rejected by the rank bound: on its 47 vertices (23 orbits and p) the 23 x 23 matrix G_odd of Proposition 6.4 has
> rank 23. No time-out.
>
> **Status at the time of writing: one program.**

Proposition 7.6 (hand statements + Computations 4.12, 7.4, 7.5). There is no configuration in case III.

Proof. Suppose there is one. By Corollary 4.13 its graph on S u {p} is III.0 or III.1 after a suitable labelling. By Lemma 7.2 its graph on Y_0 is the root of the search, and by Lemma 7.3 the search has a node of level 12. This contradicts Computation 7.4 or 7.5, by (6.2). QED

Status: hand part (Lemmas 4.1, 4.9, 5.1, 5.3-5.5, 7.1-7.3, Corollary 5.2, Proposition 6.4) plus Computations 4.12, 7.4, 7.5. For III.1 the conclusion depends on Proposition 6.4 (one node) and on a single program.

7.4 Programs, counts, and what the second implementation is

All the programs of this section are in `lemma/t20_R2`; the model (the base graph, the units, the group H_0 and the options (T2)) is in the file `cases.py` (class `CaseIII`), which **all of them import**. `check_model.py` compares the base graphs and the units of `cases.py` with a construction from the edge lists of section 4.10 typed in by hand, and finds them identical for III.0, III.1 and IIa.0 (`check_model.out`).

III.0.

- `eng3.py` and `eng4.py` (the first engine and a faster version of it; `eng4.py` writes the "plan": one line per node with the unit chosen, the number of candidates and the number kept). Each was run in 3 parts with split parameter 3 (the nodes of level 3 are split: the 7,109 tasks are their children, of level 4): the numbers of the table and all the other counters of Computation 7.4 are identical for the two (`summary_III0.out`). In these engines the choice of the unit uses a floating-point look-ahead; at the chosen unit all candidates are generated; a budget rejection is recounted in 64-bit integers and a PSD rejection is an integer vector on a sigma-block evaluated in Python integers (or exact rational elimination).
- `engv.py` (the verifier): the candidate generation is rewritten, the budgets are recounted on the whole graph for every candidate, and the PSD test uses full eigen-decompositions of the sigma-blocks, with an integer vector (or an exact rational elimination) for every rejection. It reads the unit of every node from the plan of `eng4.py` and stops with an error if its number of kept candidates differs from the plan. Result: the plan is followed to the end ("lines left unread 0, nodes with a different candidate count 0"), the same kept count at all 130,004 nodes, no node of level 10 (`v_III0_{0,1,2}of3.out`).
- 15,000 saved PSD certificates were re-checked by `check_certs.py`, a pure-Python checker that imports no engine (it re-checks that the stored graph is symmetric, that the stored involution is an automorphism, that the first 21 vertices carry III.0, and the value of the quadratic form): 0 invalid; 11,136 on the odd block of M1, 3,812 on the even block of M1, 49 and 3 on the blocks of M2 (`check_certs_III0.out`).

The working report counts this as "two implementations", with the remark "tests independent; the model file `cases.py` is shared". That is the precise status: two engines whose candidate generation and tests were written separately, one model file, and the second engine follows the units chosen by the first.

III.1. `eng5.py` (a third version of the first engine, same model `cases.CaseIII`), run in 8 parts with split parameter 3 (the nodes of level 3 are split): 21,165 tasks, of level 4, in every part, the same hash of the ordered task list in the 8 parts, no time-out (`lemma/t20_III1/summary_III1.out`, `e5_III1_{0..7}of8.json`). Counters by level, for the record:

```
    nodes            1, 13, 208, 2156, 21165, 132172, 492124, 764789, 211921, 4315, 0
    candidates       13, 212, 2939, 46605, 610429, 5386769, 25315888, 36337095, 8134160, 115296
    rejected budget  0, 0, 453, 6631, 112662, 1068846, 5747479, 8759108, 1763545, 28423
```

```
    rejected PSD     0, 4, 330, 18809, 365595, 3825799, 18803620, 27366066, 6366300, 86872
    kept             13, 208, 2156, 21165, 132172, 492124, 764789, 211921, 4315, 1
    rejected rank    0, 0, 0, 0, 0, 0, 0, 0, 0, 1
```

The per-part logs, the plan files and the sampled certificates of this run are not in git (they are in the session scratchpad, according to ‘lemma/t20_III1/README.md‘). According to the same file and to ‘T20_BOARD.md‘ (line C1), the independent verifier (‘engv.py‘ / ‘engv2.py‘, to be validated on III.0 first) over the whole tree, with a separate examination of the node rejected by rank, **is running at the time of writing**; nothing of it is used here.

Counters that depend on the program. "Candidates" are counted after the cached single and pair tests of the engine, and the numbers of nodes depend on the units chosen. They are reproducible by a program that follows the same plan, not by an arbitrary second program.

Remark (not used). For III.0 and III.1 both matrices are exactly PSD on Y_0, with ranks 26 (M1) and 25 (M2) (round 1, B3): nothing is excluded on the base alone.

8. Case II (k = 2): the configuration IIa.0

In this section the configuration is in case II. By Corollary 4.13 the orbits of the support are labelled so that the graph on S u {p} is IIa.0: the inner indices are 0 and 1, the plus edge is {2, 3}, and the four plus ends are x_2, x_3, y_2, y_3. They are words, so q(p) = 0, and for a vertex z outside S

    q(z) = |U(z) n {2, 3}| + |V(z) n {2, 3}|.

8.1 The hand lemmas [II.1 - II.6]

Lemma 8.1 [II.1]. n_0 + n_3 = 4, X_4 is empty, d(p) = 2 and q(p) = 0.

Proof. Lemma 5.3 (II), and the lines above. QED

Lemma 8.2 [II.2]. Every vertex outside S has d <= 3.

Proof. Corollary 5.2 gives d <= 4 and Lemma 5.3 (II) gives X_4 empty. (Directly, as in the working report: if d(z) >= 4, then (5.3) gives a_0(z) >= 3d - 6 >= 6, but a_0(z) <= n_0 <= 4.) QED

Lemma 8.3 [II.3]. X_3 is empty. Hence n_0 = 4, n_1 = 54, n_2 = 21, and every vertex outside S has d in {0, 1, 2}.

Proof. Suppose d(z) = 3. Then z != p (d(p) = 2), so sigma z != z, and d(sigma z) = 3: n_3 >= 2 and n_0 = 4 - n_3 <= 2. But (5.3) gives a_0(z) = a_2(z) + 2 a_3(z) + 3 * 3 + q(z) - 6 >= 3, and a_0(z) <= n_0. Contradiction. The sizes are those of Lemma 5.3 (II) with n_3 = 0. QED

Lemma 8.4 [II.4]. No vertex of N(p) \ S has d = 2 or d = 0. Consequently:

- the ten vertices of N(p) \ S all have d = 1; call their set W (five mate pairs), so N(p) = {x_0, y_0, x_1, y_1} u W;
- a_2(p) = a_0(p) = 0;
- X_0 consists of four words, forming two word orbits.

Proof. (5.3) at p (d = 2, q = 0, a_3 = a_4 = 0) gives a_0(p) = a_2(p).

Suppose f is in N(p) \ S with d(f) = 2. Its mate sigma f has d = 2 and is adjacent to f, and p is in X_2 and is adjacent to f. So a_2(f) >= 2, and (5.3) at f (d = 2) gives a_0(f) = a_2(f) + q(f) >= 2. The neighbours of f in N[p] are p and sigma f (Lemma 2.1 (b)), both in X_2; so the neighbours of f in X_0 are words, and f is adjacent to at most one vertex of each word orbit (Corollary 2.3 (b)). Hence X_0 contains at least two word orbits; as |X_0| = 4 (Lemma 8.3), X_0 is exactly two word orbits. Then no vertex of X_0 is in N(p), i.e. a_0(p) = 0. But a_2(p) >= 2 (f and sigma f). This contradicts a_0(p) = a_2(p).

So no vertex of N(p) \ S has d = 2, i.e. a_2(p) = 0, hence a_0(p) = 0: no vertex of N(p) \ S has d = 0. By Lemma 8.3 the ten vertices of N(p) \ S have d in {0, 1, 2}, so d = 1. X_0 has four elements, none in N[p] (p is in X_2); it is sigma-invariant without fixed vertex, so it is two orbits, and they are word orbits. QED

Lemma 8.5 [II.5] (the types of W). Let w be in W. Then w has exactly one neighbour x_i in S+ and one neighbour y_l in S-; write (i ; l) for its type. Then:

(a) sigma w has type (l ; i); i and l are outer indices; i != l;
(b) two different mate pairs of W never have the same unordered type {i, l};
(c) for every outer index i, the number of mate pairs of W whose unordered type contains i is 2 - r_i, where r_i is the number of neighbours of x_i among x_0, y_0, x_1, y_1.

So the five unordered types are the edges of a simple graph on the eight outer indices {2, ..., 9} in which the index i has degree 2 - r_i. For IIa.0: r_i = 0 for i = 2, 4 and r_i = 1 for i = 3, 5, 6, 7, 8, 9; so the degrees are (2, 1, 2, 1, 1, 1, 1, 1) for i = 2, ..., 9, and there are exactly **180** such graphs.

Proof. d(w) = 1 by Lemma 8.4. (a) The type of sigma w is given by section 5.5. Inside N(p) the only neighbour of w is its mate (Lemma 2.1 (b)), which is not in S; so w is adjacent to none of x_0, y_0, x_1, y_1, and i, l are outer. i != l by Lemma 5.5 (a) (w is in N(p), so U(w) n V(w) is empty). (b) Suppose two different mate pairs have the same unordered type; replacing one vertex by its mate, there are w, w' in different mate pairs, both of type (i ; l). They are distinct vertices of N(p) that are not mates, so they are non-adjacent, and they have the three common neighbours p, x_i, y_l. Contradiction. (c) x_i is a word: it has exactly two neighbours in N(p) (Corollary 2.2), r_i of them among x_0, y_0, x_1, y_1 and 2 - r_i in W. In a mate pair of W of unordered type {i, l}, exactly one vertex is adjacent to x_i (the one of type (i ; l)); a mate pair whose type does not contain i has no neighbour of x_i. So 2 - r_i mate pairs have i in their type.

For IIa.0 (section 4.10): x_i ~ y_0 exactly for i = 7, 8, 9 (minus edges 07, 08, 09) and x_i ~ y_1 exactly for i = 3, 5, 6 (minus edges 13, 15, 16); no x_i with i outer is adjacent to x_0 or x_1 (the only plus edge is {2,3}). So r_i = 1 for i = 3, 5, 6, 7, 8, 9 and r_i = 0 for i = 2, 4. Count of the graphs with degree 2 at the vertices 2 and 4 and degree 1 at the six others: if 2 ~ 4, choose the other neighbour of 2 (6 ways), the other neighbour of 4 (5 ways) and a perfect matching of the four remaining vertices (3 ways): 90; if 2 not~ 4, choose the two neighbours of 2 (15 ways), the two neighbours of 4 among the four remaining vertices (6 ways), and join the last two: 90. QED

Lemma 8.6 [II.6] (the vertices of X_2 other than p). Let s be in X_2, s != p, and r(s) = |U(s) n {0,1}| + |V(s) n {0,1}|, the number of neighbours of s among x_0, y_0, x_1, y_1.

(a) s is a word: s not~ p and s not~ sigma s. It has exactly 2 - r(s) neighbours in W, never two mates; in particular r(s) <= 2.
(b) a_2(s) = a_0(s) - q(s) and a_0(s) <= 3. So s has at most 3 - q(s) neighbours in X_2 \ {p}; in particular q(s) <= 3.
(c) A vertex w of W has a_0(w) <= 2 and a_2(w) = a_0(w) + 3 - q(w); p is one of its neighbours in X_2. So w has at most 4 - q(w) neighbours in X_2 \ {p}.
(d) For every pair {i, j} of indices exactly need(i, j) vertices s of X_2 \ {p} have U(s) = {i, j} (the units of IIa.0 in section 4.10; 20 in all). V(s) = U(sigma s), V(s) != U(s), |U(s) n V(s)| <= 1.

Proof. (a) By Lemma 8.4 the vertices of N(p) \ S have d = 1, and the vertices of N(p) n S are not outside S; so s is not in N(p), and s != p: s is a word. It has exactly two neighbours in N(p) = {x_0, y_0, x_1, y_1} u W, in two different orbits (Corollaries 2.2, 2.3). (b) (5.3) with d = 2 and a_3 = a_4 = 0 (Lemmas 8.2, 8.3): a_0(s) - a_2(s) = q(s). By Lemma 5.4 (a), 2 a_0(s) - 2 <= |X_0 \ {p}| = 4, so a_0(s) <= 3. s not~ p, so all its neighbours in X_2 are in X_2 \ {p}. (c) X_0 is two word orbits (Lemma 8.4) and w is adjacent to at most one vertex of each (Corollary 2.3 (b)): a_0(w) <= 2. (5.3) with d = 1: a_0(w) - a_2(w) = 3 + q(w) - 6. w ~ p and p is in X_2. (d) By Lemmas 8.2 and 8.3 every vertex outside S has d <= 2, so Lemma 5.5 (b) applies; the sum of need is |X_2 \ {p}| = 20. The rest is section 5.5 and Lemma 5.5 (a). QED

8.2 The determined sets and the search

    Y_0 = S u N[p] = S u {p} u W   (20 + 1 + 10 = 31 vertices).

The free vertices are F = X_2 \ {p} (20 vertices, n = 10 orbits), with m(i, j) = need(i, j). The vertices of X_0 are **not** in the vertex sets of this section.

Lemma 8.7. Y_0 is determined by IIa.0 and by the five unordered types of the mate pairs of W (a graph as in Lemma 8.5; its edges may be named in any order, the vertices of W having no other names). A state of level t is determined by these data and, for each added orbit {s, sigma s}: U(s), V(s), the set N_W(s) = N(s) n W and one pattern to each earlier orbit.

pair of classes	adjacency	reason
S u {p} with S u {p}	IIa.0	Corollary 4.13
W with p	all adjacent	W is contained in N(p)
W with W	the five mate edges only	Lemma 2.1 (b)
W with x_0, y_0, x_1, y_1	none	Lemma 2.1 (b)
W with the rest of S	the pair of type {i, l}: one vertex adjacent to x_i and y_l only, its mate to x_l and y_i only	parameter (Lemma 8.5)
s with S+, with S-	x_i for i in U(s), y_l for l in V(s)	parameters
s with p; s with sigma s	not adjacent	Lemma 8.6 (a)
s with W	exactly N_W(s)	parameter
sigma s with anything	the images under sigma	sigma is an automorphism
two different added orbits	one of the four patterns	parameter

The **model**:

- (P1) U(s) is the unit chosen at the node; V(s) is a unit different from U(s) (Lemma 8.6 (d));
- (P2) r(s) <= 2 and q(s) <= 3; N_W(s) has exactly 2 - r(s) elements, not two mates (Lemma 8.6 (a), (b));
- (P3) s has at most 3 - q(s) neighbours among the other added orbits (Lemma 8.6 (b));
- (P4) every w of W has at most 4 - q(w) neighbours among the added vertices (Lemma 8.6 (c)).

(P3) and (P4) are upper bounds only; no equality is required at level 10 (the exact numbers involve X_0, which is not in the vertex set).

Proof. Restatement of the lemmas quoted; the bounds of Lemma 8.6 (b), (c) count neighbours in the whole of X_2 \ {p}, hence bound the neighbours among the added vertices. QED

8.3 The two computations

> **Computation 8.8 (step 1: the neighbourhood of p).** Of the 180 graphs of Lemma 8.5, giving 180 graphs on the 31 vertices of Y_0: 63 have a budget violation, 30 have no budget violation and a PSD violation, and 87 are kept.

> **Computation 8.9 (step 2).** For each of the 87 graphs on Y_0 kept in step 1, the search of section 8.2 (model (P1)-(P4); no symmetry rule; rejections by budget violations and PSD violations) has no node of level 8. With the rule of the first program for the choice of the units, the numbers of nodes, summed over the 87 roots, are:
>
> | level | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
> |---|---|---|---|---|---|---|---|---|---|
> | nodes | 87 | 520 | 3,148 | 11,451 | 18,324 | 10,279 | 1,640 | 48 | 0 |
>
> Total: 45,497 nodes and 898,945 candidates, of which 201,918 are rejected by a budget violation, 651,617 by a PSD violation, and 45,410 are kept. No rejection by the rank bound. No time-out.

Proposition 8.10 (hand statements + Computations 4.12, 8.8, 8.9). There is no configuration in case II.

Proof. Suppose there is one. By Corollary 4.13 its graph on S u {p} is IIa.0 after a suitable labelling. By Lemmas 8.4, 8.5 and 8.7 its graph on Y_0 is one of the 180 graphs, and by (6.2) it is one of the 87 kept in step 1. By Lemmas 8.6, 8.7 and 6.5 the search with this root has a node of level 10, which contradicts Computation 8.9. QED

Status: hand part (Lemmas 4.6, 4.7, 5.1, 5.3-5.5, 8.1-8.7, Corollary 5.2) plus Computations 4.12, 8.8, 8.9. The rank bound is not used.

8.4 Programs and counts

All in ‘lemma/t20_R2‘.

Step 1. ‘iia_w.py‘ (which uses the tool file ‘r2c.py‘) lists the 180 graphs, builds the 31-vertex graphs and tests them; it writes the 87 kept type sets to ‘iia_w.json‘. ‘iian_verify.py types‘ does the same with its own code: "type sets with the right degrees: 180; rejected by budgets 63, by PSD (exact) 30, kept 87; identical to the list of the first program: True; certificates: integer vector 30, exact elimination 0" (‘iian_verify_types.out‘). So the 93 rejections are exact in two programs, and the two kept lists are the same list. Whether each of the 87 graphs is exactly PSD on Y_0 has not been recorded; it is not needed.

Step 2.

- First program: ‘eng5.py‘ with the model ‘cases.CaseIIaN‘ (base S u N[p], caps only), one run per type set (‘run_iian.py‘), each run writing its plan; totals in ‘summary_IIa.out‘: the table of Computation 8.9; deepest level with a node: 7; between 1 and 3,054 nodes per type set; 2 type sets have no child of the root.
- Second program: ‘iian_verify.py follow‘. It imports nothing from the engines or from ‘cases.py‘: it rebuilds IIa.0 from the structure file, builds its own base, its own units, its own candidates (with pair budgets as the only pre-filter) and its own tests (budget recount on the whole graph; integer vector for every PSD rejection). It reads the unit of every node from the plan of the first program and stops if its number of kept candidates differs. Result (‘iian_verify_0_30.out‘, ‘_30_38‘, ‘_38_44‘, ‘_44_66‘, ‘_66_87‘): 3,147 + 12,275 + 1,729 + 13,073 + 15,273 = 45,497 nodes, the same kept count at every node (45,410 in all), no plan line left, 0 complete. Its candidates are more numerous than those of the first program (1,067,833, of which 230,683 rejected by budgets and 791,740 by a PSD violation, all by integer vectors), because its pre-filter is weaker; its outputs show these differences as "candidate-count mismatches", which are not disagreements. (The sums of this paragraph were made for this text from the five output files.)
- 6,358 saved certificates of the first program (those of 5 of the 87 type sets; the certificate files of the other 82, about 1.6 GB, were deleted to save disk) were re-checked by ‘check_certs.py‘: 0 invalid (‘check_certs_IIa.out‘).

The two programs of step 2 share no code. They share: the structure file ‘structures_III_IIb_IIa.json‘ (the graph IIa.0), the list ‘iia_w.json‘ (which the second program compares with its own list in step 1), the hand model (Lemmas 8.4-8.6), and the units chosen by the first program.

9. Case I (k = 0): the graphs I.0, ..., I.5

In this section the configuration is in case I. By Lemma 4.10 and Corollary 4.13: S+ = {x_i} and S- = {y_i} are independent, S is disjoint from N[p], x_i ~ y_j if and only if i ~_B j, and B is one of the six graphs I.0, ..., I.5 of section 4.10. There is no plus end, so q(z) = 0 for every z. By Lemma 5.3 (I): X_4 is empty, p is in X_0, n_3 is even, and

    n_0 = 9 - n_3,   n_1 = 40 + 3 n_3,   n_2 = 30 - 3 n_3.

For z outside S with d = d(z), (5.2) and (5.3) read

    a_1 + 2 a_2 + 3 a_3 = 20 - 5d,   a_0 = a_2 + 2 a_3 + 3d - 6,   (9.1)

that is:

class of z	a_0	a_1
X_0	a_2 + 2 a_3 - 6	20 - 2 a_2 - 3 a_3
X_1	a_2 + 2 a_3 - 3	15 - 2 a_2 - 3 a_3
X_2	a_2 + 2 a_3	10 - 2 a_2 - 3 a_3
X_3	a_2 + 2 a_3 + 3	5 - 2 a_2 - 3 a_3

N(p) is disjoint from S and consists of the 7 orbits {a_c, b_c} (Lemma 2.1), called the **pairs** of N(p). Put delta_c = d(a_c) = d(b_c).

Sections 9.1 and 9.2 reduce the case to four sub-cases (two profiles, n_3 = 0 or 2). Sections 9.3 to 9.6 contain the other hand statements established so far; it is said in each place whether the enumeration of section 9.A uses them. Section 9.A treats n_3 = 0, section 9.B treats n_3 = 2.

9.1 No inner vertex with d = 3; n_3 is 0 or 2 [I.1, I.2]

Lemma 9.1 [I.1]. No vertex of N(p) has d = 3. So delta_c is in {0, 1, 2} for every c, and every vertex of X_3 is a word.

Proof. Suppose z is in N(p) with d(z) = 3. Its mate sigma z is in X_3 and is adjacent to z, so a_3(z) >= 1, and (9.1) gives a_0(z) = a_2(z) + 2 a_3(z) + 3 >= 5. By Lemma 5.4 (b), 2 a_0(z) - 1 <= |X_0|, so |X_0| >= 9. But |X_0| = 9 - n_3 and n_3 >= 2 (z and sigma z). Contradiction. Every vertex outside S has d <= 3 (Corollary 5.2 and X_4 empty), so delta_c <= 2. A vertex of X_3 is not in N(p), and it is not p (d(p) = 0). QED

Lemma 9.2 [I.2]. n_3 is 0 or 2.

Proof. Let z be in X_3 and T(z) = N(z) n X_0. By (9.1), |T(z)| = a_0(z) >= 3. z is a word (Lemma 9.1), so by Lemma 5.4 (a), 2 |T(z)| - 2 <= |X_0 \ {p}| = 8 - n_3. Hence n_3 <= 4, and n_3 is even. Suppose n_3 = 4.

Then |X_0 \ {p}| = 4 and |T(z)| = 3 for every z in X_3. The set X_0 \ {p} is sigma-invariant without fixed vertex, so it consists of two orbits O_1, O_2, and T(z), a 3-subset of it that does not contain p, contains one of the two orbits entirely and one vertex of the other. If T(z) contains the orbit {u, sigma u}, then z is adjacent to u and to sigma u, so by Corollary 2.3 (c) this is a word orbit and N(u) n N(sigma u) = {z, sigma z}. X_3 consists of two orbits {z, sigma z} and {z', sigma z'}. T(sigma z) = sigma T(z) contains the same full orbit as T(z). T(z) and T(z') cannot contain the same full orbit {u, sigma u}: N(u) n N(sigma u) would be both {z, sigma z} and {z', sigma z'}. So, with suitable names,

    T(z) = O_1 u {v} with v in O_2,   T(z') = O_2 u {u'} with u' in O_1,
    T(sigma z) = O_1 u {sigma v},   T(sigma z') = O_2 u {sigma u'}.

The four vertices z, sigma z, z', sigma z' are distinct, and:

- z and z' have the common neighbours v and u';
- z and sigma z' have the common neighbours v and sigma u';
- z and sigma z have the two vertices of O_1 as common neighbours; z' and sigma z' the two vertices of O_2.

Two distinct vertices have at most two common neighbours, so none of these four pairs has a common neighbour in S. Let (U ; V) be the type of z and (U' ; V') that of z'; the four sets have 3 elements, sigma z has the type (V ; U) and sigma z' the type (V' ; U'). An index in U n U' gives a common neighbour x_i of z and z'; in V n V', a common neighbour y_l of z and z'; in U n V' or in V n U', a common neighbour of z and sigma z'; in U n V, a common neighbour of z and sigma z; in U' n V', a common neighbour of z' and sigma z'. So U, V, U', V' are four pairwise disjoint 3-subsets of {0, ..., 9}: 12 > 10. Contradiction. QED

9.2 The profiles; profile (5,0,2) is impossible [A10, I.3]

For j = 0, 1, 2 let h_j be the number of c with delta_c = j. So N(p) n X_j consists of h_j pairs. Notation used in the whole section:

    A_p = N(p) n X_2   (2 h_2 vertices, the "A-pairs"),   W = N(p) n X_1   (2 h_1 vertices, the "W-pairs"),
    Z_p = N(p) n X_0   (2 h_0 vertices, the "Z-pairs"),   X_2' = X_2 \ A_p,
    R_0 = X_0 \ N[p] = X_0 \ ({p} u Z_p)   (the words of X_0).

Lemma 9.3. sum_c delta_c = 10. Hence

    h_2 = 3 + h_0,   h_1 = 4 - 2 h_0,   (9.2)

and (h_2, h_1, h_0) is one of (3,4,0), (4,2,1), (5,0,2). This triple is the **profile** of the configuration.

Proof. Each of the 10 vertices of S+ is a word, so it has exactly 2 neighbours in N(p) (Corollary 2.2). Hence sum over v in N(p) of d+(v) = 20, i.e. 2 sum_c delta_c = 20. So h_1 + 2 h_2 = 10 and h_0 + h_1 + h_2 = 7 (Lemma 9.1), which gives (9.2), and h_1 >= 0 forces h_0 in {0, 1, 2}. QED

Proposition 9.4 [I.3]. Profile (5,0,2) is impossible, for the six graphs and for n_3 = 0 and n_3 = 2.

Proof. Let the profile be (5,0,2), and let u_1, sigma u_1, u_2, sigma u_2 be the four vertices of Z_p. For each of them, (9.1) gives a_2 = a_0 + 6 - 2 a_3, and a_0 >= 2 (p and the mate); so a_2(u) >= 8 - 2 a_3(u). Inside N(p) a vertex is adjacent only to its mate (Lemma 2.1 (b)), so the neighbours of these four vertices in X_2 lie in X_2' = X_2 \ A_p, a set of 30 - 3 n_3 - 10 elements.

The four sets N(u) n X_2 have small intersections: two mates have only p as a common neighbour (Lemma 2.1 (c)), so the sets of u_1 and sigma u_1 are disjoint, and so are those of u_2 and sigma u_2; a vertex of {u_1, sigma u_1} and a vertex of {u_2, sigma u_2} are non-adjacent and have p as a common neighbour, hence at most one common neighbour in X_2. There are four such pairs. So the union of the four sets has at least (sum of a_2 over the four vertices) - 4 elements.

If n_3 = 0, then a_3 = 0 and the union has at least 32 - 4 = 28 elements; but |X_2'| = 20.
If n_3 = 2, let X_3 = {z, sigma z}. A vertex other than p is not adjacent to both u_1 and sigma u_1 (Lemma 2.1 (c)), so z has at most one neighbour in each of the two pairs, and so has sigma z: the sum of a_3 over the four vertices is at most 4. Hence the union has at least 32 - 8 - 4 = 20 elements; but |X_2'| = 14. QED

So two profiles remain, (3,4,0) and (4,2,1), each with n_3 = 0 or n_3 = 2.

9.3 General statements on the types (both values of n_3) [C1, C2, C3, C4]

Lemmas 9.5, 9.6 and 9.8 are used by the enumerations. Lemma 9.7 is used only as vocabulary, in Lemmas 9.11, 9.12.

Lemma 9.5 [C1, H0]. Let x be a vertex of S. Then x has 4 neighbours in S and 10 outside S, none of them in X_0, and

    (number of neighbours of x in X_2) + 2 (number of neighbours of x in X_3) = 6.   (9.3)

If n_3 = 0, x has exactly 6 neighbours in X_2 and 4 in X_1.

Proof. Let x be in S+ (apply sigma for S-). It has 4 neighbours in S, all in S- (Lemma 4.10), hence 10 outside S, each with d >= 1. Count the pairs (w, s) with s in S+ \ {x}, w ~ x and w ~ s: each of the 9 vertices s is non-adjacent to x, so the count is 18. A neighbour w of x in S- has 3 neighbours in S+ other than x; a neighbour w outside S has d(w) - 1. So 18 = 12 + sum over the 10 outside neighbours w of (d(w) - 1), and d(w) is 1, 2 or 3. QED

Lemma 9.6 [C2].

(a) For all indices i, l (i = l allowed) the vertices x_i and y_l have exactly 2 - [i ~_B l] common neighbours, and all of them are outside S. Equivalently: exactly 2 - [i ~_B l] vertices s outside S have i in U(s) and l in V(s).

(b) For every s outside S: every index of U(s) has at most one B-neighbour in V(s), and every index of V(s) has at most one B-neighbour in U(s).

(c) For every s outside S, s != p: U(s) n V(s) is empty if s is in N(p), and has at most one element if s is a word.

(d) If s is outside S and i != j are in U(s), then {i, j} is a deficient pair of B (codeg_B(i,j) <= 1).

(e) If n_3 = 0: for every pair {i, j}, exactly need(i, j) = 2 - codeg_B(i, j) vertices s of X_2 have U(s) = {i, j} (30 in all; the units of section 4.10), and V(s) = U(sigma s) != U(s).

Proof. (a) x_i and y_l are distinct, and adjacent if and only if i ~_B l (for i = l they are not adjacent, by Lemma 4.10 (c)). So they have 2 - [i ~_B l] common neighbours, none of which is in S (Lemma 4.10). (b) Let i be in U(s). For l in V(s) n B(i) the vertex y_l is adjacent to s and to x_i: it is a common neighbour of the adjacent pair s, x_i, which has only one. The second half is the same with the adjacent pair s, y_l. (c) is Lemma 5.5 (a) (there is no inner index). (d) s is a common neighbour of x_i and x_j outside S, and they have codeg_B(i,j) common neighbours in S and 2 in all. (e) Lemma 5.5 (b): if n_3 = 0 every vertex outside S has d <= 2, and p is not in X_2. QED

Lemma 9.7 [C3] (slots). Let w, w' be vertices of N(p) in two different pairs. They have exactly two common neighbours: p, and a second one, written slot(w, w'), which is a word. The map {w, w'} -> slot(w, w') is a bijection from the 84 sets {w, w'} of this kind onto the 84 words, and sigma slot(w, w') = slot(sigma w, sigma w'). For two different pairs c, c' of N(p) (a **cell**), the four words slot(w, w'), w in pair c, w' in pair c', form two word orbits; every word orbit lies in exactly one cell. The 12 word-neighbours of a vertex w of N(p) are the words slot(w, w'), w' outside the pair of w.

Proof. w and w' are not mates, so they are non-adjacent (Lemma 2.1 (b)) and have exactly two common neighbours. p is one. The other is not in N(p): a vertex of N(p) is adjacent, inside N(p), only to its mate, and it cannot be the mate of both. So it is a word. Conversely a word has exactly two neighbours in N(p), in two different pairs (Corollaries 2.2, 2.3), so it is slot(w, w') for exactly one set {w, w'}. There are C(14,2) - 7 = 84 such sets and 99 - 15 = 84 words. sigma maps common neighbours of w, w' to common neighbours of sigma w, sigma w'. A cell contains the four sets {a_c, a_c'}, {b_c, b_c'}, {a_c, b_c'}, {b_c, a_c'}, exchanged two by two by sigma. A vertex w of N(p) has the neighbours p, sigma w and 12 words; each of these words has w as one of its two neighbours in N(p). QED

Lemma 9.8 [C4] (the types of the pairs of N(p)). Let {w, sigma w} be a pair of N(p), with delta = d(w).

(a) If delta = 2 (an A-pair): the type of w is (U ; V) with U, V two disjoint 2-sets, and that of sigma w is (V ; U). U and V are deficient pairs of B; every index of U has at most one B-neighbour in V, and every index of V at most one B-neighbour in U. The unordered pair {U, V} is called the type of the pair.
If delta = 1 (a W-pair): the type of w is (i ; l) with i != l, that of sigma w is (l ; i); the unordered pair {i, l} is the type of the pair.
If delta = 0 (a Z-pair): w and sigma w have no neighbour in S.

(b) Every index 0..9 lies in exactly two of the seven sets attached to the pairs: U u V for an A-pair, {i, l} for a W-pair, the empty set for a Z-pair.

(c) Two vertices of N(p) in two different pairs have at most one common neighbour in S.

(d) In particular no two pairs of N(p) have the same type.

Proof. (a) U(w) n V(w) is empty by Lemma 9.6 (c); the types of the mates are exchanged (section 5.5); U and V are deficient pairs by Lemma 9.6 (d), applied to w and to sigma w; the condition on B-neighbours is Lemma 9.6 (b). (b) x_i is a word: it has exactly two neighbours in N(p), in two different pairs. In the pair {w, sigma w}, w ~ x_i iff i is in U(w), and sigma w ~ x_i iff i is in V(w); as U(w) n V(w) is empty, the pair contains a neighbour of x_i iff i is in U(w) u V(w), and then exactly one. (c) Two such vertices are non-adjacent and have p as a common neighbour. (d) Two pairs with the same type would contain two vertices with the same type, i.e. with 4 (A-pairs) or 2 (W-pairs) common neighbours in S. QED

9.4 n_3 = 0: the neighbourhoods of N(p), the words of X_2, the cells [C4, C5, C6, C7]

In this subsection n_3 = 0, so (n_0, n_1, n_2) = (9, 40, 30) and a_3 = 0 everywhere. Lemmas 9.9 and 9.10 give the upper bounds used by the enumeration of section 9.A. Lemmas 9.11 and 9.12 are not used by it.

For w in N(p) let rho(w) = |N(w) n R_0|. R_0 consists of 8 - 2 h_0 words, in 4 - h_0 orbits.

Lemma 9.9 [C4]. Let n_3 = 0 and w in N(p), delta = d(w). Then rho(sigma w) = rho(w) <= 4 - h_0, and the 12 word-neighbours of w are distributed as follows.

kind of the pair of w	in S	in R_0	in X_2'	in X_1 \ W	bound on rho(w)
A (delta = 2)	4	rho	rho	8 - 2 rho	4 - h_0
W (delta = 1)	2	rho	4 + rho	6 - 2 rho	3
Z (delta = 0)	0	rho	8 + rho	4 - 2 rho	2

Moreover the sum of rho over the seven pairs (one value per pair) is

$$\text{sum_c } \rho(a_c) = 8 - 2h_0. \tag{9.4}$$

Consequently a vertex of A_p has at most 4 - h_0 neighbours in X_2', a vertex of W at most 7, a vertex of Z_p at most 10.

Proof. |X_0| = 9 and X_0 = {p} u Z_p u R_0, so |R_0| = 8 - 2 h_0; R_0 is sigma-invariant and consists of words. rho(sigma w) = rho(w) by sigma, and w is adjacent to at most one vertex of each word orbit (Corollary 2.3 (b)), so rho(w) <= 4 - h_0. The neighbours of w are p, sigma w, delta vertices of S+, delta of S-, and 12 - 2 delta words outside S. Its neighbours in X_0 are p, its mate if delta = 0, and rho(w) vertices of R_0; so

a_0(w) = 1 + [delta = 0] + rho(w). By (9.1), a_2(w) = a_0(w) + 6 - 3 delta and a_1(w) = 20 - 5 delta - 2 a_2(w). The mate is the only neighbour of w in N(p); it is in X_2, X_1 or X_0 according to delta.

- delta = 2: a_2 = 1 + rho, of which the mate; so rho neighbours in X_2'. a_1 = 8 - 2 rho, all words.
- delta = 1: a_2 = 4 + rho, all in X_2'. a_1 = 7 - 2 rho, of which the mate; so 6 - 2 rho >= 0 and rho <= 3.
- delta = 0: a_2 = 8 + rho, all in X_2'. a_1 = 4 - 2 rho >= 0, so rho <= 2.

In each line the four numbers add up to 12. For (9.4): every vertex of R_0 is a word and has exactly two neighbours in N(p), so sum over the 14 vertices w of N(p) of rho(w) is 2 |R_0| = 16 - 4 h_0, and rho takes the same value on the two vertices of a pair. QED

Lemma 9.10 [C5]. Let n_3 = 0 and s in X_2'. Then s is a word, s not~ p, s not~ sigma s; s has exactly two neighbours in N(p), in two different pairs; and a_2(s) = a_0(s) <= 5. So s has at most 5 - |N(s) n A_p| neighbours in X_2'.

Proof. s is not in N(p) and s != p (d(p) = 0), so s is a word; Corollaries 2.2, 2.3 and Lemma 2.1 (a). a_2(s) = a_0(s) by (9.1). By Lemma 5.4 (a), 2 a_0(s) - 2 <= |X_0 \ {p}| = 8. QED

The kinds of the cells of Lemma 9.7 are named after the kinds of their two pairs: AA, AW, WW, AZ, WZ. By Lemma 9.7 every orbit of X_2' and every orbit of the support lies in exactly one cell.

Lemma 9.11 [C6] (profile (4,2,1), n_3 = 0). There are 6 cells AA, 8 cells AW, 1 cell WW, 4 cells AZ and 2 cells WZ, and X_2' consists of 11 orbits. Let n_XY be the number of orbits of X_2' in the cells of kind XY, rho_Z the value of rho on the Z-pair, rho_W the sum of the values of rho on the two W-pairs, and c the number of orbits of the support in the cell WW. Then c <= 1, rho_Z + rho_W <= 1, and (n_AA, n_AW, n_WW, n_WZ, n_AZ) is one of

    rho_Z = 0, rho_W = 0:   (0,1,2,3,5) with c = 0;   (0,2,1,4,4) with c = 0 or 1;   (1,0,2,4,4) with c = 0;
    rho_Z = 0, rho_W = 1:   (0,1,2,4,4) with c = 0;
    rho_Z = 1, rho_W = 0:   (0,0,2,4,5) with c = 0.

Proof. The numbers of cells follow from 4 A-pairs, 2 W-pairs, 1 Z-pair; |X_2'| = 30 - 8 = 22.

For a pair with vertices w, sigma w, the number of orbits of X_2' lying in the cells that contain this pair is the number of neighbours of w in X_2': an orbit of such a cell is {slot(w, w'), slot(sigma w, sigma w')} for some w', and exactly its first vertex is adjacent to w. Summing this over the pairs of each kind, with the numbers of Lemma 9.9 and (9.4) (the sum of rho over the four A-pairs is 6 - rho_Z - rho_W):

    Z-pair: n_AZ + n_WZ = 8 + rho_Z;
    W-pairs: n_AW + 2 n_WW + n_WZ = 8 + rho_W;
    A-pairs: 2 n_AA + n_AW + n_AZ = 6 - rho_Z - rho_W.

Also n_WZ <= 4 (the two cells WZ carry four word orbits) and n_WW <= 2 - c (the cell WW carries two word orbits, c of which are in the support). c <= 1: if the orbits of x_i and of x_j, i != j, were both in the cell WW, the two W-pairs would both have the type {i, j} (each contains a neighbour of x_i and one of x_j), against Lemma 9.8 (d).

The third equation minus the first gives 2 n_AA + n_AW = n_WZ - 2 - 2 rho_Z - rho_W; substituting n_AW in the second gives n_WW - n_AA = 5 + rho_W + rho_Z - n_WZ. As n_WW <= 2 and n_AA >= 0: n_WZ >= 3 + rho_Z + rho_W, and n_WZ <= 4 gives rho_Z + rho_W <= 1.

- rho_Z = rho_W = 0, n_WZ = 3: 2 n_AA + n_AW = 1, so n_AA = 0, n_AW = 1; n_WW = 2 (so c = 0); n_AZ = 5.
- rho_Z = rho_W = 0, n_WZ = 4: 2 n_AA + n_AW = 2 and n_WW = 1 + n_AA; n_AZ = 4. Either (n_AA, n_AW) = (0, 2) and n_WW = 1, or (1, 0) and n_WW = 2 (so c = 0).
- rho_W = 1, rho_Z = 0: n_WZ = 4, 2 n_AA + n_AW = 1, so (0, 1); n_WW = 2; n_AZ = 4.
- rho_Z = 1, rho_W = 0: n_WZ = 4, 2 n_AA + n_AW = 0; n_WW = 2; n_AZ = 5. QED

(The same list is printed by the script ‘hand_cells.py‘, output ‘lemma/t2O_R3_I/hand_cells.out‘. By Corollary 9.15 below, rho_Z = 0, so the last line does not occur.)

Lemma 9.12 [C7] (profile (3,4,0), n_3 = 0). There are 3 cells AA, 12 cells AW and 6 cells WW, and X_2' consists of 12 orbits. Let rho_A be the sum of the values of rho on the three A-pairs (0 <= rho_A <= 8) and c the number of orbits of the support in the cells WW. Then

    n_AW = rho_A - 2 n_AA,   n_WW = 12 - rho_A + n_AA >= 4,   n_AA + c <= rho_A.

Proof. |X_2'| = 30 - 6 = 24. As in the proof of Lemma 9.11, with Lemma 9.9 and (9.4) (the sum of rho over the seven pairs is 8, so over the four W-pairs it is 8 - rho_A):

    A-pairs: 2 n_AA + n_AW = rho_A;   W-pairs: n_AW + 2 n_WW = 16 + (8 - rho_A).

This gives the two equalities; n_WW >= 4 because rho_A <= 8. The six cells WW carry 12 word orbits, of which n_WW are in X_2' and c in the support: n_WW + c <= 12. QED

9.5 n_3 = 0: edge counts and a quadratic identity [H4, C8]

Not used by the enumeration of section 9.A. Used by Theorem 9.16.

Lemma 9.13. Let n_3 = 0. Then

    e(X_0) = e(X_2) - 27,   e(X_0, X_2) = 2 e(X_2),   e(X_0, X_1) = 180 - 4 e(X_2),
    e(X_1, X_2) = 300 - 4 e(X_2),   e(X_1) = 4 e(X_2),   (9.5)

and

    sum_{X_0} (a_2 - 6)^2 + sum_{X_1} (a_2 - 4)(a_2 - 5) + sum_{X_2} (a_2 - 2)(a_2 - 3) = 44.   (9.6)

Each of the three sums is >= 0. On X_0, a_2 - 6 = a_0. Hence sum_{X_0} a_0^2 <= 44, e(X_0) <= 9 and
27 <= e(X_2) <= 36.

Proof. Sum the lines of the table of (9.1), with a_3 = 0, over each class. On X_2: sum a_2 = 2 e(X_2); sum a_0 =
e(X_0, X_2) = 2 e(X_2); sum a_1 = e(X_1, X_2) = 300 - 4 e(X_2). On X_0: sum a_0 = 2 e(X_0) = e(X_0, X_2) - 6 * 9, so
e(X_0) = e(X_2) - 27; sum a_1 = e(X_0, X_1) = 180 - 2 e(X_0, X_2). On X_1: sum a_1 = 2 e(X_1) =
15 * 40 - 2 e(X_1, X_2) = 8 e(X_2).

For (9.6): the sum over all 99 vertices v of |N(v) n X_2|^2 is the number of triples (v, s, s') with s, s' in X_2 and
v adjacent to both, i.e. the sum over the ordered pairs (s, s') of X_2 of |N(s) n N(s')|: 14 for each of the 30
pairs with s = s', and 2 - [s ~ s'] for each of the 30 * 29 ordered pairs with s != s'. Total
420 + 1740 - 2 e(X_2) = 2160 - 2 e(X_2). Each of the 20 vertices of S has exactly 6 neighbours in X_2 (Lemma 9.5)
and contributes 36. So the sum of a_2^2 over the 79 outside vertices is 1440 - 2 e(X_2). Then, with the sums of
a_2 over the classes (2 e(X_2) on X_0, 300 - 4 e(X_2) on X_1, 2 e(X_2) on X_2) and the sizes 9, 40, 30:

    left side of (9.6) = (1440 - 2 e(X_2)) - 12 * 2 e(X_2) + 36 * 9 - 9 (300 - 4 e(X_2)) + 20 * 40
                           - 5 * 2 e(X_2) + 6 * 30
                       = (1440 + 324 - 2700 + 800 + 180) + (-2 - 24 + 36 - 10) e(X_2) = 44.

For an integer a, (a - 4)(a - 5) >= 0 and (a - 2)(a - 3) >= 0. On X_0, a_0 = a_2 - 6 by (9.1). So
sum_{X_0} a_0^2 <= 44; by the Cauchy-Schwarz inequality on the 9 terms, (2 e(X_0))^2 = (sum_{X_0} a_0)^2 <= 9 * 44 =
396, so 2 e(X_0) <= 19 and e(X_0) <= 9. Finally e(X_0) >= 0 gives e(X_2) >= 27. QED

9.6 Profile (4,2,1): the inner pair of X_0 is isolated; n_3 = 0: e(X_0) <= 8 [PROFILE_421, EDGE_BOUND]

These statements come from the Codex session (‘lemma/t20_CODEX/PROFILE_421.md‘, ‘EDGE_BOUND.md‘); according to those
files each was re-read line by line by the lab session. They are **not used** by the enumeration of section 9.A.
Theorem 9.14 holds for n_3 = 0 and for n_3 = 2 and is meant for section 9.B.

Theorem 9.14 [PROFILE_421]. Let the profile be (4,2,1) (n_3 = 0 or 2), and let u, u' = sigma u be the two
vertices of Z_p. Then no vertex of R_0 is adjacent to u or to u'. Consequently the subgraph induced on X_0 is the
disjoint union of the triangle {p, u, u'} and of the subgraph induced on R_0, with no edge between them.

Proof. R_0 = X_0 \ {p, u, u'} consists of 6 - n_3 words. Let nu = |N(u) n R_0|; by sigma, |N(u') n R_0| = nu.
We must show nu = 0.

The neighbours of u in N[p] are p and u' (Lemma 2.1 (b)), both in X_0. So a_0(u) = 2 + nu, and every neighbour of u
in X_2 is a word: the set A_u = N(u) n X_2 is contained in X_2'. sigma A_u = N(u') n X_2, and A_u n sigma A_u is
empty because the only common neighbour of the mates u, u' is p (Lemma 2.1 (c)). Put beta = a_3(u) = a_3(u')
(beta = 0 if n_3 = 0). By (9.1), |A_u| = a_2(u) = a_0(u) + 6 - 2 a_3(u) = 8 + nu - 2 beta. X_2' has
30 - 3 n_3 - 8 = 22 - 3 n_3 elements. Let

    L = X_2' \ (A_u u sigma A_u),   a sigma-invariant set with |L| = 22 - 3 n_3 - 2 (8 + nu - 2 beta).

So |L| = 6 - 2 nu if n_3 = 0, and |L| = 4 beta - 2 nu if n_3 = 2.

If n_3 = 2, write X_3 = {z, z'} with z' = sigma z. Then beta <= 1: if u ~ z and u ~ z', then (applying sigma to
u ~ z') u' ~ z, so z is adjacent to both u and u', and z != p, against Lemma 2.1 (c). If beta = 0, then
|L| = -2 nu >= 0 gives nu = 0, as wanted. So for n_3 = 2 we may assume beta = 1, and name z, z' so that u ~ z; then
u' ~ z', u not~ z' and u' not~ z.

Suppose nu > 0 and choose v in R_0 adjacent to u. v is a word. v is not adjacent to u' (Lemma 2.1 (c), v != p).
Put gamma = a_3(v) (gamma = 0 if n_3 = 0; gamma <= 2). a_0(v) >= 1. By (9.1), a_2(v) = a_0(v) + 6 - 2 gamma. Of the
two neighbours of v in N(p), one is u, which is in X_0; so at most one neighbour of v is in A_p, and

    |N(v) n X_2'| >= a_0(v) + 5 - 2 gamma.

The vertices u and v are adjacent, so they have exactly one common neighbour; every vertex of N(v) n A_u is one.
The vertices u' and v are distinct and non-adjacent, so they have exactly two common neighbours; u is one of them
(u ~ u' and u ~ v) and is not in X_2; every vertex of N(v) n sigma A_u is one. So |N(v) n A_u| <= 1 and
|N(v) n sigma A_u| <= 1. If n_3 = 2: when v ~ z, the vertex z is the common neighbour of u and v, so N(v) n A_u is
empty; when v ~ z', the vertex z' is the second common neighbour of u' and v, so N(v) n sigma A_u is empty. In all
cases

    |N(v) n A_u| + |N(v) n sigma A_u| <= 2 - gamma.

The neighbours of v in X_2' are in A_u, in sigma A_u or in L. Hence
a_0(v) + 5 - 2 gamma <= 2 - gamma + |N(v) n L| <= 2 - gamma + |L|.

- n_3 = 0 (gamma = 0, |L| = 6 - 2 nu): a_0(v) + 2 nu <= 3. As a_0(v) >= 1 and nu >= 1, a_0(v) = nu = 1 and all the
  inequalities are equalities: |L| = 4 and v is adjacent to all four vertices of L.
- n_3 = 2, beta = 1 (|L| = 4 - 2 nu): a_0(v) + 2 nu - gamma <= 1. As a_0(v) >= 1, nu >= 1 and gamma <= 2,
  a_0(v) = nu = 1, gamma = 2, and all the inequalities are equalities: |L| = 2 and v is adjacent to both vertices of
  L. Moreover gamma = 2 means v ~ z and v ~ z'.

L is sigma-invariant, so sigma v is adjacent to every vertex of L as well (apply sigma to v ~ l). For n_3 = 2,
sigma v is adjacent to sigma z = z' and to sigma z' = z. So v and sigma v have at least four common neighbours: the
four vertices of L if n_3 = 0, the two vertices of L and z, z' if n_3 = 2. But v is a word, so v and sigma v are
distinct, non-adjacent, and have exactly two common neighbours. Contradiction. Hence nu = 0. QED

The proof uses (9.1), the sizes of the classes, the profile, and lambda = 1, mu = 2. No enumeration.

Corollary 9.15. In profile (4,2,1), with u in Z_p and beta = a_3(u) (beta = 0 if n_3 = 0, beta <= 1 if n_3 = 2):

    a_0(u) = 2,   a_2(u) = 8 - 2 beta,   a_1(u) = 4 + beta,   rho(u) = 0.

Every vertex of R_0 has its two neighbours in N(p) in A_p u W. For n_3 = 0, the line "rho_Z = 1" of Lemma 9.11 does not occur.

Proof. nu = 0 in the proof above; a_1(u) = 20 - 2 a_2(u) - 3 a_3(u) by (9.1). QED

Theorem 9.16 [EDGE_BOUND]. Let n_3 = 0 (profile (3,4,0) or (4,2,1)). Then e(X_0) <= 8 and e(X_2) <= 35.

Proof. By Lemma 9.13, e(X_0) <= 9 and e(X_2) = e(X_0) + 27; so it is enough to exclude e(X_0) = 9.

By (9.1) with a_3 = 0: a_2 = a_0 + 6 on X_0, a_2 = a_0 + 3 on X_1, a_2 = a_0 on X_2. Substituting in (9.6):

    sum_{X_0} a_0^2 + sum_{X_1} (a_0 - 1)(a_0 - 2) + sum_{X_2} (a_0 - 2)(a_0 - 3) = 44.

By (9.5), sum_{X_1} a_0 = e(X_0, X_1) = 180 - 4 e(X_2) = 72 - 4 e(X_0). As (a - 1)(a - 2) = a (a - 1) - 2a + 2 and |X_1| = 40, the middle sum is sum_{X_1} a_0 (a_0 - 1) - 2 (72 - 4 e(X_0)) + 80. Hence

    sum_{X_0} a_0^2 + sum_{X_1} a_0 (a_0 - 1) + sum_{X_2} (a_0 - 2)(a_0 - 3) = 108 - 8 e(X_0).   (9.7)

The second and third sums are >= 0. Suppose e(X_0) = 9. Then sum_{X_0} a_0 = 18, so by the Cauchy-Schwarz inequality sum_{X_0} a_0^2 >= 18^2 / 9 = 36, which is the whole right side of (9.7). So there is equality everywhere:

    a_0 = 2 on X_0;   a_0 is 0 or 1 on X_1;   a_0 is 2 or 3 on X_2.

Put D_0 = {v in X_1 : a_0(v) = 0}, D_1 = X_1 \ D_0, F_3 = {v in X_2 : a_0(v) = 3}, F_2 = X_2 \ F_3. Then |D_1| = sum_{X_1} a_0 = 72 - 36 = 36 and |D_0| = 4; sum_{X_2} a_0 = e(X_0, X_2) = 2 e(X_2) = 72, so 2 |F_2| + 3 |F_3| = 72 with |F_2| + |F_3| = 30: |F_3| = 12 and |F_2| = 18.

Consider the integer vector psi = 3 * 1_{X_0} - A 1_{X_0} + 1 (1 = all-ones vector). By (1.1), A 1 = 14 * 1 and J 1_{X_0} = 9 * 1:

    A psi = 3 A 1_{X_0} - (12 * 1_{X_0} - A 1_{X_0} + 18 * 1) + 14 * 1 = -4 (3 * 1_{X_0} - A 1_{X_0} + 1) = -4 psi.

(No small-norm statement is applied to psi; it only encodes a counting identity at each vertex.) Its value at a vertex v is 3 [v in X_0] - |N(v) n X_0| + 1, that is

    2 on X_0,   1 on S (no vertex of X_0 has a neighbour in S),   1 on D_0,   0 on D_1,   -2 on F_3,   -1 on F_2.

For a vertex v and a set Y write d_Y(v) = |N(v) n Y|.

Let v be in D_0. It has 2 neighbours in S, none in X_0, and by (9.1) a_2(v) = 3 neighbours in X_2 and a_1(v) = 15 - 6 = 9 in X_1. The equation (A psi)(v) = -4 psi(v) = -4 reads

    -4 = 2 * 1 + (-2 d_{F_3}(v) - (3 - d_{F_3}(v))) + d_{D_0}(v),   i.e.   d_{F_3}(v) - d_{D_0}(v) = 3.

As d_{F_3}(v) <= 3, we get d_{F_3}(v) = 3 and d_{D_0}(v) = 0. So D_0 is independent and e(D_0, F_3) = 4 * 3 = 12.

Let v be in F_3. It has 4 neighbours in S, 3 in X_0, a_2(v) = 3 in X_2 and a_1(v) = 10 - 6 = 4 in X_1. The equation (A psi)(v) = -4 psi(v) = 8 reads

    8 = 4 * 1 + 3 * 2 + (-2 d_{F_3}(v) - (3 - d_{F_3}(v))) + d_{D_0}(v),   i.e.   d_{D_0}(v) - d_{F_3}(v) = 1.

So each of the 12 vertices of F_3 has at least one neighbour in D_0; as e(D_0, F_3) = 12, each has exactly one, and d_{F_3}(v) = 0: F_3 is independent. Then the 3 neighbours in X_2 of each vertex of F_3 are in F_2: e(F_3, F_2) = 36. Each of the 18 vertices of F_2 has a_2 = a_0 = 2 neighbours in X_2, 36 in all, and these are already accounted for by the edges between F_3 and F_2. So F_2 is independent.

Now h_2 >= 3, so there is a mate edge {a, sigma a} inside A_p, a subset of X_2. X_0 is sigma-invariant, so a_0(a) = a_0(sigma a): the two adjacent vertices a, sigma a are both in F_3 or both in F_2, which are independent sets. Contradiction. QED

Remark. Expanding (a_0 - 2)^2 in (9.7) (sum_{X_0} a_0 = 2 e(X_0)) gives the identity

    sum_{X_0} (a_0 - 2)^2 + sum_{X_1} a_0 (a_0 - 1) + sum_{X_2} (a_0 - 2)(a_0 - 3) = 16 (9 - e(X_0)),   (9.8)

and the theorem says that its right side is not 0.

Corollary 9.17. Let n_3 = 0.

(a) Profile (3,4,0): p has no neighbour in X_0, e(X_0) is even, so e(X_0) is in {0, 2, 4, 6, 8} and e(X_2) in {27, 29, 31, 33, 35}.

(b) Profile (4,2,1): e(X_0) = 3 + e(R_0), e(R_0) is in {0, 2, 4}, e(X_0) <= 7 and e(X_2) <= 34. The graph induced on the six vertices of R_0 (three word orbits) is one of: no edge; two disjoint edges and two isolated vertices; a 4-cycle and two isolated vertices; two disjoint paths with three vertices.

Proof. Let Y be a sigma-invariant set of words. sigma permutes the edges inside Y and fixes none of them: an edge {a, b} with sigma {a, b} = {a, b} would have sigma a = b (no vertex of Y is fixed), i.e. a ~ sigma a, which is false for a word. So e(Y) is even, and the edges inside Y form edge orbits {a, b}, {sigma a, sigma b} of two disjoint edges, a and b being in two different orbits of Y.

(a) Z_p is empty, so p has no neighbour in X_0 and the edges inside X_0 are the edges inside R_0 = X_0 \ {p}, a set of words. With Theorem 9.16 and (9.5). (b) By Theorem 9.14 the edges inside X_0 are the three edges of the triangle {p, u, u'} and the edges inside R_0. So e(R_0) = e(X_0) - 3 <= 5 and it is even. R_0 consists of three word orbits; an edge orbit joins two of them, as a matching of two disjoint edges. There are at most two edge orbits. None: no

edge. One: two disjoint edges. Two between the same two orbits: the complete bipartite graph on the two orbits, a 4-cycle. Two between two different pairs of orbits: the two pairs share one orbit, each of whose two vertices gets one edge of each edge orbit, and the result is two disjoint paths with three vertices. QED

9.A n_3 = 0: the enumeration [round 3]

In this subsection n_3 = 0. There are twelve cases: B is one of the six graphs, and the profile (h_2, h_1, h_0) is (3,4,0) or (4,2,1).

9.A.1 The determined sets, and the configurations of N(p)

    Y_0 = S u N[p]   (20 + 1 + 14 = 35 vertices).

The free vertices are F = X_2' = X_2 \ A_p: 30 - 2 h_2 words, i.e. n = 12 orbits for profile (3,4,0) and n = 11 orbits for profile (4,2,1).

Definition (N-configuration). Fix B. An **A-type** is an unordered pair {U, V} of disjoint deficient pairs of B such that every index of U has at most one B-neighbour in V and every index of V at most one B-neighbour in U (the working reports call it a rectangle). A **W-type** is an unordered pair {i, l} of distinct indices. A type has two **rows**, the sets of neighbours in S of the two vertices of a pair of that type: x_U u y_V and x_V u y_U for {U, V}; {x_i, y_l} and {x_l, y_i} for {i, l}. Two types are **compatible** if each row of the first has at most one vertex in common with each row of the second. An **N-configuration** for the profile (h_2, h_1, h_0) is a set of h_2 A-types and h_1 W-types, pairwise compatible, such that every index 0..9 lies in exactly two of the sets U u V, {i, l} of its types. (The h_0 Z-pairs carry no data.)

Lemma 9.18. Let n_3 = 0.

(a) The types of the pairs of N(p) form an N-configuration for the profile of the configuration.

(b) Y_0 is determined by B and this N-configuration. A state of level t (section 6.5) is determined by these data and, for each added orbit {s, sigma s}: U(s), V(s), the set N_p(s) = N(s) n N(p), and one pattern to each earlier orbit. Every adjacency is given by the following list.

pair of classes	adjacency	reason
S+ with S+, S- with S-	none	Lemma 4.10 (a)
S+ with S-	x_i ~ y_j iff i ~_B j	definition of B
p with S	none	Lemma 4.10 (b)
p with N(p)	all adjacent	definition
N(p) with N(p)	the seven mate edges only	Lemma 2.1 (b)
an A-pair of type {U, V} with S	one vertex adjacent to x_U and y_V only, its mate to x_V and y_U only	parameter (Lemma 9.8)
a W-pair of type {i, l} with S	one vertex adjacent to x_i and y_l only, its mate to x_l and y_i only	parameter (Lemma 9.8)
a Z-pair with S	none	delta = 0
s with S+, with S-	x_i for i in U(s), y_l for l in V(s)	parameters
s with p; s with sigma s	not adjacent	Lemma 9.10
s with N(p)	exactly N_p(s)	parameter
sigma s with anything	the images under sigma	sigma is an automorphism
two different added orbits	one of the four patterns	parameter

(c) The multiplicities are m(i, j) = need(i, j) - [{i, j} is one of the two deficient pairs of an A-type of the N-configuration], where need(i, j) = 2 - codeg_B(i, j). The **model**:

- (K1) U(s) is the unit chosen at the node; V(s) is a unit different from U(s);
- (K2) N_p(s) consists of two vertices of N(p) lying in two different pairs;
- (K3) s has at most 5 - |N_p(s) n A_p| neighbours among the other added orbits;
- (K4) a vertex of A_p has at most 4 - h_0 neighbours among the added vertices, a vertex of W at most 8 - h_0, a vertex of Z_p at most min(10, 12 - h_0).

  (K3) and (K4) are upper bounds only; no equality is required at the last level.

(d) Aut(B) acts on the N-configurations (through its action on the indices), and in a configuration with orbit graph B one may assume, after relabelling the ten orbits of the support, that the N-configuration is any chosen representative of its orbit under Aut(B).

Proof. (a) Lemma 9.8: (a) gives the A-types and W-types, (b) the condition on the indices, (c) the compatibility (two vertices of N(p) in different pairs have at most one common neighbour in S; applied to the four choices of one vertex in each pair). (b) The table restates the lemmas quoted; the vertices of N(p) have no names other than their types, and by Lemma 9.8 (d) the types are pairwise different. (c) By Lemma 9.6 (e) exactly need(i, j) vertices of X_2 have U = {i, j}; those in A_p are the vertices of the A-pairs, and a deficient pair is U(w) for at most one vertex w of N(p) (Lemma 9.8 (c)). (K1): Lemma 9.6 (e). (K2), (K3): Lemma 9.10 (the neighbours of s in X_2 are in A_p or in X_2'). (K4): Lemma 9.9, which gives the bounds 4 - h_0, 7 and 10; the bounds written in (K4) are those used by the programs, and they are weaker or equal (7 <= 8 - h_0 because h_0 <= 1; a Z-pair exists only if h_0 = 1, and then min(10, 12 - h_0) = 10). (d) A relabelling of the ten orbits by an element g of Aut(B) keeps B and replaces every type by its image under g. QED

9.A.2 Stage 1

> **Computation 9.19 (stage 1).** For each graph B and each of the two profiles: the number of A-types, the number of N-configurations, and the number of their orbits under Aut(B) are in the table below. For one representative of each orbit, the graph on the 35 vertices of Y_0 is built and tested (budget violations, PSD violations); the representatives that are kept are called the **bases**.
>
> | graph | Aut(B) | A-types | profile | N-configurations | orbits under Aut(B) | bases (kept) |
> |---|---|---|---|---|---|---|

> | I.0 | 16 | 114 | (4,2,1) | 178,768 | 11,322 | 3,022 |
> | I.0 | 16 | 114 | (3,4,0) | 1,229,304 | 77,391 | 11,150 |
> | I.1 | 4 | 161 | (4,2,1) | 722,524 | 180,920 | 56,300 |
> | I.1 | 4 | 161 | (3,4,0) | 3,508,550 | 878,137 | 219,589 |
> | I.2 | 10 | 140 | (4,2,1) | 412,875 | 41,441 | 13,925 |
> | I.2 | 10 | 140 | (3,4,0) | 2,301,905 | 230,609 | 55,573 |
> | I.3 | 4 | 146 | (4,2,1) | 506,294 | 126,655 | 32,022 |
> | I.3 | 4 | 146 | (3,4,0) | 2,663,180 | 666,032 | 112,368 |
> | I.4 | 4 | 144 | (4,2,1) | 495,024 | 124,367 | 21,561 |
> | I.4 | 4 | 144 | (3,4,0) | 2,606,408 | 653,570 | 76,509 |
> | I.5 | 20 | 140 | (4,2,1) | 435,540 | 22,111 | 2,710 |
> | I.5 | 20 | 140 | (3,4,0) | 2,335,670 | 117,846 | 10,676 |
>
> Totals: 17,396,042 N-configurations, 3,130,401 orbits, 615,405 bases.

The numbers of A-types, of N-configurations and of orbits are defined by the mathematics alone. The numbers of bases are sizes of kept lists (floating-point "kept").

9.A.3 Stage 2

> **Computation 9.20 (stage 2).** For each of the twelve cases and each of its bases, the search of section 6.5 with root the graph on Y_0 and the model (K1)-(K4) (no symmetry rule; rejections by budget violations, PSD violations, and, in the first program, the rank bound) has no node of level 6. With the rule of the first program for the choice of the units, the numbers of nodes, summed over the bases of each case, are:
>
> | case | nodes of level 0, 1, 2, ... | total | deepest level | rejected by the rank bound |
> |---|---|---|---|---|
> | I.0 (4,2,1) | 3022, 13440, 15198, 1843, 8 | 33,511 | 4 | 0 |
> | I.0 (3,4,0) | 11150, 59758, 118717, 56938, 3904, 37 | 250,504 | 5 | 0 |
> | I.1 (4,2,1) | 56300, 275161, 597528, 416434, 60366, 1128 | 1,406,917 | 5 | 2 |
> | I.1 (3,4,0) | 219589, 1360986, 3182403, 2207350, 304035, 5598 | 7,279,961 | 5 | 13 |
> | I.2 (4,2,1) | 13925, 78932, 188838, 125200, 14706, 172 | 421,773 | 5 | 0 |
> | I.2 (3,4,0) | 55573, 377489, 940441, 613858, 65549, 777 | 2,053,687 | 5 | 0 |
> | I.3 (4,2,1) | 32022, 115957, 179576, 89609, 10602, 218 | 427,984 | 5 | 0 |
> | I.3 (3,4,0) | 112368, 507111, 894928, 484271, 56282, 988 | 2,055,948 | 5 | 0 |
> | I.4 (4,2,1) | 21561, 72554, 97281, 41986, 3998, 28 | 237,408 | 5 | 0 |
> | I.4 (3,4,0) | 76509, 340792, 520484, 237910, 24232, 395 | 1,200,322 | 5 | 0 |
> | I.5 (4,2,1) | 2710, 10243, 11541, 2944, 86 | 27,524 | 4 | 0 |
> | I.5 (3,4,0) | 10676, 63403, 150861, 120573, 20662, 467 | 366,642 | 5 | 0 |
>
> Totals: 15,762,181 nodes; no node of level 6 or more (11 or 12 orbits are needed); 15 candidates rejected by the rank bound, all for the graph I.1: they are candidates of nodes of level 5 that pass the budget and PSD tests; on their 47 vertices (23 orbits and p) the 23 x 23 matrix G_odd of Proposition 6.4 has rank 23. No time-out.

Proposition 9.21 (hand statements + Computations 4.12, 9.19, 9.20). There is no configuration in case I with n_3 = 0.

Proof. Suppose there is one. By Corollary 4.13 its orbit graph B is one of I.0, ..., I.5 after a suitable labelling; by Lemma 9.3 and Proposition 9.4 its profile is (3,4,0) or (4,2,1). By Lemma 9.18 (a), (d) the types of the pairs of N(p) form an N-configuration, which may be taken to be the representative of its orbit under Aut(B); by Lemma 9.18 (b) and (6.2) the graph on Y_0 is one of the bases of Computation 9.19. By Lemma 9.18 (c) and Lemma 6.5 the search with this root has a node of level 11 or 12. This contradicts Computation 9.20, by (6.2). QED

Status: hand part (Lemmas 4.10, 5.1, 5.3-5.5, 9.1-9.3, Corollary 5.2, Proposition 9.4, Lemmas 9.5, 9.6, 9.8-9.10, 9.18, and Proposition 6.4 for the graph I.1) plus Computations 4.12, 9.19, 9.20. **For ten of the twelve cases the two computations rest on one program at the time of writing** (section 9.A.5).

9.A.4 Remarks on the two computations

- The table of Computation 9.20 is ‘lemma/t20_R3_I/final_table.out‘, which the working report says was recomputed from the stage-1 files and from the per-base result lines of the runs.
- In stage 1 the tests of the definition of an N-configuration (compatibility, the condition on the A-types) are budget conditions on subsets of Y_0; the test on the 35 vertices then adds the whole budget count and the two PSD matrices. In the summaries read for this text (the nine cases of the cloud run, file ‘ALL.txt‘ of its branch) no representative is rejected by a budget violation on the 35 vertices; all the rejections are PSD violations.
- The first program places the vertices of X_2’ only. The vertices of R_0 = X_0 \ N[p] and of X_1 \ W are never in a vertex set. Every search stops after at most 5 orbits of the 11 or 12, so the global statements of sections 9.4 to 9.6 (cells, edge counts, Theorems 9.14 and 9.16) could not have been used as tests and are not.
- *The rank nodes.* Without the rank bound, the 15 candidates of I.1 would be nodes of level 6 and the first program would have to search below them. The second program does this: it does not reject such a node by rank at once, but explores the subtree below it with its own tests and its own choice of the units; according to its header, only if such an exploration reached its cap (20,000 nodes or 3,600 CPU seconds) or found a complete configuration would it reject the node by an exact rank computation of its own, counted separately. It has been run, at the time of the working report, on the single base that holds the two rank nodes of I.1 (4,2,1) (base number 12632): in the words of the report, "both nodes die at once (4 candidates, all rejected by budget counts)". For the 13 rank nodes of I.1 (3,4,0) nothing but the rank bound is available at the time of writing.

9.A.5 Programs, and the number of programs of each case

All in ‘lemma/t20_R3_I‘; the cloud outputs are on the git branches named below (appendix A.5).

First program. Stage 1: ‘stage1c.py‘ (with the tool file ‘r3lib.py‘, and ‘eng.py‘ for the PSD test), run in parts and merged by ‘stage1_merge.py‘. It lists the A-types and W-types, generates every N-configuration exactly once, keeps one representative per orbit under Aut(B) (the smallest image) and checks that the sizes of the orbits of the representatives add up to the total; each representative is then tested on the 35 vertices (exact budget

count; integer vector for a PSD rejection). Stage 2: ‘run_stage2.py‘ with the engine ‘eng6.py‘ and the model ‘cases6.py‘ (class ‘CaseIN6‘); according to its header, ‘eng6.py‘ is the engine ‘eng5.py‘ of sections 7 and 8 "with lazy single-orbit certificates"; the two give identical counters on 16 bases of I.0 (4,2,1) (‘cmp_eng5_I0_421.out‘). The unit is chosen by a look-ahead restricted to three units (option ‘width=3‘ in the driver ‘cloud_case.sh‘). ‘summary2.py‘ checks that the parts tile the list of bases, that no base timed out or raised an error, and that all parts used the same list (hash).

- I.0 (4,2,1), I.0 (3,4,0), I.5 (4,2,1) were run on the laptop ("COMPLETE: True" according to the working report; the per-base files are in the session scratchpad, not in git).
- The nine other cases were run on one cloud machine (job ‘t20_I_rest‘, 128 parts per case, first program only). The result files of its branch were re-counted on the laptop by ‘check_cloud.py‘, without using the summary written by the job: for each of the nine cases, 128 parts of 128 present, every base index exactly once and in its part, no time-out, no error, the hash of the list of bases recomputed and equal in every part, 0 complete configurations (‘cloud_check_t20_I_rest.out‘). For six of these cases the list of bases had also been computed on the laptop: identical lists (‘cloud_vs_laptop_stage1.out‘); this is the same program run twice, or an earlier version of it, not a second program.

Second program. ‘in_verify.py‘. According to its header it imports nothing from the engines, from ‘cases6.py‘, ‘r3lib.py‘ or ‘stage1c.py‘: it has its own model, its own enumeration of the N-configurations (A-types first, then the W-types as a graph with prescribed degrees), its own computation of Aut(B), its own budget counts and its own PSD test (integer vector on a sigma-block, or exact rational elimination). Stage 1 is compared with the stage-1 file of the first program. Stage 2 re-traverses the search trees of the first program: the unit of every node is read from the plan stored with the results, all candidates of that unit are regenerated and tested, and the number kept must equal the plan’s. Below a node that the first program rejected by rank it searches by itself (section 9.A.4).

State of the second program **at the time of the working report** (‘lemma/t20_R3_I/REPORT.md‘):

case	stage 1	stage 2
I.0 (4,2,1)	identical numbers and identical list of bases	complete: 3,022 bases, 33,511 nodes, same kept count at every node, 1,144,830 certificates
I.5 (4,2,1)	identical numbers and identical list of bases	complete: 2,710 bases, 27,524 nodes, same kept count at every node, 1,088,445 certificates
I.0 (3,4,0)	identical numbers and identical list of bases	372 of the 11,150 bases
I.1 (4,2,1)	-	one base (number 12632, the one with the two rank nodes)
the eight other cases	-	-

So **two of the twelve cases have two programs**, and ten have one. According to the working report and to ‘T20_BOARD.md‘ (lines C2, C3, C4) the second program is running on all twelve cases (cloud jobs ‘t20_Iv_B‘ to ‘t20_Iv_F‘, and ‘t20_I1_340‘), and one part of I.1 (4,2,1) has to be re-run with the corrected version of the verifier; nothing of these runs is used here.

What the two programs share: the hand model (Lemmas 9.6, 9.8, 9.9, 9.10, 9.18), the structure file ‘structures_I.json‘ (the six graphs), and the units chosen by the first program. They share no code.

Sample of certificates. ‘sample_certs_I0_421.jsonl‘ contains ten rejection records of I.0 (4,2,1), at levels 0 to 4, each with the complete graph concerned and an integer vector of small support; ‘check_sample.py‘ re-checks them from the records alone (the graph is symmetric, sigma is an automorphism, the support is B, no budget violation, and the value of the quadratic form). The runs themselves were made without saving their certificates (the driver ‘cloud_case.sh‘ passes no certificate option to the first program; the second program only counts its certificates).

9.B n_3 = 2: the hand lemmas, and the enumeration [round 3]

Sections 9.B.1 to 9.B.5 are hand lemmas on this sub-case. Section 9.B.6 is the enumeration that excludes it (Computation 9.29, Proposition 9.30); it uses Lemma 9.22 (a) and Lemma 9.23 only. Sections 9.B.3 to 9.B.5 are kept for the record and are used by nothing.

Throughout 9.B: n_3 = 2, so (n_0, n_1, n_2) = (7, 46, 24);

    X_3 = {z, z’} with z’ = sigma z;   (U ; V) is the type of z (|U| = |V| = 3), and (V ; U) that of z’;
    X_0’ = X_0 \ {p}   (six vertices, three orbits);   T = N(z) n X_0,   T’ = sigma T = N(z’) n X_0;
    alpha = a_2(z) = a_2(z’);   eps = |U n V|.

From the earlier sections, the following apply to this sub-case as they stand: Lemmas 9.1 to 9.3 and Proposition 9.4 (the profile is (3,4,0) or (4,2,1)), Lemma 9.5 with (9.3), Lemma 9.6 (a)-(d), Lemmas 9.7 and 9.8, Theorem 9.14 and Corollary 9.15. Lemmas 9.6 (e), 9.9 to 9.13, Theorem 9.16 and Corollary 9.17 are statements for n_3 = 0 and do not apply.

9.B.1 The orbit X_3 [H1]

Lemma 9.22.

(a) z is a word; z not~ z’ and z not~ p; N(z) n N(z’) is one word orbit {w, sigma w}. a_3(z) = 0, T is contained in X_0’, and (a_0, a_1, a_2)(z) is (3, 5, 0) or (4, 3, 1). In particular alpha is 0 or 1 and |T| = 3 + alpha.

(b) eps <= 1, and exactly one of the following four cases holds.

case	the orbit N(z) n N(z’)	consequences
(i-S)	{x_i, y_i} for an index i	U n V = {i}; T n T’ is empty; alpha = 0; |T| = 3
(i-X1)	an orbit of X_1	U n V is empty; T n T’ is empty; alpha = 0; |T| = 3
(ii)	an orbit {u, sigma u} of X_0’	U n V is empty; alpha = 0; T = {u, sigma u, v}

| (iii) | an orbit {u, sigma u} of X_0' | U n V is empty; alpha = 1; T = {u, sigma u, v, v_2}, with v and v_2 in the two other orbits of X_0' |

(c) In cases (ii) and (iii) the orbit {u, sigma u} is a word orbit and N(u) n N(sigma u) = {z, z'}.

(d) In cases (i-S) and (i-X1), T contains exactly one vertex of each of the three orbits of X_0' and T' is the complementary set: a_3 = 1 on X_0'. In case (ii): a_3 = 2 on {u, sigma u}, 1 on {v, sigma v}, 0 on the third orbit. In case (iii): a_3 = 2 on {u, sigma u} and 1 on the four other vertices.

(e) For x in X_0': a_2(x) = a_0(x) + 6 - 2 a_3(x) and a_1(x) = 8 - 2 a_0(x) + a_3(x). Hence a_0(x) <= 4 if a_3(x) <= 1, and a_0(x) <= 5 if a_3(x) = 2. In cases (ii) and (iii), a_0(u) <= 2.

(f) X_0' contains exactly h_0 inner orbits (none for profile (3,4,0), one for profile (4,2,1)); the others are word orbits.

Proof. (a) z is a word by Lemma 9.1; Lemma 2.1 (a), (d) give z not~ z', z not~ p and the word orbit. The only other vertex of X_3 is z', so a_3(z) = 0. By (9.1), a_0(z) = a_2(z) + 3 and a_1(z) = 5 - 2 a_2(z). z not~ p, so T is contained in X_0'. By Lemma 5.4 (a), 2 |T| - 2 <= |X_0'| = 6, so |T| <= 4 and a_2(z) <= 1.

(b) The orbit {w, sigma w} lies in S, in X_0', in X_1 or in X_2 (it is not in X_3, and it does not contain p).

- If it is in S, it is {x_i, y_i} for an index i; z ~ x_i means i in U, and z ~ y_i means i in V. Conversely every index of U n V gives the two common neighbours x_i, y_i of z and z' (z' ~ x_i iff i in V, z' ~ y_i iff i in U). So eps <= 1, and eps = 1 exactly when the orbit is in S.
- It is not in X_2: z would have two neighbours in X_2, against (a).
- T n T' = N(z) n N(z') n X_0 is sigma-invariant and has no fixed vertex, so it is empty or it is the whole orbit {w, sigma w}. If the orbit is in S or in X_1, T n T' is empty, so 2 |T| <= 6: |T| = 3 and alpha = 0. If the orbit is {u, sigma u} in X_0', then T n T' = {u, sigma u}; for |T| = 3 this is case (ii); for |T| = 4, T = {u, sigma u, v, v_2}, and v, v_2 are not mates (otherwise they would be in T n T' too), so they are in the two other orbits: case (iii).

(c) Corollary 2.3 (c), applied to the word z and the orbit {u, sigma u}.

(d) In cases (i-S), (i-X1): T and T' = sigma T are disjoint 3-subsets of the 6-set X_0', so T' is the complement of T, and T contains no two mates. Cases (ii), (iii): read off T and T' = sigma T.

(e) (9.1) with d = 0: a_2 = a_0 + 6 - 2 a_3 and a_1 = 20 - 2 a_2 - 3 a_3 = 8 - 2 a_0 + a_3 >= 0. In cases (ii), (iii), u is a word, so u not~ p and u not~ sigma u; and u is adjacent to at most one vertex of each of the two other orbits of X_0': if u ~ v and u ~ sigma v, then sigma u ~ sigma v and sigma u ~ v, so v would be a common neighbour of u and sigma u other than z, z', against (c).

(f) The inner vertices of X_0 are those of Z_p. QED

9.B.2 The units and the exact numbers for X_2 [SPEC_N3, section 3]

Lemma 9.23.

(a) For every pair {i, j} of distinct indices, the number of vertices s of X_2 with U(s) = {i, j} is exactly

$$\text{need_z}(i, j) = 2 - \text{codeg_B}(i, j) - [i, j \text{ in } U] - [i, j \text{ in } V], \qquad (9.9)$$

and the sum of need_z over all pairs is 24. In particular the three pairs inside U and the three pairs inside V are deficient pairs of B. For s in X_2: V(s) = U(sigma s) != U(s).

(b) No vertex of X_2 is adjacent to both z and z'; so a_3(s) is 0 or 1 for s in X_2. No vertex of N(p) is adjacent to both z and z'.

(c) For s in X_2, with W_s = N(s) n X_0': a_0(s) = [s ~ p] + |W_s| <= 4, a_2(s) = a_0(s) - 2 a_3(s), and a_1(s) = 10 - 2 a_2(s) - 3 a_3(s). The number of neighbours of s in X_2 other than its mate is exactly |W_s| - 2 a_3(s), whether s is a word or an inner vertex.

(d) p has exactly 2 h_2 neighbours in X_2 (the vertices of A_p); z and z' have alpha neighbours in X_2 each; a vertex x of X_0' has exactly a_0(x) + 6 - 2 a_3(x).

(e) A vertex of A_p is adjacent to no vertex of Z_p and to at most one vertex of each word orbit of X_0'. A word of X_2 is not adjacent to both vertices of Z_p.

(f) Let w be in N(p), delta = d(w), rho(w) = |N(w) n R_0| and a_3(w) in {0, 1}. Then rho(w) <= 3 - h_0, and the number of neighbours of w in X_2' is rho(w) - 2 a_3(w) if delta = 2, 4 + rho(w) - 2 a_3(w) if delta = 1, and 8 + rho(w) - 2 a_3(w) if delta = 0; in the last case it is also at most (20 - 3 a_3(w)) / 2.

Proof. (a) x_i and x_j are non-adjacent and have exactly two common neighbours, codeg_B(i,j) of them in S (Lemma 4.10). A common neighbour outside S has d >= 2, so it is in X_2 or it is z or z'; z is one iff i, j are in U, and z' iff i, j are in V. The sum over the 45 pairs of 2 - codeg_B is 30 (section 4.10); the three pairs inside U and the three pairs inside V are six different pairs, because eps <= 1. As need_z >= 0, these six pairs have codeg_B <= 1. The last statement is section 5.5 and Lemma 5.5 (a).

(b) N(z) n N(z') is an orbit that is not in X_2 (Lemma 9.22 (b)) and is a word orbit (Lemma 9.22 (a)).

(c) The neighbours of s in X_0 are p (iff s is in N(p)) and the vertices of W_s. By Lemma 5.4, 2 a_0(s) - 2 <= 6 if s is a word and 2 a_0(s) - 1 <= 7 if s is in N(p); in both cases a_0(s) <= 4. The two formulas are (9.1) with d = 2. If s is a word, its mate is not a neighbour and a_2(s) = |W_s| - 2 a_3(s). If s is in N(p), then a_2(s) = 1 + |W_s| - 2 a_3(s), and its mate, which is in X_2, is one of these neighbours.

(d) The neighbours of p are the vertices of N(p). For z: (a) of Lemma 9.22. For x: Lemma 9.22 (e).

(e) Lemma 2.1 (b), Corollary 2.3 (b) and Lemma 2.1 (c).

(f) |X_0| = 7, so R_0 has 6 - 2 h_0 vertices, in 3 - h_0 word orbits, and rho(w) <= 3 - h_0 by Corollary 2.3 (b). a_3(w) <= 1 by (b). As in the proof of Lemma 9.9, a_0(w) = 1 + [delta = 0] + rho(w), and by (9.1) a_2(w) = a_0(w) + 6 - 3 delta - 2 a_3(w); the mate of w is in X_2 iff delta = 2. For delta = 0, a_1(w) = 20 - 2 a_2(w) - 3 a_3(w) >= 0. QED

Remark on the bounds of the working files. ‘SPEC_N3.md‘ and NOTES H9 state the bound on a_2(s) in the form "a_2(s) <= 5 if a_3(s) = 0 and a_2(s) <= 3 if a_3(s) = 1" (from a_1(s) >= 0). The bound of (c), a_0(s) <= 4, i.e. a_2(s) <= 4 - 2 a_3(s), is stronger; it is the one written in the header of ‘lemma/t20_R3_I/cases6.py‘. Both are valid. The bounds of (f) are those of the same header.

9.B.3 Profile (4,2,1): the inner pair of X_0 [PROFILE_421, TRIANGLE_REDUCTION]

By Theorem 9.14 (which covers n_3 = 2), in profile (4,2,1) the two vertices u, u’ of Z_p have no neighbour in R_0, and by Corollary 9.15, a_0(u) = 2, a_2(u) = 8 - 2 beta, a_1(u) = 4 + beta with beta = a_3(u) <= 1.

Lemma 9.24 [TRIANGLE_REDUCTION]. In profile (4,2,1) with n_3 = 2, beta = 1: each of u, u’ is adjacent to exactly one of z, z’ (after exchanging the names z, z’ if necessary: z ~ u and z’ ~ u’).

Proof. Let Delta = {p, u, u’}, a triangle. A vertex outside Delta has at most one neighbour in Delta: two of them would be adjacent, and it would be a second common neighbour of this adjacent pair, which already has the third vertex of Delta. For t in Delta let Gamma_t = N(t) \ Delta (12 vertices); the three sets are disjoint, and their union Gamma has 36 vertices. A vertex v outside Delta u Gamma is non-adjacent to each t of Delta, so it has exactly two common neighbours with t; they are neighbours of t, and they are not in Delta (v has no neighbour in Delta); so v has exactly two neighbours in each Gamma_t, six in Gamma.

No vertex of Gamma is in S (the vertices of Delta are in X_0 and have no neighbour in S), and none is in X_0 (the neighbours of p in X_0 are u and u’; those of u are p and u’, by Theorem 9.14; likewise for u’).

Suppose beta = 0. Then z is adjacent to neither u nor u’ (a_3(u) = a_3(u’) = 0), nor to p. So z is outside Delta u Gamma and has six neighbours in Gamma. None of them is in S or in X_0, and none is z’ (z not~ z’). So a_1(z) + a_2(z) >= 6. But by (9.1), a_1(z) + a_2(z) = 5 - a_2(z) <= 5. Contradiction. So beta >= 1, and beta <= 1 was shown in the proof of Theorem 9.14. QED

In the cases of Lemma 9.22 this says: in profile (4,2,1) the inner orbit Z_p of X_0’ is not the orbit with a_3 = 0 of case (ii), and it is not the orbit {u, sigma u} of cases (ii), (iii) (which is a word orbit).

9.B.4 The matching in N(z) [H3]

Lemma 9.25. Let R = N(z) n X_1 (5 - 2 alpha vertices), and let e(U, V) be the number of pairs (i, l) with i in U, l in V and i ~_B l, f the number of edges inside T, and g the number of edges inside R u (N(z) n X_2). Then

    e(U, V) + f - g = 2 + alpha.

Proof. Every neighbour v of z has exactly one neighbour inside N(z), namely the unique common neighbour of the adjacent pair z, v. So N(z) induces a perfect matching on its 14 vertices: the 3 vertices x_i (i in U), the 3 vertices y_l (l in V), the 3 + alpha vertices of T, and the 5 - alpha vertices of R u (N(z) n X_2). The edges inside x_U u y_V join an x_i to a y_l (S+ and S- are independent): there are e(U, V) of them. The vertices of T have no neighbour in S. So 6 - 2 e(U, V) vertices of x_U u y_V and 3 + alpha - 2f vertices of T have their partner in R u (N(z) n X_2), and the other vertices of this set are matched among themselves:

    5 - alpha = (6 - 2 e(U, V)) + (3 + alpha - 2f) + 2g.   QED

9.B.5 Edge counts and a quadratic identity [H4]

Lemma 9.26.

    e(X_0, X_3) = 6 + 2 alpha,   e(X_2, X_3) = 2 alpha,   e(X_1, X_3) = 10 - 4 alpha,
    e(X_0) = e(X_2) + 4 alpha - 15,   e(X_0, X_2) = 2 e(X_2) + 4 alpha,   e(X_1, X_2) = 240 - 4 e(X_2) - 6 alpha,
    e(X_0, X_1) = 122 - 4 e(X_2) - 14 alpha,   e(X_1) = 90 + 4 e(X_2) + 12 alpha,   (9.10)

and

    sum_{X_0} (a_2 - 5)^2 + sum_{X_1} (a_2 - 3)(a_2 - 4) + sum_{X_2} (a_2 - 1)(a_2 - 2) = 55 - 16 eps.   (9.11)

Each of the three sums is >= 0. On X_0’, a_2 - 5 = a_0 + 1 - 2 a_3; at p, (a_2 - 5)^2 = (2 h_2 - 5)^2, which is 1 for profile (3,4,0) and 9 for profile (4,2,1).

Proof. By Lemma 9.22 (a), each of z, z’ has 3 + alpha neighbours in X_0, alpha in X_2, 5 - 2 alpha in X_1 and none in X_3: this gives the first line. Summing the lines of the table of (9.1) over the classes: on X_2, e(X_0, X_2) = sum a_0 = 2 e(X_2) + 2 e(X_2, X_3) and e(X_1, X_2) = sum a_1 = 10 * 24 - 4 e(X_2) - 3 e(X_2, X_3); on X_0, 2 e(X_0) = sum a_0 = e(X_0, X_2) + 2 e(X_0, X_3) - 6 * 7 and e(X_0, X_1) = sum a_1 = 20 * 7 - 2 e(X_0, X_2) - 3 e(X_0, X_3); on X_1, 2 e(X_1) = sum a_1 = 15 * 46 - 2 e(X_1, X_2) - 3 e(X_1, X_3).

For (9.11): as in the proof of Lemma 9.13, the sum over all 99 vertices v of |N(v) n X_2|^2 is 24 * 14 + 24 * 23 * 2 - 2 e(X_2) = 1440 - 2 e(X_2). A vertex x_i has [i in U] + [i in V] neighbours in X_3, hence 6 - 2 ([i in U] + [i in V]) neighbours in X_2 by (9.3): 2 for the eps indices of U n V, 4 for the 6 - 2 eps indices in exactly one of U, V, and 6 for the 4 + eps others; the same holds for y_i. So S contributes 2 (4 eps + 16 (6 - 2 eps) + 36 (4 + eps)) = 480 + 16 eps, and z, z’ contribute 2 alpha^2 = 2 alpha. The sum of a_2^2 over X_0 u X_1 u X_2 is therefore 960 - 2 e(X_2) - 16 eps - 2 alpha. With the sums of a_2 over the classes (2 e(X_2) + 4 alpha on X_0, 240 - 4 e(X_2) - 6 alpha on X_1, 2 e(X_2) on X_2) and the sizes 7, 46, 24:

```
    left side of (9.11) = (960 - 2 e(X_2) - 16 eps - 2 alpha) - 10 (2 e(X_2) + 4 alpha) + 25 * 7
                            - 7 (240 - 4 e(X_2) - 6 alpha) + 12 * 46 - 3 * 2 e(X_2) + 2 * 24
                        = (960 + 175 - 1680 + 552 + 48) + (-2 - 20 + 28 - 6) e(X_2) + (-2 - 40 + 42) alpha - 16 eps
                        = 55 - 16 eps.
```

The products (a - 3)(a - 4) and (a - 1)(a - 2) are >= 0 for an integer a. On X_0', a_2 = a_0 + 6 - 2 a_3 (Lemma 9.22 (e)); a_2(p) = 2 h_2. QED

9.B.6 The enumeration: N(p) and the orbit X_3 in the base [SPEC_ROUTE_B_N3]

The determined set is the one of section 9.A with the orbit X_3 added:

```
    Y_z = S u N[p] u {z, z'}      (20 + 1 + 14 + 2 = 37 vertices).
```

The free vertices are F = X_2' = X_2 \ A_p: 24 - 2 h_2 words, i.e. n = 9 orbits for profile (3,4,0) and n = 8 orbits for profile (4,2,1). The set X_0' is not in the vertex set, and the four cases of Lemma 9.22 (b) are not separated: only Lemma 9.22 (a) and Lemma 9.23 are used, and they hold in every case. Sections 9.B.3 to 9.B.5 are not used.

The **attachment** of z is the set N_p(z) = N(z) n N(p).

Lemma 9.27. Let n_3 = 2.

(a) The types of the pairs of N(p) form an N-configuration for the profile (definition of 9.A.1).

(b) N_p(z) consists of two vertices lying in two different pairs of N(p), and N_p(z') = sigma N_p(z). At most one of the two vertices is in A_p.

(c) Y_z is determined by B, the type (U ; V) of z, the N-configuration and N_p(z). A state of level t (section 6.5) is determined by these data and, for each added orbit {s, sigma s}: U(s), V(s), N_p(s) = N(s) n N(p), the two bits [s ~ z], [s ~ z'], and one pattern to each earlier orbit. Every adjacency is given by the list of Lemma 9.18 (b), in which the line "s with p; s with sigma s" holds because s is a word, together with:

pair of classes	adjacency	reason
z with S+, with S-	x_i for i in U, y_l for l in V	parameter (the type of z)
z with p; z with z'	not adjacent	Lemma 9.22 (a)
z with N(p)	exactly N_p(z)	parameter
s with z, with z'	the two bits, not both 1	parameters; Lemma 9.23 (b)
z' with anything	the images under sigma	sigma is an automorphism

(d) The multiplicities are m_z(i, j) = need_z(i, j) - [{i, j} is one of the two deficient pairs of an A-type of the N-configuration], with need_z as in (9.9); their sum is 24 - 2 h_2. The **model**:

- (Z1) U(s) is the unit chosen at the node; V(s) is a unit different from U(s);
- (Z2) N_p(s) consists of two vertices of N(p) lying in two different pairs;
- (Z3) s is adjacent to at most one of z, z';
- (Z4) with a_3(v) = [v ~ z] + [v ~ z']: a vertex s of an added orbit has at most 4 - 2 a_3(s) - |N_p(s) n A_p| neighbours among the other added orbits; z has at most 1 - |N_p(z) n A_p| neighbours among the added vertices; a vertex w of N(p) has at most 3 - h_0 - 2 a_3(w) neighbours among the added vertices if w is in A_p, 7 - h_0 - 2 a_3(w) if w is in W, and min(11 - h_0 - 2 a_3(w), floor((20 - 3 a_3(w)) / 2)) if w is in Z_p.

  (Z4) consists of upper bounds only; no equality is required at the last level.

(e) Aut(B) acts on the data of (c) through the indices, and the exchange of the names z and z' replaces (U ; V) by (V ; U) and N_p(z) by sigma N_p(z), without changing the N-configuration. So one may assume that the type of z is a chosen representative of its class under Aut(B) and the exchange, and then that the N-configuration is any chosen representative of its orbit under the group G_z of the automorphisms of B preserving {U, V}. The two vertices of a Z-pair carry the same data, so for a Z-pair one of its two vertices is enough in N_p(z).

Proof. (a) Let w be in N(p). d(w) <= 2 (no vertex of N(p) has d = 3: section 9.1), and w is not adjacent to x_i together with y_i: then sigma w would be adjacent to y_i and x_i, and x_i would be adjacent to both vertices of a pair of N(p), which already have the common neighbour p. If d(w) = 2, with w adjacent to x_U and y_V: U and V are disjoint; the two vertices of x_U have the common neighbour w outside S, so need_z(U) >= 1 and U is a deficient pair, and likewise V; for i in U, the adjacent vertices x_i and w have at most one common neighbour, so i has at most one B-neighbour in V, and likewise with U and V exchanged. Two vertices of N(p) in two different pairs are non-adjacent and have the common neighbour p, so they have at most one common neighbour in S: this is the compatibility. Every x_i is a word and has exactly two neighbours in N(p), in two different pairs (section 2): every index lies in exactly two of the sets.

(b) z is a word (Lemma 9.22 (a)), so it has exactly two neighbours in N(p), in two different pairs (section 2). A vertex of A_p adjacent to z is a neighbour of z in X_2, and z has at most one (Lemma 9.22 (a): alpha <= 1).

(c) The lines of Lemma 9.18 (b) concern S, N[p] and the added orbits; each is proved as there, with statements that hold for n_3 = 2 as well (the list at the beginning of 9.B) and the fact that the added vertices are words. The five new lines carry their reasons.

(d) By Lemma 9.23 (a) the number of vertices s of X_2 with U(s) = {i, j} is need_z(i, j); a vertex of A_p of type {U', V'} is one of them for {i, j} = U', and its mate for {i, j} = V'. So the vertices of X_2' realise every pair exactly m_z times, and the sum is 24 - 2 h_2. (Z1): V(s) = U(sigma s) is a unit, different from U(s) (Lemma 9.23 (a)). (Z2): s is a word. (Z3): Lemma 9.23 (b). (Z4): for s, Lemma 9.23 (c): the neighbours of s in X_2 are the vertices of N_p(s) n A_p and its neighbours in X_2', and their number is |W_s| - 2 a_3(s) <= 4 - 2 a_3(s); for z, Lemma 9.22 (a) (alpha <= 1); for w, Lemma 9.23 (f) with rho(w) <= 3 - h_0.

(e) The relabellings preserve every statement used. QED

Lemma 9.28 (what the computation proves). Suppose that a configuration in case I with n_3 = 2 exists, with orbit graph B. Then, after relabelling, the type of z is one of the representatives of Computation 9.29 (a), the N-configuration is one of the representatives of (b) for that type and the profile, and the base given by them and by N_p(z) passes the budget test and has positive semidefinite matrices M1, M2 on Y_z. From that base, the search over units of section 6.5, with the model (Z1)-(Z4), reaches a state of level n (n = 9 or 8), whatever the rule choosing the unit.

Proof. The graph on S u {p, z, z'} is determined by B and the type of z, and it is an induced subgraph of G, so it passes the budget test (section 6.2): the type is in the list of Computation 9.29 (a) up to the relabelling of Lemma 9.27 (e). Lemma 9.27 (a), (b) give the form of the data, and (c) the fact that Y_z and every state are determined sets (section 6.3), so the tests of section 6 apply to them and the true ones pass. The true orbits of X_2' satisfy (Z1)-(Z4) by Lemma 9.27 (d), and a state made of the base and of true orbits, not all of them, has a unit realised fewer than m_z times; so the argument of section 6.5 applies as in section 9.A. QED

Computation 9.29 (two programs; folder `lemma/t20_R3_I`). For the six graphs:

(a) *Types of z.* The ordered pairs (U ; V) of 3-subsets, U != V, for which the graph on the 23 vertices S u {p, z, z'} passes the budget test number 8, 46, 30, 24, 24, 50 for I.0, ..., I.5; they form 1, 9, 3, 4, 5, 3 classes under Aut(B) and the exchange, 25 in all (representatives: `SPEC_ROUTE_B_N3.txt`, section 8; computed by `ztypes.py`; the same list was obtained independently in the lab's notes H2).

(b) *Stage 1: the bases.* For each of the 25 types and each of the two profiles (50 cases): all N-configurations (1,173,102 for profile (4,2,1) and 12,557,760 for profile (3,4,0), over the 25 types), one representative per orbit under G_z (686,675 and 7,461,476), and for each representative all the attachments: two vertices of N(p) in two different pairs, one vertex only for a Z-pair (84 of them, 72 when there is a Z-pair). A base is rejected by the budget test on the 37 vertices, or by an integer vector with negative form for M1 or M2; otherwise it is kept. (The last sentence of Lemma 9.27 (b) is implemented as a separate filter and rejects nothing: such attachments already fail the budget test.) Bases kept:

graph	types	profile (4,2,1)	profile (3,4,0)
I.0	1	27	199
I.1	9	5,059	30,909
I.2	3	1,420	8,138
I.3	4	1,153	9,743
I.4	5	751	5,791
I.5	3	147	1,191
all	25	8,557	55,971

The 64,528 bases are 64,498 classes under G_z and the exchange (8,552 and 55,946): in eight cases some N-configurations have a stabiliser and a class is listed twice. The type 034;178 of I.1 has no base in either profile.

(c) *Stage 2: the search.* From each of the 64,528 bases, the search over units with the model (Z1)-(Z4), the budget test and the semidefiniteness test at every node. Numbers of nodes with 0, 1, 2, 3 added orbits:

```
profile (4,2,1):    8,557 /  1,494 /   190 /   8
profile (3,4,0):   55,971 / 17,994 / 2,646 / 108
```

86,968 nodes in all. **No node has four added orbits; no state of level n is reached.** No time limit was reached, no base was abandoned, and the rank bound of section 6.4 was never used.

The two programs. First program: `stage1c.py` with the option `z=` and `stage1_merge.py` (stage 1), `cases6.py`, `eng6.py`, `run_stage2.py` (stage 2); its coverage is recomputed from its output files by `check_n3.py` (`n3_check.out`: one stage-1 file and one stage-2 part per case, every base index exactly once, the hash of the base list recomputed, no time-out, no error). Second program: `in_verify.py`, modes `basesz`, `follow`, `mergez`: its own enumeration of the N-configurations and of the attachments and its own tests; it shares no code with the first program. It finds the same classes of bases in the 50 cases, and it re-traverses the 86,968 nodes with the unit chosen by the first program and finds the same number of kept candidates at every node (`n3_verify.out`, one line per case). The two programs share the model, which is this section and `SPEC_ROUTE_B_N3.txt`. Everything ran on one laptop (about 7 CPU-hours); `python n3_all.py <out> <workers> both 340 421` reproduces both programs.

What is exact. Every rejection, in both programs, is exact: a common-neighbour count in integers, or an integer vector x with x^T M x < 0 evaluated in integers (second program: 1,950,773 such vectors in stage 1, for 362,754,336 attachments tested, and 1,978,816 in stage 2; exact rational elimination was needed 0 times). A floating-point test is only used to keep a base or a candidate. So the statement "no state of level n" rests on exact rejections; the floating-point decisions affect only the sizes of the kept lists. What the comparison of the two programs shows is this. Stage 2: the second program makes its own complete traversal, taking at each node the unit chosen by the first program (a valid remaining unit), generating and testing all the candidates of that unit with its own code; it finds the same number of kept candidates at every node, and no complete configuration. Stage 1: the canonical classes of the kept bases are compared separately, and they are the same in the 50 cases.

Proposition 9.30 (hand statements + Computations 4.12, 9.29). There is no configuration in case I with n_3 = 2.

Proof. By Lemma 9.28, a configuration would give, for its graph B (one of I.0, ..., I.5) and its profile ((3,4,0) or (4,2,1), section 9.2), a kept base of Computation 9.29 (b) from which the search reaches a state of level n = 9 or 8. By Computation 9.29 (c) no search reaches level 4. QED

9.B.7 The other route (not used)

Route A (lab session; `lemma/t20_HAND/SPEC_N3.md`, `n3_cases.py`, engine `eng5z.py`) takes the base S u {p} u X_3 u X_0' (29 vertices, 700 classes of bases) and places the twelve orbits of X_2. It was stopped on 07/10 with 183 classes closed by search (0 complete configurations) and is used by nothing in this text. The computed statements of NOTES H7, H9, H10 are not used either.

10. Conclusion and status

10.1 Conclusion as it stands today

Theorem 10.1 (hand statements + Computations 4.12, 7.4, 7.5, 8.8, 8.9, 9.19, 9.20). Let G and sigma be as in section 1, and let phi be an integer-valued sigma-odd function with A phi = -4 phi and sum phi(x)^2 = 20. Then:

1. phi takes the values 0, +1, -1 only, and S+ = {phi = +1} has 10 vertices;
2. k = 0: S+ and S- are independent and no vertex of the support is in N[p];
3. after a suitable labelling, the orbit graph B is one of I.0, ..., I.5;
4. n_3 = 2: exactly two vertices outside S, forming one orbit {z, sigma z}, have three neighbours in S+, and (n_0, n_1, n_2) = (7, 46, 24);
5. the profile of N(p) is (3,4,0) or (4,2,1).

Proof, given the Computations. 1 is Proposition 3.3. By Corollary 4.5 the configuration is in case I, II or III; case III is impossible by Proposition 7.6 and case II by Proposition 8.10. So it is in case I, which is 2 (Lemma 4.10), and 3 is Corollary 4.13. By Lemma 9.2, n_3 is 0 or 2, and n_3 = 0 is impossible by Proposition 9.21; the sizes are those of Lemma 5.3 (I). 5 is Lemma 9.3 with Proposition 9.4. QED

Theorem 10.2 (T20; Theorem 10.1 + Computation 9.29). There is no integer-valued sigma-odd function phi with A phi = -4 phi and sum phi(x)^2 = 20.

Proof. By Theorem 10.1 such a function would give a configuration in case I with n_3 = 2, and there is none by Proposition 9.30. QED

State of the verification on 07/10, 14:40 UTC: Computation 9.29 has two programs sharing the model and no code, and section 9.B.6 is refereed; Computation 7.5 (III.1) has a second verification with shared components; Computations 9.19 / 9.20 have a second program on the twelve cases (see the note at the top of this text).

10.2 What is proved by hand, what is computed, what is open

statement	content	nature	programs at the time of writing
Lemma 2.1, Corollaries 2.2, 2.3	sigma and N(p)	hand	-
Lemmas 3.1, 3.2, Proposition 3.3	integer values => 0, +1, -1, ten at +1	hand	-
Lemmas 4.1-4.4, Corollary 4.5	k in {0,2,4}; counting identities; e <= 1; parity; three cases	hand	-
Lemmas 4.6-4.8	case II: the plus edge joins two words, in a 4-cycle	hand	-
Lemmas 4.9, 4.10, 4.11	case III: words = edges of K4; case I: B; support structures	hand	-
Computation 4.12	the nine support structures	computed	2 for the four totals (no shared code); the automorphism orders, on which the completeness of the list rests, from the first program only (recomputed by the transcription script of this text)
Lemmas 5.1, 5.3-5.5, Corollary 5.2	path count; sizes of the classes; bound on X_0; units	hand	-
Lemmas 6.1, 6.2, 6.3, Proposition 6.4, Lemma 6.5	the two PSD matrices, the budget, the rank bound, the search	hand	-
Lemmas 7.1-7.3	model, determined sets and symmetry of case III	hand	-
Computation 7.4	III.0: no node of level 10	computed	2 implementations (‘eng3.py‘ / ‘eng4.py‘, and the verifier ‘engv.py‘), tests written separately, one shared model file; the second follows the units of the first
Computation 7.5	III.1: no node of level 10	computed	**1** (‘eng5.py‘); the rank bound at 1 node; verifier running
Lemmas 8.1-8.7	case II: n_3 = 0, N(p), types of W, X_2, determined sets	hand	-
Computation 8.8	IIa.0, step 1: 180 -> 87	computed	2, identical lists
Computation 8.9	IIa.0, step 2: no node of level 8	computed	2 sharing no code; the second follows the units of the first
Lemmas 9.1-9.3, Proposition 9.4	case I: n_3 in {0, 2}; profiles (3,4,0), (4,2,1)	hand	-
Lemmas 9.5, 9.6, 9.8, 9.9, 9.10, 9.18	case I, n_3 = 0: model and determined sets	hand	-
Lemmas 9.7, 9.11, 9.12, 9.13, Theorems 9.14, 9.16, Corollaries 9.15, 9.17	slots and cells; edge counts; isolation of the inner pair of X_0; e(X_0) <= 8	hand; **not used** by any computation	-
Computation 9.19	case I, n_3 = 0, stage 1	computed	2 for I.0 (4,2,1), I.0 (3,4,0), I.5 (4,2,1) (identical lists); **1** for the nine other cases
Computation 9.20	case I, n_3 = 0, stage 2: no node of level 6	computed	2 for I.0 (4,2,1) and I.5 (4,2,1); **1** for the ten other cases (second program: 372 of 11,150 bases of I.0 (3,4,0), one base of I.1 (4,2,1)); the rank bound at 15 nodes of I.1, first program
Lemmas 9.22-9.26	case I, n_3 = 2: the orbit X_3, units, a_3(u) = 1, two identities	hand	-
Computation 9.29, Proposition 9.30	case I, n_3 = 2: from the 64,528 bases no search reaches level 4	computed	2 (one laptop; shared model, no shared code; the second program follows the unit chosen by the first)

Nobody outside the working sessions has refereed the hand part. The working reports say that the hand lemmas of the Codex session (here Theorems 9.14, 9.16, Lemma 9.24, Corollary 9.17) were re-read line by line by the lab session. For this text every hand statement was derived again; see section 10.4, item 12, for the proofs that exist only here.

10.3 Dependencies

T18. The theorem T18 (no configuration of squared norm 18) is **not used**: no proof of this text invokes it, and no program of the T20 folders reads a file of the T18 folders. (Checked for this text by searching the sources of ‘lemma/t20‘, ‘t20_R2‘, ‘t20_R3_I‘: no program opens a file outside its own folder, and the names "T18" and "H16" occur in two places only: a comment of ‘in_verify.py‘ that cites Proposition 6.4 of T18_PROOF as "a general fact on the graph and sigma", and the header of ‘t20_R2/latt9.py‘, an exploratory script that looks for small values of the form of Lemma 10.2 below on the support alone; it found none, it is not mentioned in the round 2 report, and no computed line uses it.) The general facts that the two texts have in common (Lemma 2.1, Lemmas 6.1-6.3, Proposition 6.4) are proved here.

The earlier small-norm statements. The lemma of 04/10 (no configuration of squared norm 16), the statement called AUDIT_NORME1 (none of squared norm <= 14) and the statement H16 of the T18 working reports are **not used**.

The rank bound (Proposition 6.4) is used by:

- Computation 7.5 (III.1): 1 candidate, in the only program;
- Computation 9.20, graph I.1: 2 candidates for profile (4,2,1) and 13 for profile (3,4,0), in the first program. The second program replaces it by a search below the node; at the time of writing this has been done for the 2 candidates of profile (4,2,1) only (section 9.A.4).

It is not used in Computations 7.4, 8.8, 8.9, 9.19, nor in the ten other cases of Computation 9.20 (no rank rejection occurs there). Its proof (section 6.4) uses Lemma 2.1 (a) and Lemma 6.1 only.

Both PSD matrices are used by every search (the counters of the runs show rejections on M1 and on M2).

The dual-norm rule (NOTES H6) is **not used**: by no statement of sections 2 to 9, and by no program of a closed line (round 3 report: "T18, the 04/10 lemma and the dual-norm test are not used anywhere"; NOTES H9: not used by the engine of the n_3 = 2 run either). It is recorded here because it may be used for section 9.B, and because its dependency must be stated exactly. In graph language:

Lemma 10.2 (dual-norm rule; conditional; not used). Let phi be the function of a configuration, k an integer and g an integer-valued sigma-odd function on the vertices. Put

    psi = k phi - (3I - A) g,   N(k, g) = 10 k^2 + (7/2) (g^T (3I - A) g - 2k phi^T g).

Then psi is an integer-valued sigma-odd function with A psi = -4 psi and sum psi(x)^2 = 2 N(k, g); N(k, g) is a non-negative integer, congruent to 3 k^2 modulo 7, and it can be computed from the induced subgraph of G on the support of g and from the values of g on S. Consequently, **if** the following statement holds,

    (E14)   every non-zero integer-valued sigma-odd function psi with A psi = -4 psi has sum psi(x)^2 >= 16,

then N(k, g) is never one of 1, 2, ..., 9: a pair (k, g) with 1 <= N(k, g) <= 9 is a contradiction.

Proof. A commutes with the permutation matrix of sigma, so (3I - A) g is sigma-odd, and psi is integer-valued and sigma-odd. By (1.1), (A + 4I)(3I - A) = 12 I - A - A^2 = -2J, and J g = 0 because a sigma-odd function has sum 0; so A (3I - A) g = -4 (3I - A) g, and A psi = -4 psi. (3I - A) phi = 7 phi, and
(3I - A)^2 = 9I - 6A + A^2 = 7 (3I - A) + 2J, so g^T (3I - A)^2 g = 7 g^T (3I - A) g. Hence

    sum psi^2 = 20 k^2 - 14 k phi^T g + 7 g^T (3I - A) g = 2 N(k, g).

sum psi^2 is even (psi is sigma-odd and psi(p) = 0) and >= 0, so N is a non-negative integer. Modulo 7:
2 N = sum psi^2 = 20 k^2 = 6 k^2, so N = 3 k^2, which is 0, 3, 5 or 6 modulo 7. So among 1, ..., 9 the only possible values of N are 3, 5, 6 and 7, i.e. sum psi^2 = 6, 10, 12 or 14, with psi != 0: all excluded by (E14). QED

So the rule depends on (E14) only: on the statement for squared norms <= 14 (which the working reports call AUDIT_NORME1, or "min L* >= 8/7" in the lattice language), **not** on the lemma of 04/10 (squared norm 16) and **not** on T18 (squared norm 18), because 8 and 9 are not congruent to 0, 3, 5 or 6 modulo 7. (E14) is not proved in this text, and its identification with the lattice statement rests on the dictionary of section 1.4, which is outside this text.

The statements of the working reports that use the rule, and therefore (E14), are: NOTES H6 (D1), (D2), (D3), and Propositions 1, 2 and Corollary 3 of ‘lemma/t20_CODEX/HAND.md‘ with the computed tables of that file. **None of them is used here.** (The "hand proof of e(U) >= 1" in NOTES H2 does not use the rule: it exhibits a vector with negative form, i.e. a PSD violation. It is not used here either.)

10.4 What has not been independently verified, and other caveats

1. **Floating-point "kept" decisions.** In the engines (‘eng3.py‘, ‘eng4.py‘, ‘eng5.py‘, ‘eng6.py‘, ‘engv.py‘, ‘iian_verify.py‘, ‘in_verify.py‘, and the PSD test of ‘eng.py‘ used by ‘stage1c.py‘) a candidate is kept when a floating-point eigenvalue or Schur complement is above a tolerance (10^-7 in ‘eng5.py‘, ‘iian_verify.py‘, ‘in_verify.py‘; 10^-7 on the even blocks and 2 * 10^-9 on the doubled odd block in ‘eng.py‘), according to the sources read or to the headers and working reports for the others. This is never used as a proof of semidefiniteness. A rejection requires an exact witness (section 6.3). When the floating-point test is negative and no integer vector is found, the engines decide by exact rational elimination; the verifiers report how often: 0 times in the outputs read (‘iian_verify_*.out‘, ‘iian_verify_types.out‘: "exact elimination 0"). No positive statement of semidefiniteness is needed for Theorem 10.1: by (6.2) the exclusions use exact rejections only.
2. **The programs were not audited line by line.** For this text the following were read in full: ‘t20_R2/eng.py‘, ‘t20_R2/cases.py‘, ‘t20_R2/iia_w.py‘, ‘t20_R3_I/cases6.py‘; in part: ‘t20_R2/eng5.py‘ (header, rank test, main loop), ‘t20_R2/iian_verify.py‘ (header and the re-traversal), ‘t20_R3_I/stage1c.py‘ (header and the generation of the types), ‘t20_R3_I/in_verify.py‘ (header), ‘t20_R3_I/r3lib.py‘, ‘t20/t20_enum.py‘, ‘t20/t20_verify.py‘ and ‘t20/t20_common.py‘ (headers and main loops); for the others (‘eng3.py‘, ‘eng4.py‘, ‘eng6.py‘, ‘engv.py‘, ‘t20_kernel.py‘, ‘r2c.py‘, the checkers and the summary scripts) only the header, or nothing. That each program enumerates exactly the search described in the text is supported by the agreement of programs written separately where there are two, not by a proof.
3. **The second programs follow the first ones.** ‘engv.py‘, ‘engv2.py‘, ‘iian_verify.py‘ and ‘in_verify.py‘ read the unit chosen at every node from a plan written by the first program. They regenerate and test all candidates of that unit with their own code, descend into the candidates they keep themselves, and compare the number kept with the plan’s. Each of them therefore makes a complete traversal of its own with valid units (the verifier contract of section 6.5), which suffices by Lemma 6.5; the equal numbers of kept candidates are a cross-check, not a proof that the two programs keep the same candidates. The second programs do not choose the units by themselves: the node counts were never reproduced by a search with another order.
4. **Shared inputs.** The structure files ‘structures_I.json‘ and ‘structures_III_IIb_IIa.json‘ (written by ‘t20_enum.py‘) are read by ‘cases.py‘, ‘r3lib.py‘, ‘iian_verify.py‘, ‘in_verify.py‘; ‘check_model.py‘ compares the base graphs and units built from them with edge lists typed in by hand, for III.0, III.1, IIa.0. The list ‘iia_w.json‘ is read by both programs of Computation 8.9 (the second compares it with its own). For III.0 the two engines share the model file ‘cases.py‘. For case I the two programs share the hand model.
5. **Single programs.** III.1; ten of the twelve cases of n_3 = 0 (section 10.2). In particular everything about the graphs I.1, I.2, I.3, I.4 rests on one program, and for I.1 on the rank bound as well.

6. **Saved certificates.** Only samples were saved and re-checked: 15,000 for III.0, 6,358 for IIa.0 (5 type sets of 87; the others were deleted), ten for I.0 (4,2,1). The plan files, logs and certificates of III.1 and the per-base files of the three laptop cases of n_3 = 0 are not in git.
7. **Completeness of the runs** is checked by bookkeeping scripts (‘summary_III0.py‘, ‘summary_parts.py‘, ‘summary_IIa.py‘, ‘summary2.py‘, ‘check_cloud.py‘: all parts present, no time-out, same task list or same list of bases in every part), not by a rerun in one piece. The working reports mention abandoned or interrupted launches (III.1 was started three times before the run reported here; the first version of two programs of rounds 1 and 2 gave wrong or no results and was discarded); nothing of them is used.
8. **Generated and patched engines.** ‘eng4.py‘, ‘eng5.py‘, ‘eng6.py‘, ‘engv.py‘ are produced from earlier engines by scripts (‘mk_eng*.py‘, ‘patch_*.py‘). That the stored outputs were produced by exactly the versions now in the repository was not verified.
9. **The table of Computation 9.20** is copied from ‘final_table.out‘; the script that wrote it is not in the repository folder. For the nine cloud cases the same numbers appear in ‘cloud_check_t20_I_rest.out‘, which was compared for this text (all nine lines agree); for the three laptop cases they could be compared only with the working report.
10. **Computation 4.12:** see the paragraph after it. The classification into classes by the second enumerator rests on an invariant, not on an isomorphism test.
11. **Not verified at all, and not used:** the lattice dictionary and the bridge of section 1.4, (E14), the lemma of 04/10, T18 itself, the computed statements about n_3 = 2 (section 9.B.6), the statement of the working reports that B is connected with its eigenvalues other than 4 in [-3, 3].
12. **Parts of this text that only its author has checked.** The following proofs or formulations are not in the working reports, or are there in one line, and were written for this text: Corollary 2.3 (c); the equality case of Lemma 3.1 as used in Lemma 3.2; Lemma 4.6; Lemma 4.9 as a separate statement; the completeness argument after Computation 4.12; Lemma 5.5 (a) (the reports state U(s) != V(s) without proof); the formula x^T M2 x = 22 c^T (7I - G_odd) c of section 6.3; Lemma 6.5 in this form, and Lemma 7.3; Lemma 9.8 (c), (d); in Lemma 9.11, the proof of c <= 1 and the derivation of the list by hand; the definition of c in Lemma 9.12; the definition of an N-configuration and the comparison of the bounds in Lemma 9.18; Lemma 9.23 (c) (the bound a_0(s) <= 4) and (f); Lemma 10.2 in graph language. The transcription script ‘lemma/t20_PROOF/_check_text_data.py‘ rebuilds the nine structures from the lists printed in section 4.10 and confirms (S1)-(S5), the codegrees, the numbers of triangles and of automorphisms, the units, the numbers r_i and the count 180 of Lemma 8.5, the labelling by the edges of K4, the arithmetic of (9.6), (9.7), (9.8), (9.11) and the list of Lemma 9.11. It is a check of transcription, not part of the proof.

10.5 Statements of the working reports that are not used and are left out

- Round 1, B3 and B4: exact PSD on S u {p} and on S u N[p] (quoted in two remarks); the scan of the types of the vertices with d = 3 or d = 4 on 23 vertices. Superseded by Lemma 5.3 and Lemmas 9.1, 9.2.
- Round 1, A10: the inequality h_2 + 4 h_3 + n_3 / 2 <= 7 and the two profiles with h_3 = 1 (superseded by Lemma 9.1); the equivalence between PSD on S and the spectrum of B.
- Round 1, A9: the path counts from the vertices of S in case III and the numbers alpha, beta, gamma (consistent, not needed); the hand classification of the case "two triangles" (the classification is Computation 4.12).
- Round 2: the by-product "X_0 contains no 4-cycle" in case II; the explorations B4; the variants of the model of case II with X_0 in the base (‘CaseIIa‘, ‘CaseIIaW‘); the exploratory script ‘latt9.py‘ (section 10.3).
- Round 3: the paragraph "tried, closes nothing".
- NOTES: H2 (the computed list of the types of z, and the facts read off it), H5 (the description of N(p) by ten roots of D7; it is a reformulation of Lemma 9.8), H6 (D1)-(D3) and the searches for dual-norm certificates, H7, H8, H9, H10 (computed; section 9.B.6).
- ‘lemma/t20_CODEX/HAND.md‘: Propositions 1, 2, Corollary 3 and the computed tables (they depend on (E14)); ‘TRIANGLE_REDUCTION.md‘: the two tables of class sizes of the sets Delta, Gamma and their complement (correct by Corollary 9.15 and Lemma 9.24, not needed).

10.6 Where the sources disagree or are imprecise, and what was chosen

1. **III.1.** Round 2 says "open, not completed (about 900 tasks done)". ‘lemma/t20_III1/README.md‘, ‘T20_WORK.md‘ and ‘T20_BOARD.md‘, written later, say: completed by one program, 0 configurations, verifier running. Chosen: the later files (Computation 7.5, one program).
2. **Upper bounds of stage 2 of case I.** The round 3 report says that the candidates are enumerated "under the caps of C4 and C5". The headers of ‘cases6.py‘ and ‘in_verify.py‘ give 4 - h_0, 8 - h_0 and min(10, 12 - h_0); for a vertex of W in profile (3,4,0) this is 8, while C4 (Lemma 9.9) gives 7. Chosen: the bounds of the code, since they are what was run; they are weaker than those of Lemma 9.9 and therefore valid (Lemma 9.18 (c)).
3. **Progress of n_3 = 2.** 168 of 700 classes closed (NOTES H10 and ‘SPEC_N3.md‘, 11:30 UTC); 183 bases closed, run stopped at 11:43 UTC (‘T20_BOARD.md‘). Both are quoted (section 9.B.6); neither is used.
4. **Bound on a_2 of a vertex of X_2 for n_3 = 2.** ‘SPEC_N3.md‘ and NOTES H9: a_2 <= 5 (a_3 = 0), a_2 <= 3 (a_3 = 1). Header of ‘cases6.py‘: a_0 <= 4. Both valid; the second is proved in Lemma 9.23 (c).
5. **Number of configurations of N(p) for I.0 (4,2,1).** Round 2 (explorations): 1,784,786, of which 48,587 pass. Round 3: 178,768 N-configurations, 11,322 orbits, 3,022 bases. These are different objects (the first is a count of an exploratory program that, in the words of its report, proves nothing); they were not reconciled. Chosen: round 3.
6. **"Two independent programs" for the nine support structures** (round 1, header). Its section B2 says that the second program has no isomorphism test and that the automorphism orders come from the first only. Stated as such after Computation 4.12, with the argument that makes the list complete.
7. **"Two programs" for III.0** (‘T20_BOARD.md‘). Round 2 is more precise: two implementations, one shared model file. Stated as in round 2 (section 7.4).
8. **Round 3, C6 and C7.** C6 asserts c <= 1 without proof and refers to a script for the list of solutions; C7 uses c without defining it. Proof and definition supplied (Lemmas 9.11, 9.12).
9. **NOTES H6** says "N(k, m) is 0 or >= 10", quoting the three earlier statements (norms <= 14, 16, 18). By the congruence of Lemma 10.2 only the first is needed, as the same note says a few lines below.
10. **Which statement.** Round 1, section 0: the task text said squared norm 40; the statement is squared norm 20 (section 1.4).

No mathematical error was found in the working reports while writing this text.

Appendix A. Programs, outputs and commands

Paths are relative to the root of the repository ‘conway99-c2-aws‘. "python" is Python 3 with numpy; ‘t20_enum.py‘ needs numba, and the rank tests need python-flint. Each command is run in the folder of the script. The command lines are those recorded in the working reports, in the README files, in the driver scripts or in the headers of the scripts; where the exact arguments of a run were not recorded, the usage line is given and the names of the output files show how the work was split.

A.1 Computation 4.12 (the support structures), folder ‘lemma/t20‘

program	command	output
‘t20_enum.py‘ (with ‘t20_kernel.py‘, ‘t20_common.py‘)	‘python t20_enum.py I‘ ; ‘python t20_enum.py III IIb IIa‘ (the arguments are the names of the types; not recorded, deduced from the outputs)	‘enum_I.out‘, ‘structures_I.json‘; ‘enum_III_IIb_IIa.out‘, ‘structures_III_IIb_IIa.json‘ (earlier run of III and IIb, same numbers: ‘enum_III_IIb.out‘, ‘structures_III_IIb.json‘)
‘t20_verify.py‘	‘python t20_verify.py III I‘ ; ‘python t20_verify.py IIa2‘ ; ‘python t20_verify.py IIb IIa‘ ; ‘python t20_verify.py IIb2‘ (deduced likewise)	‘verify_III_I.out‘; ‘verify_IIa.out‘; ‘verify_II.out‘ (IIb complete, then cut by the time limit: "exit 124"); ‘verify_IIb.out‘. ‘verify.out‘ holds only the exit line of a discarded first version
data of section 4.10	‘python t20_survivors.py‘ ; ‘python t20_deficiency.py‘ ; ‘python t20_names.py‘	‘survivors.out‘, ‘deficiency.out‘, ‘names.out‘
transcription check of this text	‘python lemma/t20_PROOF/_check_text_data.py‘	‘_check_text_data.out‘

A.2 Computation 7.4 (III.0), folder ‘lemma/t20_R2‘

program	command	output
‘eng3.py‘, model ‘cases.py‘	‘python run_eng3.py III.0 <part> 3 <time limit> 3 e3_III0‘ (part = 0, 1, 2)	‘e3_III0_{0,1,2}of3.json‘, ‘.log‘, ‘.out‘
‘eng4.py‘, model ‘cases.py‘	‘python run_eng4.py III.0 <part> 3 <time limit> 3 e4_III0 plan‘	‘e4_III0_{0,1,2}of3.json‘, ‘.log‘, ‘.out‘; the plan files are not in git
‘engv.py‘, model ‘cases.py‘	‘python run_engv.py III.0 <part> 3 <time limit> 3 v_III0 plan=<plan file of eng4.py>‘	‘v_III0_{0,1,2}of3.json‘, ‘.log‘, ‘.out‘
totals	‘python summary_III0.py‘	‘summary_III0.out‘
certificates	‘python check_certs.py III.0 <certificate files>‘	‘check_certs_III0.out‘
model	‘python check_model.py‘	‘check_model.out‘

The time limits of these runs were not recorded; the result files record: 3 parts, split parameter 3 (tasks of level 4), the group of order 48, pointwise stabilisers, rank test enabled, no time-out.

A.3 Computation 7.5 (III.1), folders ‘lemma/t20_R2‘ (programs) and ‘lemma/t20_III1‘ (outputs)

program	command	output
‘eng5.py‘, model ‘cases.py‘	‘OPENBLAS_NUM_THREADS=1 python run_eng5.py III.1 <i> 8 80000 3 e5_III1 setwise plan certs=2500‘ (i = 0..7)	‘lemma/t20_III1/e5_III1_{0..7}of8.json‘; logs, plans and certificates not in git
totals	‘python summary_parts.py e5_III1 8‘	‘lemma/t20_III1/summary_III1.out‘

A.4 Computations 8.8 and 8.9 (IIa.0), folder ‘lemma/t20_R2‘

line	program	command	output
8.8	‘iia_w.py‘ (uses ‘r2c.py‘)	‘python iia_w.py‘	‘iia_w.json‘ (the printed output was not saved)
8.8	‘iian_verify.py‘	‘python iian_verify.py types‘	‘iian_verify_types.out‘
8.9	‘eng5.py‘, model ‘cases.CaseIIaN‘	‘python run_iian.py 0:44 <time limit> all‘ ; ‘python run_iian.py 44:87 <time limit> all‘	‘iian/all_w<k>_0of1.json‘, ‘.log‘, ‘.plan‘ (k = 0..86); ‘iian/all_summary_0-44.json‘, ‘iian/all_summary_44-87.json‘; ‘iian_all_a.out‘, ‘iian_all_b.out‘
8.9	totals	‘python summary_IIa.py‘	‘summary_IIa.out‘
8.9	‘iian_verify.py‘	‘python iian_verify.py follow <a> <b> iian all‘ for (a, b) = (0, 30), (30, 38), (38, 44), (44, 66), (66, 87)	‘iian_verify_<a>_<b>.out‘, ‘.json‘
8.9	certificates	‘python check_certs.py IIa.0 iian_certs_sample/*.jsonl‘	‘check_certs_IIa.out‘

‘hand_checks.py‘ -> ‘hand_checks.out‘ contains arithmetic guards for the hand lemmas of round 2 (not part of the proofs).

A.5 Computations 9.19 and 9.20 (case I, n_3 = 0), folder ‘lemma/t20_R3_I‘

what	program	command	output
one case, whole pipeline	‘cloud_case.sh‘	‘bash cloud_case.sh <out dir> <graph> <h2> <h1> <h0>‘ with PY and NPROC set; ‘T20_SKIP_VERIFY=1‘ stops before the second program	‘<out dir>/<graph>_<profile>/‘: ‘s1_*.json‘, ‘stage1_*.out‘, ‘stage2_*.out‘, ‘verify_*.out‘, ‘parts/‘, ‘logs/‘, ‘timing.txt‘
stage 1	‘stage1c.py‘, ‘stage1_merge.py‘ (with ‘r3lib.py‘, ‘eng.py‘)	‘python stage1c.py <graph> <h2> <h1> <h0> <prefix>.part<i>of<n>.json part=i/n‘ ; ‘python stage1_merge.py <prefix> <n> <stage1.json>‘	the stage-1 file (list of bases)
stage 2	‘run_stage2.py‘ (engine ‘eng6.py‘, model ‘cases6.py‘), ‘summary2.py‘	‘python run_stage2.py <stage1.json> <part> <nparts> 400000 <prefix> width=3‘ ; ‘python summary2.py <stage1.json> <prefix> <nparts>‘	‘<prefix>_<part>of<nparts>.jsonl‘ (one line per base, with its plan), ‘.sum.json‘; the summary
second program	‘in_verify.py‘	‘python in_verify.py bases <graph> <h2> <h1> <h0> <prefix>_<i>of<n>.json part=i/n‘ ; ‘python in_verify.py follow <stage1.json> <stage2 prefix> <part> <nparts> <out>‘ ; ‘python in_verify.py merge <stage1.json> <prefix> <n> <follow prefix>‘	‘verify_<graph>_<profile>.out‘
the nine cloud cases, first program	job ‘lemma/cloud/jobs/t20_I_rest.sh‘, run by ‘lemma/cloud/run.sh‘	-	git branch ‘lemma-run/t20_I_rest-i-02f356c70da99d700‘ (status "FINISHED exit=0"): ‘ALL.txt‘, and the folders ‘I1_340‘, ‘I1_421‘, ‘I2_340‘, ‘I2_421‘, ‘I3_340‘, ‘I3_421‘, ‘I4_340‘, ‘I4_421‘, ‘I5_340‘, each with ‘s1_*.json‘, ‘stage1_*.out‘, ‘stage2_*.out‘, ‘parts/s2_<i>of128.jsonl‘, ‘parts/s2_<i>of128.sum.json‘, ‘logs/‘
re-count of the cloud run	‘check_cloud.py‘	‘git archive origin/lemma-run/<job>-<id> | tar -x -C <dir>‘ ; ‘python check_cloud.py <dir> <out file>‘	‘cloud_check_t20_I_rest.out‘
the three laptop cases	the same programs	-	per-base files in the session scratchpad (not in git)
table of the twelve cases	-	-	‘final_table.out‘

comparisons	-	-	‘cloud_vs_laptop_stage1.out‘, ‘cmp_eng5_I0_421.out‘
sample of certificates	‘sample_certs.py‘, ‘check_sample.py‘	‘python sample_certs.py <stage1.json> <stage2 prefix> <nparts> <out.jsonl>‘ ; ‘python check_sample.py <file.jsonl>‘	‘sample_certs_I0_421.jsonl‘
cells (Lemma 9.11)	script ‘hand_cells.py‘ (not in the repository folder)	-	‘hand_cells.out‘

Runs of the second program in progress at the time of writing (**not used**): cloud jobs
‘lemma/cloud/jobs/t20_Iv_A.sh‘ to ‘t20_Iv_F.sh‘ and ‘t20_I1_340.sh‘; git branches
‘lemma-run/t20_Iv_A-i-0390fd53333c148ad‘ (status "INSTALLING" only), ‘lemma-run/t20_Iv_B-i-0a03ce79bff09fa5d‘,
‘lemma-run/t20_Iv_C-i-092f5550d84f61e09‘, ‘lemma-run/t20_Iv_D-i-09169834049067883‘,
‘lemma-run/t20_Iv_E-i-0cd9427b48ee5704e‘, ‘lemma-run/t20_Iv_F-i-0c5414832a6ac4ff5‘ and
‘lemma-run/t20_I1_340-i-019e86d399c17954f‘ (status "RUNNING" in the last commit of each that was present in the
local copy of the repository when this text was written).

A.6 Section 9.B (n_3 = 2)

Computation 9.29 (route B), folder ‘lemma/t20_R3_I‘: specification ‘SPEC_ROUTE_B_N3.txt‘; types of z ‘ztypes.py‘,
‘ztypes.json‘, ‘ztypes.out‘; one case, both programs: ‘n3_case.py‘; all cases: ‘python n3_all.py <out> <workers> both 340 421‘;
first program ‘stage1c.py‘ (option ‘z=‘), ‘stage1_merge.py‘, ‘cases6.py‘, ‘eng6.py‘, ‘run_stage2.py‘, ‘summary2.py‘;
second program ‘in_verify.py‘ (modes ‘basesz‘, ‘follow‘, ‘mergez‘); checks ‘check_n3.py‘, ‘check_n3_verify.py‘,
‘table_n3.py‘. Outputs kept in the repository: ‘n3_check.out‘ (first program, per case), ‘n3_table.out‘,
‘n3_verify.out‘ (agreement per case, totals on its last lines), ‘n3_stage1_regen_compare.out‘ (the 25 stage-1 lists
of profile (4,2,1) regenerated and identical). The base lists, the stage-2 files with plans and the second
program’s raw outputs (41 MB) are not in the repository; the command above regenerates them.

Not used by the proof. Route A: ‘lemma/t20_HAND/n3_cases.py‘ (bases), ‘eng5z.py‘ (generated by ‘mk_eng5z.py‘), ‘run_n3.py‘,
‘summary_n3.py‘, cloud job ‘lemma/cloud/jobs/t20_n3.sh‘; git branches ‘lemma-run/t20_n3-i-0b8ae938c1fc89c62‘ and
‘lemma-run/t20_n3-i-04bb3ac564f658b79‘ (both with status "RUNNING" in their last commit present locally). Small
scripts of the hand notes: ‘x3_types.py‘, ‘n3_base.py‘, ‘dual_test_S.py‘, ‘dual_test_X3.py‘, ‘check_inner.py‘.
Route B: ‘lemma/t20_R3_I/stage1c.py‘ with the option ‘z=‘, ‘cases6.py‘. Specification for a second program:
‘lemma/t20_HAND/SPEC_N3.md‘. Scripts of the Codex session: ‘lemma/t20_CODEX/*.py‘.

Appendix B. Correspondence with the labels of the working reports

label in the working reports	here
round 1 (‘t20/T20_REPORT.md‘): A0	Lemma 2.1, Corollaries 2.2, 2.3
A1	Lemma 3.1, Proposition 3.3
A2	Lemma 3.2
A3	Lemma 4.1
A4	Lemmas 4.2, 4.3
A5	Lemma 4.4, Corollary 4.5
A6	Lemma 5.1, Corollary 5.2, Lemma 5.3
A7, A8	Lemmas 4.7, 4.8 (and Lemma 4.6)
A9	Lemma 4.9, Lemma 5.3 (III), Lemma 7.1
A10	Lemma 4.10, Lemma 5.3 (I), Lemma 9.3
A11	Lemma 5.3 (II), Lemma 8.1
B1, B2; section C	Computation 4.12; section 4.10
B3, B4	not used (section 10.5)
round 2 (‘t20_R2/REPORT.md‘): (M)	Lemma 2.1, Corollary 2.3
G0, G1	Lemma 5.1, Lemma 5.4
II.1 - II.6	Lemmas 8.1 - 8.6
"Case III: model"	Lemmas 7.1, 7.2
I.1, I.2, I.3	Lemma 9.1, Lemma 9.2, Proposition 9.4
B1 (III.0); B2 (IIa.0); B3 (III.1)	Computation 7.4; Computations 8.8, 8.9; Computation 7.5 (with ‘t20_III1/‘)
round 3 (‘t20_R3_I/REPORT.md‘): C1	(9.1), Lemma 9.5
C2, C3	Lemma 9.6, Lemma 9.7
C4	Lemmas 9.8, 9.9
C5	Lemma 9.10
C6, C7	Lemmas 9.11, 9.12
C8	Lemma 9.13
C9	Lemma 6.3, Proposition 6.4
"Determined sets", stage 1, stage 2	Lemma 9.18, Computations 9.19, 9.20
lab notes (‘t20_HAND/NOTES.md‘): H0	(9.1), Lemma 9.5
H1	Lemma 9.22
H3	Lemma 9.25
H4	Lemma 9.13 (n_3 = 0), Lemma 9.26 (n_3 = 2)
H6, the rule and its dependency	Lemma 10.2
H2, H5, H6 (D1)-(D3), H7 - H10	not used (sections 9.B.6, 10.5)
‘t20_HAND/SPEC_N3.md‘, sections 2 - 4	Lemmas 9.22, 9.23, 6.5
‘t20_CODEX/PROFILE_421.md‘	Theorem 9.14, Corollary 9.15
‘t20_CODEX/EDGE_BOUND.md‘	Theorem 9.16, Corollary 9.17
‘t20_CODEX/TRIANGLE_REDUCTION.md‘	Lemma 9.24
‘t20_CODEX/HAND.md‘	not used (sections 10.3, 10.5)

Appendix C. Numbering of the vertices in the files

- All the programs: x_i = i and y_i = 10 + i (i = 0..9); p = 20. Further vertices are added in orbits (v, v + 1).
- Structure files (‘structures_I.json‘, ‘structures_III_IIb_IIa.json‘): field ‘m‘ is the 10 x 10 matrix of the states of the pairs of orbits (bit 1 = minus edge, bit 2 = plus edge), field ‘H‘ the list of the inner indices.
- Case III (‘cases.CaseIII‘): X_0 = the three mate pairs (21, 22), (23, 24), (25, 26); the added orbits of X_2 are (27 + 2q, 28 + 2q), q = 0, 1, ... in the order of the search.
- Case II (‘cases.CaseIIaN‘, ‘iian_verify.py‘): W = the five mate pairs (21, 22), ..., (29, 30); the pair number t, of type (i ; l), has 21 + 2t adjacent to p, to its mate, to x_i and to y_l; the added orbits start at 31.

```
- Case I, n_3 = 0 (‘cases6.CaseIN6‘, ‘in_verify.py‘): the seven pairs of N(p) are (21, 22), ..., (33, 34), in the
  order of the stage-1 record; a pair is ("A", U, V), ("W", i, l) or ("Z",); the added orbits start at 35.
- A plan line (first engines): level, the two indices of the chosen unit, number of candidates, number kept.
```

4 Where the computations of T18 and T20 are

File: lemma/ARTIFACTS_T18_T20.md

```
# Where every computation cited by T16, T18 and T20 lives (provenance table, 2026-10-07)

Answer to reserve G1 of ‘lemma/t20_CODEX/REFEREE_T20_PROOF.md‘. One line per computation: the programs (with the git blob id of the
version that ran), where the outputs are (a path on ‘master‘, or a branch with its commit), the line to read at the end, and the
second program. Blob ids are ‘git rev-parse <commit>:<path>‘ (first ten characters): they identify file contents.
The proof texts keep their own appendices with the commands (‘T18_PROOF.md‘ Appendix A, ‘T20_PROOF.md‘ Appendix A); this page adds
what they lacked: the run branches, the program versions, and the archive of the outputs that were only on one laptop.

## T18 and T16

| Computation | Programs and outputs | Second program |
|---|---|---|
| T18, Computations 5.x to 8.8 | ‘lemma/T18_PROOF.md‘ Appendix A.1 to A.7: folders ‘lemma/t18‘, ‘t18_B0‘, ‘t18_B0_VERIFC‘, ‘t18_B2‘, ‘
   t18_N3‘, ‘t18_STRUCT‘, ‘t18_VERIF331‘, ‘t18_REFEREE‘, ‘t18_REFEREE2‘, all on ‘master‘ with their outputs | every computation re-done
   by at least three programs, one written from the text alone (the two referee folders) |
| T16 (dual minimum > 8/7 becomes >= 9/7) | ‘lemma/T16_PROOF.md‘, folder ‘lemma/t16‘ (the dossier of 04/10 as received, the lab’s check,
    the independent enumeration) | referee pass ‘lemma/t20_CODEX/REFEREE_T16_PROOF.md‘: one defect in the dossier’s verifier (‘check_B‘
   did not test W <= set(V)); with the test added the 32 witnesses and the 240 obstructions pass again |

## T20

Program versions of ‘lemma/t20_R3_I‘ (case I). "v1" = commit 6471b9d64 (07/10, 10:35 UTC), "v2" = ea16eb42b (11:18 UTC: the
second program closes by its own search the nodes the first rejected by the rank bound), "v3" = 99eb1fb50 (13:52 UTC: the ‘z=‘
branch for n_3 = 2; the n_3 = 0 behaviour is unchanged, regression in ‘T20_PROOF.md‘ 9.B.6).

| File | v1 | v2 | v3 |
|---|---|---|---|
| ‘stage1c.py‘ | 1fc255c3d1 | 1fc255c3d1 | 78cff2c820 |
| ‘stage1_merge.py‘ | 8054146192 | 8054146192 | 3fa9647377 |
| ‘cases6.py‘ | 43a5d7e8a6 | 43a5d7e8a6 | 43a5d7e8a6 |
| ‘eng6.py‘ | 849cba6598 | cb7d0996f5 | cb7d0996f5 |
| ‘run_stage2.py‘ | 0c9aef9353 | 0c9aef9353 | 0c9aef9353 |
| ‘summary2.py‘ | d68aea1da9 | d68aea1da9 | d68aea1da9 |
| ‘in_verify.py‘ (second program) | bc657c2bad | a1e9b479f4 | 373dd5ae20 |
| ‘cloud_case.sh‘ | 97fb860b0b | 97fb860b0b | 97fb860b0b |

A cloud run clones ‘master‘ when its machine starts, so its version is the one of its launch time (column "version").
Each run of ‘cloud_case.sh‘ executes the first program AND the second program on its case, so one branch holds both.

| Computation of ‘T20_PROOF.md‘ | Case | Outputs: branch (commit) or path | Version | Line to read |
|---|---|---|---|---|
| 4.12 support structures (nine configurations) | all | ‘lemma/t20/‘ on master (645201b5b) | - | Appendix A.1; two programs (enumerator
   and verifier) |
| 7.4 | III.0 | ‘lemma/t20_R2/‘ on master (715802e33): ‘summary_III0.out‘ | - | A.2; two engines, identical counters |
| 7.5 | III.1 | ‘lemma/t20_III1/‘ on master (dff2a08f5): ‘summary_III1.out‘, ‘e5_III1_{0..7}of8.json‘, ‘VERIF.md‘ | - | A.3; second
   verification with shared components (stated in ‘VERIF.md‘) |
| 8.8, 8.9 | IIa.0 | ‘lemma/t20_R2/‘ on master | - | A.4; two independent programs |
| 9.19, 9.20 | I.1 (3,4,0) | ‘lemma-run/t20_I1_340-i-019e86d399c17954f‘ (c3d6dba726) | v1 | ‘ALL.txt‘: "STAGE 1 AGREES: True", "STAGE 2
   AGREES AT EVERY NODE, 0 complete, all bases: True" |
| 9.19, 9.20, first program alone | the nine cases of the first cloud run | ‘lemma-run/t20_I_rest-i-02f356c70da99d700‘ (937323d571) | v1
    | ‘ALL.txt‘, per case ‘stage1_*.out‘, ‘stage2_*.out‘ |
| 9.19, 9.20 | I.3 (3,4,0), I.0 (3,4,0) | ‘lemma-run/t20_Iv_B-i-0a03ce79bff09fa5d‘ (2950e59c8a) | v1 | as above, per case |
| 9.19, 9.20 | I.4 (3,4,0), I.3 (4,2,1) | ‘lemma-run/t20_Iv_C-i-092f5550d84f61e09‘ (531fb79b3d) | v1 | as above |
| 9.19, 9.20 | I.2 (3,4,0), I.4 (4,2,1) | ‘lemma-run/t20_Iv_D-i-09169834049067883‘ (685659ca90) | v1 | as above |
| 9.19, 9.20 | I.5 (3,4,0) | ‘lemma-run/t20_Iv_E-i-0cd9427b48ee5704e‘ (ab8e27e6d0) | v1 | ‘ALL.txt‘ up to "=== I.5 (3,4,0) end"; the
   machine was reclaimed during its next case, which is not used |
| 9.19, 9.20 | I.1 (4,2,1) | ‘lemma-run/t20_Iv_G-i-05703a74fbb027423‘ (f7ea733d70) | v2 | as above; 1,406,917 nodes; the 2 rank nodes
   closed by the second program’s own search |
| 9.19, 9.20 | I.2 (4,2,1) | ‘lemma-run/t20_Iv_H-i-004334efafb6ceb56‘ (951b8ff5e1) | v2 | as above |
| 9.19, 9.20 | I.0 (4,2,1) | ‘lemma-run/t20_Iv_I-i-032b7d25d335204f8‘ (bb1d5b5e00) | v2 | as above |
| 9.19, 9.20 | I.5 (4,2,1) | ‘lemma-run/t20_Iv_J-i-038ea54927e5a01c4‘ (e1f3c52bee) | v2 | as above |
| 9.29 (n_3 = 2, route B) | the 50 cases | summaries on master in ‘lemma/t20_R3_I/‘: ‘n3_check.out‘, ‘n3_table.out‘, ‘n3_verify.out‘, ‘
   n3_stage1_regen_compare.out‘; raw outputs (stage 1 lists, stage 2 plans, second program): branch ‘lemma-archive/t20_n3_routeB‘ (
   b960cd136b) | v3 | last two lines of ‘n3_verify.out‘: "TOTAL: cases 50, agreeing on both stages 50 ...", "ALL 50 CASES: SAME CLASSES
   OF BASES, SAME KEPT COUNT AT EVERY NODE, 0 COMPLETE: True" |

Not used by the proof and kept only as records: ‘lemma-run/t20_Iv_F-*‘ (reclaimed before any result), ‘lemma-run/t20_n3-*‘ and
‘lemma-run/t20_n3_sample-*‘ (route A, stopped), ‘lemma/cloud/jobs/split_pairs.sh‘ (never run).

## What this page does not certify

- The run branches are copies pushed by the machines as they worked; nobody recomputed their contents after the fact, except where
  a second program did (column "Line to read").
- The versions are inferred from the launch times and the history of ‘master‘; the machines did not record the commit they cloned.
- III.1 has a second verification that shares the model file and part of the enumeration skeleton with the first program.
```

5 Lattice 143

File: lemma/upstream/LATTICE_143.md

```
# Lattice 143 (root system a_15 a_8) hosts no frame: a two-line proof

Lab session (Claude), 2026-10-07. Replaces, for this lattice, the "component-budget test" of 28/09 (a solver plus an exact
enumeration, never reread). **Refereed by Codex on 07/10: accepted** (‘lemma/t20_CODEX/REFEREE_143_PROOF.md‘); script re-run by
the laptop session on another machine. It does not depend on the v10 run of 143 (stopped, nothing imported).

## Statement

**Proposition 143.** Let U be lattice 143 of the list (odd unimodular, rank 24, root system a_15 a_8; even neighbours A_15 D_9
and A_24). There are no vectors y_1, y_2, h_1..h_7, u_1..u_42 of U satisfying (R-ii) and (R-v) of Theorem R
(‘AUDIT_REDUCTION.md‘). In particular U hosts no frame.

Only two properties of a frame are used:
- (R-ii) the 42 vectors u_o are vectors of U of norm 3;
- (R-v) for every real v, the sum of (v,f)^2 over the 51 frame vectors f equals 7 (v,v).

The h, the y, the products between frame vectors, the new roots, T16 / T18 / T20, the pair enumeration and the search engine
are not used.

## The budget identity

Let V be a subspace of R^24 of dimension d and P_V the orthogonal projection onto it. Summing (R-v) over an orthonormal basis
e_1..e_d of V:

    sum over the 51 frame vectors f of |P_V f|^2 = sum_j sum_f (e_j,f)^2 = 7 d.          (B)

All terms are >= 0, so the 42 vectors u alone satisfy: sum_o |P_V u_o|^2 <= 7 d.

## Three facts about U (exact computation, ‘budget_143.py‘)

1. U has 312 roots, in two classes under non-orthogonality: 72 roots of rank 8 and 240 roots of rank 15 (a_8 and a_15).
   Let W be the span of the first class, dim W = 8. For every x, |P_W x|^2 = S_8(x) / 18, where S_8(x) is the sum of (alpha,x)^2
   over the 72 roots. (For an irreducible root system of rank n with N roots of norm 2, sum of alpha alpha^T is invariant under
   the Weyl group, which acts irreducibly on the span: it is c P with c = 2 N / n by the trace; here c = 18.)
2. The vectors of U orthogonal to all roots are the multiples of one primitive vector z, and (z,z) = 144. Let l = R z, the
   orthogonal complement of the span of the roots (dimension 24 - 23 = 1). For every x, |P_l x|^2 = (x,z)^2 / 144.
3. U has 4096 vectors of norm 3, of three kinds (completeness of this list: see below, it is proved by the glue description and
   does not rest on an enumerator):

   | (x,z)^2 | S_8(x) | \|P_W x\|^2 | \|P_l x\|^2 | vectors |
   |---|---|---|---|---|
   | 81 | 0 | 0 | 9/16 | 1120 |
   | 169 | 16 | 8/9 | 169/144 | 288 |
   | 9 | 36 | 2 | 1/16 | 2688 |

## Proof of Proposition 143

Let c be the number of u_o of the third kind ((u,z)^2 = 9). By (B) for V = W: 2 c <= 7 * 8 = 56, so c <= 28.
The other u_o, at least 42 - 28 = 14 of them, are of the first two kinds and have |P_l u|^2 >= 9/16. By (B) for V = l:

    7 = 7 * 1 >= sum_o |P_l u_o|^2 >= 14 * 9/16 = 63/8 > 7.

Contradiction. QED

**The same proof as one linear certificate (Codex, 07/10).** For each of the three kinds, 4 |P_W u|^2 + 16 |P_l u|^2 is 9, 67/3
or 9, hence at least 9. Summing over the 42 u and using (B) on W and on l: 378 <= 4 * 56 + 16 * 7 = 336. Contradiction. (This is
the form proposed as a first target for a Lean check: a finite sum of rationals; the table of kinds is the separate part.)

## How the three facts were obtained, and how they were checked

- ‘budget_143.py <package>‘: takes from the v10 package a basis of U and the lists of the vectors of norm 2 and 3
  (‘scripts/frame_dfs_next.lattice_data‘); asserts the sizes 312 and 4096; then, in exact integer / rational arithmetic: the two
  classes of roots and their ranks (elimination over Q), the null space of the 156 x 24 matrix of products with the roots
  (dimension 1, primitive generator z, (z,z) = 144), the table of ((x,z)^2, S_8(x)) over all vectors of norm 3 (asserted equal to
  the table above), and the identity 3 = S_8/18 + S_15/32 + (x,z)^2/144 for each of them. Output: ‘budget_143.json‘. Run on the
  lab PC on 07/10: all assertions pass (a few seconds).
- Second computation, other enumeration code: the lab’s script of 28/09 ‘exact_budget.py‘ (short vectors by the v1 engine
  ‘frame_dfs.short‘; copies in ‘q/tools/lab_2026-10-07/‘) prints the same three kinds with 560, 144, 1344 vectors up to sign,
  re-run on 07/10. Its own conclusion ("946 sums of u-types x 36 sums of h-types: INFEASIBLE") is the same statement with the h
  included; the proof above shows the u alone suffice.
- Completeness of the list of norm-3 vectors: proved for this lattice by the glue description of the next section, made exact
  by the referee (‘lemma/t20_CODEX/referee_143_checks.py‘, exact arithmetic: the reduced basis is unimodularly related to the
  input generators, Gram determinant 1, explicit A_8 and A_15 chains, z primitive of norm 144, index [U : M + Z z] = 144, the glue
  classes checked on every basis vector, the 41 possible heights enumerated). It uses no list of norm-3 vectors and no
  theta-series count. (Correction of the referee, applied here: ‘theta_odd24.py‘ takes N_3 as an input and does not prove
  N_3 = 4096; an earlier version of this note cited it for completeness. The value 4096 agrees with the count found for all
  156 lattices, hole H3 of the engine audit, but that is not what the proof rests on.)

## The table by hand: the glue description (made exact by the referee)
```

M = A_15 + A_8 is the lattice generated by the roots, det M = 16 * 9 = 144. By fact 2 and unimodularity, the primitive closure of M in U has determinant (z,z) = 144, so M is primitive, and U / (M + Z z) is cyclic of order 144. Every x of U is x = x_15 + x_8 + (t/144) z with t = (x,z), x_15 in the class u t of A_15^*/A_15 = Z/16 and x_8 in the class v t of A_8^*/A_8 = Z/9, for fixed units u, v. The norms in the class i of A_n^* are i (n+1-i)/(n+1) + 2k, k >= 0.
Integrality of the norm for t = 1 gives 9 u^2 + 16 v^2 = 1 (mod 144): u = +-3 or +-5 (mod 16), v = +-2 (mod 9). With u = +-3 the class t = 5 would contain a vector of norm 15/16 + 8/9 + 25/144 = 2, a root outside M: impossible. So u = +-5, v = +-2. Norm 3 with |t| <= 20 then has exactly the solutions
- t = +-3: classes +-1 and +-3, 15/16 + 2 + 1/16: 2 * 16 * 84 = 2688 vectors;
- t = +-9: classes +-3 and 0, 39/16 + 0 + 9/16: 2 * 560 = 1120 vectors;
- t = +-13: classes +-1 and +-1, 15/16 + 8/9 + 169/144: 2 * 16 * 9 = 288 vectors;

in all 4096 vectors. (t = (x,z) is an integer with t^2/144 <= 3, so |t| <= 20; within the class of t the norms are the minimum plus an even integer; the minimum is 0 for t = 0, 3 for t = +-3, +-9, +-13, 4 for t = +-4, +-6, +-10, +-16, 5 for t = +-1, +-5, +-7, +-19 and at least 6 for the other t: norm 3 occurs only where the minimum is 3.) This reproduces the table, counts included; the classes with minimum 4 are the kinds of norm-4 vectors found by the programs as well.

Why the packing criterion does not settle 143

C1's packing test (`c1/code/envelope_pack.py`): m(x) = r/8 - E(x) with r = 312 and E(x) = sum over the positive roots of (alpha,x)^2 = 9 |P_W x|^2 + 16 |P_{a_15} x|^2. For the three kinds: E = 39, 23, 33, so m = 0, 16, 6. There are 1120 vectors with m = 0, so the 42 smallest values of m sum to 0, against 24 * min(16, 25) - 312 = 72 new roots: the test passes trivially. The tracker field "packing: survives" of 28/09 is consistent with this.

Status

- Proof: two lines from (R-ii), (R-v) and the table. Refereed by Codex (accepted); logic also checked by the laptop session.
- Table: the exact glue description (referee's checker; actual units u = 5 mod 16, v = 7 = -2 mod 9), and two programs with different enumeration code that find the same three kinds and counts.
- The record of 28/09 (solver verdict "component budgets infeasible", `scripts/component_budget.py`) is superseded: no solver output is used here.

6 The rules of the search program, derived from the properties of a frame

File: lemma/upstream/AUDIT_ENGINE_RULES.md

```
# Section B: audit of the frame-engine rules

Reviewer: CDX, 2026-10-07. This is the section-B deliverable requested with
‘AUDIT_REDUCTION.md‘, following the single section-A pass in
‘../t20_CODEX/REFEREE_REDUCTION.md‘. No frame search or solver was run, and
no producer, engine, queue or historical result was modified. The later
T16 certificate verification is recorded separately below.

**The mathematical rules below do not use g or the index [Nmax:N].** The
replacement of ‘g in {18,20}‘ by A.4 does not invalidate them. This is
**not an unconditional validation of the executable or of the 156 exclusions**:
the completeness and certification holes below must be addressed, or shown
irrelevant to the covering records. In particular, a mathematically valid
constraint does not certify a solver’s negative answer.

## Sources and scope

Line references ‘FD:‘ refer to ‘scripts/frame_dfs_next.py‘ in the local
‘C:/Users/     /Desktop/c2_experimental/c2_cloud_run_v10‘ package, SHA256
‘1591aeeac9c0be5c529a2ff492e66d390138cd221cba2ab67aac81748de9b759‘.
‘HS‘, ‘TT‘, ‘PB‘, ‘YP‘, ‘Y2‘, ‘EP‘ denote respectively the package’s
‘scripts/hstage_stream_next.py‘, ‘scripts/twin_transfer.py‘,
‘scripts/pair_budget.py‘, ‘c1/code/y_pairs.py‘, ‘c1/code/y2_engine.py‘,
‘c1/code/envelope_pack.py‘. Other local references are relative to
‘C:/Users/     /Desktop/c2_experimental‘:

- ‘SHELL‘: ‘external_runs/shell_theorems/conway99_shell_theorems/PROOFS.md‘, sections 4, 5, 5.2 and 7 read for the twin rules. Its theta
   /moment theorems were **not** audited in this pass.
- ‘FLEX‘: ‘external_runs/flex_boundary/conway99_flex_boundary/README.md‘, sections 1-3 read for boundary points and endpoint confinement
   .
- ‘SYM2‘: ‘scripts/make_frame_dfs_next.py‘, proof in its opening docstring; ‘SYM3‘: ‘scripts/sym3_patch.py‘, opening proof.
- ‘CB‘: ‘scripts/component_budget.py‘; ‘EB‘: ‘scripts/exact_budget.py‘, read, not executed.
- ‘R‘: section A of ‘AUDIT_REDUCTION.md‘ at master ‘c10bb83e1‘; the associated referee report records its hash.

‘AUDIT_ENGINE_SOURCES.json‘ pins the inspected files. The full DFS bitset
implementation, all numerical bounds, producer coverage, Gram-matrix identities,
historical environment switches and result provenance have not received a
complete executable audit here. Sections C-E retain those responsibilities.
‘OK‘ below means the **stated mathematical rule** follows from the hypotheses;
it is subject to candidate-list completeness and correct arithmetic.

## One row per rule

All rows were reread by **CDX** in this pass. A proof may be the direct
derivation in the row; longer supporting arguments are below. No row asserts
an uninspected original C1 note was reread.

| Rule / code location | Statement and why it follows from R | Written proof / reviewer / status |
|---|---|---|
| Normal norms and orthogonality; FD:186-187, Y2:191 | Norms 7 and mutual orthogonality are R-i. Early consumer traps run before these
   assertions, so input certification remains necessary. | R-i; CDX, OK rule; certificate caveat already in ‘../BRIDGE.md‘. |
| Root pair filter; YP header, Y2:201-203 | For a root a, E(a)=14-Y>=8, hence Y<=6 and Y in {0,1,2,4,5}; neither root is in J, whose
   minimum is 7. | R-vi; CDX, OK. |
| Norm-three pair filter; YP header, Y2:197-199 | E(x)=21-Y>=8 gives Y<=13: each height is at most 3, and both cannot be 3. | R-vi; CDX,
    OK. |
| New-root pair filter; FD:188-190, Y2:192-195 | At a shadow root beta, the two y and 42 u have half-odd pairings. The baseline energy
   is 11 of 14; one absolute 3/2 costs 2, and absolute 5/2 costs 6. | R-v and A.8; CDX, OK. |
| Exact dual trap represented by root/norm-three filters | A nonzero short vector of K*=proj_K U has a lift with normal heights in
   [-3,3], of norm <8/7+18/7<4; it is a root or norm-three vector. | R-vi and primitive J; SHELL section 4, line 234; CDX, OK. |
| Weyl dominance / diagram subgroup reduction | Reflections in norm-two roots preserve integral U; a chamber representative and the
   stabilizer chamber lose no orbit. Only checked lattice automorphisms may be used for further quotienting. | R is isometry invariant;
   ‘AUDIT_REDUCTION.md‘ C, YP/Y2 headers; CDX, OK principle, enumeration coverage belongs to C. |
| Swap; HS:44-48,86-89 | R is symmetric in the two normals. Root energy and the sorted two shadow-coset hit counts are isometry/sign
   invariants. Retain I(y1)<=I(y2), with the opposite order covered by the complete producer. | ‘AUDIT_REDUCTION.md‘ C(3); CDX, OK
   conditional on C’s coverage. |
| Characteristic trap; FD:1415-1428 | For characteristic z of norm 16, 42 odd word squares and the parity of the h contribution give E(z
   )>=46, hence Y<=66. | R-v; R A.8:149-150; CDX, OK. |
| Optional MIN97 / Y=13; FD:1431-1453 | A norm-three witness with Y=13 has E=8 and gives an odd integer eigenfunction of norm 16. **R-vi
    allows E=8**; excluding it additionally requires T16. | R-vii plus the separately reviewed cube/Wagner proof; CDX, OK with T16, see
   H5 and ‘../t20_CODEX/REFEREE_T16_PROOF.md‘. |
| Candidate h/u norms and perpendicularity; FD:191-194 | The chosen h must be among norm-four vectors perpendicular to y; the u among
   norm-three vectors perpendicular to y. | R-i, R-ii; CDX, OK; shell completeness H3. |
| h-admissibility at shadow roots; FD:192-193 | The remaining energy 3 at beta allows only integer h heights 0,+/-1. | R-v, A.8; CDX, OK
   . |
| Candidate sign representatives; FD:134-140 | Changing the sign of a word is a choice of orbit representative; changing an h sign
   interchanges Same/Opp labels. Each D7 pair still has one line of each kind. | R-ii..iv up to these gauge choices; CDX, OK. |
| Exact/flex coverage targets; FD:199-204 | A y-hit has h coverage 1 and no word hitter; with no eligible word hitter coverage is 3;
   otherwise it is 1 or 3. | R-v, A.8; CDX, OK. |
```

Boundary points; FD:1456-1472	Characteristic z with norm 16 and Y=58 satisfies 4 sum a_i^2+sum(b_u^2-1)=12. Thus h coverage is 3 with no hitter, or 1 with exactly one word of absolute height 3. Any witness subset is safe.	R-v; FLEX section 2; CDX, OK.
Optional WPRUNE; FD:1473-1490	For Y>=50, 4 sum a_i^2+sum(b_u^2-1)=70-Y, with sum a_i^2 odd and >=1: no word height >=5; h height bounds 2 at Y=58 and 4 at Y=50 are safe (larger infeasible Y cannot host a frame).	R-v and parity as A.8; code comment; CDX, OK.
Endpoint confinement, signed or unsigned; FD:1533-1544	If u hits a point s, both s and s-u have one signed-unit h footprint; their difference is u's D7 footprint. A covering h must be one of its two endpoints, with the stated sign.	R-iii, R-v; FLEX section 3; CDX, OK.
Pairwise h compatibility; FD:1298-1325,1370-1412	Chosen h are orthogonal, cannot share an exact target-1 point, and require nonempty Same and Opp word cells for their pair.	R-ii, R-iii and A.8; code docstrings; CDX, OK.
Word validity / Lemma U+; FD:1016-1020,1193-1203	Delete words with forbidden h products, more than two endpoints, a y-hit, or a hit at a point already covered twice. Such violations cannot be repaired by adding h.	R-iii, A.8 and boundary lemma; CDX, OK.
Empty cells; FD:1205-1216 (and parallel kernels)	Each chosen h pair needs a Same and an Opp word, orthogonal to all other chosen h; deleting a branch with either cell empty is necessary.	R-iii, the complete D7 column set; CDX, OK.
Common-zero and star counts; FD:1217-1226	With l h remaining, l(l-1) distinct words have both endpoints among them; each chosen h needs 2l words with its other endpoint still unchosen.	R-iii; CDX, OK.
Coverage capacity and saturation; FD:1079-1103,1236-1245	Remaining required covers cannot exceed remaining h or candidates through the point; saturated points forbid further covering h. A flex point without a still-valid hitter must reach 3.	A.8/boundary and monotone word validity; CDX, OK.
Flex leaf hitter; FD:1052-1066	Coverage 1 needs an actual word hitter with exactly two h endpoints; if none remains the leaf cannot extend.	R-iii, A.8/boundary; CDX, OK.
General branching / sibling deletion; FD:1133-1137,1175-1188,1248-1253	Branch on a required point; every completion has a first chosen remaining member through it. If no point is pending, branch on all remaining h. Delete only explored sibling candidates.	Finite subset enumeration and the required-cover rules; CDX, OK principle.
Generic non-flex branching; FD:349-365	If no point is pending before depth 7, best stays -1 but through[best] is still used. This needs an extra impossibility/precondition proof.	No such proof found in inspected sources; CDX, **HOLE H2**.
Reflection symmetry at depth 0; FD:1551-1575	Fix y1,y2 and an exact target-1 point. Its unique covering h can be moved to a representative of its reflection orbit; exact coordinate checks prevent false hash edges.	R is isometry invariant; FD comment and SYM2 preamble; CDX, OK.
Reflection symmetry at depth 1; FD:1578-1601 and dfs_general_u2	Also fix the first h and an uncovered target-1 point; the same unique-cover argument applies. Earlier target-1 siblings cannot occur in a completion containing the selected h.	SYM2 opening proof ; CDX, OK.
Reflection symmetry at later depths; FD:1106-1174,1246	Only along paths previously branching at target-1 points, reflections fixing all chosen h and the next such point preserve completions. symok is disabled after other branchings. Missing reflection edges only refine orbits.	SYM3 opening proof; CDX, OK.
Lemma Z; FD:255-258,449-452	Sum R-iv: 3 sum h is an even integral combination of u, since every C column has even sum. Hence sum h is in 2U. Sign changes of h do not affect this.	R-iii, R-iv; direct proof here; CDX, OK.
Local star sums; FD:252-312,433-507	For each h, its 12 endpoint words oriented to pair +1 must sum to 3h. Partial shadow-height sums must reach the target with remaining odd increments +/-1 and at most one additional +/-2 at a flex point.	R-iv and A.8/boundary; FD docstrings; CDX, OK.
Joint cells, distinct words and hitter counts; FD:1659-1706	Choose one word per Same/Opp cell, 42 distinct lines, with all seven star equations and exactly one hitter on each coverage-1 flex point.	R-iii, R-iv, A.8; CDX, OK constraints; enumeration cap is ** HOLE H1**.
Final T identity and frame operator; FD:1709-1722	With word Gram G_u=3I-T, polarized F gives G_u^2+C^t C=7G_u, i.e. T^2+T=12I-C^t C. Coefficient outer products must sum to 7G^-1.	R-v and R-iii; CDX, OK.
Node and stored-leaf limits; FD:1049,1077,1255,1640,1653	A node stop yields INCOMPLETE; leaf overflow raises an assertion and the consumer records INVALID. Neither is an exclusion. This does not cover the separate joint cap.	Code control flow, HS assertion handler; CDX, OK contract under normal assertions.
Lattice component budgets; CB header and model	Trace F on each root component and its perpendicular space: total projected energy is 7 times dimension. Candidate type counts and relaxed nonnegative normal energies must meet these budgets; root normal energy <=3 rank follows by summing Y(root)<=6.	R-v, R-vi; CB opening proof; CDX, OK model, solver/certificate **HOLE H4**.
Pair root-class budgets; PB pair_test	Sum F over roots sharing (component,Y): h/u energy equals (14-Y) times the class size; trace the remaining orthogonal space and require 42 u,7 h.	R-v, R-vi; PB docstring; CDX, OK model, solver/certificate **HOLE H4**.
Packing and packing-admissible words; EP:80-132	The 42 actual word hit sets are disjoint in each shadow coset, so their hit counts fit. The 42 smallest counts give a lower bound; fixing one candidate uses the other 41 smallest. Equality of the two coset hit counts follows by beta -> x-beta after orienting a hit positively.	R-v, A.8; EP header and derivation below; CDX, OK rule, numerical completeness H3.
Packing connectivity/span/dual trap; EP:133-164	The actual word graph is connected; all words must lie in one candidate component of rank >=22. Every sufficiently short dual vector of its integer span lies in the normal plane; its span plus the perpendicular space has dimension <=2.	R-ii..vii, R A.5; EP comments and algebra below; CDX, OK mathematical rule.
Twin construction and transfer; TT, HS:90-95	Replacing the normal residue gluing by R=[[2,2],[-2,2]] preserves K and all frame vectors, yields another odd unimodular U with no units. Transfer is valid only when **all** classes of the computed root type are excluded.	R-i, R-v, R-vi; SHELL sections 4,5,7; CDX, OK conditional on certified target exclusions.
Twin root catalogue; TT:31-43	Reduced normal residues transform precisely (norm,Y)=(2,0),(2,5),(3,8),(3,9),(4,18) into twin roots; every twin root has Y in {0,1,2,4,5}.	R-vi, primitive J; SHELL section 5.2; CDX, OK mathematics; numeric root/rank classification is subject to H3.
Twin deferral; HS:95	Strictly greater root count gives a well-founded dependency. A deferred pair is excluded only after every possible target class is excluded. chunk_done records traversal, not discharge of that dependency.	Twin theorem; SHELL section 7; CDX, OK conditional rule.
Twin canonical rule; HS:96-100	Only for a root type unique in the 156-list, move to its twin pair when the intrinsic normal/root key strictly decreases. R^2 mod7 is a signed normal permutation, so twinning back cannot produce an endless descent.	SHELL section 5.1 plus HS kkey/own_unique checks; CDX, OK conditional on D's identification and C's coverage.

Algebra supporting the packing row

These are explicit checks of the reductions used by the source, not additional
unannounced hypotheses. Write M=Z-span of the 42 actual words. R-iv gives
3h_i in M. Polarizing F and applying it to a word gives
‘sum_i C_io h_i = -4u_o + sum_{p != o}(u_o,u_p)u_p‘ in M.
The two D7 columns for {i,j} therefore give h_i+h_j and h_i-h_j in M,
hence 2h_i in M and then h_i in M. Thus M is the whole 49-vector frame
lattice N. This step is necessary: integrality on only the words must not
silently be identified with integrality on all 49 generators.

If its word graph were disconnected, its Gram matrix would split into
orthogonal blocks. The T identity would force C_A^t C_B=0 between different

components. D7 root lines on pairs with exactly one shared endpoint have
nonzero inner product. That adjacency graph on all 42 D7 lines is connected
(also connecting the plus and minus lines via a third endpoint), a contradiction.

For a candidate component M' containing N, a vector w integral on M'
projects to N* in the frame space. If ||w||^2<8/7, A.5 forces this projection
to vanish. The short dual vectors and span(M')^perp therefore all lie in
the two-dimensional normal plane. Their spans are orthogonal, giving the
dimension test used by EP. No determinant/index restriction on N is used.

For packing, orient a hit beta of a norm-three x so (x,beta)=3/2.
Then x-beta is a shadow root in the other shadow coset, also pairing 3/2
with x; this is an involutive bijection of hit lines. The actual packing
inequality needs these directly counted hits, not the optional theta
identity `m=r/8-E`. EP records that latter identity but does not assert it.

Holes and required evidence

H1. Silent cap on joint solutions

FD:1659 sets `cap=1000`; FD:1688 stops recursing after that many joint-star
solutions. `search` (1643-1655) checks only those solutions with `full_check`
and can return `EXCLUDED ... (F) fails` with `complete=True`. The first 1000
failures do not exclude a later solution. No bound of 1000 is proved in
the inspected source, and no incomplete flag is returned by `joint`.
The stored `joint_star_sums` field counts leaves with at least one joint
solution, not how many solutions were enumerated at each leaf.

This is a hole in the general executable's completeness contract, not a
claim that a covering run hit the cap. Archived evidence that every relevant
pair died before joint enumeration, or that every entered joint call was
exhausted below the cap, can establish irrelevance for particular records.
An engine repair alone would not retrospectively validate old records.

**Follow-up, LAB bridge message 52 (2026-10-07 13:33 UTC): not exercised
according to LAB's census.** LAB reports checking 17,676 stream/Golay
record files, 356 lab-local result files and 57 O24 class records, with
zero occurrences of the only affected exclusion verdict, `joint star sums,
but (F) fails`. The reported joint-stage exclusions are twelve pairs with
an empty joint list (eight in 001, four in 002); an empty list cannot have
hit the cap. LAB's importer now classifies any `but (F) fails` verdict as
`other`, refusing it, in commit `1dde18702`; CDX inspected that guard.
The census itself is peer evidence, not an independent CDX recount, and
its durable output/path has been requested. Thus H1 is a real code hole
reported unexercised in the stored exclusions, with a guard against future
acceptance; it is not evidence that those exclusions are wrong.

H2. Generic kernel assumes a pending point exists

FD:349 initializes `best=-1`; only a positive remaining target changes it.
FD:365 nevertheless reads `through[best]`, even if all targets are already
met and fewer than seven h are chosen. The general kernels correctly have
an all-candidates fallback. Saturation has normally removed all candidates
through the final point, so this generic branch may terminate early.

No inspected proof shows that a genuine completion cannot have remaining
h with zero footprint on all supplied points. The generic kernel is selected
by absence of flex points (FD:1548,1604), which is not itself that proof.
A proof of the additional precondition, or recorded absence of zero-footprint
h candidates in each relevant run, could discharge this hole. No actual
graph or missed genuine frame is asserted here.

Follow-up, message 52: H2 remains open, with reduced scope. LAB derives
the shadow-root energy operator from the two even neighbours' root
operators. For norm-four h, the sum over signed new roots of (beta,h)^2 is
`8(H1+H2)-4 sum_j h_j ||P_j h||^2 >= 8|H1-H2|`. CDX checked this algebra
and the equality criterion: zero footprint requires H1=H2=H and h entirely
in components of U with Coxeter number H. The expression counts nonzero
pairings only for admissible h (all nonzero heights have absolute value 1);
the zero criterion holds without that restriction. LAB's table calculation
therefore rules out zero footprints for 129 lattices. CDX has not independently
recomputed the table or the Niemeier input data.

LAB's stated potentially affected list has 27 IDs:
`001,003,006,007,014,022,023,024,026,027,028,037,049,050,052,053,077,079,080,101,103,104,105,117,137,138,152`.
Removing the two said to be excluded without the engine, 117 and 152,
leaves **25**, not the 24 stated in that message. Message 56 corrects this:
001 (Golay) was omitted from that count and is audited separately.
The reported zero exposed pairs among 63,825 sampled searched pairs is
sampling, not a proof. LAB proposes complete pair-stream exposure checks
and general-kernel searches for the exposed cases; no such run was performed
or authorized by CDX through this report.

Follow-up, message 56: `measures/AUDIT_RIGID_MANIFEST_2026-10-07.json`
now records the non-Golay jobs, and `q/audit_rigid.py` provides the exposure

check and general-kernel re-search. CDX read that consumer but did not run it.
The archived ‘measures/AUDIT_RIGID_golay_001.json‘ reports a complete stream
of 117,068 pairs: 117,050 general, 18 rigid and exposed. All 18 recorded
rechecks exit at the characteristic-vector trap before the kernel. CDX
checked these counts and verdicts in the JSON; the recorded engine hash
prefix matches the inspected engine. This is evidence for 001, not an
independent replay or a completion of the remaining 24 lattice audits.
H2 remains open globally.

Follow-up, message 68: archived coverage, not a new engine replay. CDX
read ‘measures/AUDIT_COVERAGE_2026-10-07.json‘ and the coverage collector
archived at ‘q/tools/lab_2026-10-07/audit_coverage.py‘. The report records
22 of the 25 lattices covered; 003, 022 and 026 remain incomplete. Every
entry has ‘exposed_not_excluded_again=0‘. For the twelve complete plan
entries using neither fixed-normal certificates nor overlapping chunks,
the recorded audited pair totals equal the sweep totals. These are checks
of the archived report, not independently regenerated streams or searches.

For **003**, 174 of 182 first-normal representatives have accepted audit
files (‘stream_ok && ok‘); five more, ‘[157,162,174,176,181]‘, are disposed
of by the separately checked fixed-normal certificate, **conditional on
T18 and T20**. The remaining representatives are ‘[2,3,180]‘. The report
keeps these dispositions separate: it does not invent audit files or pair
counts for a theorem-covered representative. Its 17,279,427 actually
audited pairs include 313 exposed pairs, all recorded as re-excluded.
The 172 selected files can cover 174 representatives because a chunk may
contain several representatives. This H2 audit coverage is distinct from
the complete first-normal coverage of the original 003 sweep.

The collector checks coverage of the plan’s residue-class representatives;
for identity-based streams it requires all chunks from one source. It
does not itself certify the provenance of every input/plan/engine used by
those files, or the validity of every engine verdict. Those obligations
and the other holes in this report remain. The reported engine hash prefix
is ‘1591aeeac9c0be5c‘, agreeing with the inspected engine. No jobs, manifests,
or exclusion-tracker records were changed by this review.

Snapshot SHA256 pins (file bytes):

- Coverage report, LF checkout: ‘65f13c00438f1923b9cbe1bc1ec1ee72af5c53fe48c4ddc679f994e98ec74fe4‘.
  The lab’s CRLF copy has SHA256
  ‘720b7fd152026441d86d0bb961c99127fede220990c5fefde8389a1c2ed20738‘;
  the texts agree after CRLF-to-LF normalization.
- Archived collector: ‘a957667ea0e10ed1b46cebb689b0892467647a3a6012046d92f6063f7cf4c856‘.

Follow-up, message 74: the recorded H2 coverage is now complete, 25/25.
CDX parsed the replacement coverage report: every lattice is covered and
every ‘exposed_not_excluded_again‘ is zero. For 003, 179 first-normal
representatives are covered by 177 audit files; the other three,
‘[162,176,181]‘, use fixed-normal certificates conditional on T18 and T20.
All three also belong to the independently checked witnesses in
‘../t20_CODEX/CERT_003_FIXED_Y1.json‘. There are no missing representatives.
The 17,698,621 actually audited pairs include 316 exposed pairs, all
recorded as re-excluded. Adding the two recorded but theorem-disposed
chunks (19,635 and 39,056 pairs) gives the recorded sweep total 17,757,312.
No count is fabricated for representative 181, which has no pair record.
This closes the outstanding H2 coverage/re-search requirement at the level
of the archived results and conditional certificates, with the source,
verdict and provenance scope of this report unchanged. The new coverage
report’s LF-byte SHA256 is
‘2c73718a9455fc8caac31918636f8e1f6788e357c06bde084bd59db8f9d4a271‘.

H3. Exhaustive numerical catalogues are not certified by exact acceptance

FD:69-131 prunes Fincke-Pohst branches with floating Cholesky/LDL data and
fixed tolerances, then checks each returned vector’s norm exactly. That
proves acceptance, not exhaustiveness. FD:153-154 checks norm-three and
shadow-root counts; it does **not** check completeness of X4, the h-candidate
shell. SHELL section 8 documents count certification for a 078 archive,
not all inputs used by the campaign. The numerical rank/root-type operations
in TT also need exact or certified evidence when used for transfers.

Thus the formal completeness justification is missing from the inspected
engine alone. Exact shell/count certificates for every relevant input,
or a certified enclosing enumeration, could close it without redoing all
frame searches. No vector actually missed by floating arithmetic was found
or claimed. Fast floating dot products likewise need their stated
small-integer bound or an exact comparison for the used data.

LAB confirms the missing norm-four assertion in message 52 and is looking
for complete shell-count certification. SHELL section 1.1 gives the claimed
count ‘49128-12r‘ for norm-four lines; its theta derivation was outside this
audit, and agreement with that count has not been checked here for every
used lattice.

Follow-up, message 56: LAB archived
‘measures/SHELL_COUNTS_2026-10-07.json‘ and reports exact norm checks on the
engine’s returned norm-four vectors. CDX read all 156 entries (000-155)
and checked that each has ‘N3=4096‘, ‘ok=true‘, and
‘N4=expected_N4=98256-24*N2‘, the signed version of the cited line count.
Thus the recorded counts agree for all 156, rather than just the earlier
078 example. This resolves the missing norm-four count comparison **at the
level of the archived result**, subject to the theta count, distinctness of
the enumerated vectors and provenance tying the measured input/source to
the covering runs. CDX did not re-enumerate these shells or audit the theta
derivation; the measurement script was not found in the inspected q/measures
sources. This evidence does not by itself certify every floating dot product
or numerical rank/root-type decision mentioned above.

Follow-up, message 72: joint-count completeness lemma accepted. The
true shell counts satisfy ‘N4+24*N2+24*N3=196560‘. If three enumerated
counts E2,E3,E4 count distinct, exactly accepted vectors and satisfy the
same equality, then their nonnegative deficits satisfy
‘(N4-E4)+24*(N2-E2)+24*(N3-E3)=0‘; all three deficits must vanish.
This proves simultaneous completeness without assuming N3=4096 first.
CDX independently checked the theta coefficients and all 156 archived rows;
see ‘../t20_CODEX/REFEREE_SHELL_COUNTS.md‘ and its executable/output.
The cited shadow formula also gives N3=4096 and N7/2=22,892,544; its exact
theta_4 argument q^2 is now explicit in ‘theta_odd24.py‘.

This resolves the mathematical count-completeness objection when the
acceptance premises hold. This referee pass did not recheck the underlying
156 lists or historical source/input/basis provenance; LAB was asked for
the measurement script to attach its exact membership/norm/distinctness
checks to the archived table. The other numerical rank/type and floating
predicate qualifications above are not removed by this count argument.

Follow-up, message 74: fresh exact acceptance for all 156 local inputs.
LAB archived the recovered first command (which checked fewer premises)
and a replacement ‘q/tools/lab_2026-10-07/shell_counts.py‘. CDX read it,
added an explicit no-overflow bound for its int64 norm contractions, and
reran it on all 156 inputs into a fresh referee output. In 148 seconds,
all basis/Gram, norm, distinctness/sign and joint-count checks passed.
The maximum of the 468 contraction bounds is 2,928,240, below 2^63.
Counts agree with both lab tables; every input hash and basis hash matches,
and the package’s 156 generator matrices agree with the lab inputs.
The data, summary, commands and scope are in
‘../t20_CODEX/REFEREE_SHELL_COUNTS.md‘. Short-shell completeness is certified
for these rebuilt local catalogues under the stated lattice hypotheses.
This is not retroactive acceptance of every historical machine or older
engine catalogue, nor certification of the shadow-vector lists or the
other numerical decisions identified in H3.

H4. Budget model versus negative solver verdict

PB returns ‘EXCLUDED (pair budget)‘ solely for HiGHS ‘status==2‘; CB makes
the same inference. These necessary integer/rational models are correct,
but the inspected calls produce no exact infeasibility certificate.
‘FD_PREBUDGET‘ is opt-in, so first establish whether any covering record
relied on it. For lattice-level budgets, EB implements an exact finite
enumeration of a weaker necessary condition; its code is present, but this
pass has not validated a complete run or its type-table provenance for 143.
That missing evidence must not be replaced by the solver’s status string.
Nothing was run during this audit, including EB (whose import also executes CB).

Message 52 reports 12,620 ‘EXCLUDED (pair budget)‘ pairs in lab-local files
and none in the cloud records LAB counted. Thus this issue is not merely a
question about an unused option. LAB is checking whether exact re-checks
exist; CDX has not validated them, and H4 remains open.

Follow-up, message 69: the lattice-level record for 143 is replaced.
CDX reviewed ‘LATTICE_143.md‘ at master ‘48ad0cee7‘ and freshly ran its
‘budget_143.py‘: all three norm-three projection types were reproduced.
The contradiction follows just from (R-ii), (R-v) and the type table: the
A8 budget permits at most 28 of the low-complement type, leaving at least
14 vectors with squared projection at least 9/16 onto a line, whose total
budget is only 7. The separate exact check in
‘../t20_CODEX/referee_143_checks.py‘ verifies the input lattice basis,
A15 + A8 embedding, primitive norm-144 complement and cyclic glue; its
41 height classes prove completeness of that type table without a
floating enumeration or theta-count assumption. See
‘../t20_CODEX/REFEREE_143_PROOF.md‘ for the argument, runs and scope.
Thus H4 is discharged for the **lattice-level 143 exclusion**, independently
of the solver and the v10 run. The other pair-budget records remain subject
to their own exact rechecks; this is not a global H4 closure.

H5. The 9/7 rule needs a separate support-16 theorem

R and A.5 give 8/7, not 9/7. R-vii turns the Y=13 norm-three witness into
a norm-16 eigenfunction; its exclusion is exactly the additional cube/Wagner

result cited by FD:1432 and ‘../BRIDGE.md‘. This pass did not referee that
proof. Record it as a separate reviewed dependency before counting any
‘FD_MIN97=1‘ or corresponding quick-deferral exclusion. This is not a
new objection to the already stated bridge dependency, nor evidence of an
incorrect cube/Wagner theorem.

Follow-up, message 56: separate T16 referee pass completed. See
‘../t20_CODEX/REFEREE_T16_PROOF.md‘. No blocking mathematical error was found
in the hand proof or coverage sources. A fresh exact check validated all
240 A and 32 B certificate obstructions, including a necessary Hall-witness
membership check omitted by the received verifier. All shipped witnesses
satisfy that check; the verifier repair and an index wording correction are
recorded in the report. H5’s separate-dependency requirement is discharged
by T16 plus this review; the rule still does not follow from R alone.

Answer to the index question

No inspected rule requires ‘g=18‘, ‘g=20‘, or ‘[Nmax:N]‘ equal to 1 or 7.
The determinant 49 of the **primitive normal complement** follows from
unimodularity of U and primitive J=7I2. It is used in the twin mathematics,
not as a restriction on the original frame lattice. Primitivity and
the same lower dual bound follow from R. Retaining the holes above is
compatible with accepting the new all-g reduction.